

FIREFLY SOFTWARE FOUNDATION



Firefly

for Rust

by Example

Event-Driven Rust Microservices
with the Firefly Framework

 **FIREFLY** · tokio + axum · Rust 1.85+

Firefly for Rust by Example



Event-Driven Rust Microservices with the Firefly Framework

Firefly Software Foundation

Copyright © 2026 Firefly Software Foundation.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this material except in compliance with the License. You may obtain a copy of the License at <https://www.apache.org/licenses/LICENSE-2.0>.

Unless required by applicable law or agreed to in writing, software and documentation distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

First edition, 2026.

All code listings in this book were written and verified against the Firefly framework for Rust, version 26.6.x, targeting Rust 1.85+ on the `tokio` + `axum` stack.

Published by the Firefly Software Foundation.

Preface

Rust gives you fearless concurrency, zero-cost abstractions, and a compiler that refuses to ship a data race. What it does not give you is *cohesion*. Every new back-office service forces the same cascade of decisions before a single line of business logic is written: which HTTP layer, which database story, how to wire dependencies, how to handle configuration, errors, correlation IDs, metrics, and graceful shutdown. **Firefly** changes that. It brings the cohesive, convention-over-configuration experience that Spring Boot gave the JVM world, rebuilt from the ground up for Rust 1.85+ on `tokio` and `axum`.

This book teaches Firefly **by example**. You build one real application from an empty crate to a secured, observable, event-sourced service — making every concept concrete before moving to the next. The code in these pages is not illustrative pseudocode: every listing is a slice of a **real project that compiles, boots, and passes its tests** against the framework at version 26.6.x. Each snippet was lifted from the running sample and checked against the crate APIs, so what you read is what actually works. When a listing drifts from the source, the sample's build breaks and a test fails — the same guarantee the sibling PyFly, Go, and .NET books make.

Who This Book Is For

This book is for intermediate Rust developers comfortable with `async` / `await`, traits, and the basics of HTTP services. You need no prior framework expertise — if you have built anything with `axum`, `actix`, or `sqlx`, you are well prepared.

Spring Boot, WebFlux, and Project Reactor developers will feel especially at home. Wherever Firefly mirrors a concept you already know — beans and stereotypes, declarative messaging, application events, `Mono` / `Flux` — a **Spring parity** or **Reactor parity** callout draws the parallel explicitly, so you map what you already know rather than learning from zero.

What You Will Build: Lumen

Every chapter advances **Lumen**, a digital-wallet and ledger service — the Rust analog of the same Lumen the PyFly, Go, and .NET books grow. Lumen lets a customer open a wallet, deposit and withdraw money, transfer funds between wallets, and read a live balance. Behind that small surface sits the full spread of patterns a real back-office service needs: a value object that does exact money arithmetic, an aggregate that enforces invariants, CQRS with a read-side cache, domain events, an event-sourced ledger, a compensating transfer saga, JWT-secured endpoints, an actuator surface, a scheduled task, and an end-to-end test suite.

The single most important property of Lumen is its dependency list:

TOML

[dependencies]

```
firefly = { version = "26.6.3" } # the whole framework – and every macro
axum    = { version = "0.7" }   # you author the handler functions
serde   = { version = "1", features = ["derive"] }
```

One Firefly dependency. The entire framework — CQRS, dependency injection, the reactive web stack, event-driven messaging, event sourcing, saga orchestration, scheduling, resilience, security, observability — and every `#[derive(...)]` / `#[...]` macro arrives through `use firefly::prelude::*`. The chapters make a deliberate point of this: even Lumen's typed error enums hand-write `Display` + `std::error::Error` instead of pulling in `thiserror`, so the one-dependency promise holds end to end.

The journey follows a deliberate arc, one slice of Lumen at a time:

- **Part I — Foundations.** You scaffold the first Lumen service, bind typed configuration and profiles, learn how the composition root wires collaborators,

master the **Mono** / **Flux** reactive surface, and expose your first validated REST endpoints.

- **Part II — Modeling & Persisting.** You stand up a read model behind a repository, model the domain with a **Money** value object and a **Wallet** aggregate, and split reads from writes with CQRS command and query handlers dispatched through a bus.
- **Part III — Event-Driven.** The aggregate raises domain events; a **#[event_listener]** projection keeps the read model current; and an **event-sourced ledger** rebuilds every balance by folding its event stream — with the same events ready to flow out to Kafka or RabbitMQ.
- **Part IV — Into Microservices.** Lumen reaches beyond its own process: a typed HTTP client sketch shows how a wallet would call an external payments provider, and an orchestrated **transfer saga** moves money across wallets and *compensates* when the credit leg fails.
- **Part V — Secure · Observe · Ship.** You secure the endpoints with JWT bearer auth and path-based RBAC, make the service observable with metrics, tracing, and an actuator admin surface, add a read-side cache and a scheduled housekeeping task, test the whole stack in-process, and finally ship it behind the **firefly** CLI with graceful shutdown and a reactive streaming endpoint.

By the last page you have a working, tested, observable, secured, event-sourced service — and the mental model to extend it.

How to Use This Book

Read sequentially. Each chapter builds on the one before, and the Lumen codebase grows incrementally; skipping ahead leaves gaps. The **Reactive Model** chapter is the keystone — the whole reactive surface builds on **Mono** and **Flux**, so read it before the service-building chapters. The early chapters (1–5) introduce the framework with

small standalone snippets; **Lumen proper begins in Chapter 6** and grows from there. Each chapter is *additive* — it never rewrites what an earlier chapter shipped, only extends it — so the final state is exactly the companion crate.

Type every listing yourself. Reading and typing code at the same time is how the patterns stick. Resist copy-pasting until you have written each listing at least once.

Run it. Lumen really runs. From the workspace root:

Shell

```
cargo run -p firefly-sample-lumen      # boot the service (API + admin)
cargo test -p firefly-sample-lumen     # run the unit + HTTP test suite
```

Whenever a chapter adds a feature, start the app or the tests and watch it work. Seeing real JSON come back from a real endpoint — and watching a saga *compensate* a failed transfer — is worth a hundred diagrams.

Each chapter closes with a **Recap** of what changed in the Lumen codebase and a set of **Exercises** that push one step further. The exercises are optional but recommended for anything you intend to apply immediately.

Conventions in Brief

Typographic and structural conventions — code-listing captions, callout types, and the Spring-parity boxes — are demonstrated, with live examples, in the **Conventions** section that follows.

The Companion Code

The complete, runnable Lumen project lives in the framework's `samples/lumen` directory. It is a single, clean Firefly crate — one module per concern (`money`, `domain`, `ledger`, `commands`, `transfer`, `security`, `web`, `housekeeping`) — that you grow chapter by chapter; the finished source there is the destination this book walks you to.

Build it once with `cargo build -p firefly-sample-lumen`, and use it to compare your work, catch up if you fall behind, or simply run the parts you are reading about. The chapter-by-chapter map of *what code lands where* lives alongside it in `docs/book/LUMEN-ARC.md`.

Conventions

This page explains the typographic and structural conventions used throughout the book — and demonstrates each one with a live example, so the first time you meet a callout or a code caption in a chapter it already looks familiar.

Code Listings

Every multi-line code example is real, compiling Rust drawn from the Lumen companion crate. Where it helps, a listing is introduced with the **file it lives in** so you can find it in `samples/lumen`, as in "`samples/lumen/src/money.rs`". Inline code references within prose use `monospace`, as in "the `#[rest_controller]` attribute generates the wallet router."

Here is a representative listing — the constructor and exact-arithmetic core of Lumen's `Money` value object, lifted verbatim from `samples/lumen/src/money.rs`:

Rust

```
/// An exact monetary amount, expressed in integer minor units (cents).
#[derive(Debug, Clone, Copy, Default, PartialEq, Eq, PartialOrd, Ord, Serialize,
Deserialize)]
#[serde(transparent)]
pub struct Money {
    cents: i64,
}

impl Money {
    /// A zero amount – the opening balance of a brand-new wallet.
    pub const ZERO: Money = Money { cents: 0 };

    /// Returns a new `Money` that is `self + other` (immutable addition).
    #[must_use]
    pub const fn add(self, other: Money) -> Money {
        Money { cents: self.cents + other.cents }
    }
}
```

A snippet annotated `rust,ignore` or `rust,no_run` elides surrounding setup for focus, but the API names, types, and method signatures are exactly what the crates expose. A listing fenced as plain `text` is shell output, a banner, or an HTTP exchange rather than Rust source:

Text

```
$ cargo run -p firefly-sample-lumen
:: lumen :: digital-wallet & ledger (v26.6.3)
```

The One-Dependency Reminder

Because Lumen's defining property is its single Firefly dependency, every framework type you see is reached through the facade — `firefly::cqrs::Bus`, `firefly::eventsourcing::EventStore`, `firefly::reactive::Flux` — or, for the high-frequency surface and every macro, through one glob:

Rust

```
use firefly::prelude::*;
```

When a chapter introduces a new framework type, the prose names the facade path it lives behind, so you always know it came in through that one dependency.

Callouts

Five callout styles appear throughout the body. Each is a blockquote that opens with a bold label, and the design theme styles them distinctly:

NOTE

Notes provide supplementary context or clarify a subtlety in the main text. Worth reading, but not blocking.

TIP

Tips share a shortcut, idiom, or best practice that will save you time in real projects — for example, keeping money in integer cents so floating-point drift can never corrupt a balance.

WARNING

Warnings flag a common mistake or a sharp edge that causes hard-to-debug problems if ignored — for example, that Lumen's free-function CQRS handlers publish their collaborators through a process-global `OnceLock`, so a second `build_app()` in the same test binary keeps the *first* wiring.

SPRING PARITY

Spring-parity callouts map a Firefly concept directly to its Spring Boot / Firefly-Java equivalent — ideal for developers migrating from the JVM ecosystem. For example: `#[rest_controller]` is Lumen's `@RestController`; `#[event_listener]` is `@KafkaListener`; the `Saga` / `Step` API is the Java `@Saga`. You will meet these in nearly every chapter.

REACTOR PARITY

Reactor-parity callouts map Firefly's `Mono` / `Flux` surface onto Project Reactor and WebFlux, so the reactive idioms you know transfer directly. For example: `Flux::just(items)` is Reactor's `Flux.fromIterable(...)`, and returning a `Flux` from a handler is the WebFlux streaming-endpoint model — exactly how Lumen's `GET /api/v1/wallets/:id/events` works.

Mapping Tables

When a Firefly spelling has a one-to-one counterpart in a framework you already know, a mapping table lines them up so you translate by lookup rather than by guesswork:

Concept	Spring Boot	Firefly for Rust
HTTP controller	<code>@RestController</code>	<code>#[rest_controller]</code>
Message listener	<code>@KafkaListener</code>	<code>#[event_listener]</code>
Scheduled task	<code>@Scheduled</code>	<code>#[scheduled]</code>
Distributed transaction	<code>@Saga</code>	<code>Saga + Step</code>

Recap & Exercises

Each chapter closes with two fixed sections:

- A **Recap** — **what changed in Lumen** that lists the files added or extended and the one-sentence "by the end of this chapter, Lumen can ..." payoff.
- A set of **Exercises** that push one step further — usually a small, self-contained extension to the code the chapter just shipped. They are optional but recommended for anything you intend to apply immediately.

Turn the page to [Why Firefly for Rust](#), where the Lumen journey begins.

Contents

PART I Foundations

1. Why Firefly for Rust?	17
2. Quickstart: Zero to a Running Service	26
3. Configuration, Profiles & Secrets	38
4. Dependency Injection & the Application Context	50
5. Dependency Wiring & Composition	64
6. The Reactive Model — Mono & Flux	78

PART II Modeling & Persisting the Domain

7. Your First HTTP API	99
8. Persistence & Reactive Repositories	112
9. Domain-Driven Design	130
10. CQRS: Commands & Queries	147

PART III Event-Driven Architecture

11. Event-Driven Architecture & Messaging	167
12. Event Sourcing the Ledger	188

PART IV Into Microservices

13. HTTP Clients & Calling Other Services	210
14. The Experience Tier: Composing a BFF	223
15. Distributed Transactions: Sagas, Workflows & TCC	234

PART V Secure, Observe & Ship

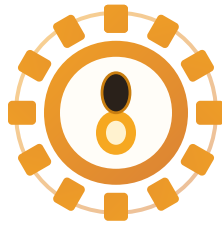
16. Security, Sessions & Identity	253
17. Observability & Health	266

CONTENTS

18. Caching & Resilience	278
19. Scheduling, Notifications & Webhooks	290
20. Declarative Services with Macros	302
21. Testing Firefly Applications	314
22. The firefly CLI	327
23. Extending Firefly & Going to Production	338

Appendices

A. Spring Boot → Firefly Cheat-Sheet	352
B. Crate & Module Index	366
Glossary	376



PART I

Foundations





CHAPTER 1

Why Firefly for Rust?

By the end of this chapter you will understand the problem Firefly solves, how its tiers fit together behind a single facade crate, and why Lumen — the digital-wallet service you grow over the rest of the book — depends on exactly **one** Firefly crate to get all of it. No code lands in Lumen yet; this chapter sets the stage and the philosophy. The next one boots the scaffold.

The cohesion problem

Picture your first day on a new Rust microservice. Before you write one line of business logic, you face a cascade of choices. Which HTTP layer — axum, actix, warp, poem? Which database story — sqlx, SeaORM, diesel, raw `tokio-postgres`? How do you wire dependencies — a hand-rolled `AppState`, a DI crate, lazy statics? How do you handle configuration, errors, correlation IDs, metrics, graceful shutdown? Every team invents its own answer.

You assemble a bespoke stack, glue it together with good intentions, and ship. Six months later a second team starts a second service and makes entirely different choices. Now you have two codebases with incompatible conventions, different error shapes, different observability stories, and no shared understanding of how anything works.

Rust gives you infinite choice. What it does not give you is cohesion.

The stack-assembly problem is not a skills failure — it is a tooling gap. Java developers solved it with Spring Boot: one opinionated framework that makes sensible choices, lets you override what matters, and enforces a consistent idiom across every service. Firefly brings that same discipline to Rust.

What is Firefly?

Firefly is a **cohesive, reactive, async-native framework** for building production-grade Rust services. It makes the cross-cutting decisions for you — HTTP middleware, configuration, caching, CQRS, messaging, security, observability — all integrated, all consistent, with production-ready defaults from the very first `cargo run`.

Under the hood Firefly delegates to battle-tested libraries — `tokio` for the runtime, `axum` / `tower` for HTTP, `serde` for serialization, `tracing` for logging, RustCrypto for crypto — but you depend on **Firefly's ports** (object-safe `async_trait` traits), and you select concrete adapters at wiring time. Swap an in-memory event store for PostgreSQL, or the in-process broker for Kafka, without touching a single line of business logic — exactly the swap Lumen is structured to make.

Firefly's defining principles:

- **Composed, not constructed.** One call wires the whole infrastructure tier — middleware chain, cache, CQRS bus, event broker, health composite, metrics,

scheduler, lifecycle. You write commands, queries, handlers, and routes; nothing more. Lumen builds its core with `WebStack::new(CoreConfig { .. })`.

- **Symmetric across runtimes.** The wire contract, the `application/problem+json` shape, the `Idempotency-Key` semantics, the saga step definitions, the event envelopes — all identical to the Java, .NET, Go, and Python siblings. The Lumen you build here is the Lumen those books build.
- **Pluggable at the adapter layer.** Each integration point (cache, broker, IDP, ECM, notification channel) is an object-safe port with multiple adapter implementations selected at wiring time as an `Arc<dyn Port>`.
- **Observable by default.** `tracing` structured logging with correlation-id enrichment, actuator health/metrics endpoints, RFC 9457 error envelopes, and a startup banner are all on out of the box.
- **Reactive to the core.** A first-class `Mono` / `Flux` reactive surface — the Rust analog of Project Reactor — runs from reactive endpoints through reactive repositories, the reactive HTTP client, and reactive EDA/CQRS.

🌿 SPRING PARITY

If you come from Spring Boot, building the Firefly core is your `@SpringBootApplication` auto-configuration: one call stands up the middleware, the bus, the broker, health, and metrics. The configuration hierarchy (defaults → profile → env vars) maps to `application.yaml` + profiles, and returning a `Mono<T>` / `Flux<T>` from a handler is the WebFlux `@RestController` model. A **Spring parity** callout appears wherever the concepts align closely enough to save you the translation.

The one-dependency facade

Here is the part that surprises people. Lumen — a service with CQRS, event sourcing, a saga, JWT security, scheduling, and an actuator surface — declares exactly one Firefly dependency. This is its real `Cargo.toml`:

TOML

[dependencies]

```
# The whole framework AND every `#[derive(...)]` / `#[...]` macro.
firefly = { version = "26.6.3" }

# The two ecosystem crates a Firefly service still writes against directly:
# axum (you author the controller handlers) and serde (your messages and
# event payloads are Serialize/Deserialize).
axum = { version = "0.7" }
serde = { version = "1", features = ["derive"] }
serde_json = "1"

# Async runtime, plus the id/clock crates the wallet domain uses.
tokio = { version = "1" }
uuid = { version = "1", features = ["v4"] }
chrono = { version = "0.4", features = ["serde"] }
```

The `firefly` crate is a **facade**: it re-exports every `firefly-*` crate behind a clean path (`firefly::cqrs`, `firefly::eventsourcing`, `firefly::reactive`, `firefly::security`, ...) and re-exports every macro at the crate root. The high-frequency surface — plus all the macros — comes in through a single glob:

Rust

```
use firefly::prelude::*;
```

That one line gives Lumen the CQRS `Bus`, the dependency-injection `Container`, the `Scheduler`, the saga `Saga` / `Step`, the lifecycle `Application`, the reactive `Mono` / `Flux`, the `WebResult` / `WebError` web types, the `FireflyError` kernel error, and every `#[derive(...)]` / `#[...]` macro the service uses.

Lumen takes the discipline one step further: even its typed error enums — `MoneyError`, `DomainError`, `CqrsError` mapping — hand-write `Display` and `std::error::Error` instead of reaching for `thiserror`. The one-dependency promise holds end to end, and the chapters point it out where it matters.

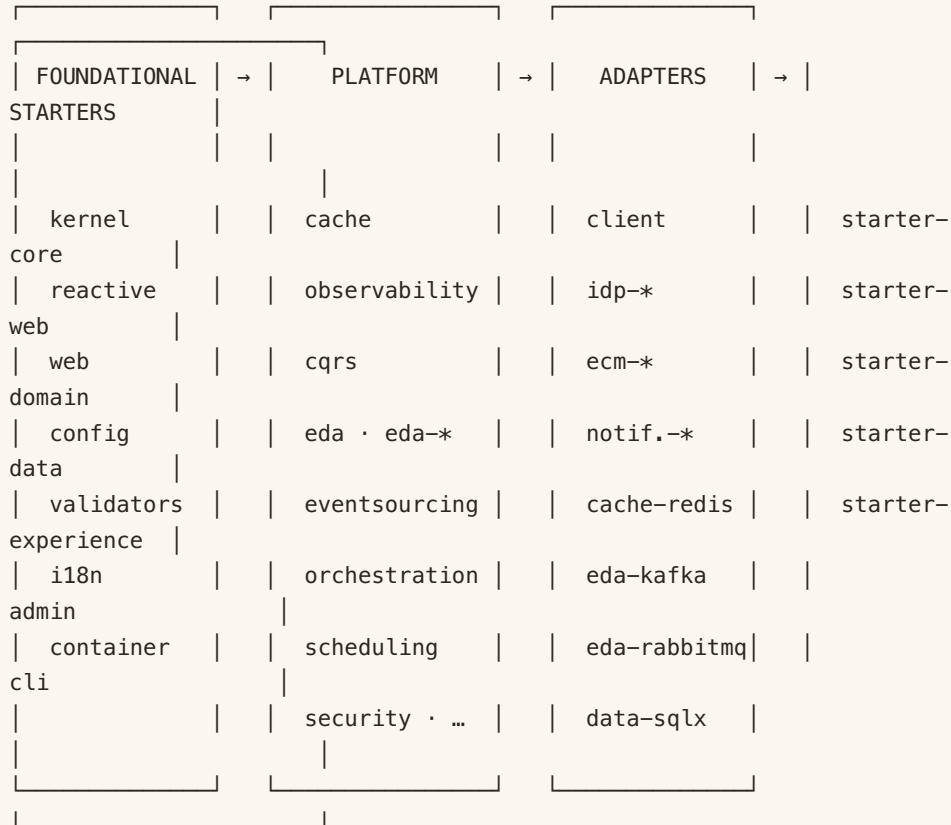
🌱 SPRING PARITY

The `firefly` facade is the spirit of a Spring Boot *starter*: one coordinate on your dependency list pulls in a curated, version- aligned stack, and the macros are your annotations. `use firefly::prelude::*;` is the equivalent of importing `org.springframework.*` and having the annotations just be there.

The tiers behind the facade

Behind that single crate the framework is organized into strictly-layered tiers, with a left-to-right dependency direction. Each tier may depend on the tiers to its left, never to its right; the Cargo crate graph enforces the layering. You rarely name these crates directly — the facade re-exports them — but knowing the shape tells you where each capability lives.

Text



- **Foundational** crates are the vocabulary: `firefly-kernel` (errors, clock, correlation scopes, DDD kit), `firefly-reactive` (`Mono` / `Flux`), `firefly-web` (middleware), `firefly-config`, `firefly-validators`, `firefly-i18n`, `firefly-container` (DI).
- **Platform** crates are the capabilities: caching, CQRS, EDA, event sourcing, orchestration, scheduling, resilience, security, observability. Lumen reaches for `firefly::cqrs`, `firefly::eventsourcing`, `firefly::orchestration`, `firefly::scheduling`, and `firefly::security` here.
- **Adapters** are the concrete integrations: the REST/reactive HTTP client, the IDP vendors, ECM, notifications, and the event transports (Kafka, RabbitMQ, Postgres).

outbox, Redis Streams). Lumen ships on the in-memory adapters and points at the production swaps in callouts.

- **Starters** bundle a sensible default stack so a service depends on one crate. Lumen's web tier is `firefly::starter_web::WebStack`, which wires the core (`firefly::starter_core`) plus the web middleware in one constructor.

For the full per-crate catalogue see the [Module Index](#).

Choosing your adapters

Lumen runs with **zero external infrastructure** — that is what makes it a good teaching baseline and a fast test target. It boots on the in-process `MemoryEventStore` and the in-process broker, so `cargo run` and `cargo test` need nothing but the crate. When you are ready for production, you change the *wiring*, not the handlers:

- **Event store.** Swap `MemoryEventStore` for a durable adapter where the `Arc<dyn EventStore>` is constructed; the `Ledger`, the projection, and every command handler are untouched.
- **Event transport.** The in-process broker that carries Lumen's domain events implements the same `Broker` port as `firefly-eda-kafka`, `-rabbitmq`, `-postgres`, and `-redis`. Swap the constructor, keep your `#[event_listener]`.
- **Cache, identity, notifications.** Code against the parent-port trait (`cache::Adapter`, `security::Verifier`, `notifications::Channel`) and pull in the concrete adapter crate at wiring time, so heavy SDKs stay out of services that do not use them.

This is the thread that runs through the whole book: Lumen is written so the in-memory baseline and the production deployment differ only at the composition root.

The road ahead: Lumen, chapter by chapter

The rest of the book is Lumen's growth, additive and in order. The early chapters introduce the framework with small standalone snippets; **Lumen proper begins in Chapter 6:**

- **Foundations** — scaffold and boot Lumen, bind its configuration and profiles, understand the composition root, master `Mono` / `Flux`, and expose the first validated REST endpoints.
- **Modeling & persisting** — a read model behind a repository, the `Money` value object and the `Wallet` aggregate, and the CQRS command/query split on a bus.
- **Event-driven** — domain events, a projection that keeps the read model current, and the event-sourced ledger that folds its stream.
- **Into microservices** — an HTTP-client sketch and the compensating transfer saga.
- **Secure · observe · ship** — JWT bearer auth and RBAC, the actuator surface, caching, a scheduled task, the test suite, and the production entry point with graceful shutdown and a reactive streaming endpoint.

By the last page, Lumen is the complete `samples/lumen` crate — and you have written every line of it.

Recap – what changed in Lumen

Nothing in code yet. This chapter framed the journey:

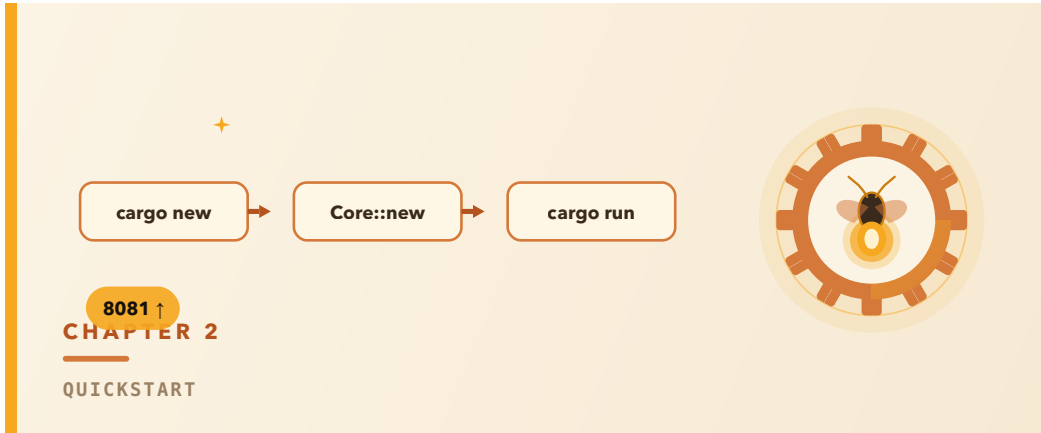
- The **cohesion problem** Firefly exists to solve, and the Spring-Boot-style answer it brings to Rust.
- The **one-dependency facade** — Lumen depends on a single `firefly` crate, and `use firefly::prelude::*;` brings in the whole high-frequency surface and every macro. Even the typed errors avoid `thiserror`, so the promise holds end to end.

- The tiers behind that facade (foundational → platform → adapters → starters) and the in-memory-to-production **adapter swap** that Lumen is built to make at the composition root.

Exercises

1. Open `samples/lumen/Cargo.toml` and confirm the dependency list: one `firefly`, plus `axum` / `serde` / `serde_json` / `tokio` / `uuid` / `chrono`. Note that no `firefly-*` sub-crate is listed directly.
2. Skim `samples/lumen/src/lib.rs`. List the eight modules it declares (`money`, `domain`, `ledger`, `commands`, `transfer`, `security`, `web`, `housekeeping`) and predict which book part introduces each.
3. Run `cargo doc -p firefly-sample-lumen --open` and read the crate-level documentation. It contains the same "building block → module → Firefly surface" table the book is organized around.
4. For each of these production swaps, find the port trait it would implement in the facade: a Postgres event store, a Kafka broker, a Redis cache. (Hint: `firefly::eventsourcing`, `firefly::eda`, `firefly::cache`.)

The next chapter gets Lumen running. Turn to the [Quickstart](#).



CHAPTER 2

Quickstart: Zero to a Running Service

*By the end of this chapter **Lumen** — the digital-wallet and ledger service you will grow across the rest of the book — exists as a real crate: it compiles, prints a banner, serves a live actuator, and shuts down gracefully. It does almost nothing yet. That is the point. Everything from here on is additive: every later chapter slices a little more out of the finished `samples/lumen` crate and folds it back into the story, and nothing you write now gets thrown away.*

This chapter takes you from an empty directory to a running Lumen process in a few minutes. Two paths get you there: the `firefly` CLI (fastest) or plain `cargo`. Either way, the destination is the same — a binary crate whose *only* Firefly dependency is the `firefly` facade.

Prerequisites

Shell

```
rustc --version    # 1.85 or later
cargo --version
```

That is all you need. Lumen's default stack requires **no external infrastructure** — the event store, the event broker, and the read model are all pure Rust, in-process. You will swap each of them for real infrastructure (Postgres, Kafka) in [Production & Deployment](#), but never before you are ready.

Path A – scaffold with the **firefly** CLI

Install the developer CLI once, then scaffold the project:

Shell

```
cargo install --path crates/cli    # from a checkout of the framework
# or: cargo install firefly-cli

firefly new lumen --archetype web-api --features web,cqrs --git
cd lumen
cargo run
```

firefly new generates a Cargo crate with a **src/** tree, a **firefly.yaml**, a **.gitignore**, a **README.md**, a **Dockerfile**, and a **tests/** directory. The **web-api** archetype is the right starting shape for Lumen: a web service with the CQRS bus already wired. See [The CLI](#) for every archetype and generator.

 TIP

Run `firefly new --list` to see every archetype (`core` , `web-api` , `web` , `hexagonal` , `library` , `cli`) and feature flag, or `firefly new lumen --dry-run` to preview the plan without writing files.

Path B – start from cargo

If you would rather see every line yourself, create the crate by hand. This is exactly the shape `samples/lumen` has, so the rest of the book lines up with it listing for listing.

Shell

```
cargo new lumen
cd lumen
```

Lumen's `Cargo.toml` makes the one-dependency story concrete. The whole framework — CQRS, dependency injection, the reactive web stack, event sourcing, saga orchestration, scheduling, security, observability — and *every* `#[derive(...)]` / `#[...]` macro arrive through a single crate:

TOML

```
# Cargo.toml
[dependencies]
# The one-dependency front door: the `firefly` facade re-exports the whole
# framework AND every macro. Generated code resolves runtime types through the
# facade, so Lumen never lists the underlying `firefly-*` crates.
firefly = "26.6.3"

# The two ecosystem crates a Firefly service still writes against directly:
# axum (you author the controller handlers) and serde (your messages and event
# payloads are Serialize/Deserialize). serde_json encodes the event payloads.
axum = "0.7"
serde = { version = "1", features = ["derive"] }
serde_json = "1"

# The async runtime, and the id/clock crates the domain uses later.
tokio = { version = "1", features = ["rt-multi-thread", "macros", "net",
"signal"] }
uuid = { version = "1", features = ["v4"] }
chrono = "0.4"
async-trait = "0.1"

[features]
# The reactive streaming endpoint is feature-gated so the teaching baseline
# stays lean; chapter 20 turns it on. It needs nothing beyond `firefly`.
default = []
streaming = []
```

🌱 SPRING PARITY

A Spring Boot service pulls in a constellation of `spring-boot-starter-*` artifacts, and pyfly enables subsystems with `@enable_domain_stack`. Firefly for Rust collapses all of that into one `firefly` line. There is no starter to forget and no version skew between `firefly-web` and `firefly-cqrs` — they ship as one calendar-versioned release and you depend on the facade.

Lumen's two entry points

A Firefly service has a *composition root* (where the application is assembled) and a *process entry point* (`main`, where it is run). Keeping them apart is what lets the tests drive the fully-wired app in-process without binding a socket — you will lean on that hard in [Testing](#).

Lumen names the assembled application `LumenApp` and builds it in `src/web.rs::build_app`. For now the body is almost empty; later chapters give it a controller, a CQRS bus, an event-sourced ledger, and a security chain. What matters in this chapter is the shape and the two constants that name the service:

Rust

```
// src/web.rs
use firefly::prelude::*;
use firefly::starter_web::WebStack;

/// Lumen's application name (banner + `/actuator/info`).
pub const APP_NAME: &str = "lumen";

/// The released framework version, surfaced in the banner.
pub const VERSION: &str = firefly::VERSION;

/// The fully-assembled Lumen application. Right now it carries only the
/// web-tier stack; the bus, ledger, and read model arrive in later chapters.
pub struct LumenApp {
    /// The web-tier starter (CORS / security-headers / correlation / metrics
    /// on by default), which `Deref`s to the infrastructure `Core`.
    pub web: WebStack,
}

/// Assembles a `LumenApp` over in-memory infrastructure – the default for
/// tests and a no-infra `cargo run`.
pub async fn build_app() -> LumenApp {
    let web = WebStack::new(firefly::starter_web::CoreConfig {
        app_name: APP_NAME.into(),
        app_version: VERSION.into(),
        ..Default::default()
    });
    LumenApp { web }
}
```

`WebStack::new` is the one call that wires the whole web tier: the RFC 9457 problem renderer, correlation-id propagation, idempotency replay, the in-process cache, the CQRS bus, the event broker, the health and metrics registries, the scheduler, plus the web batteries (CORS, security headers, request metrics, the access log) — all from

those defaulted `CoreConfig` fields. `WebStack` derefs to the infrastructure `Core`, so `app.web.bus`, `app.web.broker`, and friends are right there when later chapters need them.

SPRING PARITY

`WebStack::new(CoreConfig { .. })` is the Rust spelling of `@SpringBootApplication` + auto-configuration (and of pyfly's `@enable_web_stack`): one declaration stands up the entire managed context. The difference is that nothing is reflective or hidden — `CoreConfig` is a plain struct whose fields *are* the knobs, so "what is wired" is exactly "what the struct says."

The process entry point lives in `src/main.rs`. It builds the app, turns on logging, prints the banner, serves the public API and the actuator on separate ports, and runs under the lifecycle `Application` with graceful shutdown:

Rust

```
// src/main.rs
use firefly_sample_lumen::web::{build_app, APP_NAME, VERSION};

/// Default bind address of the public API server.
const DEFAULT_ADDR: &str = "127.0.0.1:8080";
/// Default bind address of the admin (actuator) server.
const DEFAULT_ADMIN_ADDR: &str = "127.0.0.1:8081";

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let app = build_app().await;
    // Best-effort: a test harness may already own the global subscriber.
    let _ = app.web.init_logging();

    let api = app.router();
    let admin = app.web.actuator_router(Vec::new());

    app.web.print_banner();
    println!("{}", APP_NAME) :: digital-wallet & ledger (v{VERSION});

    let api_addr = std::env::var("LUMEN_ADDR").unwrap_or_else(|_|
DEFAULT_ADDR.to_owned());
    let admin_addr =
        std::env::var("LUMEN_ADMIN_ADDR").unwrap_or_else(|_|
DEFAULT_ADMIN_ADDR.to_owned());

    let application = app
        .web
        .new_application()
        .on_server("api", move |shutdown| async move {
            let listener = tokio::net::TcpListener::bind(&api_addr).await?;
            axum::serve(listener, api)
                .with_graceful_shutdown(shutdown.wait())
                .await?;
            Ok(())
        })
        .on_server("admin", move |shutdown| async move {
            let listener = tokio::net::TcpListener::bind(&admin_addr).await?;
```

```

        axum::serve(listener, admin)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    });

    if let Err(err) = application.run().await {
        if !err.is_cancelled() {
            eprintln!("application failed: {err}");
            std::process::exit(1);
        }
    }
    Ok(())
}

```

A few things to notice, because they recur in every chapter:

- `app.router()` is the public surface. In this chapter it is empty; in [Your First HTTP API](#) it gains the wallet routes, and in [Security](#) it gains the JWT layer. `main` never changes when that happens — the composition root absorbs the growth.
- `app.web.actuator_router(...)` is the management surface, served on a *separate* port (`8081` by default) so it never leaks onto the public network.
- `LUMEN_ADDR` / `LUMEN_ADMIN_ADDR` override the bind addresses from the environment — your first taste of the typed configuration story in [Configuration](#).
- `application.run()` traps SIGINT/SIGTERM and drains in-flight requests before exiting. A cancelled run is a clean shutdown, not an error.

Run it

Shell

```
cargo run
```

You will see the Firefly banner, then Lumen's own line:

Text

```
:: lumen :: digital-wallet & ledger (v26.6.3)
```

Even with no business routes yet, the actuator is live on the admin port:

Shell

```
# Liveness / readiness – on the admin port, never the public one.
curl localhost:8081/actuator/health
# {"status":"UP", ...}

# Build metadata – note app_name and app_version flow straight from CoreConfig.
curl localhost:8081/actuator/info
# {"app":{"name":"lumen","version":"26.6.3"}, ...}
```

What you got for free

Without writing any of it yourself, Lumen already has:

- **RFC 9457 problem responses.** Any handler error renders as `application/problem+json`, and a panic is caught and rendered as a 500 problem. (You will use this from the very first endpoint in chapter 6.)
- **Correlation IDs.** Every response echoes an `X-Correlation-Id`; an incoming one is honored and scoped through the whole request.
- **Idempotency.** Every `POST` / `PUT` / `PATCH` carrying an `Idempotency-Key` header is recorded; repeating the request replays the stored response, and reusing the key with a different body is a `409`.
- **A management surface.** `/actuator/{health,info,metrics,...}` on a separate listener.

- Graceful shutdown. `application.run()` traps SIGINT/SIGTERM and drains.

🌿 SPRING PARITY

This is the Spring Boot Actuator experience — health, info, metrics on a management port, plus production-grade request middleware — but stood up by a single `WebStack::new`, with no `application.properties` to author first and no annotations to remember.

Recap – what changed in Lumen

Before	After this chapter
empty directory	a compiling <code>firefly-sample-lumen</code> crate with one Firefly dependency
no entry point	<code>build_app()</code> composition root + a <code>main</code> that runs under the lifecycle <code>Application</code>
nothing to run	a live actuator on <code>:8081</code> , a (still-empty) public API on <code>:8080</code> , graceful shutdown
—	<code>APP_NAME</code> / <code>VERSION</code> constants that name the service and feed <code>/actuator/info</code>

Lumen is now a real, runnable service that happens to have no business logic. Every subsequent chapter fills that emptiness in — never by rewriting what you have, only by extending the composition root.

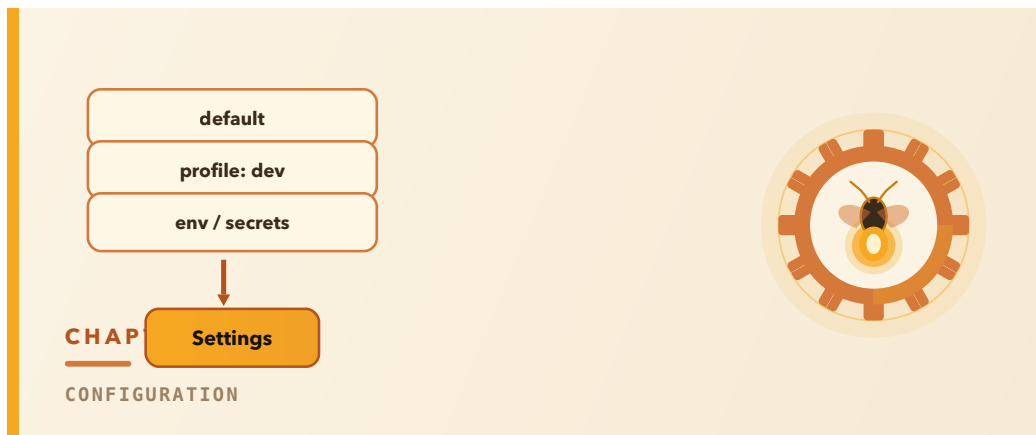
Exercises

1. Move the ports. Start Lumen with `LUMEN_ADDR=127.0.0.1:9090` `LUMEN_ADMIN_ADDR=127.0.0.1:9091` `cargo run`, then `curl localhost:9091/actuator/health`. Confirm the public and admin surfaces moved independently — this is the seam [Configuration](#) builds on.

2. **Read your own metadata.** `curl localhost:8081/actuator/info` and find the `app.name` / `app.version` values. Change `APP_NAME` in `web.rs`, re-run, and watch the banner and `/actuator/info` both follow — one constant, two surfaces.
3. **Provoke graceful shutdown.** Run Lumen, then press `Ctrl-C`. Notice the process exits cleanly (no stack trace): `application.run()` treated the signal as a shutdown, not a failure.
4. **Preview the scaffold.** Even if you took Path B, run `firefly new lumen2 --archetype web-api --dry-run` and compare the generated plan to the files you wrote by hand.

Where to go next

- Add typed, layered, profile-aware configuration in [Configuration](#) — and replace those raw `std::env::var` calls.
- Learn how the composition root resolves collaborators in [Dependency Wiring](#).
- Give Lumen its first real endpoints in [Your First HTTP API](#).



CHAPTER 3

Configuration, Profiles & Secrets

By the end of this chapter Lumen reads its identity and its bind addresses from configuration instead of hard-coding them: the `app_name` and `app_version` that flow into the banner and `/actuator/info`, and the `LUMEN_ADDR` / `LUMEN_ADMIN_ADDR` overrides `main` already honors. You will also see the typed, layered, profile-aware machinery Lumen grows into as it moves toward production.

In the last chapter Lumen named itself with two `pub const` strings and pulled its ports straight off the environment with `std::env::var`. That is the right starting point — but a real wallet service runs in dev, in CI, and in prod, and each environment wants different ports, log levels, and (eventually) database URLs. `firefly-config` brings Spring Boot–style **typed, layered configuration binding** to Rust: you declare a `serde`-deserializable struct, call `load` / `load_from_profile`, and the loader merges sources in precedence order, resolves the active profile, resolves `${...}` placeholders, and binds the flat dot-keyed map onto your struct.

SPRING PARITY

This is `@ConfigurationProperties` plus the `application.yaml` → profile → environment hierarchy, re-expressed for Rust — and the Rust analog of pyfly's `pyfly.yaml` + `Settings`. The binding rules below are deliberately identical across the Java, Go, .NET, and Python ports, so one `application.yaml` flattens to the same keys everywhere.

Where Lumen is today: app identity

Recall the composition root from the Quickstart:

Rust

```
// src/web.rs
let web = WebStack::new(firefly::starter_web::CoreConfig {
    app_name: APP_NAME.into(),           // "lumen"
    app_version: VERSION.into(),         // firefly::VERSION
    ..Default::default()
});
```

`CoreConfig` is itself plain configuration: every field is a knob, and the two Lumen sets — `app_name` and `app_version` — are exactly the values `/actuator/info` reports and the banner prints. The remaining fields default (in-memory cache, in-process broker, a fresh CQRS bus), which is why a bare `cargo run` needs no infrastructure. Promoting any of those to real infrastructure is a one-field change you will make in [Production & Deployment](#); the config story in this chapter is how those values stop being literals and start coming from files and the environment.

Defining configuration

A configuration struct is plain `serde`. Nested structs become nested dot-keyed sections (`web.port`, `cache.adapter`). Here is the shape Lumen would adopt as it outgrows the two constants:

Rust

```

use serde::Deserialize;

#[derive(Debug, Deserialize)]
struct Web {
    /// Public API bind address – the typed home of LUMEN_ADDR.
    addr: String,
    /// Admin/actuator bind address – the typed home of LUMEN_ADMIN_ADDR.
    admin_addr: String,
}

#[derive(Debug, Deserialize)]
struct LumenConfig {
    name: String,
    web: Web,
    tags: Vec<String>,
}

```

The binder is **type-driven**: `"9090"` binds onto a `u16`, `"alpha,beta"` splits onto a `Vec<String>`, `"true"` parses onto a `bool`, and missing keys produce zero values — so plain `#[derive(Deserialize)]` structs bind without `#[serde(default)]`.

Loading with profiles

The canonical helper reads `application.yaml`, then the profile-specific `application-{profile}.yaml`, then `LUMEN_*` environment variables:

Rust

```

use firefly_config::{load_from_profile, ConfigError};

fn main() -> Result<(), ConfigError> {
    // dir, app basename, fallback profile (FIREFLY_PROFILE overrides).
    let cfg: LumenConfig = load_from_profile("/etc/lumen", "application",
"dev");
    println!("public API on {}", cfg.web.addr);
    Ok(())
}

```

`FIREFLY_PROFILE` selects the profile file at runtime — `FIREFLY_PROFILE=prod` reads `application-prod.yaml`. A comma-separated value (`FIREFLY_PROFILE=dev,cloud`) overlays one file per profile in order. This is how Lumen would carry an in-memory event store in `dev` and a Postgres event store in `prod` without a single `if` in the wiring code.

Source precedence

`Layered::new(vec![s1, s2, ...])` merges from left to right — last write wins. The canonical chain is:

Order	Source	Beats
1	Defaults — <code>StaticSource::new(name, entries)</code>	nothing
2	Base YAML — <code>from_optional_yaml("application.yaml")</code>	defaults
3	Profile YAML — <code>from_optional_yaml("application-prod.yaml")</code>	base
4	Environment — <code>from_env("LUMEN")</code>	YAML files
5	CLI flags — <code>FlagSource::new().set("web.addr", "0.0.0.0:80")</code>	everything

So an environment override (`LUMEN_WEB_ADDR=0.0.0.0:80`) always beats a YAML file, and a CLI override always beats both. That precedence is precisely why `main` can read `LUMEN_ADDR` and have it win over any baked-in default. Build the chain explicitly when you need full control:

Rust

```
use firefly_config::{from_env, from_optional_yaml, load, Source, StaticSource};

let sources: Vec<Box<dyn Source>> = vec![
    Box::new(StaticSource::new("defaults", [("web.addr".into(),
"127.0.0.1:8080".into())])),
    Box::new(from_optional_yaml("application.yaml")),
    Box::new(from_env("LUMEN")),
];
let cfg: LumenConfig = load(&sources)?;
```

YAML subset and value rules

Files are parsed by a line-by-line YAML-subset scanner (no general-purpose YAML dependency), so the flattened output is identical across the Java/Go/.NET/Rust ports for any given `application.yaml`:

YAML

```
name: lumen
web:
  addr: 127.0.0.1:8080
  admin-addr: 127.0.0.1:8081
tags: wallet, ledger, demo  # sequences of scalars are comma-joined
```

- nested mappings become dot-joined, lower-cased keys;
- scalar lexemes are preserved verbatim until the binder parses them against the target field type;

- duplicate keys follow last-write-wins;
- aliases, anchors, multi-doc, tags, and flow sequences are deliberately not interpreted.

Supported leaf kinds: `String`, `bool` (Go `ParseBool` syntax), every integer width, `f32` / `f64`, `char`, unit enums (by variant name), `Option<T>`, sequences of scalars, and `HashMap<String, _>` subtrees. For durations, use an `i64` field plus a conversion: `Duration::from_millis(cfg.cache.ttl as u64)`.

NOTE

Keys are normalized kebab ↔ snake, so `admin-addr:` in YAML binds an `admin_addr` serde field.

Placeholders

`load` / `bind` run a post-merge pass resolving `${...}` placeholders in values (also exposed standalone as `resolve_placeholders(&flat)`):

YAML

```
name: lumen
datasource:
  url: ${DATABASE_URL:postgres://localhost/lumen}  # env, else default
  pool: ${name}-pool                             # config reference
```

- `${ENV_VAR}` — a literal environment variable;
- `${name}` — a config reference, resolved recursively with a depth-10 guard against cycles;
- `${key:default}` — a fallback when neither environment nor config resolves `key`;
- **environment beats config**: `${name}` honors `FIREFLY_NAME` before the merged map.

An unresolvable placeholder without a default raises `ConfigError::Placeholder`.

Binding config straight into a bean –

`#[derive(ConfigProperties)]`

Loading a struct by hand is fine for `main`. But Lumen's services want their configuration *injected*, not threaded through every constructor. The `#[derive(ConfigProperties)]` macro turns a `serde` struct into a container-managed, prefix-bound bean — the exact pattern the dependency-injection chapter builds on:

Rust

```
use firefly::prelude::*;
use serde::Deserialize;

/// Binds the `lumen.web.*` config subtree into an injectable bean.
#[derive(Deserialize, ConfigProperties, Default)]
#[firefly(prefix = "lumen.web")]
pub struct WebProperties {
    pub addr: String,
    #[serde(default)]
    pub admin_addr: String,
}
```

Any `#[derive(Service)]` bean can then `#[autowired] props: Arc<WebProperties>` and receive the bound values — no manual `load`, no global. You will wire one in [Dependency Wiring](#). For one-off scalars there is an even lighter touch: a `#[firefly(value = "${lumen.web.addr:127.0.0.1:8080}")]` field injects a single resolved value with a default, the Rust spelling of Spring's `@Value`.

SPRING PARITY

`#[derive(ConfigProperties)]` with `#[firefly(prefix = "...")]` is `@ConfigurationProperties(prefix = "...")`, and the `#[firefly(value = "${...}")]` field is `@Value("${...}")`. Both bind against the same merged, profile-resolved, placeholder-expanded map described above.

Profile expressions

`accepts_profiles(&active, &exprs)` evaluates the Spring Boot 2.4+ profile-expression grammar against an active-profile list — useful for gating a bean that should exist only in some environments:

Rust

```
use firefly_config::{accepts_profiles, active_profiles};

let active = active_profiles("dev");           // e.g. ["prod", "cloud"]
accepts_profiles(&active, &["prod & cloud"]);  // AND
accepts_profiles(&active, &["prod | qa"]);      // OR
accepts_profiles(&active, &["!test"]);          // negation
accepts_profiles(&active, &["(prod & cloud) | qa"]); // grouping
```

It returns `true` when any expression matches; a malformed expression evaluates to `false` (it never panics). The dependency-injection chapter shows how a bean declares `#[firefly(profile = "prod")]` so the container applies exactly this rule at scan time.

Runtime reload – the `/actuator/refresh` contract

`ReloadableConfig<T>` replays the full merge → placeholder-resolution → bind pipeline and atomically swaps the snapshot; a failed reload keeps the previous snapshot. This is the hook a `POST /actuator/refresh` endpoint wires up — so an operator could re-point Lumen's datasource without a restart.

Rust

```
use firefly_config::ReloadableConfig;

let cfg: ReloadableConfig<LumenConfig> = ReloadableConfig::load(sources)?;
let snapshot = cfg.get(); // Arc<LumenConfig>
let mut rx = cfg.subscribe(); // tokio watch receiver
let changed: Vec<String> = cfg.reload()?; // sorted, changed top-level keys
```

`Arc<ReloadableConfig<T>>` coerces to `Arc<dyn Refresher>` — the object-safe trait the actuator refresh endpoint depends on.

Property-source introspection and masking

`Layered::property_sources()` returns ordered, origin-attributed `PropertySourceView`s (highest precedence first, Spring `/actuator/env` style), with secrets masked: keys naming secrets (`password`, `secret`, `token`, `credential`, `*key`) mask as `*****`, and URI userinfo passwords are redacted (`postgresql://user:*****@host`). The `mask` module exposes `mask_value`, `is_sensitive_key`, and `sanitize_uri` directly. This matters for Lumen the moment it has a JWT signing key (chapter 14) and a datasource URL — neither should ever appear in plaintext on `/actuator/env`.

In-process application events

`ApplicationEventBus` is a synchronous, in-process, `TypeId`-dispatched, `@order`-sorted pub/sub — Spring's `ApplicationEventPublisher` model. This is distinct from the asynchronous `firefly-eda` broker Lumen uses for domain events (no transport, no topics; listeners run on the publishing thread):

Rust

```
use firefly_config::{ApplicationEventBus, ApplicationReadyEvent};

let bus = ApplicationEventBus::new();
bus.subscribe:<ApplicationReadyEvent, _>(|_e| { /* on ready */ });
bus.publish(&ApplicationReadyEvent);
```

Lifecycle events ship: `ContextRefreshedEvent`, `ApplicationReadyEvent`, `ContextClosedEvent`. Any `'static` type can be published as a domain event.

NOTE

Do not confuse this with Event-Driven Architecture: the `ApplicationEventBus` is a *local* lifecycle/notification channel; Lumen's wallet domain events ride the `firefly-eda Broker` over a topic, with a real Kafka/RabbitMQ adapter waiting behind the in-memory default.

Pulling config from a config server

`ConfigClient` fetches a Spring-Cloud-Config document and flattens it into a `StaticSource` you slot into your chain above the defaults:

Rust

```
use firefly_config::ConfigClient;

let remote = ConfigClient::new("http://config:8888", "lumen")
    .with_profile("prod")
    .with_label("main")
    .with_basic_auth("user", "pass")
    .fetch_source()           // fail-fast; .fetch_source_or_empty() = soft
                              fallback
    .await?;
sources.insert(1, Box::new(remote)); // above defaults, below env/flags
```

A non-2xx response logs a warning and yields an empty map (soft miss); transport or decode failures raise `ConfigError::Remote`. The standalone config server lives in `firefly-config-server`.

Recap – what changed in Lumen

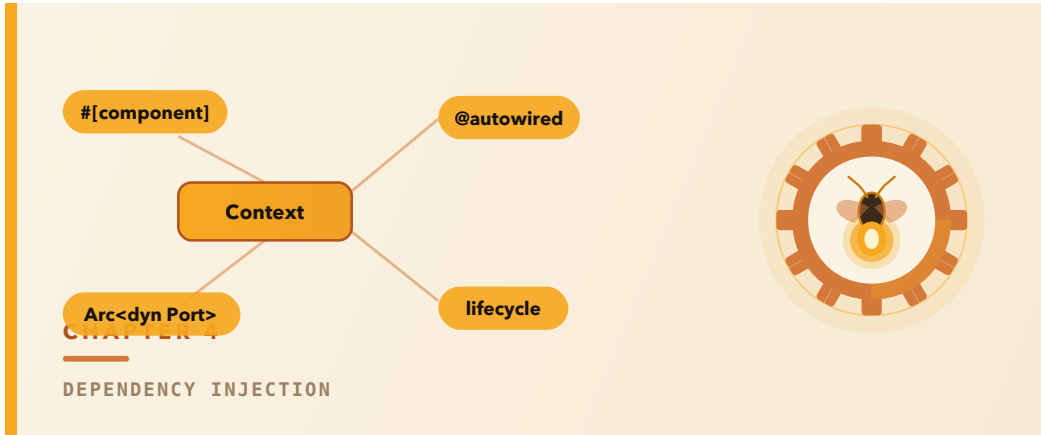
Before	After this chapter
identity hard-coded in two <code>pub const</code> strings	the same values understood as <code>CoreConfig</code> knobs that feed the banner and <code>/actuator/info</code>
ports read with bare <code>std::env::var</code>	the typed home of <code>LUMEN_ADDR</code> / <code>LUMEN_ADMIN_ADDR</code> , sitting at the top of a documented precedence chain
no path to per-environment settings	profiles, placeholders, and <code>#[derive(ConfigProperties)]</code> ready for injection in the next chapter
secrets unconsidered	masking + <code>/actuator/env</code> redaction in place before Lumen ever holds a signing key

Exercises

- Promote the ports to YAML.** Write an `application.yaml` with `web.addr` / `web.admin-addr`, load it with `load_from_profile`, and confirm a `LUMEN_WEB_ADDR` environment variable still wins (precedence row 4 beats row 2).
- Add a profile.** Create `application-prod.yaml` that overrides `web.addr` to `0.0.0.0:80`, run with `FIREFLY_PROFILE=prod`, and verify the prod value takes effect while `dev` keeps the localhost binding.
- Bind a `ConfigProperties` bean.** Define the `WebProperties` struct above, set its keys via a `ConditionContext`, and resolve it from a `Container` (you will recognize this pattern in the next chapter's DI tests).

4. **Mask a secret.** Add a `datasource.password` key and call `Layered::property_sources()`; confirm the value renders as `*****` rather than in plaintext.

Next, see how Lumen's composition root resolves its collaborators — explicitly today, with the best-in-class DI container as the scan-driven alternative — in [Dependency Wiring](#).



CHAPTER 4

Dependency Injection & the Application Context

The [previous chapter](#) wired Lumen the idiomatic-Rust way: an explicit composition root (`build_app`) that constructs each collaborator and hands it to the controller. That style is the default, and Lumen keeps it for teachability — the dependency graph is visible in code and checked by the compiler. But Firefly also ships the *other* style, the one a Spring or pyfly developer reaches for instinctively: a **dependency-injection container** where beans declare what they need with `#[autowired]` , and the container resolves the graph for you.

By the end of this chapter you will know how Lumen's exact collaborators — `ReadModel` , `Ledger` , `WalletApi` — would be expressed as **beans**: declared with stereotype derives, wired by constructor injection, disambiguated with `#[firefly(primary)]` , gated by profiles and conditions, bracketed by lifecycle hooks, and discovered by component scanning. This is the full Spring-DI parity story, told against code you already understand.

 SPRING PARITY

This chapter is the Rust port of Spring's `@Component` / `@Autowired` / `@Bean` / `@Primary` / `@Profile` / `@ConditionalOn*` / `@PostConstruct` model (and of pyfly's `@service` / `@repository` stereotype set). What changes is the discovery mechanism: Rust has no runtime reflection, so "scanning" is **link-time** (`inventory`) and autowiring is generated by a derive macro rather than inferred at runtime. The *programming model* is identical: declare intent, let the container provide the instances.

Two styles, one mental model

Lumen's `build_app` (chapter 4) is constructor injection written by hand:

Rust

```
let read_model = Arc::new(ReadModel::default());
let ledger = Ledger::new(Arc::clone(&store), Arc::clone(&broker));
// ... hand the collaborators to the controller state.
```

The container inverts the *who calls whom*: instead of `build_app` calling each constructor, every bean declares its dependencies and the container calls the constructors in dependency order. The collaborators are the same; only the wiring moves from a function into declarations next to each type.

 TIP

Prefer explicit construction (chapter 4) for a small service like Lumen — it compiles faster and the graph is right there in `build_app`. Reach for the container when the graph grows large, when you want a central registry the admin dashboard can introspect (`/beans`), or when you are porting a Spring/pyfly service and want the wiring to read the way it did before.

Stereotypes: declaring your beans

A **bean** is any value the container builds, wires, and owns. You make a type a bean with a **stereotype derive** — a thin annotation that generates a `firefly_register(&Container)` method *and* submits a link-time scan thunk. Five stereotypes ship, all functionally equivalent, differing only in the architectural intent they document and the label they carry into `/beans`:

Derive	Documents	pyfly / Spring
<code>#[derive(Component)]</code>	a generic managed bean	<code>@component</code> / <code>@Component</code>
<code>#[derive(Service)]</code>	business-logic / use-case	<code>@service</code> / <code>@Service</code>
<code>#[derive(Repository)]</code>	data access / a port	<code>@repository</code> / <code>@Repository</code>
<code>#[derive(Configuration)]</code>	a holder of <code>#[bean]</code> factories	<code>@configuration</code> / <code>@Configuration</code>
<code>#[derive(Controller)]</code>	a web controller bean	— / <code>@Controller</code>

Lumen's read model is a data-access bean, so it would carry `#[derive(Repository)]`:

Rust

```
use firefly::prelude::*;
use std::sync::Mutex;
use std::collections::HashMap;

#[derive(Repository, Default)]
pub struct ReadModel {
    rows: Mutex<HashMap<String, WalletView>>,
}

// generated: ReadModel::firefly_register(&container) + a component-scan thunk
```

 SPRING PARITY

`#[derive(Repository)]` is `@Repository`, `#[derive(Service)]` is `@Service`, and so on — same five names, same "pick the one that documents the layer" guidance. Choosing `Repository` over `Component` costs nothing technically but tells every reader exactly what `ReadModel` is for.

Constructor injection with `#[autowired]`

A bean's dependencies are its `#[autowired]` fields. The container resolves each one by type and injects it before the bean is built. The field type controls the *shape* of the injection:

Field type	Resolves via	Spring analog
<code>Arc<T></code>	<code>resolve::<T>()</code> (required)	<code>@Autowired T</code>
<code>Option<Arc<T>></code>	<code>resolve::<T>().ok()</code> (optional)	<code>@Autowired(required=false)</code>
<code>Vec<Arc<T>></code>	<code>resolve_all::<T>()</code> (all implementations)	<code>List<T></code> injection
<code>Provider<T></code>	a deferred handle (<code>container.provider::<T>()</code>)	<code>ObjectProvider<T></code>

Lumen's `Ledger` depends on an event store and a broker; as a bean it autowires them, and a `WalletApi` controller bean autowires the `Bus` and the `Ledger`:

Rust

```

use firefly::prelude::*;
use std::sync::Arc;

#[derive(Service)]
pub struct Ledger {
    #[autowired] store: Arc<dyn firefly::eventsourcing::EventStore>,
    #[autowired] broker: Arc<dyn firefly::eda::Broker>,
}

#[derive(Controller)]
pub struct WalletApi {
    #[autowired] bus: Arc<Bus>,
    #[autowired] ledger: Arc<Ledger>,
}

```

When the container builds `WalletApi`, it builds `Ledger` first (recursively resolving its store and broker), exactly as `build_app` does by hand — but the order is derived from the field types, not written out. A field with no attribute is built with `Default::default()`; a `#[firefly(value = "${...}")]` field is bound from config (see *Config properties* below).

🌿 SPRING PARITY

`#[autowired]` is constructor injection — functionally identical to Spring's `@Autowired` on a single constructor (which modern Spring infers, just as the derive does). A missing required dependency is a loud `ContainerError::NoSuchBean` at resolve time, with fuzzy "did you mean..." suggestions, rather than a `None` panic three frames deep.

Resolution rules, `#[firefly(primary)]`, and ports

Lumen depends on *ports* — `EventStore`, `Broker` — and selects an adapter at wiring time. The container expresses "depend on the port, get the adapter" with a trait-object **binding**. Register the concrete type, bind the trait to it, then resolve the trait:

Rust

```
let c = Container::new();
firefly::register_all!(&c, [MemoryEventStore]);
c.bind::<dyn firefly::eventsourcing::EventStore, MemoryEventStore>(|a| a);
let store: Arc<dyn firefly::eventsourcing::EventStore> = c.resolve().unwrap();
```

When a single implementation is bound, it resolves immediately. When **several** adapters back one port — the classic in-memory-vs-Redis split — exactly one must be marked primary, or resolution fails loudly:

Rust

```
#[derive(Repository)]
#[firefly(primary)] // the default adapter
pub struct MemoryEventStore { /* ... */ }

#[derive(Repository)]
pub struct PostgresEventStore { /* ... */ } // activated by profile/condition
```

The resolution rules, in strict priority order:

1. **Direct registration** — `T` registered directly resolves to it.
2. **Single binding** — one implementation bound to a trait resolves to it.
3. `#[firefly(primary)]` — among several bindings, the primary one wins.
4. **Error** — `NoSuchBean` when nothing matches; `NoUniqueBean` (naming every competing candidate) when several match with no primary.

 SPRING PARITY

`#[firefly(primary)]` is `@Primary`, and the `NoUniqueBean` error that names every candidate is Spring's `NoUniqueBeanDefinitionException`. Moving `primary` from one adapter to the other is the *only* change needed to swap Lumen's backing store — nothing in `Ledger` changes.

`#[firefly(order)]` and named beans

`#[firefly(order = N)]` controls initialization sequence and the order

`resolve_all::<T>()` returns implementations (lower runs first); the constants

`HIGHEST_PRECEDENCE` / `LOWEST_PRECEDENCE` mark the extremes. When two beans share a type — say a primary and a read-replica store — give one a name and select it with a **qualifier**:

Rust

```

#[derive(Repository)]
#[firefly(name = "replica")]
pub struct ReplicaStore { /* ... */ }

#[derive(Service)]
pub struct ReportService {
    #[firefly(qualifier = "replica")] store: Arc<ReplicaStore>,
}

```

🌿 SPRING PARITY

`#[firefly(order = N)]` is `@Order`, and `#[firefly(qualifier = "replica")]` is `@Qualifier("replica")`. A mistyped qualifier name pointing at the wrong type is a clear `NoSuchBean`, not a silently-wrong injection.

Bean factories: `#[derive(Configuration)]` + `#[bean]`

Not every dependency is a type you own — a third-party client needs constructor arguments, an interface needs a hand-built adapter. For these, a

`#[derive(Configuration)]` holder exposes `#[bean]` factory methods. Each method is keyed by its **return type**; its `Arc<Dep>` arguments are resolved from the container (so a factory can depend on other beans). This is how Lumen would produce its `InMemoryBroker` behind the `Broker` port:

Rust

```

use firefly::prelude::*;
use std::sync::Arc;

#[derive(Configuration)]
pub struct LumenInfraConfig;

#[firefly::bean]
impl LumenInfraConfig {
    #[bean(primary)]
    fn broker(&self) -> firefly::eda::InMemoryBroker {
        firefly::eda::InMemoryBroker::new()
    }

    #[bean]
    fn ledger(&self, store: Arc<dyn firefly::eventsourcing::EventStore>) ->
    Ledger {
        Ledger::new(store, /* ... */)
    }
}

// generated: LumenInfraConfig::firefly_register_beans(&container)

```

A `#[bean]` method returns a **concrete (sized) type** — that is the bean's key. To expose it behind a port, add `#[firefly(provides = "dyn Broker")]` to the holder or call `Container::bind` after registration. Per-method options are `#[bean(name = "...", scope = "...", primary, profile = "...")]`.

 SPRING PARITY

`#[derive(Configuration)] + #[bean]` is `@Configuration + @Bean`. The return-type-as-key rule is identical: the container registers the produced value *under its return type*, so a downstream `#[autowired] ledger: Arc<Ledger>` finds it. Swapping `InMemoryBroker` for a Kafka adapter is one line in this holder; the rest of Lumen is untouched.

Scopes

Every bean has a **scope** controlling how long its instance lives. Pass it as

```
#[firefly(scope = "...")]:
```

Scope	Behaviour	Use for
<code>singleton</code> (default)	one instance, cached after first resolve	stateless services, pools, caches
<code>transient</code>	a fresh instance on every resolve	per-operation state (a saga's scratch context)
<code>request</code>	one per HTTP request (needs a request <code>ScopeHandler</code>)	the authenticated user, a request trace id
<code>session</code>	one per session (needs a session <code>ScopeHandler</code>)	per-session state

Rust

```
#[derive(Component)]
#[firefly(scope = "transient")]
pub struct TransferContext {                // a fresh scratch pad per transfer
    pub steps: Vec<String>,
}
```

🌿 SPRING PARITY

The scope names map directly to Spring's `singleton` / `prototype` (Firefly: `transient`) / `request` / `session`. Almost every Lumen bean is a singleton; the `request` and `session` scopes resolve through a `ScopeHandler` the web tier installs (`register_request_scope`), the Rust adaptation of Spring's request/session context.

Lifecycle: post-construct and pre-destroy

A bean often needs one-time setup after its dependencies are injected — Lumen's projection, for instance, **subscribes to the broker** once at startup — and a clean teardown on shutdown. Name a method per hook on the struct attribute:

Rust

```
#[derive(Service)]
#[firefly(post_construct = "on_start", pre_destroy = "on_stop")]
pub struct ProjectionListener { /* ... */ }

impl ProjectionListener {
    fn on_start(&self) { /* subscribe to wallets.events */ }
    fn on_stop(&self) { /* unsubscribe, flush */ }
}
```

`on_start` runs after construction (all `#[autowired]` fields set), so it can safely query collaborators. `Container::destroy()` (the `ApplicationContext`'s `close()`) runs every `pre_destroy` hook in **reverse** construction order, so a listener that started after the read model is stopped before it.

🌿 SPRING PARITY

`#[firefly(post_construct = "...")]` / `#[firefly(pre_destroy = "...")]` are `@PostConstruct` / `@PreDestroy`, including the reverse-order teardown. (Note these are *attribute keys on the struct*, not standalone macros.)

Conditional beans and profiles

Conditions answer "should this bean exist *at all*, given the environment?" — the mechanism that lets one Lumen codebase run on in-memory infrastructure in tests and real infrastructure in production with no `if` in the service code. They are evaluated by `Container::scan` in two passes:

Pass 1 (config/profile facts, knowable before any bean is built):

Rust

```
#[derive(Repository)]
#[firefly(profile = "prod", condition_on_property =
  "lumen.store.postgres=true")]
pub struct PostgresEventStore { /* ... */ }
```

Pass 2 (registry-dependent, knowable only after pass 1 settles) — the **default-with-override** pattern: ship a fallback that yields to any user-provided implementation:

Rust

```
#[derive(Repository)]
#[firefly(condition_on_property = "lumen.store.postgres=true")]
pub struct PostgresEventStore { /* ... */ } // real store when configured

#[derive(Repository)]
#[firefly(condition_on_missing_bean = "EventStore")]
pub struct MemoryEventStore { /* ... */ } // fallback whenever none is wired
```

The full conditional family: `condition_on_property = "key=value"`, `condition_on_class = "label"`, `condition_on_bean = "Type"`, `condition_on_missing_bean = "Type"`, `condition_on_single_candidate = "Type"`, plus `profile = "expr"` (which supports the Spring Boot 2.4+ grammar — `prod & cloud`, `dev | test`, `!staging`, parentheses).

 SPRING PARITY

These map one-for-one to `@ConditionalOnProperty`, `@ConditionalOnClass`, `@ConditionalOnBean`, `@ConditionalOnMissingBean`, `@ConditionalOnSingleCandidate`, and `@Profile`. The two-pass design matches Spring Boot's: pass-1 facts (config, feature labels) settle first, then pass-2 conditions evaluate against the resulting bean registry — which is how *all* of Firefly's own auto-configuration backs off when you provide your own.

Component scanning vs. `register_all!`

`Container::scan()` is the link-time analog of Spring's `@ComponentScan` / pyfly's `scan_packages`: every non-generic stereotype derive submits an `inventory` thunk, and `scan` collects them across the whole crate graph, applies conditions and profiles, and registers the survivors. Drive it through the `ApplicationContext`, which builds the condition context from the active profiles and config:

Rust

```
use firefly::prelude::*;

let ctx = ApplicationContext::builder()
    .profiles(["prod"])
    .property("lumen.store.postgres", "true")
    .build();                                // scans + warms singletons
let store: Arc<dyn firefly::eventsourcing::EventStore> =
    ctx.resolve().unwrap();                  // -> PostgresEventStore (prod)
println!("{}", beans_registered, ctx.bean_count());
```

Generic beans cannot be inventoried (the monomorphization is chosen at the use site), so register those with the explicit-list fallback:

Rust

```
let c = Container::new();
firefly::register_all!(&c, [ReadModel, Ledger, WalletApi]); // calls each
firefly_register
let api = c.resolve::<WalletApi>().unwrap();
```

🌱 SPRING PARITY

`Container::scan()` is `@ComponentScan`; `firefly::scan(&c)` reads as pyfly's `scan_package`. The difference is *when* discovery happens — at link time via `inventory`, not at runtime via reflection — so a bean is only discoverable if its crate is actually linked. `register_all!` is the explicit fallback Spring never needs but Rust does for generics.

Introspection – the `/beans` view

The container is observable. `container.beans()` returns a `BeanDescriptor` per registration (name, type, scope, stereotype, primary, initialized, resolution count) and `bean_stats()` aggregates counts per stereotype — exactly what the admin dashboard's `/beans` page and Spring Boot's `/actuator/beans` render, and what `firefly beans --url ...` (the [CLI chapter](#)) prints. Errors carry diagnostics too: `fuzzy_suggestions(name)` powers the "did you mean..." hints, and `CircularDependency` is caught with a per-thread resolution stack.

What changed in Lumen

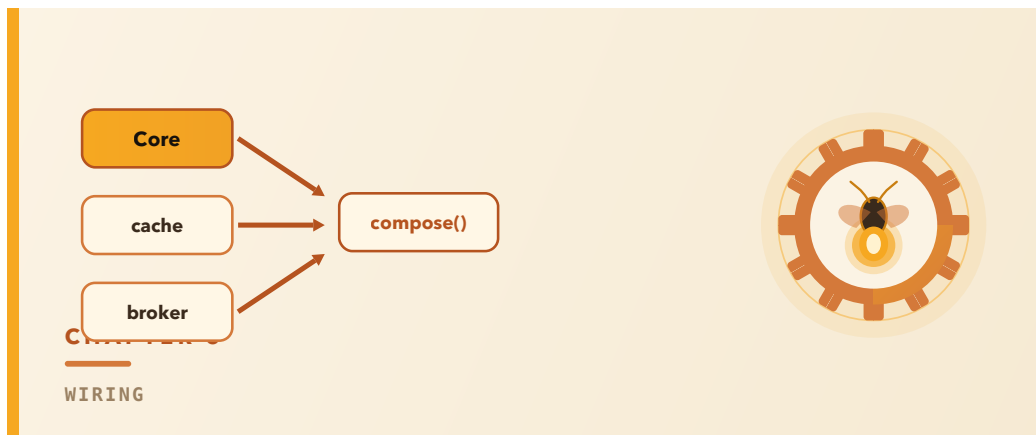
Nothing in `samples/lumen` — Lumen deliberately keeps the explicit composition root from chapter 4 for teachability. What this chapter showed is the *other valid wiring*, told against Lumen's real collaborators: `ReadModel` as a `#[derive(Repository)]`, `Ledger` / `WalletApi` autowiring their dependencies, `MemoryEventStore` vs. a hypothetical `PostgresEventStore` disambiguated by `#[firefly(primary)]` and gated by `profile` / `condition_on_missing_bean`, a `#[derive(Configuration)]` +

`#[bean]` factory producing the broker, lifecycle hooks bracketing the projection, and `Container::scan()` / `register_all!` discovering it all. Every attribute shown — `autowired`, `primary`, `order`, `qualifier`, `scope`, `profile`, the `condition_on_*` family, `post_construct`, `pre_destroy`, `provides`, `value` — is a real option on the stereotype derives.

Exercises

1. **Express Lumen as beans.** Take `ReadModel`, `Ledger`, and `WalletApi` and add the stereotype derives + `#[autowired]` fields shown above. Wire them with `register_all!(&c, [ReadModel, Ledger, WalletApi])` and `resolve::<WalletApi>()`. Confirm the container builds the graph in dependency order.
2. **Default-with-override.** Give `MemoryEventStore` `#[firefly(condition_on_missing_bean = "EventStore")]` and `PostgresEventStore` `#[firefly(condition_on_property = "lumen.store.postgres=true")]`. Scan with and without the property set; confirm which store resolves each time.
3. **Primary swap.** Bind two adapters to one port without a primary and observe the `NoUniqueBean` error naming both candidates. Add `#[firefly(primary)]` to one and watch resolution succeed — with no change to the consuming bean.
4. **Lifecycle order.** Add `post_construct` / `pre_destroy` hooks to two beans where one depends on the other; call `ApplicationContext::close()` and confirm the dependent is torn down first.

You now have both wiring styles in hand: explicit construction (the default Lumen uses) and the full DI container (the Spring/pyfly parity surface). The reactive model underpins everything that follows — continue to [The Reactive Model](#).



CHAPTER 5

Dependency Wiring & Composition

By the end of this chapter you will understand Lumen's **composition root** — the `build_app()` function that resolves every collaborator and hands the wallet controller its state — and you will know both ways Firefly lets you wire a service: explicit construction (Lumen's choice, for teachability) and the best-in-class DI container with component scanning (`#[derive(Service)]`, `#[autowired]`, `firefly::scan`). Lumen keeps an explicit root and shows the scan alternative, so you can pick the style that fits your team.

Every service has a moment where the pieces come together: the cache, the bus, the repositories, the services that depend on them. Firefly calls that the *composition root*, and it gives you two ways to write it. The explicit style builds collaborators with plain constructors and passes them where they are needed — idiomatic Rust, checked entirely by the compiler. The container style is a Spring-grade dependency-injection engine: declare beans with stereotype derives, mark dependencies `#[autowired]`, and let `firefly::scan` discover and wire the whole graph. This chapter covers both, using Lumen as the worked example, and is honest about when to reach for each.

Lumen's explicit composition root

Lumen wires its application explicitly, in one function, on purpose: a reader can see the entire dependency graph in a single screen, and the borrow checker proves it is correct before the program runs. Here is the shape `build_app` grows into (later chapters add the bus middleware, the ledger, and the security chain — the *structure* is what matters now):

Rust

```

// src/web.rs
use std::sync::Arc;
use firefly::cQRS::QueryCache;
use firefly::eventsourcing::MemoryEventStore;
use firefly::prelude::*;
use firefly::starter_web::WebStack;

/// The fully-assembled Lumen application: the web-tier stack, the CQRS bus, the
/// ledger application service, and the read model.
pub struct LumenApp {
    pub web: WebStack,
    pub bus: Arc<Bus>,
    pub ledger: Ledger,
    pub read_model: Arc<ReadModel>,
    pub query_cache: QueryCache,
}

/// Assembles a `LumenApp` over **in-memory** infrastructure – the default for
/// tests and a no-infra run.
pub async fn build_app() -> LumenApp {
    let web = WebStack::new(firefly::starter_web::CoreConfig {
        app_name: APP_NAME.into(),
        app_version: VERSION.into(),
        ..Default::default()
    });

    /// Reuse the WebStack's already-wired collaborators – no duplication,
/// no globals. `web` Derefs to the Core, so `web.bus` / `web.broker`
/// are the same instances the middleware and actuator see.
    let bus = Arc::clone(&web.bus);
    let store: Arc<dyn firefly::eventsourcing::EventStore> =
Arc::new(MemoryEventStore::new());
    let broker = Arc::clone(&web.broker);
    let read_model = Arc::new(ReadModel::default());

    let ledger = Ledger::new(Arc::clone(&store), Arc::clone(&broker));
    let query_cache = QueryCache::new();

```

```
LumenApp { web, bus, ledger, read_model, query_cache }
}
```

Three ideas carry the whole design:

- **The root reuses, it does not re-create.** `web.bus` and `web.broker` are the instances `WebStack::new` already wired; `build_app` clones the `Arc` s rather than standing up a second bus. There is exactly one of each, and everything downstream shares it.
- **Ports are `Arc<dyn Trait>` fields.** `store: Arc<dyn EventStore>` is "depend on the interface, inject the implementation" with no machinery. Swap `MemoryEventStore` for a Postgres-backed store and *only this line* changes — the ledger, the handlers, and the controller never notice.
- **The controller gets its state from the root.** When the wallet routes arrive in [Your First HTTP API](#), `LumenApp::router()` constructs the `WalletApi` controller from these resolved collaborators and hands it to the macro-generated router as axum `State`.

💡 TIP

Explicit wiring keeps the dependency graph visible in code and checked by the compiler. There is no reflection and no startup-time magic; if it compiles, it is wired. For an application as focused as Lumen, this is the clearest choice — and it is why the book uses it.

🌱 SPRING PARITY

This is constructor injection done by hand: the `LumenApp { .. }` literal is the place where dependencies are "injected." Where Spring infers the graph from a single constructor and pyfly reads `__init__` type hints, Lumen spells it out. The trade-off is verbosity for transparency — and for many Rust services, transparency wins.

The container as a composition root

`WebStack` / `Core` is itself a wired bundle of the components a typical service needs. You read them straight off the struct (Lumen does), or pass overrides in `CoreConfig`:

Field / accessor	Type
<code>web.bus</code>	<code>Arc<cqrs::Bus></code> (validation pre-installed)
<code>web.cache</code>	<code>Arc<dyn cache::Adapter></code> (Memory by default)
<code>web.broker</code>	<code>Arc<dyn eda::Broker></code> (InMemory by default)
<code>web.scheduler</code>	<code>Arc<scheduling::Scheduler></code>
<code>web.metrics</code>	<code>Arc<actuator::MetricRegistry></code>
<code>web.health</code>	<code>Arc<actuator::HealthComposite></code>

Rust

```
let web = WebStack::new(firefly::starter_web::CoreConfig {
    app_name: "lumen".into(),
    cache: Some(Arc::new(my_redis_adapter)), // Arc<dyn cache::Adapter>
    broker: Some(Arc::new(my_kafka_broker)), // Arc<dyn eda::Broker>
    ..Default::default()
});
```

Override any field and everything downstream uses your choice — the same "swap the adapter, keep the code" move you will make per subsystem throughout the book.

The best-in-class DI container – `firefly-container`

When you would rather *declare* beans and let the framework discover and wire them — the Spring / pyfly experience — Firefly ships a full dependency-injection container with **component scanning**, stereotype derives, constructor-style `#[autowired]` injection,

qualifier/primary/order disambiguation, `Vec` and `Provider` injection, `#[bean]` factories, lifecycle hooks, and conditional/profile gating. It is `TypeId`-keyed, `Send + Sync`, and shareable as `Arc<Container>`.

Stereotypes – declaring your beans

You make a type visible to the container by deriving a **stereotype**. All five register the type as a managed bean; the difference is the architectural role each name communicates (and that the web layer uses to find controllers):

Derive	Role
<code>#[derive(Service)]</code>	Business-logic layer: use-case orchestration.
<code>#[derive(Component)]</code>	Generic managed bean with no specific role.
<code>#[derive(Repository)]</code>	Data-access layer: databases, external storage, ports.
<code>#[derive(Configuration)]</code>	A factory holder that can carry <code>#[bean]</code> methods.
<code>#[derive(Controller)]</code>	HTTP layer (<code>#[rest_controller]</code> builds on this).

🌿 SPRING PARITY

`#[derive(Service / Component / Repository / Configuration / Controller)]` map one-to-one onto Spring's `@Service`, `@Component`, `@Repository`, `@Configuration`, `@Controller` and pyfly's `@service`, `@component`, `@repository`, `@configuration`, `@rest_controller`. The stereotype is recorded on each bean so the admin dashboard's `/beans` view can group beans by layer, exactly as the JVM and Python ports do.

`#[autowired]` – constructor injection without a constructor

Mark a field `#[autowired]` and the container fills it in by type. This is the Rust spelling of constructor injection — you declare *what* a bean needs; the container supplies it. Here is how Lumen's ledger-and-read-model pair would look in container style:

Rust

```

use std::sync::Arc;
use firefly::prelude::*;

#[derive(Repository, Default)]
struct WalletReadModel { /* in-memory rows */ }

#[derive(Service)]
struct WalletService {
    #[autowired]
    read_model: Arc<WalletReadModel>, // resolved by type, recursively
}

```

When the container constructs `WalletService` it first constructs `WalletReadModel`, then injects it. A dependency that does not exist surfaces as a clear resolution error at startup — not a panic three frames deep at runtime.

`#[autowired]` injects more than a single `Arc<T>`:

- `#[autowired] widgets: Vec<Arc<Widget>>` injects **every** registered `Widget`, ordered by each bean's `order` — Spring's `List<T>` injection.
- `#[autowired] maybe: Option<Arc<Thing>>` injects `Some` when a `Thing` is registered and `None` when it is not — an optional dependency.
- `#[autowired] tickets: Provider<Ticket>` injects a **deferred** handle: `tickets.get()` resolves a fresh instance on each call, the way you would reach for a transient bean inside a singleton.

Component scanning – `firefly::scan`

Rust has no runtime package introspection, so discovery is **link-time**: every stereotype derive emits an `inventory` thunk, and `firefly::scan(&container)` (equivalently `container.scan()`) collects every submitted thunk across the whole crate graph and registers them — honoring conditionals and profiles as it goes. The usual entry point is the `ApplicationContext`, which wraps the container with the full startup sequence:

Rust

```
use firefly::prelude::*;

let ctx = ApplicationContext::builder()
    .profiles(["test"])
    .property("feature.audit", "on")
    .build();
let c = ctx.container();

// Every stereotype-derived bean in the crate graph is discovered and wired.
let svc = c.resolve:::<WalletService>().expect("scan registered the service");
```

🌿 SPRING PARITY

`firefly::scan` / `ApplicationContext::builder()` is the Rust analog of `@ComponentScan` and pyfly's `scan_packages`. The semantics match: every stereotype-derived type in the linked crate graph is registered, subject to its conditions and the active profiles. The one Rust-specific note: generic types can't be inventoried (the monomorphization is chosen at the use site), so you register those with the explicit `register_all!` fallback below.

Interface auto-binding, `primary`, `order`, and qualifiers

Bind a trait object to an implementation right on the derive, and resolve the trait afterward — "depend on the port, get the adapter":

Rust

```

trait Clock: Send + Sync { fn now(&self) -> u64; }

#[derive(Component, Default)]
#[firefly(provides = "dyn Clock", primary)]
struct SystemClock;
impl Clock for SystemClock { fn now(&self) -> u64 { 42 } }

// elsewhere: c.resolve::<dyn Clock>() yields the SystemClock instance.

```

When several beans satisfy the same interface, `#[firefly(... primary)]` picks the default for `resolve`, and `resolve_all::<dyn Trait>()` returns *all* of them ordered by `order` — Spring's `@Primary` and `@Order`. For the rare case where you need a *specific* named instance rather than any satisfying one, the container supports qualifier-by-name resolution.

`#[bean]` factories – wiring things you do not own

Not every dependency is a type you can annotate. Third-party clients need constructor arguments; some beans are clearest as a factory. A `#[derive(Configuration)]` holder with `#[firefly::bean]` methods produces beans by their **return type** — the same move pyfly's `@configuration` / `@bean` makes:

Rust

```

use firefly::prelude::*;

#[derive(Configuration, Default)]
struct LumenInfraConfig;

#[firefly::bean]
impl LumenInfraConfig {
    // Registered as `dyn EventStore` — swap MemoryEventStore for a Postgres
    // store here and nothing else in Lumen changes.
    #[bean(primary)]
    fn event_store(&self) -> Arc<dyn firefly::eventsourcing::EventStore> {
        Arc::new(firefly::eventsourcing::MemoryEventStore::new())
    }
}

```

`#[bean]` methods may declare parameters; the container resolves each by type before the method runs. A `#[bean(profile = "prod")]` method registers only when the `prod` profile is active — the factory-level twin of the conditional gating below.

Lifecycle hooks – `#[post_construct]` / `#[pre_destroy]`

Real infrastructure beans need to *act* once wired (open a pool, subscribe to a topic) and undo it on shutdown. Name the methods on the derive:

Rust

```

#[derive(Service, Default)]
#[firefly(post_construct = "started", pre_destroy = "stopped")]
struct ProjectionSubscriber { /* ... */ }

impl ProjectionSubscriber {
    fn started(&mut self) { /* subscribe the read-model projection */ }
    fn stopped(&self)     { /* drain and unsubscribe */ }
}

```

`post_construct` runs after construction and injection complete; `pre_destroy` runs on `container.destroy()` in reverse construction order, so a subscriber started after the store is torn down before it. This is precisely the lifecycle Lumen drives by hand in `main` today (subscribe the projection, then run); the container would manage it for you.

🌿 SPRING PARITY

`#[firefly(post_construct = "...", pre_destroy = "...")]` are `@PostConstruct` / `@PreDestroy` (JSR-250) and pyfly's `@post_construct` / `@pre_destroy`, with the same "destroy in reverse init order" guarantee.

Conditional and profile gating – the same codebase in every environment

Conditions answer "should this bean exist at all, given the environment?" — how one codebase runs with cheap in-memory adapters in dev and real infrastructure in prod, without an `if` in your service code:

Rust

```
// Registered only when the property is present and not false.
#[derive(Service, Default)]
#[firefly(condition_on_property = "feature.audit=on")]
struct AuditService;

// Registered only under the named profile.
#[derive(Service, Default)]
#[firefly(profile = "prod")]
struct PostgresHealthCheck;
```

`firefly::scan` evaluates these as it collects each thunk, so the resolved container holds exactly the beans the environment calls for. This is the mechanism behind the "swap the adapter" callouts throughout the book — and the reason Lumen can stay in-memory for teaching while a production deployment flips to real infrastructure through configuration alone.

`register_all!` – the explicit fallback

When you want an explicit list (for generics that can't be scanned, or simply to keep wiring local to one test), `register_all!` registers a known set on a container:

Rust

```
let c = Container::new();
firefly::register_all!(&c, [WalletReadModel, WalletService]);
let svc = c.resolve::<WalletService>().expect("service resolves");
```

Errors and introspection

The error taxonomy mirrors Spring's: a missing bean, a non-unique bean with no `primary`, and a detected circular dependency each surface as a distinct, named error at resolution time. For diagnostics, the container can list its registered beans and report per-bean resolution stats — the data the admin dashboard's `/beans` view renders.

Choosing a style

Both styles are first-class; neither is required by the core or the starters.

- **Explicit construction** (Lumen's choice) keeps the graph visible and compiler-checked, compiles faster, and reads top-to-bottom. Prefer it for a focused service whose wiring fits on a screen.
- **The container** shines as a service grows many beans, many adapters, and many environment-specific variations — when "declare the bean, let scan find it" removes

more boilerplate than the indirection costs. Reach for it when you want the Spring/pyfly authoring experience or the `/beans` introspection.

You can even mix them: keep an explicit root and resolve a scanned sub-graph from a `Container` field on `LumenApp`.

Recap – what changed in Lumen

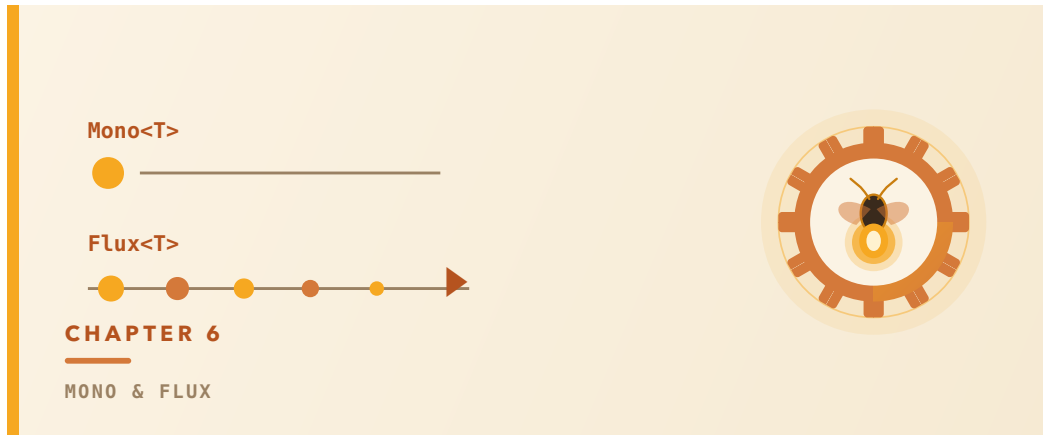
Before	After this chapter
<code>build_app</code> understood only as "the thing <code>main</code> calls"	named as the composition root : it reuses the <code>WebStack</code> 's wired collaborators and hands them down
ports felt abstract	seen concretely as <code>Arc<dyn EventStore></code> / <code>Arc<dyn Broker></code> fields — one line to swap an adapter
only one wiring style implied	both styles in hand: explicit (Lumen's) and the full DI container with scan, stereotypes, <code>#[autowired]</code> , <code>#[bean]</code> , lifecycle, and conditions

Exercises

1. **Trace the single bus.** In `build_app`, print `Arc::strong_count(&bus)` before and after constructing `LumenApp`. Confirm there is exactly one `Bus` shared by the middleware, the controller, and the handlers — never a second one.
2. **Scan a two-bean graph.** Recreate the `WalletReadModel` / `WalletService` pair above, build an `ApplicationContext`, and `resolve::<WalletService>()`. Then add `#[firefly(condition_on_property = "wallet.enabled=true")]` to the service and watch it disappear from the container until you set the property.

3. **Auto-bind a port.** Define a `Clock` trait, give `SystemClock` `#[firefly(provides = "dyn Clock", primary)]`, and resolve `dyn Clock`. Add a second implementation without `primary` and observe the non-unique-bean error; move `primary` to fix it.
4. **Produce a bean from a factory.** Move Lumen's `MemoryEventStore` behind a `#[derive(Configuration)]` holder with a `#[bean]` method returning `Arc<dyn EventStore>`, and resolve the trait object. Note that swapping in a real store is now a one-method change — the same seam the explicit root has.

With the wiring understood, the reactive primitives underpin everything that follows — read [The Reactive Model](#) next, then give Lumen its first endpoints in [Your First HTTP API](#).



CHAPTER 6

The Reactive Model – Mono & Flux

This is the keystone chapter. `firefly-reactive` is a faithful, production-grade **Reactor / WebFlux-style reactive core** — the Rust analog of Project Reactor's `Mono` and `Flux`, the engine behind Spring WebFlux and the Java Firefly framework. Every reactive surface in the framework — reactive HTTP endpoints, reactive repositories, the reactive `WebClient`, reactive EDA and CQRS — is built on the two types you will learn here. Read this before the service-building chapters.

*By the end of this chapter, Lumen will have the vocabulary it leans on twice over: the `Flux<WalletEvent>` that backs its NDJSON / SSE `stream-a-wallet's-events` endpoint (turned on in *Production & Deployment*), and the lazy `Mono<R>` that `Bus::send_mono` / `Bus::query_mono` return so a wallet command can be composed into a reactive pipeline. No Lumen file lands yet — but every reactive shape in the chapters ahead is one of the two publishers below.*

Mono and Flux

Two publishers, mirroring Reactor exactly:

- **Mono<T>** — a producer of *at most one* value (0-or-1, plus a terminal error). The reactive analog of "an async function that returns a **T**."
- **Flux<T>** — a producer of *0..N* values plus a terminal completion-or-error. The reactive analog of "an async stream of **T**."

Both are **lazy**: building a pipeline does nothing; work runs only when you subscribe, block, or await. Both are **Send + 'static'**, so a **Mono** or **Flux** drops directly into an axum handler.

The error type is fixed to **firefly_kernel::FireflyError**, exactly as WebFlux models everything as a **Throwable**. Fixing the error keeps the operator surface ergonomic (no error type parameter) and wires straight into the framework's RFC 9457 problem responses.

Rust

```

use firefly_reactive::{Flux, Mono};

# async fn ex() {
    // Mono: one value, lazily transformed, then awaited.
    let n = Mono::just(20)
        .map(|x| x + 1)
        .filter(|x| *x > 10)
        .default_if_empty(0)
        .block()
        .await
        .unwrap();
    assert_eq!(n, Some(21));

    // Flux: a stream of values, filtered + mapped, collected to a Vec.
    let xs = Flux::range(1, 5)
        .filter(|x| x % 2 == 1)
        .map(|x| x * 10)
        .collect_list()
        .block()
        .await
        .unwrap() // Result -> Option
        .unwrap(); // Option -> Vec (collect_list always yields a list)
    assert_eq!(xs, vec![10, 30, 50]);
# }

```

 REACTOR PARITY

`Mono::block()` here is *not* the thread-parking `Mono.block()` of the JVM. It is `async` and never parks a Tokio worker; it resolves the publisher in place and returns `Result<Option<T>, FireflyError>`.

Reactor → firefly-reactive concept map

Project Reactor	firefly-reactive
<code>Mono<T></code>	<code>Mono<T></code>
<code>Flux<T></code>	<code>Flux<T></code>
<code>Throwable</code> (error signal)	<code>firefly_kernel::FireflyError</code> (fixed)
<code>Mono.empty()</code> / <code>onComplete</code>	<code>Ok(None)</code> from a <code>Mono</code>
<code>onError(t)</code>	<code>Err(FireflyError)</code> (terminal)
<code>Mono.block()</code>	<code>Mono::block</code> (async — never parks a thread)
<code>Mono.subscribe(..)</code> / <code>Flux.subscribe(..)</code>	<code>Mono::subscribe</code> / <code>Flux::subscribe</code>
<code>Schedulers.immediate()</code>	<code>Scheduler::Immediate</code>
<code>Schedulers.parallel()</code>	<code>Scheduler::Parallel</code>
<code>Schedulers.boundedElastic()</code>	<code>Scheduler::BoundedElastic</code>
<code>subscribeOn</code> / <code>publishOn</code>	<code>subscribe_on</code> / <code>publish_on</code>
<code>FluxSink</code> / <code>Flux.create</code>	<code>FluxSink</code> / <code>Flux::create</code>
<code>Retry.backoff(..)</code>	<code>Backoff</code> + <code>*::retry_backoff</code>
<code>Mono.toFuture()</code> / <code>await</code>	<code>Mono::into_future</code> / <code>.await</code>
<code>Flux.toStream()</code> (escape hatch)	<code>Flux::to_stream</code> / <code>Flux::into_stream</code>

Creating publishers

`Mono` constructors:

Constructor	Produces
<code>Mono::just(v)</code>	exactly <code>v</code>
<code>Mono::just_or_empty(opt)</code>	<code>v</code> if <code>Some</code> , empty if <code>None</code>
<code>Mono::empty()</code>	completes with no value (<code>Ok(None)</code>)
<code>Mono::error(e)</code>	terminal error
<code>Mono::from_future(fut)</code>	awaits a <code>Future<Output = T></code>
<code>Mono::from_result_future(fut)</code>	awaits a <code>Future<Output = Result<T, _>></code>
<code>Mono::from_callable(f)</code>	runs a <code>FnOnce() -> Result<Option<T>, _></code> on subscribe
<code>Mono::defer(factory)</code>	builds the <code>Mono</code> fresh per subscription

`Flux` constructors:

Constructor	Produces
<code>Flux::just(vec)</code>	each element of the <code>Vec</code>
<code>Flux::from_iter(iter)</code>	each element of an iterator
<code>Flux::range(start, count)</code>	<code>start, start+1, ...</code> (count items)
<code>Flux::empty()</code> / <code>Flux::never()</code>	completes immediately / never emits
<code>Flux::error(e)</code>	terminal error
<code>Flux::from_stream(s)</code>	a <code>Stream<Item = Result<T, FireflyError>></code>
<code>Flux::from_value_stream(s)</code>	a <code>Stream<Item = T></code>
<code>Flux::create(producer)</code>	imperative push via a <code>FluxSink</code> (see below)
<code>Flux::interval(period)</code>	<code>0, 1, 2, ...</code> on a timer
<code>Flux::generate(seed, step)</code>	stateful generation

Operator quick reference

The operator surface mirrors Reactor. `Mono` and `Flux` share most names; the differences reflect cardinality.

Category	Mono	Flux
transform	<code>map</code> <code>map_async</code> <code>flat_map</code> <code>flat_map_many</code> <code>filter</code>	<code>map</code> <code>map_async</code> <code>flat_map(n)</code> <code>concat_map</code> <code>filter</code> <code>scan</code> <code>index</code> <code>flat_map_iterable</code>
reduce/term	<code>then</code> <code>then_return</code> <code>zip_with</code>	<code>reduce</code> <code>collect_list</code> <code>collect_map</code> <code>count</code> <code>all</code> <code>any</code> <code>then</code> <code>last</code> <code>next</code> <code>single</code> <code>element_at</code>
limit/slice	—	<code>take</code> <code>take_while</code> <code>take_last</code> <code>skip</code> <code>skip_while</code> <code>distinct</code> <code>distinct_until_changed</code>
combine	<code>when</code> <code>zip</code>	<code>merge_with</code> <code>concat_with</code> <code>zip_with</code> <code>combine_latest</code> <code>start_with</code> <code>switch_if_empty</code> <code>default_if_empty</code>
error	<code>on_error_return</code> <code>on_error_resume</code> <code>on_error_map</code> <code>retry</code> <code>retry_backoff</code>	<code>on_error_resume</code> <code>on_error_continue</code> <code>retry</code> <code>retry_backoff</code>
time	<code>timeout</code> <code>delay_element</code>	<code>timeout</code> <code>delay_elements</code> <code>sample</code> <code>debounce</code> <code>interval</code>
backpressure	—	<code>on_backpressure_buffer</code> <code>on_backpressure_drop</code> <code>on_backpressure_latest</code> <code>limit_rate</code>
window	—	<code>buffer</code> <code>window</code> <code>group_by</code>
side-effect	<code>do_on_next</code> <code>do_on_success</code> <code>do_on_error</code> <code>do_on_finally</code>	<code>do_on_next</code> <code>do_on_complete</code> <code>do_on_error</code> <code>do_on_finally</code>
schedule	<code>subscribe_on</code> <code>publish_on</code>	<code>subscribe_on</code> <code>publish_on</code>
cache/view	<code>cache</code> <code>as_flux</code>	—

Transforming and chaining

Rust

```
use firefly_reactive::{Flux, Mono};

# async fn ex() {
// flat_map: chain a Mono onto the result of another (sequential dependency).
let total = Mono::just(3)
    .flat_map(|seed| Mono::just(seed * 10))
    .map(|x| x + 1)
    .block()
    .await
    .unwrap();
assert_eq!(total, Some(31));

// flat_map on a Flux runs up to N inner publishers concurrently.
let doubled = Flux::range(1, 3)
    .flat_map(2, |n| Mono::just(n * 2).as_flux())
    .collect_list()
    .block()
    .await
    .unwrap()
    .unwrap();
assert_eq!(doubled.len(), 3);
# }
```

Combining

Rust

```
use firefly_reactive::{zip, Mono};

# async fn ex() {
  // zip two Monos into a tuple – both run, then combine.
  let pair = zip(Mono::just("alice"), Mono::just(42))
    .block()
    .await
    .unwrap();
  assert_eq!(pair, Some(("alice", 42)));
# }
```

Error semantics

An **Err** item is **terminal** in a **Flux**: every operator short-circuits on the first error and propagates it downstream — there is no per-element error channel. To recover:

- **Mono::on_error_return(fallback)** — substitute a value;
- **Mono::on_error_resume(f)** / **Flux::on_error_resume(f)** — switch to a fallback publisher, keeping items emitted before the error;
- **Flux::on_error_continue(handler)** — drop the failing element and keep the rest (for operators that re-signal per item);
- **Mono::on_error_map(f)** — translate the error.

retry and **retry_backoff** re-subscribe to a **factory closure**, since Rust streams and futures are single-use:

Rust

```

use std::time::Duration;
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use firefly_reactive::{Backoff, Mono};
use firefly_kernel::FireflyError;

# async fn ex() {
let calls = Arc::new(AtomicUsize::new(0));
let c = calls.clone();
let value = Mono::retry_backoff(
    move || {
        let c = c.clone();
        Mono::from_callable(move || {
            let n = c.fetch_add(1, Ordering::SeqCst);
            if n < 2 { Err(FireflyError::internal("flaky")) } else {
Ok(Some(n)) }
        })
    },
    Backoff::new(5, Duration::from_millis(10)),
)
.block()
.await
.unwrap();
assert_eq!(value, Some(2));
# }

```

`Mono::timeout` / `Flux::timeout` map a missed deadline to a 504 `FireflyError` (code `REACTIVE_TIMEOUT`), which renders as an RFC 9457 problem response.

Imperative emission with `FluxSink`

When values arrive from a callback or an imperative loop, push them into a `Flux` with `Flux::create`:

Rust

```
use firefly_reactive::Flux;

# async fn ex() {
let flux = Flux::create(|sink| {
    for i in 1..=3 {
        sink.next(i);
    }
    sink.complete();
});
let out = flux.collect_list().block().await.unwrap();
assert_eq!(out, Some(vec![1, 2, 3]));
# }
```

Schedulers

A `Scheduler` decides *where* work runs. `subscribe_on` hops the source onto a scheduler; `publish_on` switches the thread for everything downstream.

Rust

```
use firefly_reactive::{Flux, Scheduler};

# async fn ex() {
let out = Flux::range(1, 3)
    .subscribe_on(Scheduler::Parallel) // run the source on the Tokio worker
pool
    .map(|x| x * 2)
    .collect_list()
    .block()
    .await
    .unwrap();
assert_eq!(out, Some(vec![2, 4, 6]));
# }
```


`Scheduler::Immediate` runs inline, `Scheduler::Parallel` uses the Tokio worker pool for CPU-bound work, and `Scheduler::BoundedElastic` is for blocking calls.

Reactive HTTP endpoints

`firefly-web` ships responders that turn a `Mono` / `Flux` into an axum response — the Rust analog of returning `Mono<T>` / `Flux<T>` from a WebFlux `@RestController`. They reuse `firefly-sse`'s wire format, so every reactive response is byte-compatible across the ports.

Spring WebFlux	firefly-web
<code>Mono<T></code> handler return	<code>MonoJson(Mono<T>)</code>
<code>Mono<T></code> empty → <code>404</code>	<code>Ok(None)</code> → 404 problem+json
<code>Mono<T></code> error → problem	<code>Err(FireflyError)</code> → that error's RFC 9457 response
<code>Flux<T></code> + <code>APPLICATION_NDJSON_VALUE</code>	<code>NdJson(Flux<T>)</code>
<code>Flux<ServerSentEvent<T>></code>	<code>Sse(Flux<T>)</code> / <code>SseEvents(Flux<Event>)</code>

Rust

```

use axum::{routing::get, response::IntoResponse, Router};
use firefly_reactive::{Flux, Mono};
use firefly_web::{MonoJson, NdJson, Sse};

async fn one_order() -> impl IntoResponse {
    // Ok(Some) -> 200 application/json; Ok(None) -> 404 problem+json;
    // Err -> that error's problem response.
    MonoJson(Mono::just(serde_json::json!({ "id": "o1" })))
}

async fn stream_orders() -> impl IntoResponse {
    // application/x-ndjson, one line per element, backpressured.
    NdJson(Flux::just(vec![1, 2, 3]))
}

async fn live_orders() -> impl IntoResponse {
    // text/event-stream, one `data:` frame per element.
    Sse(Flux::just(vec![1, 2, 3]))
}

let app: Router = Router::new()
    .route("/orders/one", get(one_order))
    .route("/orders", get(stream_orders))
    .route("/orders/live", get(live_orders));

```

The responders, precisely:

- **MonoJson(Mono<T>)** resolves the **Mono** : **Ok(Some)** → **200 application/json** ; **Ok(None)** → **404 application/problem+json** ; **Err** → that error's problem response.
- **NdJson(Flux<T>)** streams **application/x-ndjson** — one compact JSON doc + **'\n'** per element, flushed incrementally with real backpressure. The **Flux**'s **Stream** is bridged straight into an axum streaming **Body** ; the whole stream is **never** buffered. An **Err** item mid-stream terminates the body cleanly.

- `Sse(Flux<T>)` streams `text/event-stream` — each element serialized to a bare `data: <json>\n\n` frame, byte-identical to the `firefly-sse` writer.
- `SseEvents(Flux<Event>)` streams pre-built `firefly_sse::Event` values — use it when you need `id` / `event` / `retry` fields.

⚠ WARNING

Backpressure is real, not cosmetic. A slow client throttles the producer; nothing is buffered up front. This is what lets a `NdJson` endpoint stream a million rows without the response landing fully in memory.

How Lumen will use this

Lumen's optional `GET /api/v1/wallets/:id/events` endpoint is exactly this shape. It replays a wallet's persisted event stream as a `Flux<WalletEvent>` and hands it to `NdJson` (or `Sse` with `?format=sse`). The whole handler — drawn verbatim from `samples/lumen/src/web.rs`, feature-gated behind `streaming` — is the responders above applied to the wallet domain:

Rust

```
// samples/lumen/src/web.rs – the reactive streaming handler (feature
`streaming`).
use firefly::reactive::Flux;
use firefly::web::{NdJson, Sse};

async fn stream_events(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    axum::extract::Query(params): axum::extract::Query<StreamParams>,
) -> Response {
    // A missing wallet is decided before the streaming head is committed.
    let events = match api.ledger.load_events(&id).await {
        Ok(events) => events,
        Err(e) => return WebError::from(domain_to_web(e)).into_response(),
    };
    let items: Vec<WalletEvent> =
events.iter().map(WalletEvent::from_domain).collect();
    let flux = Flux::just(items);
    if params.format.as_deref() == Some("sse") {
        Sse(flux).into_response()
    } else {
        NdJson(flux).into_response()
    }
}
```

Two details worth carrying forward. First, the *not-found* decision happens **before** the `Flux` is built, so a 404 still renders as a clean problem response rather than a half-open stream. Second, Lumen reaches the reactive types through the one-dependency facade — `firefly::reactive::Flux` and `firefly::web::{NdJson, Sse}` — never the underlying `firefly-reactive` / `firefly-web` crates. The full endpoint, including the route wiring, returns in [Production & Deployment](#).

The reactive `WebClient`

The reactive HTTP client — the Rust analog of WebFlux's `WebClient` — hands its terminal operators back as `Mono` / `Flux`, so an outbound call drops straight into a reactive pipeline and composes end-to-end with the `NdJson` / `Sse` responders above. Full treatment in [HTTP Clients](#); the shape:

Rust

```
use firefly_client::WebClientBuilder;
use http::Method;
use serde::Deserialize;

#[derive(Deserialize)]
struct Order { id: String }
#[derive(Deserialize)]
struct Tick { seq: u64 }

# async fn ex() {
let client = WebClientBuilder::new("https://api.example.com").build();

// body_to_mono – the whole body decoded as one T.
let _order: firefly_reactive::Mono<Order> =
    client.get().uri("/orders/o1").retrieve().body_to_mono:::<Order>();

// body_to_flux – a streamed NDJSON/SSE body decoded element-by-element,
// lazily and with backpressure.
let _ticks: firefly_reactive::Flux<Tick> = client
    .get()
    .uri("/ticks")
    .header("Accept", "application/x-ndjson")
    .retrieve()
    .body_to_flux:::<Tick>();
# }
```

 REACTOR PARITY

Like WebFlux, the `WebClient` has no baked-in retry. Compose `Mono::retry` / `Mono::retry_backoff` on the returned publisher, just as WebFlux composes `retryWhen(..)` rather than baking retry into the client.

Reactive repositories, EDA, and CQRS

The same two types thread through the rest of the framework — and through Lumen:

- **Repositories** — `ReactiveCrudRepository<T, ID>` returns `Mono` / `Flux`; the SQL adapters stream rows out of `find_all()` as a `Flux` so a huge table never lands fully in memory. See [Persistence](#).
- **EDA** — `InMemoryBroker::subscribe_reactive(topic)` yields a `Flux<Event>`, and `publish_mono(event)` is a cold reactive publish. Lumen's ledger publishes every wallet event to a `Broker`; see [EDA](#).
- **CQRS** — `Bus::send_mono` / `Bus::query_mono` wrap the dispatch in a lazy `Mono<R>`, running the same handler lookup and middleware chain. Lumen's wallet commands ride this bus; see [CQRS](#).

A taste of the reactive bus, the shape a Lumen `OpenWallet` could take when composed reactively (it dispatches the same handler the controller's `bus.send(..)` does, only lazily):

Rust

```

use std::sync::Arc;
use firefly::cQRS::Bus;

// `send_mono` / `query_mono` take `&Arc<Bus>` so the lazy Mono can own the bus.
let bus: Arc<Bus> = /* the WebStack's bus */;
let balance = bus
    .query_mono::<_, WalletView>(GetWallet { id: wallet_id })
    .map(|view| view.balance)
    .block()
    .await?;           // Ok(Some(<cents>))

```

 REACTOR PARITY

Because `firefly-reactive` fixes its error channel to `FireflyError`, a failed dispatch is mapped from the bus's `CQRSError` into a status-faithful `FireflyError` (validation → 422, authorization → 403, missing handler → 500) with the original error preserved as `source()` — so a reactive command flows straight into the RFC 9457 problem stack, exactly as a WebFlux `Mono` flows into Spring's problem handler.

Interop with raw `Stream` / `Future`

The reactive types are not a walled garden. Convert in and out at the edges:

- **In:** `Flux::from_stream` (a `Stream<Item = Result<T, FireflyError>>`), `Flux::from_value_stream` (a `Stream<Item = T>`), `Mono::from_future`, `Mono::from_result_future`.
- **Out:** `Flux::to_stream` / `Flux::into_stream`, `Mono::into_future` (or just `.await` the `Mono`).

What changed in Lumen

No Lumen source file lands in this chapter — but Lumen now has the two publishers every reactive surface it touches is built from:

- `Flux<T>` is the engine behind the `GET /api/v1/wallets/:id/events` streaming endpoint: a wallet's events replay through `Flux::just(items)` and stream out as `NdJson` (or `Sse`), with the not-found check resolved *before* the stream head is committed. Lumen reaches it as `firefly::reactive::Flux` + `firefly::web::{NdJson, Sse}` — one dependency, no crate sprawl.
- `Mono<R>` is the lazy result of `Bus::send_mono` / `Bus::query_mono`, the reactive face of the wallet command/query bus you wire in `CQRS`. Its `FireflyError` channel is the same one the RFC 9457 problem responses render from.

Exercises

1. **Map a balance.** Build `Mono::just(1_250_i64)` (a balance in cents), `map` it to a major-unit `f64` (`cents as f64 / 100.0`), and `block().await` it. Confirm you get `Some(12.5)`.
2. **Stream wallet events as a `Flux`.** Make a `Vec<i64>` of signed balance deltas (`[1000, 50, -25]`), wrap it with `Flux::just`, `scan` a running balance, and `collect_list` it. Verify the running balances are `[1000, 1050, 1025]` — a hand-rolled version of what the Lumen streaming endpoint emits per event.
3. **Recover from a flaky source.** Write a `Mono::from_callable` that returns `Err(FireflyError::internal("flaky"))` the first two times and `Ok(Some(n))` afterward, then wrap it in `Mono::retry_backoff(factory, Backoff::new(5, Duration::from_millis(10)))`. Assert it resolves to a value — the retry factory pattern Lumen's HTTP client would use against an external FX provider.

4. **Pick a responder.** Given a `Flux<WalletEvent>`, decide which responder a real-time dashboard wants (`Sse`) versus a bulk export (`NdJson`), and explain in one sentence why backpressure matters for the export case.

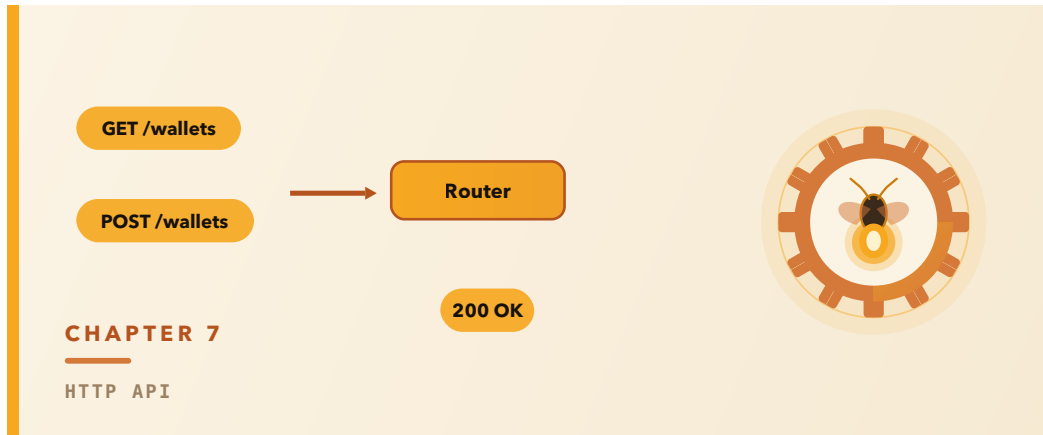
You now have the vocabulary the rest of the book builds on. Next, put it to work in [Your First HTTP API](#).



PART II

Modeling & Persisting the Domain





CHAPTER 7

Your First HTTP API

By the end of this chapter Lumen has a real HTTP surface: a `WalletApi` controller declared with `#[rest_controller]`, a `POST /api/v1/wallets` endpoint that opens a wallet and answers `201 Created`, a `GET /api/v1/wallets/:id` that returns a `WalletView`, and typed errors that render as RFC 9457 problem documents — with an in-process test that drives the whole router without binding a socket. This is the chapter where Lumen stops being a banner and starts being a service.

So far Lumen compiles, runs, and serves an actuator, and you know how its composition root resolves collaborators. Now you give it endpoints. The HTTP layer is axum; Firefly contributes the controller macro, the problem rendering, and the correlation/idempotency middleware you met in the Quickstart, all woven in by the `WebStack`. You write handlers; the framework supplies the wiring.

The `#[rest_controller]` macro

Lumen's wallet endpoints live on one type, `WalletApi`, whose `impl` block carries `#[rest_controller]`. The macro reads each verb attribute and generates a `WalletApi::routes(state) -> axum::Router` function — so a controller is just an `impl` with annotated methods, and the routing table is derived from the code rather than maintained beside it.

Rust

```
// src/web.rs
use std::sync::Arc;
use axum::extract::{Path, State};
use axum::Json;
use firefly::prelude::*;
use firefly::web::{WebError, WebResult};

use crate::commands::{GetWallet, OpenWallet};
use crate::domain::WalletView;

/// The wallet HTTP surface. It carries the CQRS `Bus` it dispatches through;
/// the controller stays thin and delegates every decision to a handler.
#[derive(Clone)]
pub struct WalletApi {
    pub bus: Arc<Bus>,
    // (the ledger and query cache fields arrive in later chapters)
}

/// `#[rest_controller(path = "...")]` generates `WalletApi::routes(state) ->`
/// `axum::Router`. Each method carries one verb mapping and returns
/// `WebResult<T>`, so a handler error renders as RFC 9457
/// `application/problem+json`.
#[rest_controller(path = "/api/v1")]
impl WalletApi {
    /// `POST /api/v1/wallets` - open a wallet. Validation failures surface as
    /// 422 problems; success answers `201 Created` with the view.
    #[post("/wallets")]
    async fn open(
        State(api): State<WalletApi>,
        Json(body): Json<OpenWallet>,
    ) -> WebResult<(axum::http::StatusCode, Json<WalletView>)> {
        let view: WalletView = api.bus.send(body).await.map_err(cqrs_to_web)?;
        Ok((axum::http::StatusCode::CREATED, Json(view)))
    }

    /// `GET /api/v1/wallets/:id` - fetch the read-model view. An unknown id
    /// renders as a 404 problem.
    #[get("/wallets/:id")]

```

```

async fn get(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
) -> WebResult<Json<WalletView>> {
    let view: WalletView = api.bus.query(GetWallet {
id }).await.map_err(cqrs_to_web)?;
    Ok(Json(view))
}
}

```

Three things to read here:

- **The path is composed.** `#[rest_controller(path = "/api/v1")]` is the prefix; `#[post("/wallets")]` and `#[get("/wallets/:id")]` are the suffixes. The macro joins them into `/api/v1/wallets` and `/api/v1/wallets/:id`.
- **Each handler is a plain axum handler.** `State`, `Path`, and `Json` are axum's own extractors — Firefly does not replace them. You write the function; the macro only registers it on the router.
- **The controller is thin.** `open` and `get` translate HTTP into a message and hand it to the CQRS `Bus`, then translate the result (or error) back into an HTTP response. The wallet *logic* lives behind the bus, where `CQRS` puts it. Treat `api.bus.send(...)` / `api.bus.query(...)` here as "dispatch to the handler that knows how"; the bus, the commands, and the read model are the subjects of chapters 7 through 11.

🌿 SPRING PARITY

`#[rest_controller(path = "/api/v1")]` is `@RestController` + `@RequestMapping("/api/v1")`, and `#[get]` / `#[post]` are `@GetMapping` / `@PostMapping` (pyfly's `@rest_controller` + `@get` / `@post`). The macro even emits a route descriptor per endpoint for the actuator `/mappings` view and the OpenAPI generator — the Rust equivalent of Spring's `RequestMappingHandlerMapping`.

The wire shape – `WalletView`

The view a handler returns is a plain `serde` struct. It is the *read model* projection of a wallet — flat, query-optimized, and decoupled from the internal aggregate. The balance travels as an integer count of minor units (cents), so `€10.00` is the JSON number

`1000`:

Rust

```
// src/domain.rs
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct WalletView {
    pub id: String,
    pub owner: String,
    /// The current balance, in minor units (cents).
    pub balance: i64,
    /// The aggregate version (number of events applied).
    pub version: i64,
}
```

The request body Lumen accepts on `POST /api/v1/wallets` is just as ordinary — the `OpenWallet` message, with a `#[serde(rename)]` so the JSON field is `openingBalance` while the Rust field stays `snake_case`:

JSON

```
{ "owner": "alice", "openingBalance": 1000 }
```

Typed errors → RFC 9457 problems

A handler that returns `WebResult<T>` turns any error into the right `application/problem+json` response via `?`. `WebResult<T>` is an alias whose error arm is a `WebError`, and the framework knows how to render it. Lumen's controller maps the bus's error channel onto a precise HTTP status with one helper:

Rust

```
// src/web.rs
use crate::domain::DomainError;

/// Maps a bus `CqrsError` onto the precise HTTP problem the domain implies:
/// a validation failure → 422, a not-found detail → 404, an
/// insufficient-funds / non-positive detail → 422, otherwise 500.
fn cqrs_to_web(err: CqrsError) -> WebError {
    match err {
        CqrsError::Validation(detail) =>
            WebError::from(FireflyError::validation(detail)),
        CqrsError::Handler(detail) => {
            if detail.ends_with("not found") {
                WebError::from(FireflyError::not_found(detail))
            } else if detail == DomainError::InsufficientFunds.to_string()
                || detail == DomainError::NonPositiveAmount.to_string()
                || detail == DomainError::OwnerRequired.to_string()
            {
                WebError::from(FireflyError::validation(detail))
            } else {
                WebError::from(FireflyError::not_found(detail))
            }
        }
        other => WebError::from(FireflyError::internal(other.to_string())),
    }
}
```

The `FireflyError` constructors map straight to HTTP status — pick the one that matches the failure and the renderer does the rest:

Constructor	Status	Use
<code>FireflyError::bad_request(detail)</code>	400	malformed input
<code>FireflyError::unauthorized(detail)</code>	401	missing/invalid credentials
<code>FireflyError::forbidden(detail)</code>	403	authenticated but not allowed
<code>FireflyError::not_found(detail)</code>	404	absent resource
<code>FireflyError::conflict(detail)</code>	409	state conflict
<code>FireflyError::validation(detail)</code>	422	semantic validation failure
<code>FireflyError::internal(detail)</code>	500	server fault

A rendered problem for an unknown wallet looks like this — note the dedicated `application/problem+json` content type, which the tests assert on:

JSON

```
{
  "type": "https://fireflyframework.org/problems/not-found",
  "title": "Not Found",
  "status": 404,
  "detail": "wallet wlt_does_not_exist not found"
}
```

🌿 SPRING PARITY

`WebResult<T> + FireflyError` is the `ResponseEntity` / `@ExceptionHandler` story, but the problem rendering is built in: you never write an error-to-status mapping for the framework's own errors. The same RFC 9457 contract holds across the Java, Go, .NET, and Python ports — a 404 from Lumen looks like a 404 from pyfly's Lumen.

Mounting the routes

The controller's `routes(state)` function returns a plain `axum::Router`, which the composition root wraps in the `WebStack` middleware chain. `LumenApp::router` is that one place — construct the controller from the resolved collaborators, call `WalletApi::routes`, and hand it to `apply_middleware`:

Rust

```
// src/web.rs
impl LumenApp {
    /// Builds the public router: the macro-generated wallet routes wrapped in
    /// the web middleware chain.
    pub fn router(&self) -> axum::Router {
        let state = WalletApi { bus: Arc::clone(&self.bus) };
        let routes = WalletApi::routes(state);
        self.web.apply_middleware(routes)
    }
}
```

Every request to a wallet route now passes through the canonical chain you got for free in the Quickstart — the RFC 9457 problem layer, correlation-id propagation, and idempotency replay — before it reaches your handler. You wrote the two handlers; the rest of the request lifecycle is the framework's.

NOTE

`LumenApp::router` is the *only* function that changes as Lumen grows. The streaming endpoint merges into it in [Production](#); the JWT security layer wraps it in [Security](#). `main` keeps calling `app.router()` and never learns the difference — the composition root absorbs every addition.

Proving it works – an in-process round-trip

Because `router()` is a self-contained `axum::Router`, Lumen's tests drive it **in-process** with `tower::ServiceExt::oneshot` — no socket bound, no port to race on. This is the first end-to-end test, the open-then-get round-trip:

Rust

```
// tests/http.rs
use axum::body::Body;
use axum::http::{Request, StatusCode};
use firefly_sample_lumen::build_router;
use firefly_sample_lumen::domain::WalletView;
use http_body_util::BodyExt;
use tower::ServiceExt;

#[tokio::test]
async fn open_then_get_round_trips_through_cqrs() {
    // POST /api/v1/wallets → 201 Created with the opened view.
    let res = build_router()
        .await
        .oneshot(
            Request::post("/api/v1/wallets")
                .header("content-type", "application/json")
                .body(Body::from(
                    serde_json::to_vec(&serde_json::json!({
                        "owner": "alice", "openingBalance": 1_000
                    })))
                .unwrap(),
        ))
        .unwrap(),
    )
    .await
    .unwrap();
    assert_eq!(res.status(), StatusCode::CREATED, "open should 201");

    let bytes = res.into_body().collect().await.unwrap().to_bytes();
    let opened: WalletView = serde_json::from_slice(&bytes).unwrap();
    assert_eq!(opened.owner, "alice");
    assert_eq!(opened.balance, 1_000);
    assert_eq!(opened.version, 1);

    // GET /api/v1/wallets/:id → 200 OK with the same view.
    let res = build_router()
        .await
        .oneshot(
```

```

        Request::get(&format!("/api/v1/wallets/{}", opened.id))
            .body(Body::empty())
            .unwrap(),
    )
    .await
    .unwrap();
    assert_eq!(res.status(), StatusCode::OK);
}

```

`build_router()` is the testable composition root: it is `build_app().await` followed by `.router()`, returning exactly the `axum::Router` `main` serves. The test exercises the macro-generated routes, the JSON contract, and the status-code mapping — the same code path a real client hits, minus the network.

The error paths are tested the same way. An empty `owner` is a `422` problem; an id that was never opened is a `404` problem — and both assert the `application/problem+json` content type, so the RFC 9457 contract is part of the test suite, not just the prose:

Rust

```

#[tokio::test]
async fn unknown_wallet_is_404_problem() {
    let res = build_router()
        .await
        .oneshot(
            Request::get("/api/v1/wallets/wlt_does_not_exist")
                .body(Body::empty())
                .unwrap(),
        )
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::NOT_FOUND);
    let ct = res.headers().get("content-type").unwrap().to_str().unwrap();
    assert!(ct.contains("application/problem+json"));
}

```

 SPRING PARITY

`oneshot` against `build_router()` is `MockMvc` / `WebTestClient` (pyfly's `TestClient`): the whole controller stack runs in the test process with no live server. `Testing` builds this into a full strategy.

Recap – what changed in Lumen

Before	After this chapter
an empty public router	a <code>WalletApi</code> controller declared with <code>#[rest_controller]</code> and two real endpoints
no client contract	<code>POST /api/v1/wallets → 201 + WalletView</code> , <code>GET /api/v1/wallets/:id → 200 / 404</code> , all JSON
errors unconsidered	typed <code>FireflyError</code> → RFC 9457 <code>application/problem+json</code> with the right status
nothing to test	a <code>tower::oneshot</code> round-trip that drives the full router in-process, content-type assertions included

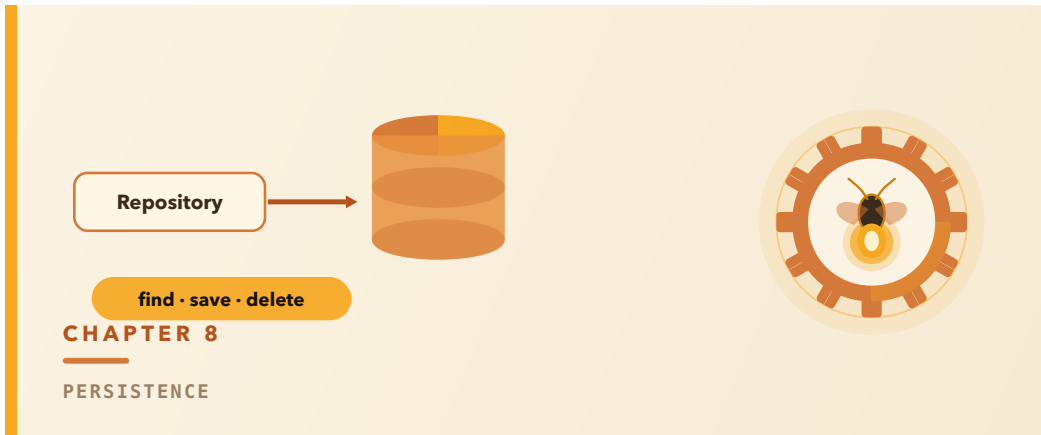
The controller is deliberately thin: it speaks HTTP and delegates the wallet logic to the bus. That seam is what the next several chapters fill in — the read model the `GET` serves, the domain that enforces the rules, and the CQRS handlers the `POST` dispatches to.

Exercises

1. Add a route. Give `WalletApi` a `#[get("/wallets")] list` method that returns `WebResult<Json<Vec<WalletView>>>`. Watch `WalletApi::routes` pick it up automatically — you never touch a routing table.

2. **Shape an error.** Make `cqrs_to_web` (or a small handler of your own) return `FireflyError::conflict("wallet already closed")` and confirm the response is a `409` with `application/problem+json`. Try `bad_request` and `forbidden` too, and read the rendered `type / title / status` for each.
3. **Honor idempotency.** `POST /api/v1/wallets` twice with the same `Idempotency-Key` header and identical body; confirm the second response carries `Idempotent-Replay: true`. Then change the body under the same key and observe the `409`. You wrote none of this — it came with `apply_middleware`.
4. **Write the round-trip yourself.** Copy the `open_then_get` test, change the owner and opening balance, and assert the returned `balance` matches. Run `cargo test -p firefly-sample-lumen` and watch it pass against the real router.

Next, give the `GET` endpoint a real backing store with [Persistence & Reactive Repositories](#), then put the rules behind the bus in [Domain-Driven Design](#) and [CQRS](#).



CHAPTER 8

Persistence & Reactive Repositories

Lumen has a wallet API that returns a `WalletView` — but where does that view come from? At the end of [Your First HTTP API](#) the answer was "an in-memory map." This chapter gives Lumen a real persistence vocabulary: the **repository pattern** as the seam between the query side and storage, the in-memory baseline that keeps the teaching build infrastructure-free, and the pluggable reactive-repository / SQL upgrade path that swaps that baseline for Postgres, MySQL, SQLite, or MongoDB without touching a call site.

By the end of this chapter, Lumen will have its query-side read store — the `ReadModel` — framed as a repository: a map of wallet id → `WalletView` that the `GetWallet` query reads from and the event projection (built in [EDA](#)) writes to. You will see the in-memory baseline that ships in `samples/lumen`, and exactly which `firefly-data` adapter you would drop in to make it durable — the book's recurring swap the adapter move.

`firefly-data` provides the framework's persistence vocabulary: a composable `Filter` query DSL, a `Page<T>` paged-result envelope, a blocking `Repository<T, K>` contract, and — built on `firefly-reactive` — a reactive **CRUD** surface that is the Rust analog of Spring Data R2DBC. This chapter covers both, with a real streaming SQL repository.

🌿 SPRING PARITY

This is the Repository pattern as Spring Data popularized it, translated to idiomatic Rust. `Repository<T, K>` is the analog of `JpaRepository<T, ID>`; `ReactiveCrudRepository<T, ID>` is the analog of Spring Data R2DBC's `ReactiveCrudRepository<T, ID>`. You depend on the trait; the adapter supplies the SQL — exactly the JVM contract.

Lumen's read store, as a repository

Lumen splits its write model from its read model (the CQRS shape you build out in `CQRS`). The write side is the event-sourced `Ledger`; the read side is a flat, query-optimized `ReadModel` that `GET /api/v1/wallets/:id` serves. In `samples/lumen` the read model is an in-memory map — small, exact, and dependency-free, the right baseline for teaching:

Rust

```
// samples/lumen/src/ledger.rs – the CQRS query side.
use std::collections::HashMap;
use std::sync::Mutex;

use crate::domain::WalletView;

/// The in-memory read model: a map of wallet id → WalletView, upserted by the
/// projection and served by the GetWallet query. A real service would back this
/// with firefly's reactive repository over Postgres; an in-memory map keeps the
/// teaching baseline dependency-free.
#[derive(Debug, Default)]
pub struct ReadModel {
    rows: Mutex<HashMap<String, WalletView>>,
}

impl ReadModel {
    /// Upserts a projected view, replacing any previous row for the id.
    pub fn upsert(&self, view: WalletView) {
        self.rows
            .lock()
            .expect("read model lock")
            .insert(view.id.clone(), view);
    }

    /// Looks a projected view up by id.
    pub fn find(&self, id: &str) -> Option<WalletView> {
        self.rows.lock().expect("read model lock").get(id).cloned()
    }
}
```

Two things are deliberate. First, the surface is a **repository in miniature**:

`upsert(view)` and `find(id)` are the only operations the query side needs, so those are the only operations it exposes. Second, the keys and values are the plain domain

types — a `WalletView` keyed by its `id` — so when you swap the map for a database adapter, the *shape* the rest of Lumen sees does not move: a `find` still returns `Option<WalletView>`, an `upsert` still takes one.

That is the whole point of treating the read store as a repository: the `GetWallet` handler depends on "give me the view for this id," not on a `HashMap`. Below is the production surface you would lower this onto — same contract, a real database behind it.

The `Page<T>` envelope

`Page<T>` is the canonical paged result, wire-identical to the Java/.NET/Go/Python `Page<T>` so SDK clients deserialize it uniformly:

Rust

```
pub struct Page<T> {
    pub content: Vec<T>,
    pub number: usize,           // zero-based page index
    pub size: usize,
    pub total_elements: u64,
    pub total_pages: usize,     // derived
}
```

A *list wallets* endpoint — a natural Lumen extension — returns a `Page<WalletView>` so a client can page through accounts without ever loading the whole table.

The `Filter` DSL

A `Filter` composes predicates, sorts, and a page window, and renders to a parameterized `WHERE` clause via `to_sql()`:

Rust

```

use firefly_data::{Direction, Filter, Op, Predicate};
use serde_json::json;

let filter = Filter::default()
    .where_eq("owner", json!("alice"))
    .add(Predicate { field: "balance".into(), op: Op::Gte, value: json!(
100_000) })
    .order_by("version", Direction::Desc)
    .paged(0, 20);

let (where_clause, args) = filter.to_sql();
// where_clause: a parameter-indexed " WHERE ..." fragment
// args:          the bound values, in order
assert!(where_clause.contains("WHERE"));
assert_eq!(args.len(), 2);

```

The operators (`Op`) cover `Eq`, `Ne`, `Lt`, `Lte`, `Gt`, `Gte`, `Like`, `ILike`, `In`, and `IsNil` — the last skips an argument slot, so a predicate list and its argument list stay aligned. A "rich wallets" query (`balance >= 100_000`, newest first) is exactly the filter above.

The blocking `Repository` contract

`Repository<T, K>` is the object-safe `async_trait` port; `MemoryRepository` implements it for tests, and you back it with your driver in production:

Rust

```

#[async_trait]
pub trait Repository<T, K>: Send + Sync {
    async fn find_by_id(&self, id: &K) -> Result<T, DataError>;
    async fn find(&self, filter: &Filter) -> Result<Page<T>, DataError>;
    // save / delete / count ...
}

```

This is the contract Lumen's `ReadModel` is a hand-rolled, two-method subset of. Lower it onto `Repository<WalletView, String>` and `find` / `find_by_id` / `save` come from the framework.

The reactive CRUD surface

On top of the blocking contract, `firefly-data` adds a **reactive** surface — the Spring Data R2DBC analog — built on `Mono` / `Flux` (the publishers from [The Reactive Model](#)). It is purely additive: nothing about the existing `Repository` API changes.

Spring Data R2DBC	firefly-data reactive
<code>ReactiveCrudRepository<T, ID></code>	<code>ReactiveCrudRepository<T, ID></code>
<code>Flux<T> findAll()</code>	<code>find_all() -> Flux<T></code>
<code>Flux<T> findAllById(ids)</code>	<code>find_all_by_id(ids) -> Flux<T></code>
<code>Mono<T> findById(id)</code>	<code>find_by_id(id) -> Mono<T></code>
<code>Mono<Boolean> existsById(id)</code>	<code>exists_by_id(id) -> Mono<bool></code>
<code>Mono<T> save(e)</code>	<code>save(e) -> Mono<T></code>
<code>Flux<T> saveAll(es)</code>	<code>save_all(es) -> Flux<T></code>
<code>Mono<Void> deleteById(id)</code>	<code>delete_by_id(id) -> Mono<()></code>
<code>Mono<Void> deleteAll()</code>	<code>delete_all() -> Mono<()></code>
<code>Mono<Long> count()</code>	<code>count() -> Mono<u64></code>
<code>findAll(Specification, Pageable)</code>	<code>ReactiveSpecificationRepository</code>

A "no row" `find_by_id` maps to an empty `Mono` (Reactor's `Mono.empty()`), exactly as Spring Data signals a missing `findById` — the reactive equivalent of Lumen's `ReadModel::find` returning `None`.

In-memory, for tests

`ReactiveMemoryRepository` is the reactive twin of `MemoryRepository`. Drive the publishers with `block()` / `collect_list()`. Here it is holding wallet views — the reactive version of Lumen's read store:

Rust

```
use firefly_data::{ReactiveCrudRepository, ReactiveMemoryRepository};

#[derive(Clone, PartialEq, Debug)]
struct WalletView { id: String, owner: String, balance: i64, version: i64 }

#[tokio::main]
async fn main() {
    let repo = ReactiveMemoryRepository::new(|w: &WalletView| w.id.clone());

    // save -> Mono<T>
    repo.save(WalletView { id: "wlt_1".into(), owner: "alice".into(), balance:
1000, version: 1 })
        .block().await.unwrap();

    // find_all -> Flux<T>, collected to a Vec
    let all = repo.find_all().collect_list().block().await.unwrap().unwrap();
    assert_eq!(all.len(), 1);

    // find_by_id miss -> empty Mono (Lumen's `ReadModel::find` returning None)
    assert_eq!(repo.find_by_id("ghost".into()).block().await.unwrap(), None);
    assert_eq!(repo.count().block().await.unwrap(), Some(1));
}
```

Swapping `ReadModel` for this repository is mechanical: `upsert` becomes `save`, `find` becomes `find_by_id(...).block().await`, and the query handler keeps its `Option<WalletView>` shape.

Real SQL, streaming rows as a **Flux**

`PostgresReactiveRepository` is a production repository over `tokio-postgres`. Reads drive the driver's `query_raw` row stream, so each row is decoded by a `RowMapper` and emitted the moment it arrives over the wire — a million-row table never lands fully in memory. Writes use a per-entity `inserter` closure that renders a `T` to an upsert whose `RETURNING` projects the configured columns. Backing Lumen's read model with it looks like this:

Rust

```

use std::sync::Arc;
use firefly_data::{PostgresReactiveRepository, ReactiveCrudRepository,
TableConfig};
use firefly_kernel::FireflyError;
use tokio_postgres::{Row, types::ToSql, NoTls};

#[derive(Clone, PartialEq, Debug)]
struct WalletView { id: String, owner: String, balance: i64, version: i64 }

# async fn ex() -> Result<(), Box<dyn std::error::Error>> {
let (client, conn) =
    tokio_postgres::connect("postgres://localhost/lumen", NoTls).await?;
tokio::spawn(async move { let _ = conn.await; });
let client = Arc::new(client);

let repo: PostgresReactiveRepository<WalletView, String> =
PostgresReactiveRepository::new(
    Arc::clone(&client),
    TableConfig::new("wallet_views", "id", ["id", "owner", "balance",
"version"]),
    // RowMapper: decode a WalletView from each streamed row.
    |row: &Row| Ok(WalletView {
        id: row.try_get("id").map_err(|e|
FireflyError::internal(e.to_string()))?,
        owner: row.try_get("owner").map_err(|e|
FireflyError::internal(e.to_string()))?,
        balance: row.try_get("balance").map_err(|e|
FireflyError::internal(e.to_string()))?,
        version: row.try_get("version").map_err(|e|
FireflyError::internal(e.to_string()))?,
    }),
    // inserter: upsert RETURNING the projected columns (the projection's
upsert).
    |w: &WalletView| (
        "INSERT INTO \"wallet_views\" (\"id\", \"owner\", \"balance\",
\"version\") \
        VALUES ($1, $2, $3, $4) \
        ON CONFLICT (\"id\") DO UPDATE SET \"owner\" = EXCLUDED.\"owner\", \

```



```

    \"balance\" = EXCLUDED.\"balance\", \"version\" = EXCLUDED.\"version\"
  \
  RETURNING \"id\", \"owner\", \"balance\", \"version\".to_string(),
  vec![
    Box::new(w.id.clone()) as Box<dyn ToSql + Sync + Send>,
    Box::new(w.owner.clone()) as Box<dyn ToSql + Sync + Send>,
    Box::new(w.balance) as Box<dyn ToSql + Sync + Send>,
    Box::new(w.version) as Box<dyn ToSql + Sync + Send>,
  ],
),
);

// Rows stream lazily out of find_all() as a Flux — a *list wallets* endpoint.
let all = repo.find_all().collect_list().block().await?.unwrap();
# Ok(())
# }

```

Use `stream_query(sql, params)` for custom derived queries: any `SELECT` projecting the configured columns is streamed row-by-row through the same `RowMapper`. This `Flux` plugs directly into a `NdJson` / `Sse` endpoint, so a database read streams to the client end-to-end with backpressure — no collect-then-emit step anywhere in the path. (That is the same `Flux → NdJson` seam Lumen's events endpoint uses, only sourced from rows instead of an event store.)

Reactive specification / paging

`ReactiveSpecificationRepository` runs a composable `Specification` predicate with an optional `Pageable` window and **streams** the matches as a `Flux` — the reactive analog of `findAll(Specification, Pageable)`, but with no intermediate `Page<T>` envelope, so it plugs straight into an NDJSON / SSE endpoint with backpressure.

Pluggable databases – a new DB is a new adapter

`firefly-data` is the `ports` crate: it owns no driver and implies no SQL engine. The `Filter` DSL, the composable `Specification`, the repository traits, and the auditing / soft-delete policies are all storage-agnostic. The lowering surface is what makes this hexagonal:

- a `SqlDialect` trait with three shipped impls — `PostgresDialect`, `MySqlDialect`, `SqliteDialect` — so `Filter::to_sql_with(&dialect)` and `Specification::to_sql_with(&dialect)` render the *same* query tree per backend, getting placeholder style (`$1` vs `?`), identifier quoting (`"id"` vs ``id``), `IN`-list shape, and case-insensitive `LIKE` right for each. (`Filter::to_sql` / `Specification::to_sql` stay the PostgreSQL default.)
- a `Specification::to_mongo()` / `Filter::to_mongo()` that lowers the same tree to a MongoDB `$`-operator filter document.

Two adapter crates implement those ports so you code once and swap backends.

🌿 SPRING PARITY

This is hexagonal architecture / "ports & adapters" as Spring practices it: your service depends on the repository *port*; the *adapter* (the relational starter, the Mongo module) is chosen by configuration. Swapping Postgres for MySQL is the Rust analog of swapping a JDBC driver and dialect — a config change, not a code change.

Relational – `firefly-data-sqlx` (Postgres / MySQL / SQLite)

`SqlxReactiveRepository` (and the blocking `SqlxRepository`) serve all three relational backends from one codebase. A `Db` enum tags a `PgPool` / `MySqlPool` / `SqlitePool` with its `Backend`, and the repository picks the matching `SqlDialect` at runtime — so "new relational DB = new pool", not "new adapter". `UPSERT` is dialect-

aware (`ON CONFLICT ... DO UPDATE` for Postgres/SQLite, `ON DUPLICATE KEY UPDATE` for MySQL), reads stream off sqlx's row stream into a `Flux`, and an optional `Auditor` / `SoftDeletePolicy` auto-stamps and hides rows on every write/read.

SQLite-in-memory is the **no-infrastructure default** — the same role the in-memory map plays in `samples/lumen`, but exercising the real adapter:

Rust

```

use firefly_data::{ReactiveCrudRepository, TableConfig};
use firefly_data_sqlx::{AnyRow, ColumnValue, Db, SqlxReactiveRepository};
use firefly_kernel::FireflyError;

#[derive(Debug, Clone, PartialEq)]
struct WalletView { id: String, owner: String, balance: i64 }

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async
{
    let pool = sqlx::SqlitePool::connect("sqlite::memory:").await.unwrap();
    sqlx::query(r#"CREATE TABLE "wallet_views" ("id" TEXT PRIMARY KEY, "owner" TEXT
NOT NULL, "balance" BIGINT NOT NULL)"#)
        .execute(&pool).await.unwrap();

    let repo: SqlxReactiveRepository<WalletView, String> =
SqlxReactiveRepository::new(
    Db::Sqlite(pool),
    TableConfig::new("wallet_views", "id", ["id", "owner", "balance"]),
    // RowMapper: decode by column name – backend-agnostic via AnyRow.
    |row: &AnyRow| Ok:::<_, FireflyError>(WalletView {
        id: row.get_str("id")?,
        owner: row.get_str("owner")?,
        balance: row.get_i64("balance")?,
    }),
    // RowWriter: the entity's (column, value) pairs.
    |w: &WalletView| vec![
        ColumnValue::new("id", w.id.clone()),
        ColumnValue::new("owner", w.owner.clone()),
        ColumnValue::new("balance", w.balance),
    ],
);
repo.save(WalletView { id: "wlt_1".into(), owner: "alice".into(), balance:
1000 })
    .block().await.unwrap();
# });

```

Switching to Postgres or MySQL is `Db::Postgres(pg_pool)` / `Db::MySql(my_pool)` — the repository call sites do not change. That is the upgrade path the `samples/lumen` comment promises: the in-memory `ReadModel` becomes a `SqLxReactiveRepository<WalletView, String>`, and the `GetWallet` handler is none the wiser.

Document – `firefly-data-mongodb` (MongoDB)

`MongoRepository<T, ID>` puts a MongoDB collection behind the same `ReactiveCrudRepository` + `ReactiveSpecificationRepository` traits, lowering a `Specification` via `Specification::to_mongo()` exactly as the relational adapters lower it via `to_sql`. A `BaseDocument` mixin (embedded with `#[serde(flatten)]`) carries the audit stamps and soft-delete column, and `with_soft_delete(policy)` makes every read inject a `{"<column>": null}` guard while `delete_by_id` becomes a logical delete. Reads stream lazily off the driver cursor as a `Flux`.

Rust

```

use firefly_data::{ReactiveCrudRepository, Specification};
use firefly_data_mongodb::{BaseDocument, MongoRepository};
use mongodb::bson::{Bson, Document};
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
struct WalletDocument {
    #[serde(rename = "_id")] id: String,
    owner: String,
    balance: i64,
    #[serde(flatten)] base: BaseDocument,
}

# async fn run() -> Result<(), Box<dyn std::error::Error>> {
let client = mongodb::Client::with_uri_str("mongodb://localhost:27017").await?;
let collection =
client.database("lumen").collection::<Document>("wallet_views");
let repo: MongoRepository<WalletDocument, String> =
    MongoRepository::new(collection, |w: &WalletDocument|
Bson::String(w.id.clone()));

repo.save(WalletDocument {
    id: "wlt_1".into(), owner: "alice".into(), balance: 1000, base:
BaseDocument::new(),
}).block().await?;
# Ok(())
# }

```

Because all four backends sit behind the same ports, a service that codes against `Repository` / `ReactiveCrudRepository` / `Specification` can move from Postgres to MySQL, SQLite, or MongoDB by swapping the adapter constructor — and adding a *new* database is "write a `firefly-data-<tech>` crate that implements the ports", not "rewrite the data layer." Both adapters are tested against real Postgres, MySQL, SQLite, and MongoDB.

One-dependency note. *Lumen pulls none of these adapters in its default build — they are opt-in cargo features on the `firefly` facade (`firefly = { version = "26.6", features = ["data-sqlx"] }`), re-exported as `firefly::data_sqlx` / `firefly::data_mongodb`. The teaching build stays lean; the production build adds exactly the driver it needs.*

Schema migrations

`firefly-migrations` is a forward-only SQL migration runner. Migration files are named `V{version}_{description}.sql` (e.g. `V001_init.sql`); each runs once, in version order, inside a transaction. The runner works against any store behind the small synchronous `Database` port:

Rust

```
use firefly_migrations::{run, DirSource};

let src = DirSource { dir: "migrations".into() };
run(&mut db, &src)?; // applies pending migrations in order
let status = firefly_migrations::inspect(&mut db, &src)?; // applied + pending
```

The `CLI` wraps this: `firefly db init`, `firefly db migrate -m "create wallet_views"`, `firefly db upgrade --url sqlite://lumen.db`, and `firefly db status`. A `V001_wallet_views.sql` creating the read-model table is all the schema Lumen's durable read side needs.

Transactions

`firefly-transactional` provides `with_tx(ctx, db, f)` over pluggable `Database` / `Transaction` ports, so a unit of work commits or rolls back as a whole. Bind the ports to your driver and run your repository calls inside the closure. (The write side of

Lumen reaches durability differently — through the event store's optimistic-concurrency `append`, covered in [Event Sourcing](#) — but a read model upserted from many events at once is a natural unit of work.)

What changed in Lumen

Lumen now has a clear persistence story, even though its teaching build stays infrastructure-free:

- **The read store is a repository.** `ReadModel` (in `samples/lumen/src/ledger.rs`) is a `Mutex<HashMap<String, WalletView>>` exposing exactly `upsert(view)` and `find(id)` — the two operations the query side needs, keyed by the `WalletView`'s own id. The `GetWallet` handler depends on the *contract*, not the map.
- **The baseline is in-memory by choice.** The map keeps the dependency footprint at one Firefly crate; the comment in the source names the upgrade explicitly — "a real service would back this with firefly's reactive repository over Postgres."
- **The upgrade is an adapter swap.** Lowering `ReadModel` onto `ReactiveCrudRepository<WalletView, String>` — `SqlxReactiveRepository` for Postgres/MySQL/SQLite, `MongoRepository` for Mongo — turns `upsert` into `save` and `find` into `find_by_id`, with the query handler's `Option<WalletView>` shape unchanged. A new database is a new pool (relational) or a new adapter crate, never a rewrite.

Exercises

1. Re-skin `ReadModel` as a trait. Define `trait WalletViews { fn upsert(&self, v: WalletView); fn find(&self, id: &str) -> Option<WalletView>; }`, implement it for the in-memory `ReadModel`, and have the `GetWallet` handler take `&dyn WalletViews`. Confirm the rest of Lumen still compiles — proof the query side depends on the contract, not the map.

2. **Back the read model with SQLite.** Using the `SqlxReactiveRepository` listing above, create a `wallet_views` table in `sqlite::memory:`, `save` two views, and `find_all().collect_list()` them. Assert both come back. This is the production read store, exercised end to end against the real adapter.
3. **Page the wallets.** Build a `Filter` that selects wallets with `balance >= 100_000` ordered by `version` descending, page `(0, 20)`, and print its `to_sql()`. Then describe (one sentence each) how the same filter would render under `MySqlDialect` and `SqliteDialect`.
4. **Trace the swap.** List the exact lines in `samples/lumen/src/ledger.rs` that would change if `ReadModel` became a `SqlxReactiveRepository<WalletView, String>`, and which lines in `samples/lumen/src/commands.rs` (the `GetWallet` handler) would *not* — confirming the adapter boundary holds.

With persistence framed, the next chapter models the business itself — the `Money` value object and the `Wallet` aggregate. See [Domain-Driven Design](#).



CHAPTER 9

Domain-Driven Design

Lumen can open wallets and read them back, and it has a place to put the read model. But look closely and something is missing: nothing yet *owns* the rules. Where is "you cannot withdraw more than you hold"? Where is "an amount must be positive"? Where is "a wallet must have an owner"? Right now those would live as scattered `if`-statements in a handler — exactly the kind of rule a future developer can bypass by writing to the store directly.

Domain-Driven Design fixes that by making the model responsible for its own invariants. This chapter builds Lumen's domain core: the `Money` value object (immutable, integer cents, exact arithmetic) and the `Wallet` aggregate that guards the overdraft, positive-amount, and owner-required rules — each command raising the domain event that records what happened. Both files are drawn verbatim from `samples/lumen`.

By the end of this chapter, Lumen will have `src/money.rs` and the heart of `src/domain.rs`: a `Money` value object closed under `add` / `subtract`, a `Wallet` aggregate with `open` / `deposit` / `withdraw` that enforce the invariants and `raise` events, and a typed `DomainError` family whose `Display` strings surface verbatim as RFC 9457 problem details. The aggregate carries `#[derive(AggregateRoot)]`; nothing carries `thiserror`.

SPRING PARITY

This is the DDD vocabulary you know from the JVM — value objects, aggregates, aggregate roots, domain events — expressed as idiomatic Rust.

`#[derive(AggregateRoot)]` is the analog of jMolecules's `@AggregateRoot` / Spring Data's `AbstractAggregateRoot`, generating the event-buffer plumbing so your code holds only the rules.

The `Money` value object

A value object is defined entirely by its attributes — it has no identity — and it is **immutable**: every operation returns a *new* value rather than mutating in place. Money is the textbook example, and getting it right matters more here than almost anywhere: amounts are stored as integer **minor units** (cents), so the arithmetic is exact. No binary floating-point drift, the classic correctness bug a money type exists to prevent.

Here is Lumen's `Money`, the whole core of `src/money.rs`:

Rust

```
// samples/lumen/src/money.rs
use std::fmt;
use serde::{Deserialize, Serialize};

/// An exact monetary amount, expressed in integer minor units (cents).
#[derive(Debug, Clone, Copy, Default, PartialEq, Eq, PartialOrd, Ord, Serialize,
Deserialize)]
#[serde(transparent)]
pub struct Money {
    cents: i64,
}

impl Money {
    /// A zero amount – the opening balance of a brand-new wallet.
    pub const ZERO: Money = Money { cents: 0 };

    /// Builds a `Money` from a raw minor-unit (cent) count.
    pub const fn cents(cents: i64) -> Self {
        Money { cents }
    }

    /// Builds a `Money` from a whole-currency unit count (`from_units(10)` is €10.00).
    pub const fn from_units(units: i64) -> Self {
        Money { cents: units * 100 }
    }

    /// The amount in minor units (cents) – the wire representation.
    pub const fn cents_value(self) -> i64 {
        self.cents
    }

    /// Whether this amount is strictly positive (`> 0`).
    pub const fn is_positive(self) -> bool {
        self.cents > 0
    }

    /// Returns a new `Money` that is `self + other` (immutable addition).

```

```

#[must_use]
pub const fn add(self, other: Money) -> Money {
    Money { cents: self.cents + other.cents }
}

/// Returns `self - other`, or `MoneyError::Overdraw` if that would go below
zero.
pub fn subtract(self, other: Money) -> Result<Money, MoneyError> {
    if other.cents > self.cents {
        return Err(MoneyError::Overdraw);
    }
    Ok(Money { cents: self.cents - other.cents })
}

/// Validates that this amount is strictly positive, returning it unchanged
on success.
pub fn require_positive(self) -> Result<Money, MoneyError> {
    if self.is_positive() {
        Ok(self)
    } else {
        Err(MoneyError::NonPositive)
    }
}
}

```

Several design choices are load-bearing:

- **Integer cents, never a float.** The single field is `cents: i64`, kept private so the only way to a `Money` is through a constructor. €10.00 is `Money::cents(1_000)`; €12.50 after an addition is `Money::cents(1_250)`. The math is exact by construction.
- **Immutable.** `add` is `#[must_use]` and `const`; it returns a fresh `Money` and leaves the operands untouched. So does `subtract`. There is no `add_assign` — a value object is replaced, not edited.

- **Closed under the wallet's operations.** `add` for credits, `subtract` (fallible, guarding against overdraw) for debits, and `require_positive` for the guard every mutating command runs before raising an event.
- `#[serde(transparent)]`. `Money` serializes as the bare cent integer — a balance of €10.00 is the JSON number `1000`, the contract the read model and the event payloads share. No `{ "cents": 1000 }` wrapper.

`Display` renders the human form (`1250` cents → `"12.50"`) for logs and the banner, and the error type is hand-written:

Rust

```
/// The typed error a `Money` operation can fail with.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum MoneyError {
    /// An amount was expected to be strictly positive (> 0) but was not.
    NonPositive,
    /// A subtraction would drop the balance below zero.
    Overdraw,
}

impl fmt::Display for MoneyError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            MoneyError::NonPositive => f.write_str("amount must be positive"),
            MoneyError::Overdraw => f.write_str("amount exceeds balance"),
        }
    }
}

impl std::error::Error for MoneyError {}
```

One-dependency note. `MoneyError` hand-writes `Display` and `std::error::Error` rather than deriving them with `thiserror`. That is on purpose: *Lumen* depends on exactly one *Firefly* crate plus `axum` and `serde`, and the book keeps that promise all the way down — even the error enums. Two trait impls per error type is a small price for an honest dependency list.

SPRING PARITY

A frozen, value-equal `Money` maps to a Java `record` (the JVM's value type) or *jMolecules*'s `@ValueObject`. The "minor units, never a float" discipline is the same one Spring teams reach for `BigDecimal` or integer cents to enforce — Rust just makes the immutability a compiler guarantee.

The `Wallet` aggregate

`Money` solves *representation*. The `Wallet` aggregate owns *behavior*: it is the consistency boundary, the single entry point through which every change to a wallet must flow, so the invariants cannot be bypassed.

Lumen's `Wallet` is event-sourced (the full event-store machinery arrives in [Event Sourcing](#)), but the DDD shape is visible already. The aggregate embeds the framework's `AggregateRoot` — the uncommitted-event buffer plus a version — and `#[derive(AggregateRoot)]` generates the `AGGREGATE_TYPE` constant and the `aggregate()` / `aggregate_mut()` accessors:

Rust

```
// samples/lumen/src/domain.rs
use firefly::eventsourcing::{AggregateRoot, DomainEvent};
use firefly::prelude::*;

use crate::money::{Money, MoneyError};

/// The aggregate-type discriminator stamped onto every event a Wallet raises.
pub const AGGREGATE_TYPE: &str = "Wallet";

/// The event-sourced wallet aggregate.
#[derive(Debug, Clone, AggregateRoot)]
#[firefly(aggregate_type = "Wallet")]
pub struct Wallet {
    /// The framework aggregate root – uncommitted-event buffer + version.
    pub root: AggregateRoot,
    /// The owner's display name.
    pub owner: String,
    /// The current balance as a `Money` value object.
    pub balance: Money,
    /// Whether the wallet has been opened (an empty stream is "absent").
    pub opened: bool,
}
```

The aggregate's projected state — `owner`, `balance`, `opened` — is the *result* of applying its events. The behavior methods enforce the rules.

The factory: `open`

`open` is the sole way to bring a wallet into existence. It validates inputs, constructs the aggregate, and `raise` s the opening event:

Rust

```

impl Wallet {
    /// Opens a fresh wallet, raising a `WalletOpened` event.
    pub fn open(
        id: impl Into<String>,
        owner: impl Into<String>,
        opening_balance: Money,
    ) -> Result<Self, DomainError> {
        let id = id.into();
        let owner = owner.into();
        if owner.trim().is_empty() {
            return Err(DomainError::OwnerRequired);
        }
        if opening_balance.cents_value() < 0 {
            return Err(DomainError::NonPositiveAmount);
        }
        let mut wallet = Wallet {
            root: AggregateRoot::new(&id, AGGREGATE_TYPE),
            owner: owner.clone(),
            balance: Money::ZERO,
            opened: false,
        };
        wallet.raise(
            WalletOpened::EVENT_TYPE,
            &WalletOpened { wallet_id: id, owner, opening_balance:
opening_balance.cents_value() },
        );
        wallet.balance = opening_balance;
        wallet.opened = true;
        Ok(wallet)
    }
}

```

Two invariants are enforced before any event is raised: the owner must be non-blank (`OwnerRequired`), and the opening balance must not be negative (a *zero* opening balance is allowed). Using a factory rather than a public constructor guarantees the `WalletOpened` event is never forgotten — there is no back-channel that produces a wallet without recording its birth.

The behavior methods: `deposit` and `withdraw`

The two mutating commands follow one shape: check the wallet exists, validate the amount, apply the `Money` operation, raise the event, update state. The overdraft rule is enforced exactly once — by `Money::subtract`:

Rust

```

impl Wallet {
    /// Credits `amount` to the wallet, raising a `MoneyDeposited` event.
    pub fn deposit(&mut self, amount: Money) -> Result<(), DomainError> {
        self.require_opened()?;
        let amount = amount.require_positive()?;
        self.raise(
            MoneyDeposited::EVENT_TYPE,
            &MoneyDeposited { wallet_id: self.root.id.clone(), amount:
amount.cents_value() },
        );
        self.balance = self.balance.add(amount);
        Ok(())
    }

    /// Debits `amount` from the wallet, raising a `MoneyWithdrawn` event.
    pub fn withdraw(&mut self, amount: Money) -> Result<(), DomainError> {
        self.require_opened()?;
        let amount = amount.require_positive()?;
        let remaining = self.balance.subtract(amount)?; // Overdraw ->
InsufficientFunds
        self.raise(
            MoneyWithdrawn::EVENT_TYPE,
            &MoneyWithdrawn { wallet_id: self.root.id.clone(), amount:
amount.cents_value() },
        );
        self.balance = remaining;
        Ok(())
    }

    fn require_opened(&self) -> Result<(), DomainError> {
        if self.opened {
            Ok(())
        } else {
            Err(DomainError::NotFound(self.root.id.clone()))
        }
    }
}

```

Read `withdraw` carefully — it is where the consistency boundary earns its keep. The order is *validate, then mutate*: `require_opened`, `require_positive`, and the `subtract` overdraft check all run **before** the event is raised. If the withdrawal would overdraw, `Money::subtract` returns `MoneyError::Overdraw`, the `?` converts it to `DomainError::InsufficientFunds` (via the `From` impl below), and the method returns *without raising anything*. The aggregate's invariant — "balance never goes below zero" — is physically unreachable from outside, because the only path to a withdrawal runs this gauntlet first.

The typed `DomainError` family

The errors are a closed enum with stable `Display` strings — stable because tests assert on them and they surface verbatim as the RFC 9457 problem `detail` once mapped at the HTTP boundary (see [CQRS](#) and [First HTTP API](#)):

Rust

```

/// The typed domain-error family.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum DomainError {
    /// A command referenced an amount that was not strictly positive.
    NonPositiveAmount,
    /// A withdrawal (or transfer debit) exceeded the available balance.
    InsufficientFunds,
    /// A command targeted a wallet that was never opened.
    NotFound(String),
    /// The owner name was empty when opening a wallet.
    OwnerRequired,
}

impl std::fmt::Display for DomainError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            DomainError::NonPositiveAmount => f.write_str("amount must be
positive"),
            DomainError::InsufficientFunds => f.write_str("insufficient funds"),
            DomainError::NotFound(id) => write!(f, "wallet {id} not found"),
            DomainError::OwnerRequired => f.write_str("owner is required"),
        }
    }
}

impl std::error::Error for DomainError {}

impl From<MoneyError> for DomainError {
    fn from(e: MoneyError) -> Self {
        match e {
            MoneyError::NonPositive => DomainError::NonPositiveAmount,
            MoneyError::Overdraw => DomainError::InsufficientFunds,
        }
    }
}

```

The `From<MoneyError> for DomainError` impl is what lets `withdraw` write `self.balance.subtract(amount)?` and have an overdraw surface as `InsufficientFunds` — the value object reports the arithmetic fact, the aggregate translates it into domain language. Like `MoneyError`, `DomainError` hand-writes its `Display` and `Error` impls: no `thiserror`, one dependency.

🌿 SPRING PARITY

Where a Spring `Wallet` aggregate would throw a `BusinessRuleViolation` (mapped to HTTP 422) and an `AggregateNotFound` (mapped to 404), Lumen returns a typed `DomainError`. `InsufficientFunds` / `NonPositiveAmount` / `OwnerRequired` become 422 problems and `NotFound` becomes a 404 — the same status mapping, decided by a `match` at the web boundary instead of an exception-to-status table.

The read-model view

The aggregate is the *write* model. What `GET /api/v1/wallets/:id` returns is a flat, query-optimized `WalletView` — the read model the previous chapter framed as a repository. The aggregate produces it on demand:

Rust

```

impl Wallet {
    /// The current read-model view of this aggregate.
    pub fn view(&self) -> WalletView {
        WalletView {
            id: self.root.id.clone(),
            owner: self.owner.clone(),
            balance: self.balance.cents_value(),
            version: self.root.version,
        }
    }
}

/// The read-model projection of a wallet – the wire shape served by
/// `GET /api/v1/wallets/:id` and stored in the read-model repository.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct WalletView {
    pub id: String,
    pub owner: String,
    /// The current balance, in minor units (cents).
    pub balance: i64,
    /// The aggregate version (number of events applied).
    pub version: i64,
}

```

Keeping `Wallet` (rich, rule-enforcing, event-raising) and `WalletView` (flat, serializable, no behavior) as separate types is the same domain/persistence separation the previous chapter drew around the repository: the aggregate never serializes itself, and the wire shape never carries an invariant. The `version` field lets a client detect staleness under the eventual consistency CQRS introduces.

Proving the invariants

Because the aggregate is a plain struct with plain methods, you can exercise every rule with no database and no HTTP — the unit tests that ship in `samples/lumen/src/domain.rs`:

Rust

```
#[test]
fn open_validates_owner_and_balance() {
    assert_eq!(Wallet::open("w1", " ", Money::cents(100)).unwrap_err(),
DomainError::OwnerRequired);
    assert_eq!(Wallet::open("w1", "alice", Money::cents(-1)).unwrap_err(),
DomainError::NonPositiveAmount);
    let w = Wallet::open("w1", "alice", Money::ZERO).unwrap();
    assert!(w.opened);
    assert_eq!(w.balance, Money::ZERO);
}

#[test]
fn withdraw_rejects_overdraft() {
    let mut w = Wallet::open("w1", "alice", Money::cents(100)).unwrap();
    assert_eq!(w.withdraw(Money::cents(101)).unwrap_err(),
DomainError::InsufficientFunds);
    // The failed command raised no event beyond the open.
    assert_eq!(w.root.uncommitted().len(), 1);
}

#[test]
fn deposit_and_withdraw_update_balance_and_raise_events() {
    let mut w = Wallet::open("w1", "alice", Money::cents(100)).unwrap();
    w.deposit(Money::cents(50)).unwrap();
    assert_eq!(w.balance, Money::cents(150));
    w.withdraw(Money::cents(30)).unwrap();
    assert_eq!(w.balance, Money::cents(120));
}
```


The `withdraw_rejects_overdraft` test makes the consistency boundary concrete: after a rejected withdrawal, the aggregate's uncommitted-event buffer still holds exactly one event — the `WalletOpened` from the factory. The overdraft never produced a `MoneyWithdrawn`, so nothing partial can ever be persisted. That is the difference between a service-level guard (a convention) and an aggregate invariant (a physical constraint): the model exposes no mechanism to violate it.

What changed in Lumen

Lumen now has a domain core that owns its rules:

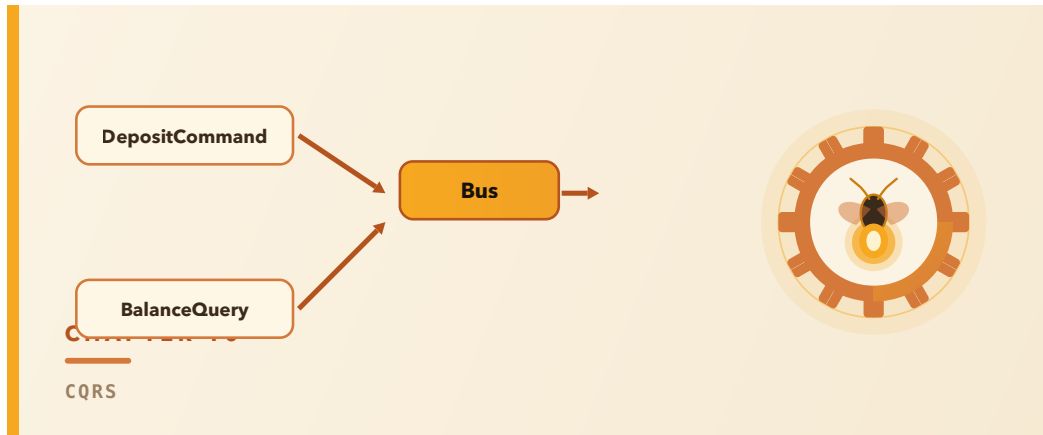
- `src/money.rs` — a `Money` value object: immutable, integer cents, `#[serde(transparent)]` so it rides the wire as a bare number, closed under `add` / `subtract` / `require_positive`, with a hand-written `MoneyError` (no `thiserror`).
- `src/domain.rs` — the `Wallet` aggregate carrying `#[derive(AggregateRoot)]` (which generates `AGGREGATE_TYPE` and the `aggregate()` accessors). `open` / `deposit` / `withdraw` enforce the three invariants — owner required, amounts positive, no overdraft — *before* raising an event, so a rejected command leaves the event buffer untouched.
- The typed `DomainError` family with stable `Display` strings that surface as RFC 9457 problem details, plus the `From<MoneyError>` bridge that turns an arithmetic overdraft into `InsufficientFunds`.
- `WalletView` — the flat read-model projection the aggregate hands out via `view()`, kept a separate type from the rule-enforcing aggregate.

The event payloads (`WalletOpened` / `MoneyDeposited` / `MoneyWithdrawn`) and the `rehydrate` / `apply` fold that reconstruct a wallet from its stream get their full treatment in [Event Sourcing](#). Here, the shape that matters is the DDD one: a value object you cannot corrupt and an aggregate you cannot bypass.

Exercises

1. **Break an invariant, watch it hold.** In a `#[cfg(test)]` block, open a wallet with `Money::cents(100)` and call `withdraw(Money::cents(200))`. Assert the error is `DomainError::InsufficientFunds` and that `w.root.uncommitted().len()` is still `1` — proving the rejected command raised no event.
2. **Add a `transfer_within` rule (domain only).** Write a free function `fn transfer(from: &mut Wallet, to: &mut Wallet, amount: Money) -> Result<(), DomainError>` that calls `from.withdraw(amount)?` then `to.deposit(amount)?`. Test that a transfer exceeding the source balance fails on the withdraw leg and leaves the *target* balance unchanged. (The real, persisted version becomes the saga in [Sagas](#).)
3. **Confirm the wire shape.** Serialize a freshly opened wallet's `view()` with `serde_json` and assert it equals `{"id":"wlt_1","owner":"alice","balance":250,"version":1}` — verifying that `Money`'s `#[serde(transparent)]` and `WalletView`'s field order produce the contract the read model and clients share.
4. **Justify the hand-written error.** In two sentences, explain why `MoneyError` and `DomainError` implement `Display` / `Error` by hand instead of deriving with `thiserror`, and what it would cost the book's one-dependency promise to add the crate.

Next, separate the write path from the read path with the command/query bus. See [CQRS](#).



CHAPTER 10

CQRS: Commands & Queries

Lumen's `Wallet` aggregate enforces its own rules, and the read model has a home. But the controller still needs a way to *deliver* an instruction to the write side and a *question* to the read side — and to do it without the two paths sharing a code path, so reads can be cached and writes can be validated independently.

CQRS — Command Query Responsibility Segregation — draws that bright line. Writes become **commands** (`OpenWallet`, `Deposit`, `Withdraw`); reads become a **query** (`GetWallet`). Each travels a typed `Bus`, matched to its handler by `std::any::TypeId`, through a middleware chain that validates commands and caches queries. This chapter wires Lumen's bus end to end, exactly as `samples/lumen` does.

By the end of this chapter, Lumen will have `src/commands.rs`: the `OpenWallet` / `Deposit` / `Withdraw` commands and the `GetWallet` query as `#[derive(Command)]` / `#[derive(Query)]` structs, free-`fn` handlers marked `#[command_handler]` / `#[query_handler]`, the bus wired with validation and query-cache middleware, and the read-after-write cache invalidation that keeps a balance from going stale after a deposit.

SPRING PARITY

The `Bus` is the framework's command/query dispatcher. `#[command_handler]` / `#[query_handler]` are the analog of Axon's `@CommandHandler` / `@QueryHandler` (or Spring Modulith message handlers); `bus.send` / `bus.query` are the gateway dispatch; the middleware chain is the analog of Spring's handler interceptors. The difference: Lumen's handlers are free functions a macro registers, not annotated methods a proxy wraps.

Commands, queries, and the `Message` trait

Every command and query implements `Message`. Hand-writing it is one line, but the trait's optional methods — `validate`, `cache_ttl` — are overridable defaults that the matching middleware picks up automatically:

Rust

```
pub trait Message: Clone + Serialize + Send + Sync + 'static {
    fn validate(&self) -> Result<(), CqrsError> { Ok(()) } //
    ValidationMiddleware
    fn cache_ttl(&self) -> Option<Duration> { None } // QueryCache
}
```

`Clone` stands in for pass-by-value handler invocation; `Serialize` seeds the query cache key. Lumen never writes that impl by hand — it derives it.

Lumen's commands and query

The four messages are plain structs carrying `#[derive(Command)]` / `#[derive(Query)]`, which generate the `Message` impl. The `#[firefly(validate)]` field attribute makes a field required (the generated `validate()` rejects an empty `String` or a non-positive number), and `#[firefly(cache_ttl = "...")]` is reflected on the query's generated `cache_ttl`:

Rust

```
// samples/lumen/src/commands.rs
use firefly::prelude::*;
use serde::{Deserialize, Serialize};

/// `POST /api/v1/wallets` command – open a new wallet.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct OpenWallet {
    /// The wallet owner's display name – required.
    #[firefly(validate)]
    pub owner: String,
    /// The opening balance, in minor units (cents); must be `>= 0`.
    #[serde(rename = "openingBalance")]
    pub opening_balance: i64,
}

/// `POST /api/v1/wallets/:id/deposit` command – credit a wallet.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct Deposit {
    /// The wallet to credit – required.
    #[firefly(validate)]
    #[serde(rename = "walletId")]
    pub wallet_id: String,
    /// The amount to credit, in minor units (cents); must be `> 0`.
    #[firefly(validate)]
    pub amount: i64,
}

/// `POST /api/v1/wallets/:id/withdraw` command – debit a wallet.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct Withdraw {
    #[firefly(validate)]
    #[serde(rename = "walletId")]
    pub wallet_id: String,
    #[firefly(validate)]
    pub amount: i64,
}
```

```

}

/// `GET /api/v1/wallets/:id` query – cached for 30 seconds.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetWallet {
    /// The wallet id to fetch.
    pub id: String,
}

```

A few choices echo the domain chapter. The commands carry `i64` cents, not a `Money` — the handler constructs the value object, keeping the wire contract a bare number and the validation simple. `#[firefly(validate)]` on `amount` rejects a zero or negative amount *before* the handler runs, so the aggregate is never even called with structurally wrong data. And `#[serde(rename = ...)]` keeps the JSON camelCase (`openingBalance`, `walletId`) while the Rust fields stay `snake_case`.

SPRING PARITY

`#[derive(Command)]` + `#[firefly(validate)]` is the declarative analog of a Bean-Validation-annotated command record (`@NotBlank owner`, `@Positive amount`) whose constraints a validating interceptor enforces — except the check is generated by a derive macro, not reflected at runtime. `#[firefly(cache_ttl = "30s")]` mirrors a `@Cacheable(ttl = ...)` on the query path.

The free-`fn` handlers

A handler is a free `async fn` marked `#[command_handler]` or `#[query_handler]`. The macro reads the argument type as the dispatch key and generates a `register_<fn>(bus)` helper that installs it. The handler turns the bus's `CqrsError` channel into the precise outcome; the web layer (next) restores the HTTP status:

Rust

```

use crate::domain::{DomainError, Wallet, WalletView};
use crate::ledger::{Ledger, ReadModel};
use crate::money::Money;

/// Handles `OpenWallet`. `#[command_handler]` generates
`register_open_wallet(bus)`.
#[command_handler]
pub async fn open_wallet(cmd: OpenWallet) -> Result<WalletView, CqrsError> {
    if cmd.opening_balance < 0 {
        return Err(CqrsError::validation("openingBalance must be >= 0"));
    }
    state()
        .ledger
        .open(&cmd.owner, Money::cents(cmd.opening_balance))
        .await
        .map_err(to_cqrs)
}

/// Handles `Deposit`. Generates `register_deposit(bus)`.
#[command_handler]
pub async fn deposit(cmd: Deposit) -> Result<WalletView, CqrsError> {
    state()
        .ledger
        .deposit(&cmd.wallet_id, Money::cents(cmd.amount))
        .await
        .map_err(to_cqrs)
}

/// Handles `Withdraw`. Generates `register_withdraw(bus)`.
#[command_handler]
pub async fn withdraw(cmd: Withdraw) -> Result<WalletView, CqrsError> {
    state()
        .ledger
        .withdraw(&cmd.wallet_id, Money::cents(cmd.amount))
        .await
        .map_err(to_cqrs)
}

```



```

/// Handles `GetWallet`. Generates `register_get_wallet(bus)`.
#[query_handler]
pub async fn get_wallet(q: GetWallet) -> Result<WalletView, CqrsError> {
    if let Some(view) = state().read_model.find(&q.id) {
        return Ok(view);
    }
    let events = state().ledger.load_events(&q.id).await.map_err(to_cqrs)?;
    Ok(Wallet::rehydrate(&q.id, &events).view())
}

```

Each command handler constructs the `Money` value object from the command's `i64`, delegates to the `Ledger` application service (which rehydrates the aggregate, runs the domain command, and persists — see [Event Sourcing](#)), and maps a `DomainError` onto the bus's `CqrsError` channel:

Rust

```

fn to_cqrs(e: DomainError) -> CqrsError {
    CqrsError::handler(e.to_string())
}

```

The `get_wallet` query is the read-after-write pattern in miniature: it serves from the projected `ReadModel` first, and *only* if the projection has not yet caught up does it fall back to folding the event stream. That fallback is what keeps a read immediately after a write from returning a stale balance under the eventual consistency the projection introduces.

Why free functions, not handler structs? Rust free functions cannot capture state, so the resolved collaborators live in a module-local static and the handler reads them through `state()`. That is the next section — and it is the one piece of CQRS wiring Lumen does by hand.

Publishing collaborators: the `OnceLock` pattern

Because a `#[command_handler]` free function can't own a `Ledger` or a `ReadModel`, Lumen publishes the resolved collaborators once at startup into a process-global `OnceLock`, and the handlers reach them through `state()`:

Rust

```
use std::sync::{Arc, OnceLock};

/// The collaborators the CQRS handlers need.
struct HandlerState {
    ledger: Ledger,
    read_model: Arc<ReadModel>,
}

static STATE: OnceLock<HandlerState> = OnceLock::new();

/// Publishes the handlers' collaborators and returns the effective state —
/// the one actually held by the process-global OnceLock.
pub fn bind(ledger: Ledger, read_model: Arc<ReadModel>) -> (Ledger,
Arc<ReadModel>) {
    let effective = STATE.get_or_init(|| HandlerState { ledger, read_model });
    (effective.ledger.clone(), Arc::clone(&effective.read_model))
}

fn state() -> &'static HandlerState {
    STATE.get().expect("commands::bind must run before a handler dispatches")
}
```

The subtle part is the return value. The global binds only **once**, so a second `build_app()` in the same test binary keeps the *first* ledger and read model. `bind` returns the *effective* pair — whichever the `OnceLock` actually holds — so the composition root can wire the controller's saga, the event projection, and the free-fn

handlers all against the **same** collaborators, no matter how many times the app is built. (This is why the HTTP test suite can call `build_router()` per test and still share one ledger.)

SPRING PARITY

In Spring this problem does not exist: a `@CommandHandler` is a bean, and the container injects its collaborators. Rust free-fn handlers trade that injection for a one-line `bind` at startup — the same macro-quickstart pattern every Firefly-Rust service with free-fn handlers uses. If you prefer constructor injection, the DI container's `#[derive(Component)]` route (see [Dependency Wiring](#)) gives you the Spring shape; Lumen keeps the explicit `bind` for teachability.

Wiring the bus

The composition root in `src/web.rs` builds the bus, layers the middleware, binds the collaborators, and installs the handlers via the generated `register_*` helpers (gathered behind one `register(&bus)` fn):

Rust

```
// samples/lumen/src/commands.rs
/// Installs every generated handler-registration helper on `bus`.
pub fn register(bus: &Bus) {
    register_open_wallet(bus);
    register_deposit(bus);
    register_withdraw(bus);
    register_get_wallet(bus);
}
```

Rust

```
// samples/lumen/src/web.rs - build_app(), the relevant slice.
let bus = Arc::clone(&web.bus);
let read_model = Arc::new(ReadModel::default());

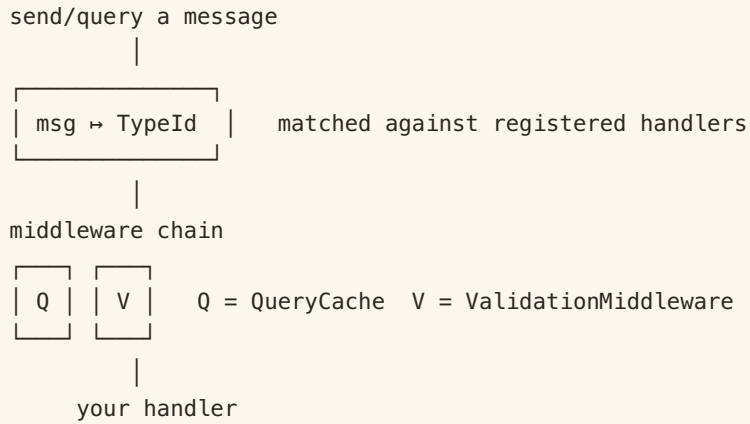
// Read-side caching on the bus (honours GetWallet's 30s cache_ttl).
let query_cache = QueryCache::new();
bus.use_middleware(query_cache.middleware());
// Validation middleware enforces the `#[firefly(validate)]` checks.
bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());

// Publish the handler collaborators, then install the generated registrations.
let (ledger, read_model) = crate::commands::bind(
    Ledger::new(Arc::clone(&store), Arc::clone(&broker)),
    read_model,
);
crate::commands::register(&bus);
```

Middleware runs first-registered = outermost. Two ship in this chain (a third, authorization, arrives with [Security](#)):

Middleware	Behaviour
<code>QueryCache::middleware()</code>	memoises results for messages whose <code>cache_ttl</code> is <code>Some</code>
<code>ValidationMiddleware</code>	calls <code>Message::validate</code> before dispatch, short-circuits on error

Text



Dispatching from the controller

The `#[rest_controller]` (built in [First HTTP API](#)) holds the `Bus` and dispatches through `send` / `query`. `Bus::query` is a readability synonym for `send`. A failed dispatch is a `CqrsError`, which the web layer maps to the right RFC 9457 status:

Rust

```
// samples/lumen/src/web.rs - WalletApi handlers.
#[post("/wallets")]
async fn open(
    State(api): State<WalletApi>,
    Json(body): Json<OpenWallet>,
) -> WebResult<(axum::http::StatusCode, Json<WalletView>)> {
    let view: WalletView = api.bus.send(body).await.map_err(cqrs_to_web)?;
    Ok((axum::http::StatusCode::CREATED, Json(view)))
}

#[get("/wallets/:id")]
async fn get(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
) -> WebResult<Json<WalletView>> {
    let view: WalletView = api.bus.query(GetWallet {
        id }).await.map_err(cqrs_to_web)?;
    Ok(Json(view))
}
```

`cqrs_to_web` is the seam where a domain failure becomes an HTTP status. It reads the `CqrsError` and its detail string — which, recall, is the `DomainError`'s stable `Display` text from the previous chapter — and chooses the status:

Rust

```

fn cQRS_to_web(err: CQRSError) -> WebError {
    match err {
        CQRSError::Validation(detail) =>
WebError::from(FireflyError::validation(detail)),
        CQRSError::Handler(detail) => {
            if detail.ends_with("not found") {
                WebError::from(FireflyError::not_found(detail)) //
404
            } else if detail == DomainError::InsufficientFunds.to_string()
                || detail == DomainError::NonPositiveAmount.to_string()
                || detail == DomainError::OwnerRequired.to_string()
            {
                WebError::from(FireflyError::validation(detail)) //
422
            } else {
                WebError::from(FireflyError::not_found(detail))
            }
        }
        other => WebError::from(FireflyError::internal(other.to_string())), //
500
    }
}

```

This is why the domain chapter insisted the `Display` strings be *stable*: they are the contract `cQRS_to_web` matches on to recover the precise status.

Read-after-write: invalidating the cache

`GetWallet` is cached for 30 seconds. Without care, a deposit would update the balance while a cached `GetWallet` kept serving the old one for up to 30 seconds. Lumen closes that gap by invalidating the cached query family after every mutation:

Rust

```
// samples/lumen/src/web.rs - deposit handler.
#[post("/wallets/:id/deposit")]
async fn deposit(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    Json(body): Json<AmountBody>,
) -> WebResult<Json<WalletView>> {
    let cmd = Deposit { wallet_id: id, amount: body.amount };
    let view: WalletView = api.bus.send(cmd).await.map_err(cqrs_to_web)?;
    api.query_cache.invalidate_type::<GetWallet>(); // read-after-write
    Ok(Json(view))
}
```

`QueryCache::invalidate_type::<GetWallet>()` evicts every cached result for exactly that query type. The withdraw handler does the same, and the transfer saga ([Sagas](#)) — which touches two wallets — invalidates the whole `GetWallet` family. The query cache's backend swap (Redis / Postgres) and event-driven invalidation get their own treatment in [Caching](#); here, the point is that the *bus* is where read-after-write consistency lives, not the handler.

The reactive bus

The bus also exposes a Reactor / WebFlux-style surface that wraps the eventual result in a lazy `Mono<R>` — the same handler lookup, the same middleware chain, run only when the `Mono` is subscribed, blocked, or awaited. The methods take `&Arc<Bus>` so the `Mono` can own the bus:

Method	Returns	Reactor analog
<code>Bus::send_mono(cmd)</code>	<code>Mono<R></code>	<code>Mono<R> bus.send(cmd)</code>
<code>Bus::query_mono(q)</code>	<code>Mono<R></code>	<code>Mono<R> bus.query(q)</code>
<code>Bus::send_mono_with_context</code>	<code>Mono<R></code>	context-carrying send
<code>Bus::query_mono_with_context</code>	<code>Mono<R></code>	context-carrying query

A reactive `GetWallet`, composing on the `Mono` from [The Reactive Model](#):

Rust

```
use std::sync::Arc;
use firefly::cQRS::Bus;

let balance = bus
    .query_mono::<_, WalletView>(GetWallet { id: wallet_id })
    .map(|view| view.balance)
    .block()
    .await?;           // Ok(Some(<cents>))
```

Because `firefly-reactive` fixes its error channel to `FireflyError`, a failed dispatch is mapped from `CQRSError` into a status-faithful `FireflyError` (validation → 422, missing handler → 500), with the original `CQRSError` preserved as `source()`. So a reactive command flows straight into the RFC 9457 problem stack while staying inspectable.

Proving the bus

Lumen's `src/commands.rs` exercises the full dispatch path with no HTTP — the test that ships in the crate:

Rust

```

#[tokio::test]
async fn handlers_dispatch_through_the_bus() {
    let ledger = Ledger::new(Arc::new(MemoryEventStore::new()),
Arc::new(InMemoryBroker::new()));
    bind(ledger, Arc::new(ReadModel::default()));
    let bus = Bus::new();
    bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());
    register(&bus);

    let opened: WalletView = bus.send(OpenWallet { owner: "alice".into(),
opening_balance: 100 }).await.unwrap();
    assert_eq!(opened.balance, 100);

    let after: WalletView = bus.send(Deposit { wallet_id: opened.id.clone(),
amount: 50 }).await.unwrap();
    assert_eq!(after.balance, 150);

    let fetched: WalletView = bus.query(GetWallet { id:
opened.id.clone() }).await.unwrap();
    assert_eq!(fetched.id, opened.id);
}

```

And the validation derive is testable on its own — no bus needed, because

`#[derive(Command)]` generates `validate()` directly on the type:

Rust

```

#[test]
fn deposit_validates_required_fields() {
    assert!(Deposit::default().validate().is_err());
    assert!(
        Deposit { wallet_id: "wlt_1".into(), amount: 0 }.validate().is_err(),
        "zero amount fails the #[firefly(validate)] check"
    );
    assert!(Deposit { wallet_id: "wlt_1".into(), amount:
10 }.validate().is_ok());
}

#[test]
fn get_wallet_carries_cache_ttl() {
    assert!(GetWallet::default().cache_ttl().is_some());
}

```

What changed in Lumen

Lumen's read and write paths are now separate, typed, and bus-dispatched:

- **src/commands.rs** — **OpenWallet** / **Deposit** / **Withdraw** carry `#[derive(Command)]` with `#[firefly(validate)]` on required fields; **GetWallet** carries `#[derive(Query)]` with `#[firefly(cache_ttl = "30s")]`. The derives generate the **Message** impl, the `validate()` checks, and the query's `cache_ttl`.
- Free-**fn** handlers marked `#[command_handler]` / `#[query_handler]` generate `register_open_wallet` / `register_deposit` / `register_withdraw` / `register_get_wallet`. Command handlers build the **Money** value object and delegate to the **Ledger**; the query serves the **ReadModel** and falls back to folding the stream for read-after-write freshness.
- The **OnceLock** bind pattern publishes the resolved **Ledger** + **ReadModel** once and returns the *effective* pair, so every collaborator across repeated builds shares one ledger and one read model.

- The **bus** is wired with `QueryCache::middleware()` and `ValidationMiddleware`, dispatched from the `#[rest_controller]` via `bus.send` / `bus.query`, with `cqrs_to_web` mapping a `CqrsError` (carrying the domain `Display` string) to the right RFC 9457 status — 422 for business rules, 404 for not-found.
- Read-after-write is enforced at the bus boundary:
`query_cache.invalidate_type::<GetWallet>()` runs after every mutation.

Exercises

1. **Watch validation short-circuit.** In a test, build a `Bus`, add `ValidationMiddleware::new()`, `register` the handlers, and `bus.send` a `Deposit { wallet_id: "wlt_1".into(), amount: 0 }`. Assert the result is a `CqrsError::Validation` and that the ledger was never touched (open a wallet first, then deposit zero, and confirm its balance is unchanged).
2. **Prove the cache, then bust it.** With the full `build_app()` bus, `query(GetWallet { id })` twice and confirm the second is served from cache (instrument the `ReadModel::find` or trace a counter). Deposit into the wallet, then `query` again — assert the new balance comes back, proving `invalidate_type::<GetWallet>()` did its job.
3. **Add a `CloseWallet` command.** Define `CloseWallet { #[firefly(validate)] wallet_id: String }` with `#[derive(Command)]`, write a `#[command_handler]` `close_wallet` that returns a `WalletView`, add `register_close_wallet(bus)` to `register`, and dispatch it. (You do not need a domain `close` yet — returning the current view is enough to exercise the wiring.)
4. **Reactive compose.** Rewrite the `get` controller handler to use `bus.query_mono::<_, WalletView>(GetWallet { id }).map(|v| v.balance)` and return just the balance as JSON. Note where the `FireflyError` channel takes over from `CqrsError`.

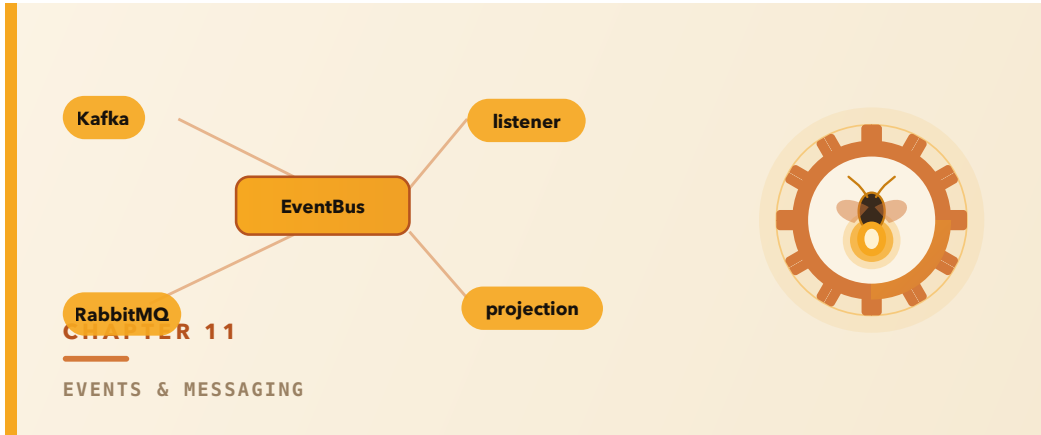
The bus dispatches *within* the service. To propagate what happened *between* collaborators — the read-model projection, external subscribers — fan out domain events. Continue to [Event-Driven Architecture & Messaging](#).



PART III

Event-Driven Architecture





CHAPTER 11

Event-Driven Architecture & Messaging

By the end of the [CQRS chapter](#), Lumen could open a wallet, deposit, withdraw, and read a balance — but the command side and the query side were quietly cheating. The `Wallet` aggregate raised crisp domain events (`WalletOpened` , `MoneyDeposited` , `MoneyWithdrawn`), the `Ledger` persisted them, and then nothing carried them anywhere. The read model the `GetWallet` query serves had to be repaired on the fly by re-folding the event stream.

By the end of *this* chapter, Lumen closes the loop. Every event the ledger persists is also **published** to a `Broker` , and a read-model **projection** — declared with one `#[event_listener]` attribute — consumes those events and keeps the query side current without the write side knowing it exists. That is event-driven architecture: a fact is published once, and any number of independent reactions subscribe to it. The audit trail, the welcome notification, the balance read model — each becomes a subscriber you can add months later without touching a single command handler.

`firefly-eda` is the framework's **event-driven architecture port**. It defines the `Event` envelope every Firefly event flows through, the `Publisher` / `Subscriber` / `Broker` ports, an in-process `InMemoryBroker`, and the messaging machinery — glob topics, consumer groups, retry/DLQ, event filters, and a reactive `Flux` subscription surface. The production transports (Kafka, RabbitMQ, Postgres outbox, Redis Streams) implement the same ports and slot in at wiring time, so Lumen's projection never changes when the broker does.

SPRING PARITY

The `Broker` port is the Spring Cloud Stream binder abstraction; publishing to it is `ApplicationEventPublisher.publishEvent` / `StreamBridge.send`.
`#[event_listener(topic = ...)]` is `@KafkaListener` / `@EventListener`.
`wrap_listener`'s retry/DLQ is `@RetryableTopic`; the glob subscription is `bus.subscribe("user.*", ...)`.

Two kinds of "event" in one wallet

Before wiring anything, it is worth being precise about the word *event*, because Lumen ends up using it for two different things and confusing them leads to the wrong port.

A **domain event** in the event-sourcing sense — `firefly::eventsourcing`'s `DomainEvent` — is the durable, versioned fact the `Wallet` aggregate raises and the [next chapter](#) makes the source of truth. It lives in the event *store*.

A **messaging event** — `firefly::eda`'s `Event` — is the wire envelope that carries a fact *to subscribers*. It lives on the *broker*. Lumen bridges the two with one function (`to_envelope`, below): the ledger persists a `DomainEvent`, then maps it onto an `Event` and publishes it. This chapter is about the second kind — getting the fact onto the wire and reacting to it. The first kind is the next chapter's subject.

The **Event** envelope

Event is wire-compatible across the Java/.NET/Go/Python/Rust ports — the same JSON field names and omission rules. Construct one with **Event::new**, which also stamps **correlation_id** from the kernel's task-local correlation scope:

Rust

```
use firefly_eda::Event;

let ev = Event::new(
    "orders.created",    // topic
    "OrderCreated",     // event type
    "orders-svc",       // source
    Some(br#"{"id":"o1"}"#.to_vec()), // payload (base64 on the wire)
)
.with_header("x-tenant", "acme")
.with_key(b"customer-42".to_vec()); // partition / routing key
```

The **key** is what Kafka uses for partitioning and RabbitMQ for routing; it is omitted from the wire when absent, so older events stay byte-for-byte identical.

Lumen's domain-event-to-envelope bridge

Lumen never builds an **Event** by hand in a handler. The ledger owns one mapping function that turns a persisted **DomainEvent** into the canonical envelope, carrying the JSON-encoded domain event as the payload and the wallet id as the partition key so a real broker keeps per-wallet events ordered:

Rust

```

use firefly::eda::Event;
use firefly::eventsourcing::DomainEvent;

/// The EDA topic every wallet domain event is published to.
pub const EVENTS_TOPIC: &str = "wallets.events";
/// The logical EDA source stamped on published events.
pub const EVENT_SOURCE: &str = "lumen";

/// Maps a persisted `DomainEvent` onto the canonical EDA `Event` envelope.
pub fn to_envelope(event: &DomainEvent) -> Event {
    let payload = serde_json::to_vec(event).expect("domain event serialises");
    Event::new(
        EVENTS_TOPIC,
        event.event_type.clone(),
        EVENT_SOURCE,
        Some(payload),
    )
    .with_key(event.aggregate_id.clone().into_bytes())
    .with_header("aggregateType", "Wallet")
    .with_header("aggregateId", event.aggregate_id.clone())
    .with_header("version", event.version.to_string())
}

```

Three design choices repay attention. The **topic** (`wallets.events`) is a shared constant — the publisher and the projection key off the same value, so the channel name can never drift. The **key** is the wallet id, so on a partitioned broker every event for one wallet lands on the same partition and stays in order. The **headers** (`aggregateId`, `version`) carry just enough routing metadata for a subscriber to find and re-fold the affected aggregate without decoding the payload — which is exactly what Lumen's projection does below.

 SPRING PARITY

`to_envelope` is the hand-written equivalent of a Spring Cloud Stream message converter: the domain object becomes the payload, and the `KafkaHeaders.MESSAGE_KEY` / custom headers carry the routing metadata.

Publishing from the ledger

The `Ledger` is the single write path every command and the transfer saga call. After it appends an aggregate's uncommitted events to the store with optimistic concurrency, it publishes each one — `to_envelope` then `broker.publish` — so the projection downstream can react:

Rust

```

use std::sync::Arc;
use firefly::eda::Broker;
use firefly::eventsourcing::{EventSourcingError, EventStore};

use crate::domain::{DomainError, Wallet};

/// Appends the aggregate's uncommitted events at `expected_version`
/// (optimistic concurrency) then publishes each to the EDA broker.
async fn commit(&self, wallet: &mut Wallet, expected: i64) -> Result<(),
DomainError> {
    let events = wallet.take_uncommitted();
    if events.is_empty() {
        return Ok(());
    }
    self.store
        .append(&wallet.root.id, expected, events.clone())
        .await
        .map_err(|e| match e {
            EventSourcingError::Concurrency => {
                DomainError::NotFound(format!("{}: concurrent modification",
wallet.root.id))
            }
            other => DomainError::NotFound(format!("{}: {other}",
wallet.root.id)),
        })?;
    for event in &events {
        self.broker
            .publish(to_envelope(event))
            .await
            .map_err(|e| DomainError::NotFound(format!("publish failed:
{e}")))?;
    }
    Ok(())
}

```

Notice the ordering: **append before publish**. A subscriber must never see a fact that did not persist — the same "save before you publish" discipline a Spring service follows with `@TransactionalEventListener(phase = AFTER_COMMIT)`. If the append fails (including the optimistic-concurrency race) the loop is never reached, so no event is broadcast. The store backing this `Ledger` is the in-memory `MemoryEventStore`; the [next chapter](#) is where that store earns the name *event-sourced*.

SPRING PARITY

Publishing after the unit of work commits mirrors Spring's `@TransactionalEventListener(phase = AFTER_COMMIT)`. The gap between append and publish — where a crash could persist a fact but drop the broadcast — is what the transactional outbox in the next chapter eliminates.

The in-process broker

`InMemoryBroker` is the default — fan-out delivery, glob topic matching, and per-`(topic, group)` round-robin, with no external dependency. It is the broker Lumen's `WebStack` exposes as `web.broker`, and it is everything the teaching build (and the test suite) needs. Subscribe a handler, publish an event:

Rust

```

use firefly_eda::{handler, Event, InMemoryBroker};

#[tokio::main]
async fn main() {
    let broker = InMemoryBroker::new();

    broker
        .subscribe(
            "wallets.events",
            handler(|ev: Event| async move {
                println!("observed {} for {}", ev.event_type,
                    ev.headers.get("aggregateId").map(String::as_str).unwrap_or("?"));
                Ok(())
            }),
        )
        .unwrap();

    let ev = Event::new("wallets.events", "WalletOpened", "lumen",
        Some(br#"{"wallet_id":"wlt_1"}"#.to_vec()));
    broker.publish(ev).await.unwrap();
    broker.close().unwrap();
}

```

`InMemoryBroker::publish` awaits each subscribed handler sequentially on the publisher's task; the first handler error short-circuits and is returned to the publisher. After `close()`, publish and subscribe fail with `EdaError::Closed`.

The read-model projection – `#[event_listener]`

Here is where Lumen closes the CQRS loop. The **projection** is a free `async fn` carrying one attribute. The `#[event_listener(topic = "wallets.events")]` macro generates a `subscribe_project_wallet_event(broker)` helper that subscribes the function to the topic; for each delivered event it reloads the affected wallet's stream, folds it into a `WalletView`, and upserts it into the read model:

Rust

```
use firefly::eda::Event;
use firefly::prelude::*;

use crate::domain::Wallet;

/// The read-model projection. `#[event_listener]` generates
/// `subscribe_project_wallet_event(broker)`, which subscribes this fn to
/// `EVENTS_TOPIC`. For each event it reloads the wallet's stream, folds it into
/// a `WalletView`, and upserts it – the idempotent rebuild-from-stream pattern.
#[event_listener(topic = "wallets.events")]
pub async fn project_wallet_event(ev: Event) -> FireflyResult<()> {
    let Some(state) = projection_state() else {
        return Ok(());
    };
    let Some(wallet_id) = ev.headers.get("aggregateId") else {
        return Ok(());
    };
    // A transient store miss is swallowed so one poison message never stalls
    // the projection – the EDA at-least-once contract.
    if let Ok(events) = state.store.load(wallet_id).await {
        let view = Wallet::rehydrate(wallet_id, &events).view();
        state.read_model.upsert(view);
    }
    Ok(())
}
```

Two properties make this a *good* projection rather than just a working one.

It is **idempotent**. Rather than mutating the read-model row from the single delivered event (`balance += amount`), it reloads the wallet's full stream and rebuilds the view from scratch. Under EDA's at-least-once delivery a redelivered `MoneyDeposited` would double-count if you applied the delta — but re-folding the same stream converges on the same `WalletView` no matter how many times the event arrives. The header carries the `aggregateId` ; that is all the projection needs to find the stream.

It is **decoupled**. `project_wallet_event` imports no command, calls no handler, and has no idea a deposit was processed. It reacts purely to the published fact. You can add a `FraudDetector` or a `WelcomeNotifier` subscriber next to it without touching a line of the command path.

SPRING PARITY

`#[event_listener(topic = "wallets.events")]` →
`subscribe_project_wallet_event(broker)` is the macro analog of Spring's
`@KafkaListener(topics = "wallets.events")` and pyfly's
`@event_listener(event_types=[...])` : the framework wires the subscription for you;
 you write only the reaction.

Wiring state into a free-fn listener

Spring would inject the store and read model into a `@Component` listener bean. A Rust free fn cannot capture wiring state, so Lumen uses the same publish-collaborators-once pattern its CQRS handlers use: the resolved collaborators are placed in a process-global `OnceLock` at startup, and the listener reads them back through a small accessor:

Rust

```

use std::sync::{Arc, OnceLock};
use firefly::eventsourcing::EventStore;

use crate::ledger::ReadModel;

/// The collaborators the free-fn projection needs.
struct ProjectionState {
    store: Arc<dyn EventStore>,
    read_model: Arc<ReadModel>,
}

static PROJECTION: OnceLock<ProjectionState> = OnceLock::new();

/// Publishes the projection's collaborators and returns the *effective* state
/// (the first call wins, so repeated builds in one test binary share one
state).
pub fn bind_projection(
    store: Arc<dyn EventStore>,
    read_model: Arc<ReadModel>,
) -> (Arc<dyn EventStore>, Arc<ReadModel>) {
    let effective = PROJECTION.get_or_init(|| ProjectionState { store,
read_model });
    (
        Arc::clone(&effective.store),
        Arc::clone(&effective.read_model),
    )
}

fn projection_state() -> Option<&'static ProjectionState> {
    PROJECTION.get()
}

```

The composition root ties it all together: it binds the projection's collaborators to the *same* store and read model the command handlers use, then awaits the generated subscription so the events the handlers publish are exactly the events the projection consumes:

Rust

```
// in build_app() - crate::web
ledger::bind_projection(Arc::clone(ledger.store()), Arc::clone(&read_model));
crate::commands::register(&bus);
// Subscribe the projection to the effective ledger's broker.
ledger::subscribe_project_wallet_event(ledger.broker().as_ref())
    .await
    .expect("projection subscription");
```

With that one `await`, every `POST /api/v1/wallets/:id/deposit` flows command → ledger → store → broker → projection → read model, and the next `GET /api/v1/wallets/:id` is served from the projected view. The HTTP test suite proves the loop converges: it deposits, withdraws, then reads back the balance and `version` the projection folded — no manual repair needed.

Glob topics and consumer groups

A subscription topic is a glob pattern (`*`, `?`, `[..]`, `{a,b}`); a published event is delivered to every subscription whose pattern matches its topic. Lumen subscribes to the exact `wallets.events`, but a multi-event service could fan a single listener across a family:

Rust

```
broker.subscribe("wallets.*", handler(|ev| async move { Ok(()) })).unwrap();
// matches wallets.events, wallets.audit, ...
```

Consumer groups give competing-consumer delivery: within a group each matching event goes to exactly **one** member (round-robin); distinct groups each get their own copy:

Rust

```
broker.subscribe_group("wallets.events", "projections", handler1).unwrap();  
broker.subscribe_group("wallets.events", "projections", handler2).unwrap();  
// each event reaches exactly one of handler1/handler2
```

This is how you would scale Lumen's projection horizontally: run several projector instances in one group and the broker shares the partitions among them, each instance owning a slice of the wallet space.

Retry and dead-letter

`wrap_listener(handler, publisher, policy)` is the adapter-agnostic retry/DLQ wrapper. A failing delivery is retried up to `retries` times with linear backoff (`retry_delay * attempt`); on exhaustion the event is republished to the dead-letter topic (when set), carrying the original payload/key/headers plus `x-original-topic` and `x-exception` diagnostic headers:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly_eda::{handler, wrap_listener, InMemoryBroker, ListenerPolicy};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async {
    let broker = Arc::new(InMemoryBroker::new());
    let inner = handler(|_ev| async {
        Err(firefly_kernel::FireflyError::internal("boom")) });
    let wrapped = wrap_listener(
        inner,
        broker.clone(),
        ListenerPolicy::with_retries(3)
            .retry_delay(Duration::from_millis(50))
            .dead_letter_topic("wallets.events.DLT"),
    );
    broker.subscribe("wallets.events", wrapped).unwrap();
# });

```

Lumen's projection takes a gentler path — it *swallows* a transient store miss and returns `Ok(())` rather than failing the delivery, so a single poison message never stalls the stream. That is the right call for a rebuild-from-stream projection (the next redelivery, or the next event for the wallet, converges anyway). A side-effecting listener — one that sends an email or calls an external API — is where `wrap_listener` and a dead-letter topic earn their keep.

For an inspectable record of failures (rather than a routing topic), wire an `EdaDeadLetterStore` via `ListenerPolicy::dead_letter_store`: an exhausted event is captured into the store (queryable with `list` / `get` / `remove`).

Event filters

`EventFilter` is a per-envelope delivery gate layered over topic matching. Where the broker decides *which* subscriptions a topic reaches, a filter decides whether a reached subscription actually *runs*. Two ship — a header regex filter and an arbitrary predicate filter. Lumen's envelopes carry an `aggregateType` header, so a header filter could restrict a subscriber to `Wallet` events:

Rust

```
use firefly_eda::{handler, with_filters, Event, HeaderEventFilter,
InMemoryBroker};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async {
    let broker = InMemoryBroker::new();
    let inner = handler(|_ev: Event| async { Ok(()) });
    let gated = with_filters(inner, [HeaderEventFilter::new("aggregateType",
r"^Wallet$").unwrap()]);
    broker.subscribe("wallets.events", gated).unwrap();
    # });
```

An event must pass *every* filter to be delivered; a non-matching event is dropped before the handler body runs.

The reactive subscription surface

`InMemoryBroker::subscribe_reactive(topic)` is the reactive twin of `subscribe` — a `Flux<Event>` that emits every event delivered to the topic, composing with the whole Reactor operator set. `publish_mono(event)` is the cold reactive publish (nothing happens until the `Mono` is subscribed):

Rust

```

use std::sync::Arc;
use firefly_eda::{Event, InMemoryBroker};

#
tokio::runtime::Builder::new_current_thread().enable_all().build().unwrap().block_on(async {
{
let broker = Arc::new(InMemoryBroker::new());
let flux = broker.subscribe_reactive("wallets.*").unwrap();

broker
    .publish_mono(Event::new("wallets.events", "WalletOpened", "lumen", None))
    .block()
    .await
    .unwrap();
broker.close().unwrap(); // terminates the Flux

let events = flux.take(1).collect_list().block().await.unwrap().unwrap();
assert_eq!(events[0].topic, "wallets.events");
# });

```

Deliveries are buffered through a bounded channel; when the downstream consumer falls behind, the newest events are dropped (`onBackpressureDrop`) rather than blocking or failing the publisher — extending "a slow consumer never fails publishers" to the reactive surface. This is the same `Flux` Lumen's optional streaming endpoint composes over (see [Production & Deployment](#)).

Production transports

Each transport crate implements the same `Broker` port; swap the constructor and keep every handler. Code against `firefly_eda::Broker` and select the adapter at wiring time. For Lumen this is a one-line change in `build_app`: replace the in-memory broker with a Kafka one and the projection, the ledger, and every command keep compiling unchanged.

Crate	Backend	Constructor
<code>firefly-eda-kafka</code>	Apache Kafka	<code>new_kafka_broker(KafkaConfig)?</code>
<code>firefly-eda-rabbitmq</code>	RabbitMQ	<code>RabbitMqBroker::new(RabbitMqBrokerConfig)</code>
<code>firefly-eda-postgres</code>	Postgres outbox	<code>PostgresBroker::new(PostgresConfig::new(dsn))</code>
<code>firefly-eda-redis</code>	Redis Streams	<code>RedisStreamsBroker::connect(RedisConfig::new(url))?</code>

Kafka, for example — note the handler body is identical to Lumen's:

Rust

```

use firefly_eda::{handler, Event};
use firefly_eda_kafka::{new_kafka_broker, KafkaConfig};

# async fn ex() -> firefly_eda::EdaResult<()> {
let broker = new_kafka_broker(KafkaConfig {
    brokers: vec!["kafka:9092".into()],
    client_id: "lumen".into(),
    consumer_group: "lumen-projections".into(),
    ..Default::default()
})?;

broker
    .subscribe("wallets.events", handler(|ev: Event| async move {
        println!("observed {}", ev.event_type);
        Ok(())
    }))
    .await?;

let ev = Event::new("wallets.events", "WalletOpened", "lumen", None);
broker.publish(ev).await?;
# Ok(())
# }

```

Redis Streams uses a connect-then-start lifecycle:

Rust

```

use firefly_eda::{handler, Event};
use firefly_eda_redis::{RedisConfig, RedisStreamsBroker};

# async fn ex() -> firefly_eda::EdaResult<()> {
let broker = RedisStreamsBroker::connect(
    RedisConfig::new("redis://localhost:6379/0")
        .with_streams(["wallets.events"])
        .with_group("lumen-projections"),
)?;
broker.subscribe("wallets.*", handler(|ev: Event| async move {
    println!("got {}", ev.event_type);
    Ok(())
})).await?;
broker.start().await?;
# Ok(())
# }

```

NOTE

The Postgres broker is a **transactional outbox**: events are written in the same transaction as your state change and drained to consumers via **LISTEN / NOTIFY**, giving at-least-once delivery without a separate broker. That closes the append-then-publish gap discussed above; the [next chapter](#) covers the outbox primitive directly.

Broker health

EventPublisherHealthIndicator adapts any broker implementing the **BrokerHealth** ping probe to a **firefly_observability::Indicator**, surfacing broker liveness on **/actuator/health** under the **eventPublisher** id — so when Lumen graduates to a real broker, its readiness shows up alongside the rest of the service's health (see [Observability](#)).

Recap – what changed in Lumen

The CQRS loop is closed. Where Chapter 9's command handlers persisted events and left the read side to repair itself, Lumen now publishes every persisted event and projects it back automatically.

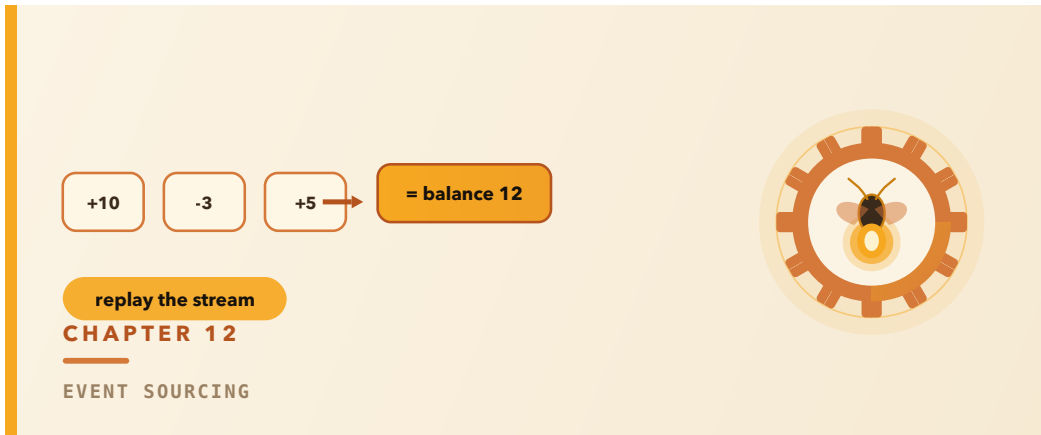
Piece	Role
<code>EVENTS_TOPIC / EVENT_SOURCE</code>	Shared constants the publisher and listener agree on
<code>to_envelope(&DomainEvent)</code>	Bridges a persisted domain event to the wire <code>Event</code> (key = wallet id, headers carry routing)
<code>Ledger::commit</code>	Appends, then publishes each event — save before you publish
<code>#[event_listener(topic = "wallets.events")]</code>	Generates <code>subscribe_project_wallet_event(broker)</code>
<code>project_wallet_event</code>	The idempotent rebuild-from-stream projection that feeds the read model
<code>bind_projection / projection_state</code>	Publish-once wiring so the free-fn listener reaches its collaborators
<code>web.broker (InMemoryBroker)</code>	The default transport — swap the constructor for Kafka/RabbitMQ/Redis, keep the listener

Three principles carry forward: **save before you publish** so a subscriber never sees an uncommitted fact; **make projections idempotent** so at-least-once redelivery is harmless (Lumen re-folds the stream rather than applying a delta); and **depend on the Broker port, not the adapter** so the in-memory broker becomes Kafka with a one-line change.

The events Lumen publishes here are still backed by a transient in-memory store. The [next chapter](#) makes those events the *source of truth* — durable, replayable, the canonical record from which every balance is recomputed.

Exercises

1. Add a `WelcomeNotifier` listener. Write a second `#[event_listener(topic = "wallets.events")]` free fn that reacts only to `WalletOpened` (check `ev.event_type`) and logs a welcome line carrying the `aggregateId` header. Subscribe it next to the projection in `build_app` and confirm — via an `InMemoryBroker` unit test that publishes a `WalletOpened` envelope — that it fires, while the existing command handlers stay untouched.
2. Prove idempotency. In a test, build a `Ledger` over a `MemoryEventStore` and an `InMemoryBroker`, subscribe the projection, open a wallet, and deposit twice. Then publish the *same* `MoneyDeposited` envelope a second time with `broker.publish(to_envelope(&event))` and assert the read-model `WalletView` balance is unchanged — the rebuild-from-stream fold absorbs the redelivery.
3. Gate by aggregate type. Wrap the projection's handler with `with_filters` and a `HeaderEventFilter::new("aggregateType", r"^Wallet$")`, then publish an envelope whose `aggregateType` header is `"Account"` and confirm the projection does not run for it. Explain why a header filter is a cheaper guard than checking inside the handler body.
4. Swap in a real broker (sketch). Add the `eda-kafka` feature to the crate and write the `build_app` variant that constructs `new_kafka_broker(...)` instead of using `web.broker`. You do not need a running Kafka — the point is to confirm the projection, the ledger, and the command handlers compile unchanged against the `Broker` port.



CHAPTER 12

Event Sourcing the Ledger

The [last chapter](#) left one question politely unasked. Lumen's `Ledger` persists wallet events and the projection rebuilds the read model by re-folding the stream — but *what stream*? So far the wallet's canonical state has been implied. By the end of this chapter it is explicit and load-bearing: the `Wallet` aggregate holds **no stored balance at all**. Its balance is a pure function of an append-only stream of `WalletOpened`, `MoneyDeposited`, and `MoneyWithdrawn` events, recomputed every time the aggregate is loaded.

That is **event sourcing**: instead of storing current state and discarding each change, you store the *sequence of changes* and derive state by replaying them. A financial ledger is the ideal domain for it — accountants have known for centuries that a ledger's authority comes from its entries, not from the running total at the foot of the column. The total is a *derived fact*; the entries are the *source of truth*. By the end of this chapter, an auditor asking "what was wallet `wlt_...`'s balance after the third movement?" gets an answer Lumen can *prove* from the stream, not merely report from a column.

`firefly-eventsourcing` provides the framework's **event-sourced aggregate** primitives: an `AggregateRoot` that tracks uncommitted events, an `EventStore` with optimistic concurrency, snapshots, projections, a global cross-aggregate stream, a transactional outbox, and multi-tenancy. Where the [EDA chapter](#)'s `Event` envelope was the *transport* for a fact, the `DomainEvent` here is the *record* of it — the durable truth from which state is rebuilt.

🌿 SPRING PARITY

This is the `firefly-event-sourcing` starter / Axon-style model: `AggregateRoot::raise`, an `EventStore` with optimistic concurrency, and projections that build read models.

`raise ~ AggregateLifecycle.apply`; the `apply` fold `~ @EventSourcingHandler`. The `DomainEvent` JSON is wire-compatible across every port.

State storage vs event storage

The clearest way to feel the shift is to compare what Lumen's storage *holds* in each model.

In the **state-storage model**, the store keeps only the wallet's current state:

id	owner	balance	version
wlt_a1	alice	120	3

Every deposit and withdrawal overwrites `balance`. The history is gone — you know the wallet holds 120 cents now; you cannot know how it got there.

In the **event-storage model**, the store keeps the stream:

aggregate_id	version	event_type	payload
wlt_a1	1	WalletOpened	<code>{"wallet_id":"wlt_a1","owner":"alice","opening_bala</code>
wlt_a1	2	MoneyDeposited	<code>{"wallet_id":"wlt_a1","amount":50}</code>
wlt_a1	3	MoneyWithdrawn	<code>{"wallet_id":"wlt_a1","amount":30}</code>

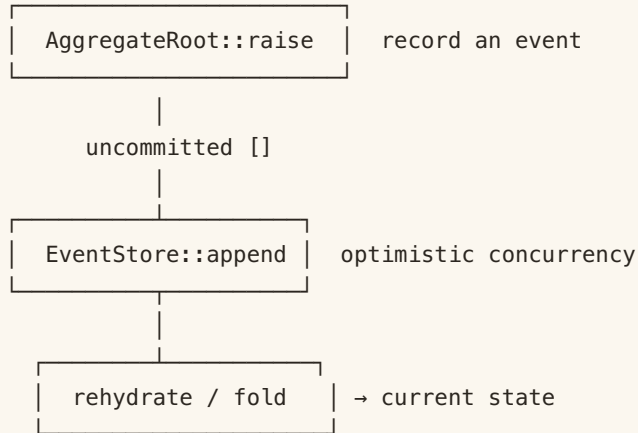
The current balance is still 120 cents — but now you can read every decision that led to it, replay to any version, and audit the lot. The trade-off is real: reads cost a replay (mitigated by **snapshots**) and events are immutable (schema evolution handled by **upcasters**). Both have first-class support below.

 NOTE

Event sourcing is not the same as the [previous chapter](#)'s EDA. There, the aggregate stored its state and *published* events as a side effect. Here the events *are* the state: there is no **balance** column to keep in sync — the balance is computed by folding the stream every time the aggregate loads.

The mental model

Text



The Wallet's domain events

In Lumen the three events are plain payload structs carrying `#[derive(DomainEvent)]`. The derive stamps each with a stable `EVENT_TYPE` discriminator (its struct name) and a `to_domain_event(...)` conversion onto the framework wire event — so the event type is never spelled as a bare string literal at the call sites:

Rust

```

use firefly::eventsourcing::DomainEvent;
use serde::{Deserialize, Serialize};

/// Payload of the event raised when a wallet is opened.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct WalletOpened {
    pub wallet_id: String,
    pub owner: String,
    /// The opening balance, in minor units (cents).
    pub opening_balance: i64,
}

/// Payload of the event raised when money is credited to a wallet.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct MoneyDeposited {
    pub wallet_id: String,
    pub amount: i64,
}

/// Payload of the event raised when money is debited from a wallet.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct MoneyWithdrawn {
    pub wallet_id: String,
    pub amount: i64,
}

```

`#[derive(DomainEvent)]` generates, for each struct, a `pub const EVENT_TYPE:` `&'static str` equal to the struct name (`"WalletOpened"`, ...), an `event_type()` accessor, and a `to_domain_event(aggregate_id, aggregate_type, version)` that JSON-encodes the payload into a framework `DomainEvent`. That generated `EVENT_TYPE` is the only thing the aggregate and its `apply` fold reference, so a rename of the struct flows through automatically.

 SPRING PARITY

`#[derive(DomainEvent)]` is the macro analog of an Axon event class plus its registration: the struct name becomes the routing type, and serialization to the store's `DomainEvent` is generated rather than hand-written. The wire JSON matches pyfly's `DomainEvent` field-for-field.

The Wallet aggregate – raise, then apply

The `Wallet` carries `#[derive(AggregateRoot)]`, which finds the embedded framework `AggregateRoot` field (`root`) and generates `Wallet::AGGREGATE_TYPE` plus `aggregate()` / `aggregate_mut()` accessors. The projected state (`owner`, `balance`, `opened`) is *not* stored — it is folded from the stream:

Rust

```
use firefly::eventsourcing::{AggregateRoot, DomainEvent};

use crate::money::Money;

#[derive(Debug, Clone, AggregateRoot)]
#[firefly(aggregate_type = "Wallet")]
pub struct Wallet {
    /// The framework aggregate root – uncommitted-event buffer + version.
    pub root: AggregateRoot,
    pub owner: String,
    /// Folded from the stream; never stored.
    pub balance: Money,
    /// Whether the wallet has been opened (an empty stream is "absent").
    pub opened: bool,
}
```

Every command follows the canonical event-sourcing shape: validate the invariant, **raise** the matching event onto the embedded root, then apply it to in-memory state. The write path and the replay path run the *same* **apply** code — that symmetry is the correctness guarantee of event sourcing.

Rust

```
/// Credits `amount` to the wallet, raising a `MoneyDeposited` event.
pub fn deposit(&mut self, amount: Money) -> Result<(), DomainError> {
    self.require_opened()?;
    let amount = amount.require_positive()?;
    self.raise(
        MoneyDeposited::EVENT_TYPE,
        &MoneyDeposited {
            wallet_id: self.root.id.clone(),
            amount: amount.cents_value(),
        },
    );
    self.balance = self.balance.add(amount);
    Ok(())
}

/// Serialises a `#[derive(DomainEvent)]` payload and raises it onto the embedded
/// root under `event_type` – the discriminator from the generated `EVENT_TYPE`.
fn raise<P: Serialize>(&mut self, event_type: &str, payload: &P) {
    let bytes = serde_json::to_vec(payload).expect("domain event payload serialises");
    self.root.raise(event_type, bytes);
}
```

AggregateRoot::raise buffers the event (so the ledger can persist it) and bumps the version. **withdraw** is the same shape, with one extra guard: it computes the remaining balance *first* and lets **Money::subtract** reject an overdraw — so a failed withdrawal raises **no** event at all, leaving the stream clean. That overdraft guard is the trigger the transfer saga relies on in [Sagas, Workflows & TCC](#).

Rehydration – folding the stream

Rehydration is the load path: rebuild a wallet by folding its full ordered stream through the same `apply` the commands use. An empty stream yields an unopened wallet — which is how the ledger distinguishes "absent" from "exists":

Rust

```

/// Rebuilds a wallet by folding `events` (its full ordered stream).
pub fn rehydrate(id: &str, events: &[DomainEvent]) -> Self {
    let mut wallet = Wallet {
        root: AggregateRoot::new(id, AGGREGATE_TYPE),
        owner: String::new(),
        balance: Money::ZERO,
        opened: false,
    };
    for event in events {
        wallet.apply(event);
        // Keep the root version in lock-step with the stream head so a
        // subsequent command appends at the right expected version.
        wallet.root.version = event.version;
    }
    wallet
}

/// Folds one persisted event into the projected state.
fn apply(&mut self, event: &DomainEvent) {
    match event.event_type.as_str() {
        WalletOpened::EVENT_TYPE => {
            if let Ok(p) =
serde_json::from_slice::<WalletOpened>(&event.payload) {
                self.owner = p.owner;
                self.balance = Money::cents(p.opening_balance);
                self.opened = true;
            }
        }
        MoneyDeposited::EVENT_TYPE => {
            if let Ok(p) =
serde_json::from_slice::<MoneyDeposited>(&event.payload) {
                self.balance = self.balance.add(Money::cents(p.amount));
            }
        }
        MoneyWithdrawn::EVENT_TYPE => {
            if let Ok(p) =
serde_json::from_slice::<MoneyWithdrawn>(&event.payload) {
                self.balance = Money::cents(self.balance.cents_value() -

```

```

p.amount);
    }
}
_ => {}
}
}

```

The folding logic in `apply` is matched on the *same* `EVENT_TYPE` constant the commands raise under, so the two halves can never disagree about an event's name. Lumen's unit tests prove the replay law directly: open + deposit + withdraw on a *writer* wallet, take its uncommitted stream, then `Wallet::rehydrate` a fresh wallet from that stream and assert the rebuilt balance, owner, and version match — state recomputed from events, never stored.

SPRING PARITY

`raise` + `apply` is Axon's `AggregateLifecycle.apply(...)` + `@EventSourcingHandler`. The discipline is identical: the command applies the event, the handler mutates the fields, and load replays the same handlers to rebuild state. Lumen registers no handler table — it `match`es on the generated `EVENT_TYPE` const, which is the Rust-idiomatic spelling of the same idea.

Raising and appending events

The framework `AggregateRoot` accumulates `DomainEvent`s as you `raise` them; you `take_uncommitted` and `append` them to the store. `append` enforces optimistic concurrency: pass the version you loaded, and a concurrent writer's append fails with `EventSourcingError::Concurrency`:

Rust

```

use firefly_event sourcing::{AggregateRoot, EventStore, MemoryEventStore};

#[tokio::main]
async fn main() {
    let store = MemoryEventStore::new();

    let mut user = AggregateRoot::new("u1", "User");
    user.raise("UserCreated", br#"{"name":"alice"}"#);
    user.raise("UserRenamed", br#"{"name":"bob"}"#);

    let events = user.take_uncommitted();
    // expected_version 0 -> this is a brand-new aggregate.
    if let Err(err) = store.append(&user.id, 0, events).await {
        eprintln!("append failed (raced): {err}");
    }

    assert_eq!(store.load("u1").await.unwrap().len(), 2);
}

```

The `EventStore` port:

Rust

```

#[async_trait]
pub trait EventStore: Send + Sync {
    async fn append(&self, aggregate_id: &str, expected_version: i64,
        events: Vec<DomainEvent>) -> Result<(), EventSourcingError>;
    async fn load(&self, aggregate_id: &str) -> Result<Vec<DomainEvent>,
        EventSourcingError>;
    async fn load_after(&self, aggregate_id: &str, since_version: i64)
        -> Result<Vec<DomainEvent>, EventSourcingError>;
    async fn stream_all(&self, after_event_id: Option<&str>, limit: usize,
        tenant: Option<&str>)
        -> Result<Vec<StreamedEvent>, EventSourcingError>;
}

```

The default is `MemoryEventStore` — the in-process store Lumen runs on, ideal for development and tests. `SqlEventStore` backs it with a SQL store over the `firefly-transactional Database` port for production; swapping it is a one-line change in `build_app`, exactly like swapping the broker in the last chapter.

The Ledger ties it together

Lumen's `Ledger` is the application service that owns the store (and the broker from the [last chapter](#)). Every command rehydrates, runs the domain method, and commits with optimistic concurrency. Here is `deposit` and the load path — the version the wallet rehydrated to is the `expected_version` the append must match:

Rust

```

/// Credits `amount` to `wallet_id`, persisting + publishing `MoneyDeposited`.
pub async fn deposit(&self, wallet_id: &str, amount: Money) ->
Result<WalletView, DomainError> {
    let mut wallet = self.load(wallet_id).await?;
    let expected = wallet.root.version;
    wallet.deposit(amount)?;
    self.commit(&mut wallet, expected).await?;
    Ok(wallet.view())
}

/// Rehydrates the aggregate from its persisted stream.
async fn load(&self, wallet_id: &str) -> Result<Wallet, DomainError> {
    let events = self.load_events(wallet_id).await?;
    Ok(Wallet::rehydrate(wallet_id, &events))
}

/// Loads the full event stream, mapping an absent aggregate to a domain 404.
pub async fn load_events(&self, wallet_id: &str) -> Result<Vec<DomainEvent>,
DomainError> {
    match self.store.load(wallet_id).await {
        Ok(events) => Ok(events),
        Err(EventSourcingError::AggregateNotFound) => {
            Err(DomainError::NotFound(wallet_id.to_string()))
        }
        Err(e) => Err(DomainError::NotFound(format!("{wallet_id}: {e}"))),
    }
}

```

The `commit` method (shown in full in the [last chapter](#)) appends at `expected` then publishes each event. The two chapters meet here: this one supplies the durable, replayable store; that one carries each appended event onto the wire so the projection can react.

Optimistic concurrency in practice

Two concurrent requests — say a deposit from the app and a fee withdrawal from a job — can both load wallet `wlt_a1` at version 3, each apply a change, and each try to append at `expected_version = 3`. The first append wins and the stream advances to 4; the second now mismatches and the store returns

`EventSourcingError::Concurrency`. Lumen maps that to a `DomainError::NotFound` detail ("concurrent modification") so the caller retries from a fresh load. You never manage version numbers by hand — the version the wallet rehydrated to is the token, and the store enforces it.

SPRING PARITY

`append(id, expected_version, events)` is Axon's optimistic-locking append; the `Concurrency` error is its `ConcurrencyException`. The rule is the same in both: catch it and retry the load-mutate-save cycle (or surface a 409), never swallow it.

Typed aggregates and the repository

Lumen folds the stream by hand in `Wallet::apply` because it teaches the mechanic clearly. For larger aggregates the framework offers a thinner path: implement `EventSourcedAggregate` — a typed `apply_event` plus optional snapshot serialization — and let `EventSourcedRepository` tie `load` (snapshot + replay) and `save` (append + snapshot policy) together:

Rust

```

use firefly_event sourcing::{
    AggregateRoot, DomainEvent, EventSourcedAggregate, EventSourcedRepository,
    EventSourcingError, MemoryEventStore,
};
use std::sync::Arc;

#[derive(Default)]
struct Wallet { root: AggregateRoot, balance: i64 }

impl EventSourcedAggregate for Wallet {
    const AGGREGATE_TYPE: &'static str = "Wallet";
    fn root(&self) -> &AggregateRoot { &self.root }
    fn root_mut(&mut self) -> &mut AggregateRoot { &mut self.root }
    fn apply_event(&mut self, event: &DomainEvent) -> Result<(),
EventSourcingError> {
        if event.event_type == "Credited" {
            let amount: i64 = serde_json::from_slice(&event.payload)
                .map_err(|e| EventSourcingError::Projection(e.to_string()))?;
            self.balance += amount;
        }
        Ok(())
    }
}

# async fn ex() -> Result<(), EventSourcingError> {
    let repo =
EventSourcedRepository::<Wallet>::new(Arc::new(MemoryEventStore::new()));

    let mut w = Wallet::default();
    w.root_mut().raise("Credited", b"500");
    repo.save(&mut w).await?; // append uncommitted

    let reloaded = repo.load(&w.root.id).await?; // snapshot + replay
    assert!(reloaded.is_some());
    # Ok(())
    # }

```

`EventSourcedRepository::with_snapshots(store, snapshots, interval)` enables periodic state captures so rehydration does not replay the entire history.

Snapshots – when streams get long

Event sourcing trades write simplicity for read cost: a wallet with 10,000 movements replays 10,000 events every load. **Snapshots** cut that down. A snapshot is a serialized checkpoint of the aggregate's state at a version; on load, the repository deserializes the latest snapshot and replays only the events after it. A snapshot at version 9,000 turns a 10,000-event replay into 1,000.

Lumen's wallets are short-lived enough that the in-memory store's full replay is fine, so the sample does not wire snapshots — but the seam is there: `with_snapshots(store, MemorySnapshotStore::new(), 100)` would checkpoint every time a wallet's stream crosses a 100-event boundary. Snapshots are an optimization, never a correctness requirement: remove them and the system is slower but still correct.

SPRING PARITY

`MemorySnapshotStore` + `snapshot_interval` is Axon's snapshotting with a `SnapshotTriggerDefinition`. The interval-crossing trigger handles a batch that straddles the threshold (version 95 → 105 still snapshots).

Projections – building read models

A **Projection** is a read-side handler. Register projections on a `ProjectionRunner` and replay an aggregate's events through them. This is the event-store sibling of the [last chapter](#)'s event-bus listener: Lumen's `project_wallet_event` reacts to events as they are published, whereas a `ProjectionRunner` can replay history from the beginning to rebuild a read model from scratch:

Rust

```

use std::sync::Arc;
use firefly_event sourcing::{FunctionProjection, ProjectionRunner};

let runner = ProjectionRunner::new();
runner.register(Arc::new(FunctionProjection::new("balances", |event| async move
{
    // update a read-model row from the event ...
    Ok(())
})));

runner.replay(&store, "wlt_a1").await?; // replay one aggregate's stream

```

This rebuildability is unique to event sourcing. If Lumen's read model is ever lost or its schema changes, you stop the projector, clear the read model, and replay every stream — the history is right there in the store. A state-storage model cannot do this; it discarded the history at write time.

The global stream

`EventStore::stream_all` exposes the global, cross-aggregate, ordered event stream with a resumable cursor — the engine for read models that span many aggregates (think: "all movements across all wallets, in order"). The runner consumes it in batches, at-least-once and in-order:

Rust

```

// Drive one batch; returns the next cursor + any per-event error.
let (next_cursor, err) = runner
    .drive_once(&store, None, 100, None)
    .await?;

// Or replay the whole global stream from a start cursor.
let cursor = runner.replay_all(&store, None, 100, None).await?;

```

The transactional outbox

The [last chapter](#) noted a gap in `Ledger::commit`: it appends, then publishes, and a crash *between* the two persists the fact but drops the broadcast. `TransactionalOutbox` closes that gap. Instead of publishing directly, a writer `enqueue`s the `DomainEvent`; a background relay polls and forwards each pending record to an `OutboxSink`, retrying up to `max_attempts`. The default `EdaSink` bridges each `DomainEvent` to a `firefly_eda::Event` and publishes it — the same `to_envelope`-shaped bridge, but durable:

Rust

```
use std::sync::Arc;
use firefly_eventsourcing::{EdaSink, TransactionalOutbox};

let outbox = TransactionalOutbox::new(Arc::new(EdaSink::new(broker)))
    .with_max_attempts(5);

outbox.enqueue(some_event).await;           // a writer enqueues
outbox.start().await;                       // background relay forwards + retries
// ... later
let dead = outbox.dead_letters().await;     // exhausted records, for inspection
outbox.stop().await;
```

Exhausted records become dead letters — excluded from the publish loop and surfaced for inspection or manual retry. This is the upgrade path for a production Lumen: enqueue into the outbox inside the same store transaction as the append, and let the relay guarantee at-least-once delivery to the broker even across crashes — which is exactly why the projection was built to be **idempotent** in the last chapter.

 SPRING PARITY

`TransactionalOutbox` is the portable equivalent of Spring Modulith's `EventPublicationRegistry` and `@TransactionalEventListener(phase = AFTER_COMMIT)`: record the event durably *before* dispatching it. Axon Server's stored log fulfils the same role for Axon apps.

Schema evolution – upcasters

`EventUpcaster` migrates events on the **read** paths only, so consumers always observe current-schema events while the stored history stays untouched. Suppose Lumen later needs a **reference** field on every deposit for reconciliation: new events carry it, old `MoneyDeposited` events do not, and an upcaster fills the gap on load:

Rust

```
use std::sync::Arc;
use firefly_eventsourcing::{EventUpcaster, MemoryEventStore};

let store = MemoryEventStore::with_upcasters(vec![Arc::new(MyUpcaster)]);
// every event returned by load / load_after passes through applicable upcasters
```

Old data becomes readable without a migration; new data is written in the current schema. The events themselves stay immutable — you never rewrite history.

Multi-tenancy

An optional `DomainEvent::tenant_id` (stamped from `AggregateRoot::with_tenant`, persisted and filterable, omitted from JSON when `None`) is threaded through `append` / `load` / `stream_all`, so one store serves many tenants with per-tenant isolation on the global stream — the route a multi-bank Lumen deployment would take to keep each tenant's wallet streams separate.

Recap – what changed in Lumen

The wallet's balance is no longer a stored value — it is a *computation* over an immutable stream, and the stream is the system of record.

Piece	Role
<code>#[derive(DomainEvent)]</code>	Generates <code>EVENT_TYPE</code> + <code>to_domain_event(...)</code> for each payload struct
<code>#[derive(AggregateRoot)]</code>	Generates <code>AGGREGATE_TYPE</code> + <code>aggregate()</code> / <code>aggregate_mut()</code> over the embedded <code>root</code>
<code>Wallet::raise</code> / <code>apply</code>	Command applies the event; the same fold runs on write and on replay
<code>Wallet::rehydrate</code>	Rebuilds a wallet by folding its full stream — empty stream = unopened
<code>EventStore</code> / <code>MemoryEventStore</code>	The append-only log; <code>SqlEventStore</code> for production
<code>append(id, expected_version, ...)</code>	Optimistic concurrency — the rehydrated version is the token
<code>ProjectionRunner</code>	Rebuilds read models from history (the store-side sibling of the EDA listener)
<code>TransactionalOutbox</code>	Closes the append-then-publish gap with at-least-once relay

Three ideas carry forward. **The events are the truth** — there is no balance column to drift. **Write and replay share one fold** — `apply` runs the same way whether a command just raised the event or a load is rebuilding from history, which is the correctness guarantee. **Depend on the `EventStore` port** — the in-memory store becomes SQL with a one-line swap, just as the broker became Kafka.

When a business process spans multiple aggregates and needs compensation — moving money from one wallet to another, atomically — folding a single stream is no longer enough. Continue to [Sagas, Workflows & TCC](#), where the transfer saga drives two wallets and rolls the debit back when the credit fails.

Exercises

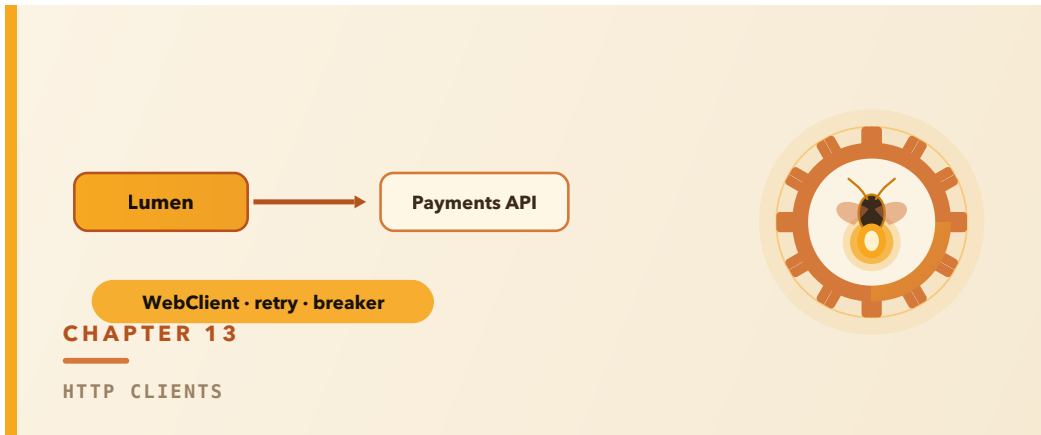
1. **Replay to a point in time.** Open a wallet and make three deposits. Load the raw stream with `ledger.load_events(&id)`, take only the events with `version <= 2`, and `Wallet::rehydrate` a fresh wallet from that slice. Assert the balance equals opening + first deposit only — the "time-travel query" a state-storage model cannot answer.
2. **Prove the overdraft guard raises no event.** Open a wallet with 100 cents, attempt to `withdraw` 101, and assert it errors with `DomainError::InsufficientFunds`. Then call `root.uncommitted()` and assert the buffer still holds exactly one event (the `WalletOpened`) — the failed command left the stream clean.
3. **Force an optimistic-concurrency conflict.** Append the open event for a wallet at `expected_version = 0`. Then, without reloading, raise a second event and append it *also* at `expected_version = 0`. Assert the second append returns `EventSourcingError::Concurrency`, and explain why a fresh load (which advances `expected` to 1) would have succeeded.
4. **Add a `ProjectionRunner` rebuild.** Register a `FunctionProjection` that tallies the count of `MoneyDeposited` events per wallet into an in-memory map, `replay` one wallet's stream through it, and assert the count. Then clear the map and replay again — confirming the read model is rebuildable from the store alone, with no live event traffic.



PART IV

Into Microservices





CHAPTER 13

HTTP Clients & Calling Other Services

By the end of this chapter, you will know how Lumen reaches *out* — how a wallet service settles a transfer through an external **Payments** processor, and how an **experience tier** sits in front of Lumen and composes it with its neighbors into one journey-shaped API. Lumen itself stays deliberately self-contained (every leg of its transfer drives the in-process **Ledger**), so this chapter is the "next adapter you would add": a typed outbound client wired into the transfer flow, plus the BFF that fans out across services.

When a transfer needs to settle against a real payment rail, the credit leg stops being a local **Ledger::deposit** and becomes a network call. That call can time out, fail halfway, or land on an overloaded service — failure modes a local method call never had. **firefly-client** gives you a typed client for that call instead of a hand-rolled **request** session threaded with retry and timeout logic.

The crate ships two HTTP clients that share the same automatics — default `Accept / Content-Type`, correlation-id and W3C trace-context propagation, and RFC 7807 problem decode into a typed `FireflyError`:

- the eager `RestClient` (built with `RestBuilder`) — an `async fn` that awaits a `Result`, with a built-in retry budget;
- the reactive `WebClient` — the Rust analog of WebFlux's `WebClient`, whose terminal operators hand back `Mono` / `Flux`.

The crate also ships scaffolds for SOAP, gRPC, GraphQL, and WebSocket clients. Everything is reachable through the one `firefly` facade as `firefly::client`.

SPRING PARITY

`RestClient` is `RestTemplate / RestClient`; `WebClient` is the WebFlux `WebClient` — fluent builder, `body_to_mono` / `body_to_flux` terminals, `exchange` for the raw response. Where pyfly and the JVM generate a client *implementation* from an annotated interface (`@service_client`, `@FeignClient`), Rust has no reflection — you build the client value with a fluent builder, and the resilience decorators (covered below) wrap the call.

The eager `RestClient`

Build a client with `RestBuilder`, then call `request` with a method, path, and optional body. The retry budget, timeout, and default headers are configured on the builder. Here Lumen builds the Payments client it would call from the credit leg of a transfer:

Rust

```

use std::time::Duration;
use firefly::client::RestBuilder;
use http::Method;
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct SettleTransfer { wallet_id: String, amount: i64, reference: String }
#[derive(Deserialize)]
struct Payment { id: String, status: String }

#[tokio::main]
async fn main() {
    let payments = RestBuilder::new("https://payments.internal")
        .with_header("X-Tenant", "lumen")
        .with_timeout(Duration::from_secs(5))
        .with_retries(3)
        .build();

    let req = SettleTransfer {
        wallet_id: "wlt_alice".into(),
        amount: 300,
        reference: "transfer-42".into(),
    };
    match payments.request:::<_, Payment>(Method::POST, "/payments",
Some(&req)).await {
        Ok(payment) => println!("settled {} ({})", payment.id, payment.status),
        Err(err) => {
            // Upstream RFC 7807 problems are decoded into a typed FireflyError.
            if let Some(fe) = err.as_firefly() {
                eprintln!("payments upstream {}: {}", fe.status, fe.detail);
            }
        }
    }
}

```

A non-2xx `application/problem+json` response is decoded into a `FireflyError`, so an upstream failure carries the upstream's status and detail straight through Lumen's own error stack. `err.as_firefly()` is the typed accessor — the analog of the JVM's `errors.As(&FireflyError)` — and `ClientError` also offers predicate helpers like `is_not_found()`, `is_unprocessable_entity()`, and `is_retryable()` so a caller can branch on the failure class without matching raw status codes.

Where this plugs into the saga. In `Sagas` the credit leg was `ledger.deposit(&to, amount)`. In a split deployment that becomes `payments.request::<_, Payment>(Method::POST, "/settle", ...)`. The saga shape does not change — it is still a `Step` with the debit's compensation — only the body of the credit step now does I/O across the network, which is exactly why the compensation (refund the debit) matters more than ever.

Idempotency on the wire

A transfer's settlement call must be idempotent: if `POST /payments` times out and the saga's retry fires it again, Payments must not create two payments. Carry a stable `Idempotency-Key` — typically the transfer id — so the upstream deduplicates a re-delivered request. Set it as a default header on a per-call builder, or thread it through the request:

Rust

```
let payments = RestBuilder::new("https://payments.internal")
    .with_header("Idempotency-Key", &transfer_id) // stable per business op
    .with_timeout(Duration::from_secs(2))
    .build();
```

The deduplication itself is the upstream's job; the client's job is to forward the key *consistently* across retries.

The reactive `WebClient`

The reactive client returns `Mono` / `Flux`, so an outbound call drops straight into a reactive pipeline and composes end-to-end with the `NdJson` / `Sse responders` Lumen's streaming endpoint uses. The fluent chain is the WebFlux spelling:

Rust

```

use firefly::client::WebClientBuilder;
use http::Method;
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct SettleTransfer { wallet_id: String, amount: i64 }
#[derive(Deserialize)]
struct Payment { id: String }
#[derive(Deserialize)]
struct LedgerTick { seq: u64 }

#[tokio::main]
async fn main() {
    let client = WebClientBuilder::new("https://payments.internal")
        .with_header("X-Tenant", "lumen")
        .build();

    // body_to_mono - a single value -> Mono<Payment>.
    let _payment = client
        .method(Method::POST)
        .uri("/payments")
        .body(&SettleTransfer { wallet_id: "wlt_alice".into(), amount: 300 })
        .retrieve()
        .body_to_mono::<Payment>();

    // body_to_flux - a streamed NDJSON OR SSE body, decoded lazily
    // element-by-element with backpressure.
    let _ticks = client
        .get()
        .uri("/ledger/ticks")
        .header("Accept", "application/x-ndjson")
        .retrieve()
        .body_to_flux::<LedgerTick>();

    // exchange - raw status + headers without raising on a non-2xx.
    let _resp = client.get().uri("/health").retrieve().exchange();
}

```

The terminal operators:

Operator	Returns	Behavior
<code>body_to_mono::<T>()</code>	<code>Mono<T></code>	the whole body decoded as one <code>T</code>
<code>body_to_flux::<T>()</code>	<code>Flux<T></code>	a streamed NDJSON/SSE body, element-by-element
<code>exchange()</code>	<code>Mono<WebClientResponse></code>	the raw status + headers + body, no raise

Streaming semantics

`body_to_flux` consumes the byte stream chunk-by-chunk and decodes one element per frame, lazily and with backpressure — a slow downstream throttles the producer, and `.take(n)` stops pulling early. The decoder is chosen from the response `ContentType`:

- `application/x-ndjson` (and any non-SSE type) → one JSON document per newline-terminated line;
- `text/event-stream` → SSE frames separated by a blank line; the `data:` lines are concatenated and comment / `event:` / `id:` lines are ignored.

A malformed element terminates the stream with a decode `FireflyError` (Reactor's first-error-is-terminal semantics). This is the *consumer* side of the same wire format Lumen's own `GET /api/v1/wallets/:id/events` endpoint produces: one service streams the wallet's event log, another reads it back element by element.

Inspecting the raw response

`exchange()` hands back a `WebClientResponse` **without** raising on a non-2xx, so you can inspect it and decide:

Rust

```

let resp = client.get().uri("/
health").retrieve().exchange().block().await?.unwrap();
if resp.is_success() {
    let body: serde_json::Value = resp.body_json()?;
} else if let Some(problem) = resp.problem() {
    // a decoded RFC 7807 FireflyError, if the body was a problem document
}

```

No baked-in retry **REACTOR PARITY**

Unlike `RestBuilder::with_retries`, the `WebClient` has **no** retry budget. Compose retries on the returned publisher with `Mono::retry` / `Mono::retry_backoff`, exactly as WebFlux composes `retryWhen(..)` rather than baking retry into the client:

Rust

```

use firefly::reactive::{Backoff, Mono};
use std::time::Duration;

let payment = Mono::retry_backoff(
    || client.get().uri("/payments/p1").retrieve().body_to_mono::<Payment>(),
    Backoff::new(3, Duration::from_millis(100)),
);

```

Composing with resilience

Both clients are deliberately small. For circuit breaking, rate limiting, or bulkheads, wrap calls in `firefly-resilience` decorators (covered in [Caching](#) and applied the same way to outbound calls). The circuit breaker is what keeps a sick Payments service from dragging Lumen down with it:

Rust

```

use firefly::resilience::{CircuitBreaker, CircuitConfig};

// CircuitBreaker::execute returns the operation's value (Result<T, _>), so the
// guarded call still yields the Payment. (Chain::execute is for guarded ops
// whose value you discard – it returns Result<(), _>.)
let breaker = CircuitBreaker::new(CircuitConfig::default());

let payment = breaker.execute(|| async {
    payments.request::<_, Payment>(Method::POST, "/payments", Some(&req)).await
}).await?;

```

When repeated calls fail, the breaker opens and rejects subsequent calls immediately with `ResilienceError::CircuitOpen` instead of waiting on a timeout — so one slow upstream cannot exhaust Lumen's task pool. Resilience belongs at the *client* layer, configured once, not scattered through every handler.

The experience tier: a Lumen BFF

A mobile or web frontend rarely wants a single domain service's raw shape — it wants a *journey*: "show me this wallet's balance **and** its pending payments, in one call." Calling Lumen for the balance and Payments for the pending list and merging in the client means two round trips, two failure domains, and the frontend leaking knowledge of both services' internals. The **Backend-for-Frontend (BFF)** pattern moves that composition server-side.

Firefly ships a dedicated starter for this tier: `firefly-starter-experience`. It builds on `WebStack` (so it inherits CORS, security headers, request metrics, correlation, and the actuator surface) and adds the BFF building blocks:

- `DomainClients` — a registry of named `RestClient`s for the downstream domain services, the Rust spelling of pyfly's `ClientFactory`;

- `SignalService` — `@WaitForSignal`-style gates for long-running, signal-driven workflow steps (the experience-tier `Workflow` from `Sagas`);
- a Redis-capable `WorkflowState` keyed by correlation id, plus a `WorkflowQueryService` for journey-status reads.

A Lumen experience service registers its downstream clients up front, then composes them:

Rust

```
use firefly::starter_experience::{ExperienceStack, DomainClients};
use firefly::starter_web::CoreConfig;

let bff = ExperienceStack::new(CoreConfig {
    app_name: "lumen-mobile-bff".into(),
    ..Default::default()
});

// Register the domain SDKs this BFF composes. `register` returns an
// Arc<RestClient> already wired with correlation + trace propagation.
let wallets = bff.clients.register("wallets", "https://lumen.internal");
let payments = bff.clients.register("payments", "https://payments.internal");
```

The composition root then fans out across the registered clients — both calls go out concurrently, so the composite latency is bounded by the slower upstream rather than their sum — and degrades gracefully if one upstream is circuit-open (show the balance, leave the pending list empty) instead of failing the whole response.

*The tier boundary is strict. The dependency direction is `channel` → `experience` → `domain` → `core`. An experience service **never** owns a database, **never** calls a core service directly, and **never** calls a sibling experience service — it composes domain SDKs only. Lumen is a domain/core-style service built on the `firefly` facade; the BFF is a separate crate that depends on*

`firefly-starter-experience` and on Lumen's published client. That separation is why the experience starter is not bundled into the one-dependency facade — a domain service does not need it.

🌿 SPRING PARITY

The BFF mirrors the Spring Cloud Gateway / BFF approach where a thin application aggregates several microservices. `Mono::zip` (and the concurrent fan-out above) plays the role of WebFlux's `Mono.zip()` and pyfly's `asyncio.gather`. `DomainClients` is the `ClientFactory`; the signal-driven `Workflow` is `@WaitForSignal`. The team-ownership model is identical: domain teams own stable, fine-grained contracts; the frontend team owns the BFF that adapts them.

Other protocols

The crate ships builders/scaffolds for the protocols a back-office platform needs — SOAP (CXF-style envelope), gRPC, GraphQL, and WebSocket — selected by feature so heavy dependencies stay out of services that do not use them. The REST and GraphQL surfaces are fully wired; SOAP and the streaming protocols are feature-gated.

Outbound calls inherit the caller's correlation id automatically, so a request that fans out to three upstreams stitches together in your traces.

What changed in Lumen

- We sketched the **Payments client** Lumen would build to settle a transfer's credit leg over the network — `RestBuilder::new(...).with_retries(...).build()` — and showed how an upstream RFC 7807 problem decodes into a typed `FireflyError` via `err.as_firefly()`.

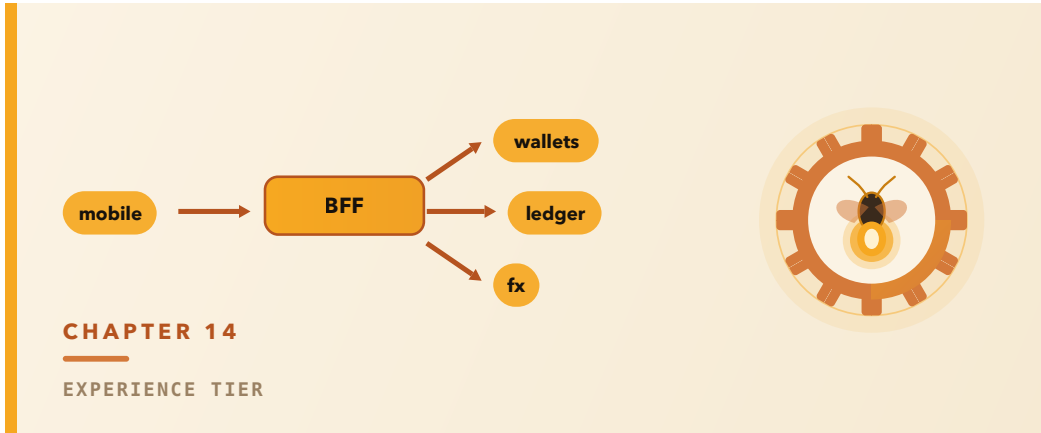
- We saw how the saga's credit step changes from a local `Ledger::deposit` to a resilient outbound call, why an `Idempotency-Key` is mandatory across retries, and why the debit's compensation matters more once I/O can fail.
- We introduced the reactive `WebClient` (`body_to_mono` / `body_to_flux` / `exchange`), its streaming decode (the consumer side of Lumen's own NDJSON/SSE endpoint), and the rule that retries are *composed* on the returned `Mono`, not baked into the client.
- We met the **experience tier** — `firefly-starter-experience` with `DomainClients`, `SignalService`, and `WorkflowState` — and saw how a Lumen BFF composes the wallet and payments services into one journey-shaped API, with the strict `experience → domain → core` dependency direction.

Exercises

1. **Decode an upstream problem.** Stand up a stub that answers `POST /payments` with a `422 application/problem+json` body. Call it through a `RestClient` and assert that `err.as_firefly()` returns `Some`, that `fe.status == 422`, and that `err.is_unprocessable_entity()` is `true` — so the saga can map the upstream rejection onto Lumen's own `422` instead of a `500`.
2. **Wrap the credit leg in a circuit breaker.** Take the transfer saga's credit step and replace `ledger.deposit(...)` with a `CircuitBreaker`-guarded `payments.request(...)`. Drive the stub to fail enough times to trip the breaker, and assert the next call returns `ResilienceError::CircuitOpen` *immediately* (no timeout) — and that the saga still compensates the debit.

3. **Compose a BFF summary.** Build a tiny `ExperienceStack`, register a `wallets` and a `payments` client against two local stubs, and write a handler that fetches the balance and the pending list concurrently. Make the payments stub return an error and assert the handler still returns the balance with an empty pending list — proving partial degradation rather than a `500`.

The next chapter secures the inbound side — JWT bearer auth and path-based RBAC on Lumen's mutating routes. Continue to [Security](#).



CHAPTER 14

The Experience Tier: Composing a BFF

Lumen, as the book has built it, is a self-contained service: it owns the wallet domain *and* the HTTP API that fronts it. Real Firefly systems split that responsibility across **three service tiers**, and Lumen sits at the bottom two. By the end of this chapter you will know where a frontend-facing API for Lumen belongs — the **experience tier** — and you will build a small Backend-for-Frontend that composes Lumen as a downstream domain SDK, drives a multi-step "fund a wallet and confirm" journey with a signal-gated workflow, and survives a client disconnect by persisting journey state.

This is the one Firefly tier Lumen itself never enters, so the chapter introduces it from first principles and then wires it against the service you already have.

 SPRING PARITY

The experience tier is the Firefly Java `firefly-starter-application` -driving- `@Workflow` s pattern (and pyfly's `transactional/workflow` + `client` building blocks composed into a BFF). Its Rust home is the `firefly-starter-experience` crate. A signal-gated step is Spring's `@WaitForSignal` ; the domain-SDK registry is the `ClientFactory` .

The three service tiers

Firefly services are layered into three **service** tiers (distinct from the crate-graph tiers in the architecture docs). The dependency direction is strict and one-way: `channel` → `experience` → `domain` → `core` .

Service tier	Owns	Talks to	Rust starter
core	the database (R2DBC/sqlx, migrations, CRUD)	nothing below	<code>firefly-starter-core</code> / <code>firefly-starter-data</code>
domain	sagas, CQRS, event sourcing, third-party adapters	core SDKs	<code>firefly-starter-domain</code>
experience (BFF)	signal-driven workflows, stateless aggregation, atomic REST	domain SDKs <i>only</i>	<code>firefly-starter-experience</code>

Lumen, with its event-sourced ledger, CQRS bus, and transfer saga, is a **domain** service. An **experience** service is a Backend-for-Frontend: it aggregates several domain SDKs into APIs shaped for *one* frontend or channel. It **never** owns a database, **never** calls a core service directly, and **never** calls a sibling experience service — it composes domain SDKs (over `firefly-client`) and nothing else.

 SPRING PARITY

The strict `channel → experience → domain → core` direction is the same layering Spring Cloud microservice estates enforce: a BFF/edge service calls domain services; domain services own their data. The Rust starters make the boundary a *type*: an experience stack can register domain SDKs and nothing else.

ExperienceStack – batteries for a BFF

`ExperienceStack::new(cfg)` builds a full `WebStack` (so it inherits every web battery from the [production chapter](#) — CORS, security headers, request metrics, access log, correlation, idempotency, the actuator surface) and adds the four experience-tier building blocks:

Rust

```
use firefly_starter_experience::{CoreConfig, ExperienceStack};

let bff = ExperienceStack::new(CoreConfig {
    app_name: "lumen-bff".into(),
    app_version: "1.0.0".into(),
    ..CoreConfig::default()
});
```

The stack exposes five public fields (`Bff` is an alias for `ExperienceStack`):

Field	Type	Role
<code>clients</code>	<code>DomainClients</code>	the domain-SDK registry (the <code>ClientFactory</code> equivalent)
<code>signals</code>	<code>Arc<SignalService></code>	<code>@WaitForSignal</code> -style gates a workflow step parks on
<code>state</code>	<code>WorkflowState</code>	Redis-capable persisted journey state, keyed by correlation id
<code>query</code>	<code>Arc<WorkflowQueryService></code>	the journey-status query surface
<code>children</code>	<code>Arc<ChildWorkflowService></code>	child-workflow composition for nested journeys

`ExperienceStack` `Deref`s to its `WebStack` (which derefs to `Core`), so every web + core method you met building Lumen — `apply_middleware`, `actuator_router`, `new_application`, `with_security`, `cache`, `bus`, `scheduler` — is reachable directly on the BFF value. There is also a pyfly-parity bootstrap pair, `register_experience_stack(cfg)` (`== ExperienceStack::new`) and `enable_experience_stack(cfg)` (stamps the tier defaults onto a `CoreConfig`), so a migrating service reaches the tier by the spelling it already knows.

🌿 SPRING PARITY

`register_experience_stack` / `enable_experience_stack` mirror pyfly's `register_*_stack` / `@enable_*_stack` (and .NET's `services.AddFireflyExperience`). `ExperienceStack` deref-ing to `WebStack` is the Rust analog of Go's embedded `*WebStack` — the BFF is a web service plus the journey machinery.

Composing domain SDKs – `DomainClients`

A BFF reaches its downstream domain services through named `RestClient` s, one per dependency. `DomainClients` is the registry; register Lumen under a logical name and resolve it by that name from any handler or workflow step:

Rust

```
// Experience -> Domain only. Register Lumen as a downstream domain SDK.
bff.clients.register("wallets", "https://lumen.internal");

// later, in a handler or workflow step:
let wallets = bff.clients.get("wallets").expect("wallets SDK");
// wallets is an Arc<RestClient> with correlation-id propagation, JSON codec,
// RFC 7807 error decoding, and retry/backoff all inherited from firefly-client.
```

`register(name, base_url)` builds a default client; `register_client(name, client)` takes a pre-tuned `RestClient` (custom timeout, headers, retry policy). Re-registering a name replaces it (last wins), and `names()` lists every registered SDK.

🌿 SPRING PARITY

`DomainClients` is the `ClientFactory` : instead of threading a `RestBuilder` through every call site, a step resolves the right client by logical name. The clients carry the same correlation propagation and RFC 7807 decoding the [HTTP-clients chapter](#) covered.

Signal-driven journeys

A BFF journey is rarely one request. "Fund a wallet, wait for the customer to confirm the amount, then commit the transfer" is three interactions over time. A **workflow** with a **signal gate** models exactly that: steps that call domain SDKs, and a gate that parks until an atomic endpoint delivers a named signal.

The journey is a `Workflow` of `Node` s; `Node::wait_for_signal` parks on the stack's `SignalService` until the signal arrives:

Rust

```
use std::sync::Arc;
use firefly_starter_experience::{Node, Workflow};

let signals = Arc::clone(&bff.signals);
let journey_id = "j-1".to_string();

let workflow = Workflow::new("fund-and-confirm")
    // 1. reserve: call the Lumen "wallets" SDK to open/lock the funds.
    .node(Node::new("reserve", || async { Ok(()) }))
    // 2. await-confirm: park until POST /journeys/{id}/confirm delivers
    "confirmed".
    .node(
        Node::wait_for_signal("await-confirm", &signals, journey_id.clone(),
        "confirmed")
        .depends_on(["reserve"]),
    )
    // 3. commit: call the Lumen "wallets" SDK to run the transfer.
    .node(Node::new("commit", || async { Ok(()) }).depends_on(["await-
confirm"]));
```

`Workflow::run().await` executes the nodes in dependency order; when it reaches `await-confirm` it parks. An atomic endpoint later calls `bff.signals.deliver(&journey_id, "confirmed", payload)` and the parked node resumes. Delivery is **buffered** — if the signal arrives before the gate parks, there is no lost wakeup. `signals.list_active()` lists the journeys currently parked on a gate.

🌱 SPRING PARITY

`Node::wait_for_signal(...)` is the engine spelling of `@WaitForSignal("confirmed")`; `signals.deliver(...)` is the external event that satisfies the gate. The workflow is the same DAG-with-compensation engine from the [sagas chapter](#) (`firefly-orchestration`), here driven by signals rather than run to completion in one call.

Persisting journey state – `WorkflowState`

Because a journey spans several requests, its state must outlive any one of them.

`WorkflowState` round-trips a workflow run's `StepContext` snapshot through the stack's cache `Adapter`, keyed by correlation id. The in-memory adapter is the default; point it at `firefly-cache-redis`'s `RedisAdapter` for cross-restart durability — the convention the experience tier is built around.

Rust

```
use firefly_starter_experience::CoreConfig;
use firefly_orchestration::StepContext;

// Save when a journey parks:
let ctx = StepContext::new();
ctx.set_correlation_id("j-1");
ctx.set_variable("phase", serde_json::json!("AWAITING_CONFIRM"));
bff.state.save(&ctx).await?;

// Rehydrate from a later request to advance it:
if let Some(ctx) = bff.state.load("j-1").await? {
    // ... advance the journey ...
}

// Discard when the journey completes:
bff.state.delete("j-1").await?;
```

A miss on an unknown journey is `Ok(None)`, not an error — so a status check on a journey that never existed is a clean 404, not a 500.

🌿 SPRING PARITY

`WorkflowState` is the experience-tier analog of `DurableWorkflowState`, but persisted through the `cache Adapter` — the Firefly convention of "hold workflow state in Redis." A parked journey saves its `StepContext`; a later request loads it and resumes, surviving the client disconnect an in-memory waiter would not.

Querying journey status – `WorkflowQueryService`

The frontend polls "where is my journey?" while it waits. `WorkflowQueryService` holds the live `StepContext` per run and answers named queries against it — the main recovery mechanism when a client reconnects:

Rust

```
bff.query.register(&journey_id, ctx.clone());           // on start
bff.query.register_query(&journey_id, "phase", |ctx| { // a named query
    ctx.variable("phase").unwrap_or(serde_json::json!("UNKNOWN"))
});
let phase = bff.query.query(&journey_id, "phase"?);    // GET /journeys/{id}
bff.query.unregister(&journey_id);                     // on completion
```

The atomic-endpoint contract

Put together, an experience controller exposes one request per journey phase — the **atomic REST** shape:

Method & path	Does
<code>POST /journeys</code>	start the workflow (calls the "wallets" SDK to reserve), persist <code>WorkflowState</code> , park on the gate, return the journey id
<code>POST /journeys/:id/confirm</code>	deliver the <code>confirmed</code> signal — the parked workflow resumes and commits the transfer via the "wallets" SDK
<code>GET /journeys/:id</code>	report the persisted phase (or 404 if unknown)

Each phase is one HTTP request; state lives in the cache (Redis-capable), so a client can resume a journey after a disconnect. Because the controller runs through `bff.apply_middleware(routes)`, every response inherits the web batteries — the start response carries `X-Frame-Options: DENY` and a correlation id just as Lumen's own responses do, and the same `ExperienceStack::with_security(chain)` filter chain from the [security chapter](#) guards the mutating routes.

This is exactly the contract the crate's own boot test proves end to end (its checkout journey uses an `AWAITING_PAYMENT` phase rather than Lumen's `AWAITING_CONFIRM`, but the shape is identical): two mock domain SDKs composed through a signal-gated workflow, driven with `tower::oneshot` — start parks on the gate, the status endpoint reports the persisted phase, delivering the signal advances the workflow off-task, and the final status flips to `COMPLETED`.

Where Lumen fits

Drawn as the full estate, Lumen is one of the domain services a BFF composes:

Text

```

[ web / mobile app ]           ← channel tier
    |
[ lumen-bff (experience) ]     ← this chapter: DomainClients + signals + state
    |   Experience → Domain only
[ lumen (domain) ]           ← the service the book built (ledger, CQRS,
saga)
    |
[ accounts (core) ]           ← owns the database

```

The BFF never reaches into Lumen's event store or bus — it speaks to Lumen's public HTTP API through the registered `"wallets"` SDK, exactly as any external client would, and adds only the journey orchestration a frontend needs.

What changed in Lumen

Lumen itself is unchanged — it is a *domain* service, and this chapter is about the tier *above* it. What you built is the mental model and the wiring for a frontend-facing BFF: an `ExperienceStack` inheriting Lumen's web batteries, `DomainClients` registering Lumen as the `"wallets"` SDK (Experience → Domain only), a signal-gated `Workflow` modeling a multi-request "fund and confirm" journey, `WorkflowState` persisting that journey through a Redis-capable cache adapter, and `WorkflowQueryService` answering status polls — every API drawn from the real `firefly-starter-experience` surface.

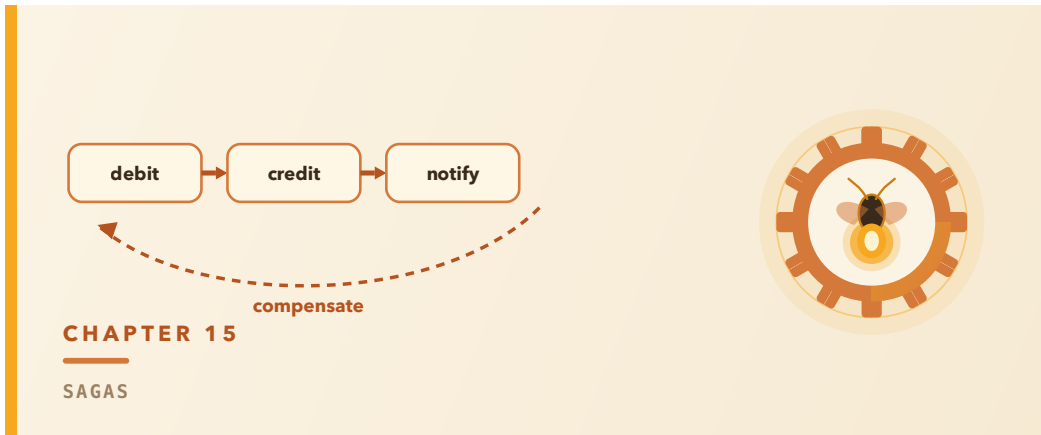
Exercises

1. **Register Lumen as a domain SDK.** Build an `ExperienceStack`, call `bff.clients.register("wallets", "http://localhost:8080")`, and confirm `bff.clients.get("wallets")` returns a client and `bff.clients.names()` lists it.
2. **Park and resume.** Build a two-node workflow whose second node is a `Node::wait_for_signal` gate. Run it on a task, assert

`bff.signals.list_active()` contains the journey id, then `deliver` the signal and confirm the workflow completes.

3. **Persist a journey.** Save a `StepContext` with a `phase` variable via `bff.state.save`, `load` it back from a fresh handle, and confirm the variable survives. Then `delete` it and confirm `load` returns `Ok(None)`.
4. **Atomic endpoints.** Wire the three-route controller above with `bff.apply_middleware(routes)` and drive it with `tower::oneshot`: start, poll status (`AWAITING_CONFIRM`), confirm, poll again (`COMPLETED`). Assert the start response carries the inherited `X-Frame-Options` header.

With Lumen in production (the previous chapter) and its place in the tiered estate clear, the capstone re-reads the whole service through the declarative-macro lens. Continue to [Declarative Services with Macros](#).



CHAPTER 15

Distributed Transactions: Sagas, Workflows & TCC

By the end of this chapter, Lumen can **move money between two wallets** — and do it *safely*. A transfer is not a single command: it debits one wallet, then credits another, and those are two independent writes to two independent event streams. If the credit leg fails after the debit already committed, the source owner is out of pocket with nothing on the other side. There is no **BEGIN ... COMMIT** that spans two aggregates, so Lumen reaches for the pattern that does the job: a **saga** that compensates the debit when the credit fails.

We build the **POST /api/v1/transfers** endpoint on top of the **Ledger** the CQRS handlers already use, so a transfer raises *real* **MoneyWithdrawn** / **MoneyDeposited** events on both streams — and a refund raises a real **MoneyDeposited** on the source stream. Everything stays observable on the event-sourced ledger you built in chapter 11.

`firefly-orchestration` ships the three classic **distributed-transaction engines** every Firefly platform agrees on. Each composes async steps, runs as a plain future on the caller's task, applies a per-step retry policy, threads a typed context blackboard, and respects cooperative cancellation.

Engine	Topology	Compensation
<code>Saga</code>	Sequential steps	Reverse-order, configurable policy
<code>Workflow</code>	DAG with parallel branches	Reverse-order, configurable policy
<code>Tcc</code>	Try-all then Confirm-all	Cancel-tried-on-Try-failure

SPRING PARITY

This is the `firefly-common-domain` orchestration model: orchestrated multi-step processes with compensation, the same step/compensation shapes you know from the JVM's `@Saga` / `@SagaStep` (and pyfly's `@saga` / `@saga_step`), expressed here as async closures rather than a bean the container discovers.

The problem with distributed writes

Make the failure modes concrete before writing a line of code. A Lumen transfer has two legs:

1. **Debit the source** — `withdraw(amount)`, which enforces `balance >= 0`.
2. **Credit the destination** — `deposit(amount)`.

Each leg is an independent `Ledger` call that appends to that wallet's own event stream. A missing destination wallet, or an overdraft on the source, fails one leg after the other may already have committed. Retrying the *whole* operation is unsafe — you might debit twice. Silently skipping the failed leg leaves the balances inconsistent.

The principled answer is **eventual consistency with explicit compensation**. Each leg commits to its own stream independently, and you design a recovery path — a *compensating transaction* — for every step that could succeed before a later one fails. A compensation is not a database rollback; it is a *semantic undo*: "re-credit the source" is a brand-new **deposit** that restores the balance, and it leaves an auditable refund event behind.

Saga – sequential with compensation

A **Saga** runs steps in order; on any step failure it compensates the completed steps in reverse order. Attach a compensation to each step with **with_compensation**:

Rust

```
use firefly_orchestration::{CompensationPolicy, Saga, SagaStatus, Step};

let saga = Saga::new("checkout")
    .policy(CompensationPolicy::BestEffort) // or StopOnError
    .step(
        Step::new("reserve", || async { Ok(()) })
            .with_compensation(|| async { Ok(()) }),
    )
    .step(
        Step::new("charge", || async { Ok(()) })
            .with_compensation(|| async { Ok(()) }),
    )
    .step(Step::new("ship", || async { Ok(()) }));

let outcome = tokio::runtime::Runtime::new()
    .unwrap()
    .block_on(saga.run())
    .expect("saga completes");

assert_eq!(outcome.status, SagaStatus::Completed);
assert_eq!(outcome.steps_executed, ["reserve", "charge", "ship"]);
```

On success the outcome's `status` is `Completed` and `steps_executed` lists the steps that ran. On failure, `run` returns a `SagaFailure` carrying both the error (`failure.error()`) and the fully-populated `Outcome` (with `steps_rolled` naming the compensations that ran, reverse order). The status is then `Compensated` (rollback succeeded) or `Failed`.

`CompensationPolicy`:

- `BestEffort` (default) — log and continue compensating remaining steps even if one compensation fails.
- `StopOnError` — abort rollback at the first compensation failure and surface a `SagaError::Compensation` wrapping the original.

Lumen's transfer saga

Lumen's transfer is a two-step saga: `debit` the source, then `credit` the destination. The debit's compensation refunds the source; the credit is the last leg, so it needs no compensation — a failure there rolls back only the debit. The whole thing lives in `src/transfer.rs`, and it threads the `Ledger` through both legs.

First, the wire types — the request body and the result `POST /api/v1/transfers` returns:

Rust

```

use serde::{Deserialize, Serialize};

/// `POST /api/v1/transfers` command – move `amount` (cents) from `from` to
/// `to`.
#[derive(Debug, Clone, Default, PartialEq, Eq, Serialize, Deserialize)]
#[serde(default)]
pub struct TransferRequest {
    pub from: String,
    pub to: String,
    /// The amount to move, in minor units (cents); must be `> 0`.
    pub amount: i64,
}

/// The result of a completed (or compensated) transfer.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct TransferResult {
    /// `"completed"` when both legs succeeded – the lowercase `SagaStatus`.
    pub status: String,
    pub from: String,
    pub to: String,
    pub amount: i64,
    #[serde(rename = "stepsExecuted")]
    pub steps_executed: Vec<String>,
    #[serde(rename = "stepsRolledBack")]
    pub steps_rolled_back: Vec<String>,
}

```

The result echoes the `SagaStatus` as a lowercase string plus the two step lists, so the API tells the caller *exactly* what the engine did — which steps ran and which were rolled back. A transfer also has a typed error that distinguishes a malformed request from a clean, compensated business failure:

Rust

```

/// The typed error a transfer surfaces to its caller.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum TransferError {
    /// The request was malformed (same wallet, non-positive amount).
    Invalid(String),
    /// The transfer failed and was rolled back; the inner string is the
    /// failing leg's domain error (e.g. `insufficient funds`).
    Compensated(String),
}

impl std::fmt::Display for TransferError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            TransferError::Invalid(detail) => f.write_str(detail),
            TransferError::Compensated(detail) => write!(f, "transfer rolled
back: {detail}"),
        }
    }
}

impl std::error::Error for TransferError {}

```

One dependency, even here. `TransferError` hand-writes `Display` + `std::error::Error` instead of pulling in `thiserror` — the same discipline the rest of Lumen keeps, so the whole framework and every typed error still arrive through the single `firefly` facade.

Building and running the saga

`run_transfer` validates the request, builds the two steps, and runs the saga. The interesting part is how the *typed* domain cause escapes the saga engine's generic `BoxError`: each leg stashes the failing `DomainError` in a shared `Mutex` so we can surface "insufficient funds" to the caller verbatim rather than an opaque boxed error string.

Rust

```

use std::sync::{Arc, Mutex};

use firefly::orchestration::{Saga, SagaStatus, Step};

use crate::domain::DomainError;
use crate::ledger::Ledger;
use crate::money::Money;

pub async fn run_transfer(
    ledger: &Ledger,
    req: &TransferRequest,
) -> Result<TransferResult, TransferError> {
    if req.amount <= 0 {
        return Err(TransferError::Invalid("amount must be > 0".into()));
    }
    if req.from == req.to {
        return Err(TransferError::Invalid("from and to must differ".into()));
    }
    let amount = Money::cents(req.amount);

    // Captures the domain error of the failing leg so the saga's generic
    // BoxError can be surfaced as a typed cause to the caller.
    let cause: Arc<Mutex<Option<DomainError>>> = Arc::new(Mutex::new(None));

    // Step 1 – debit the source; compensation refunds it.
    let debit = {
        let ledger = ledger.clone();
        let from = req.from.clone();
        let refund_ledger = ledger.clone();
        let refund_from = req.from.clone();
        let cause = Arc::clone(&cause);
        Step::new("debit", move || {
            let ledger = ledger.clone();
            let from = from.clone();
            let cause = Arc::clone(&cause);
            async move {
                ledger.withdraw(&from, amount).await.map_err(|e| {
                    *cause.lock().expect("cause lock") = Some(e.clone());
                })
            }
        })
    }
}

```



```

        box_err(e)
    }?;
    Ok(())
}
})
.with_compensation(move || {
    let ledger = refund_ledger.clone();
    let from = refund_from.clone();
    async move {
        // A refund is a normal deposit, so it raises a real
        // MoneyDeposited event on the source stream.
        ledger.deposit(&from, amount).await.map_err(box_err)?;
        Ok(())
    }
})
};

// Step 2 – credit the destination (no compensation; it is the last leg,
// so a failure here rolls back only the debit).
let credit = {
    let ledger = ledger.clone();
    let to = req.to.clone();
    let cause = Arc::clone(&cause);
    Step::new("credit", move || {
        let ledger = ledger.clone();
        let to = to.clone();
        let cause = Arc::clone(&cause);
        async move {
            ledger.deposit(&to, amount).await.map_err(|e| {
                *cause.lock().expect("cause lock") = Some(e.clone());
                box_err(e)
            })?;
            Ok(())
        }
    })
};

let saga = Saga::new("money-transfer").step(debit).step(credit);

match saga.run().await {

```

```

Ok(outcome) => Ok(TransferResult {
    status: SagaStatus::Completed.to_string(),
    from: req.from.clone(),
    to: req.to.clone(),
    amount: req.amount,
    steps_executed: outcome.steps_executed,
    steps_rolled_back: outcome.steps_rolled,
}),
Err(failure) => {
    let detail = cause
        .lock()
        .expect("cause lock")
        .clone()
        .map(|e| e.to_string())
        .unwrap_or_else(|| failure.error().to_string());
    Err(TransferError::Compensated(detail))
}
}
}

/// Boxes a `DomainError` as the saga engine's `BoxError`.
fn box_err(e: DomainError) -> firefly::orchestration::BoxError {
    Box::<dyn std::error::Error + Send + Sync>::from(e.to_string())
}

```

How it works. Each `Step::new(name, action)` takes a closure returning a future that resolves to `Result<(), BoxError>`. The debit leg clones the `Ledger` (it is cheap to clone — `Arc` s inside) so the step owns what it captures; `with_compensation` takes a *second* closure that runs only if a later step fails. The credit leg has no compensation: it is the last step, so the only thing to undo on its failure is the debit, which the engine handles automatically by running the debit's compensation.

The `cause` `Mutex` is the bridge between the saga's untyped error channel and Lumen's typed `DomainError`. When a leg fails, it records the `DomainError` *before* boxing it; if `saga.run()` returns `Err`, we read the captured cause back out and wrap it in `TransferError::Compensated`. That is how `POST /api/v1/transfers` can answer with `insufficient funds` instead of a stringly-typed box.

🌿 SPRING PARITY

On the JVM you would annotate a bean with `@Saga` and each method with `@SagaStep(compensate = "refundDebit")`, and the `WorkflowBeanPostProcessor` would discover and register it. Rust has no reflection, so Lumen builds the `Saga` value explicitly with `Step::new / with_compensation`. The shape is identical — forward step, named compensation, reverse-order rollback — but the wiring is a value you construct, not a bean the container scans.

The endpoint

The controller method in `src/web.rs` is thin: drive the saga, then translate the typed outcome into the HTTP contract. A clean rollback is a *business* failure, so it surfaces as a `422` problem carrying the cause — not a `500`:

Rust

```

/// `POST /api/v1/transfers` – run a money transfer as a saga.
#[post("/transfers")]
async fn transfer(
    State(api): State<WalletApi>,
    Json(body): Json<TransferRequest>,
) -> WebResult<Json<TransferResult>> {
    let result = run_transfer(&api.ledger, &body)
        .await
        .map_err(|e| match e {
            TransferError::Invalid(detail) =>
                WebError::from(FireflyError::validation(detail)),
            TransferError::Compensated(detail) => {
                WebError::from(FireflyError::validation(detail))
            }
        })?;
    // A transfer touches both wallets' views; invalidate the family.
    api.query_cache.invalidate_type::<GetWallet>();
    Ok(Json(result))
}

```

Note the `invalidate_type::<GetWallet>()` at the end: a transfer changed two balances, so the cached `GetWallet` views must be dropped to keep a read after the write honest. That cache and its invalidation are the subject of [Caching](#); the transfer is just one more mutation that has to play by its rules.

What the saga does on each path

The tests in `src/transfer.rs` exercise all three paths, and they are the best documentation of the behavior. The happy path moves funds and rolls back nothing:

Rust

```

let result = run_transfer(
    &ledger,
    &TransferRequest { from: src.id.clone(), to: dst.id.clone(), amount: 300 },
)
.await
.unwrap();

assert_eq!(result.status, "completed");
assert_eq!(result.steps_executed, ["debit", "credit"]);
assert!(result.steps_rolled_back.is_empty());
assert_eq!(balance(&ledger, &src.id).await, 700);
assert_eq!(balance(&ledger, &dst.id).await, 300);

```

The **overdraft** path short-circuits at the debit — the source never has the funds, so the withdraw fails *before* anything applies. There is nothing to compensate, and both balances are left intact:

Rust

```

let err = run_transfer(
    &ledger,
    &TransferRequest { from: src.id.clone(), to: dst.id.clone(), amount: 500 },
)
.await
.unwrap_err();

assert_eq!(err, TransferError::Compensated("insufficient funds".into()));
assert_eq!(balance(&ledger, &src.id).await, 100); // untouched
assert_eq!(balance(&ledger, &dst.id).await, 0);   // untouched

```

The **credit-failure** path is where compensation earns its keep. The debit applied, then the credit failed (the destination does not exist), so the engine runs the debit's compensation — a refund deposit. The source's net balance is restored, and the stream records *both* the debit and its refund, an audit trail of exactly what happened:

Rust

```

let err = run_transfer(
    &ledger,
    &TransferRequest { from: src.id.clone(), to: "wlt_missing".into(), amount:
400 },
)
.await
.unwrap_err();
assert!(matches!(err, TransferError::Compensated(_)));

// open(1000) - withdraw(400) + refund(400) = 1000, with 3 events on the stream.
let src_events = ledger.load_events(&src.id).await.unwrap();
assert_eq!(Wallet::rehydrate(&src.id, &src_events).view().balance, 1_000);
assert_eq!(src_events.len(), 3); // open + withdraw + refund-deposit

```

Sagas are eventually consistent. A saga does not give you serializability. Between the moment the source is debited and the moment the credit commits (or the refund runs), another request could read the source and see a balance lower than it will ultimately be. That is the trade-off for operating across independent aggregates without a distributed lock: consistency in the end — all legs committed, or all compensated — not at every instant.

Passing data between steps

Lumen's transfer steps are self-contained — each captures what it needs — but many sagas need a later step to consume an earlier step's output. A step built with `Step::with_context(name, |ctx| ...)` reads the typed `StepContext` blackboard; run the saga with `run_with_context(&ctx)`. Per-step `with_retry(policy)` adds max attempts, exponential backoff, jitter, and a per-attempt timeout, so a flaky leg can recover on its own before the engine declares it failed.

Workflow – a DAG

When a process has *independent* steps that should run in parallel, reach for a **Workflow**: a DAG of **Node**s with **depends_on** declarations. Independent nodes run concurrently within a wave; the first node error short-circuits the run:

Rust

```
use firefly_orchestration::{Node, Workflow};

let workflow = Workflow::new("approval")
    .node(Node::new("credit-check", || async { Ok(()) }))
    .node(Node::new("fraud-scan", || async { Ok(()) }))
    .node(
        Node::new("approve", || async { Ok(()) })
            .depends_on(["credit-check", "fraud-scan"]),
    );

let result = tokio::runtime::Runtime::new()
    .unwrap()
    .block_on(workflow.run());
assert!(result.is_ok());
```

credit-check and **fraud-scan** run in parallel; **approve** waits for both. Duplicate node names and unknown dependencies are rejected up-front, and an unreachable node aborts the run (**"no progress (dependency cycle?)"**).

Nodes get the same superpowers as saga steps: **Node::with_compensation** rolls back in reverse completion order under the configurable **CompensationPolicy**, **Node::with_context** reads prior results, **Node::when(expr)** skips a node on a false predicate, and **Node::fire_and_forget** schedules it without blocking the wave.

Where Lumen would grow this. A large transfer that needs a parallel compliance check (a credit check and a fraud scan that both feed an approval gate) is the textbook **Workflow**. The experience-tier starter, **firefly-starter-experience**, builds on exactly this engine with

signal-driven workflow steps that park until an external caller delivers a named signal — the Rust spelling of pyfly's `@wait_for_signal` and the JVM's `@WaitForSignal`. We return to that tier in [HTTP Clients](#).

TCC – Try / Confirm / Cancel

Tcc runs a two-phase protocol: Try all participants, then Confirm all on success; on any Try failure, Cancel the tried participants in reverse order (best-effort):

Rust

```
use firefly_orchestration::{Tcc, TccParticipant};

let tcc = Tcc::new("transfer")
    .participant(
        TccParticipant::new("debit", || async { Ok(()) }, || async { Ok(()) })
            .with_cancel(|| async { Ok(()) }),
    )
    .participant(
        TccParticipant::new("credit", || async { Ok(()) }, || async { Ok(()) })
            .with_cancel(|| async { Ok(()) }),
    );

let result = tokio::runtime::Runtime::new()
    .unwrap()
    .block_on(tcc.run());
assert!(result.is_ok());
```

`TccParticipant::new(name, try_fn, confirm_fn)` takes the Try and Confirm phases; `with_cancel` adds the compensation for a failed Try.

TCC vs. Saga. Lumen models its transfer as a **Saga** because each leg commits locally and a refund cleanly undoes the debit. A TCC framing would instead reserve (hold the funds, pre-authorize the credit), then confirm both or cancel both — better when a participant can cheaply hold a reservation and you want all-or-nothing semantics with no window where one side has committed and the other has not.

Cancellation

All three engines respect a **CancellationToken** — the Rust analog of the Go port's **context.Context** cancellation and pyfly's cooperative cancellation. Pass one to **run_cancellable(&token)** and cancel it from elsewhere to drain the run:

Rust

```
use firefly_orchestration::CancellationToken;

let token = CancellationToken::new();
let handle = token.clone();
// ... cancel from a timeout or a shutdown signal:
handle.cancel();

let outcome = saga.run_cancellable(&token).await;
```

The engines depend only on **futures**, so any executor (Tokio included) drives them.

Choosing an engine

Need	Engine
Linear process, undo on failure	Saga
Parallel branches that join	Workflow
Reserve-then-commit across resources	Tcc

Lumen's money-transfer is a **Saga** (debit → credit, with a debit refund). A multi-check approval (credit + fraud + KYC, then decision) is a **Workflow**. A reserve-funds-then-capture flow across a wallet and a card processor is a **Tcc**.

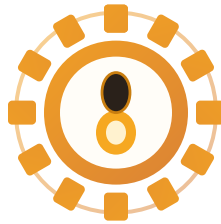
What changed in Lumen

- Lumen grew a **transfer saga** in `src/transfer.rs`: a **Saga** named `"money-transfer"` with a **debit** step (compensation: refund) and a **credit** step (no compensation — it is last).
- `run_transfer` validates the request, runs the saga, and translates the outcome into the typed `TransferResult` / `TransferError`, surfacing the *typed* failing `DomainError` (e.g. `insufficient funds`) by capturing it in a shared **Mutex** before the engine boxes it.
- The `POST /api/v1/transfers` endpoint in `src/web.rs` drives the saga, renders a clean rollback as a **422** business problem, and invalidates the `GetWallet` cache because the transfer changed two balances.
- The three behaviors — happy path (funds move), overdraft (short-circuit, both balances intact), and credit failure (debit refunded, three events on the source stream) — are pinned by tests, so the prose can never drift from the code.
- `TransferError` keeps the one-dependency discipline: hand-written `Display` + `Error`, no `thiserror`.

Exercises

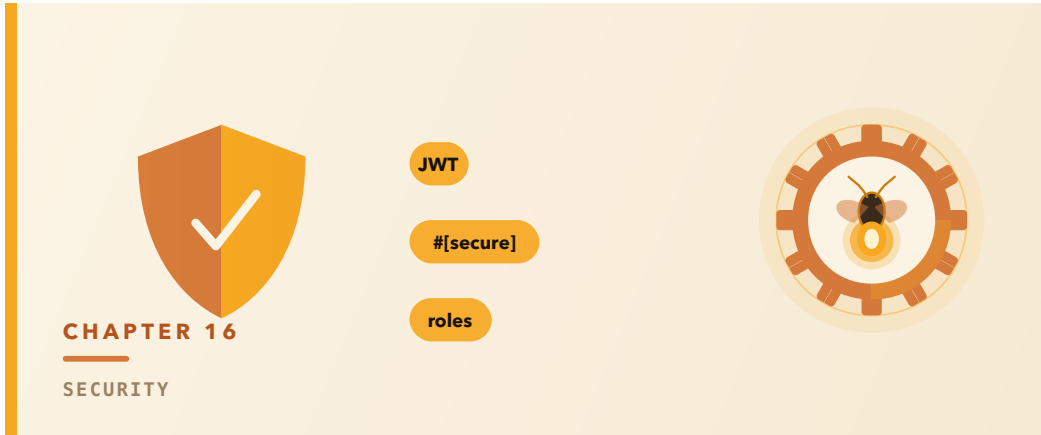
1. Add a `notify` step. Append a third, no-op `Step::new("notify", ...)` to the saga that "sends a receipt" (just `Ok(())` for now). Assert that on the happy path `steps_executed == ["debit", "credit", "notify"]`, and that when the credit fails the notify step never runs and only the debit is rolled back.
2. Make the debit retry. Give the `debit` step a `with_retry(policy)` with a small number of attempts and a short backoff. Wrap a flaky `Ledger` that fails the first withdraw and succeeds the second, and assert the transfer still completes — proving per-step retry recovers a transient failure before compensation is ever considered.
3. Switch the compensation policy. Build a three-step saga where two completed steps both have compensations and the third step fails. Run it once with `CompensationPolicy::BestEffort` and once with `StopOnError`, making one compensation itself fail, and assert how `steps_rolled` and the returned `SagaError` differ between the two policies.

To call the external services these steps coordinate — a Payments processor, an FX provider — you need an HTTP client. Continue to [HTTP Clients](#).



PART V

Secure, Observe & Ship



CHAPTER 16

Security, Sessions & Identity

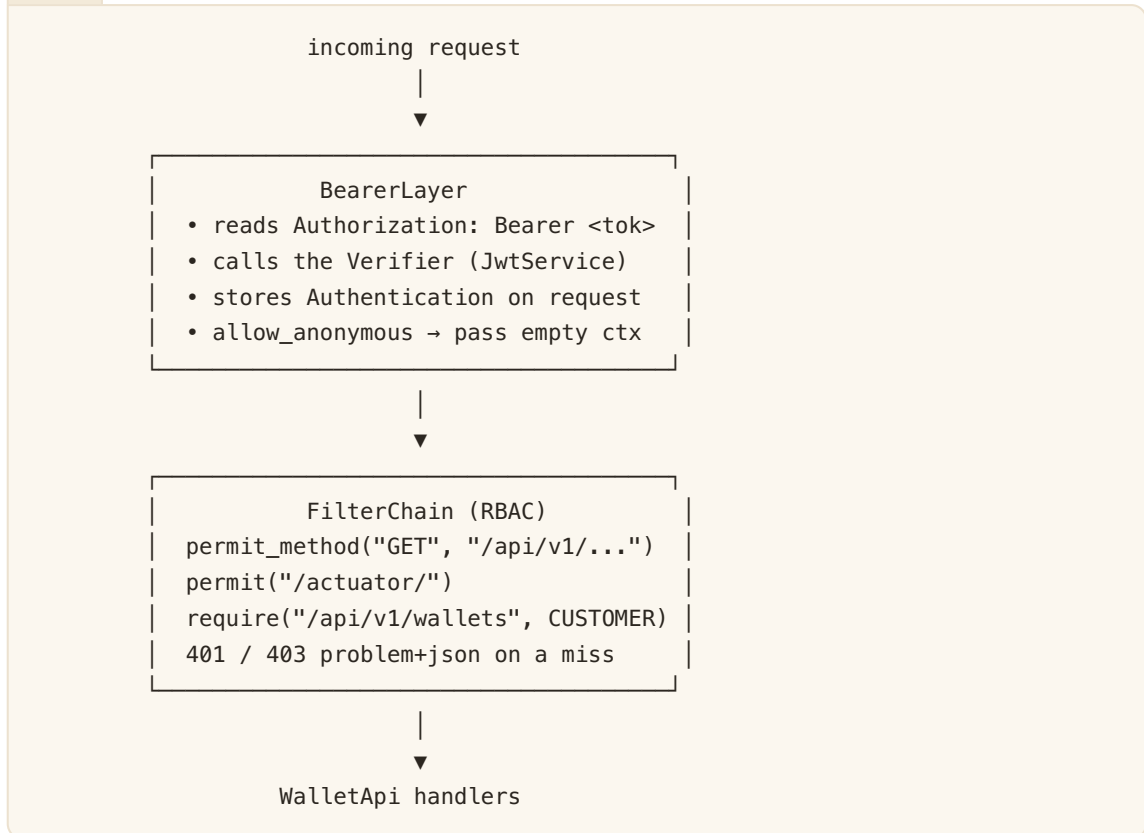
In Chapter 13 you saw how Lumen *would* call an external payments or FX provider. Lumen itself, though, is still wide open: any caller can open a wallet, deposit, withdraw, or move money between wallets. Before Part V can ship Lumen to production, you have to close that door.

By the end of this chapter Lumen will **authenticate** every request with a signed JWT, **authorize** the mutating routes with a path-based RBAC filter chain, and leave the public reads and the management surface open. The whole thing is built on `firefly-security`, reached through the one `firefly` facade — no new dependency, no hand-rolled crypto, and (true to Lumen's promise) not even a `thiserror` derive.

🍃 SPRING PARITY

`firefly-security` is Spring Security: a resource server (`BearerLayer` + a `Verifier` , the analog of a JWT decoder), URL-based authorization (`FilterChain` ~ `HttpSecurity.authorizeHttpRequests()`), method guards, JWKS validation, an OAuth2 client + authorization server, a role hierarchy, CSRF, and a bcrypt encoder. The wire formats — RFC 9457 problems on a 401/403 — are byte-identical across the Java, .NET, Go, and Python ports.

The mental model

Text

`firefly-security` carries far more than Lumen uses — JWKS, OAuth2 (client + authorization server), `RoleHierarchy`, method-level `guards`, `CsrfLayer`, `BcryptPasswordEncoder`. We tour those at the end; first, the four pieces Lumen actually wires.

Minting and verifying tokens – JwtService

Lumen is a stateless API. Sessions would need sticky routing or a shared store on every replica; a signed JWT lets each request carry its own credential, so the service scales horizontally with no shared state. The framework's `JwtService` both **mints** the demo tokens and **verifies** the incoming ones, using a symmetric HS256 key — which is exactly what makes Lumen runnable and testable with no external IdP.

Here is the whole of Lumen's token surface, from `src/security.rs`:

Rust

```

use firefly::security::{
    BearerConfig, BearerLayer, FilterChain, JwtService, SecurityError,
    Verifier, VerifierFn,
};
use serde_json::json;

/// The demo signing key. A real service reads this from configuration / a
/// secret store; it is inlined here so the sample is runnable as-is.
pub const DEMO_SIGNING_KEY: &[u8] = b"lumen-demo-signing-key-change-me";

/// The role every mutating wallet command requires.
pub const CUSTOMER_ROLE: &str = "CUSTOMER";

/// The shared HS256 service that both signs the demo tokens and verifies
/// incoming bearer tokens.
fn jwt_service() -> JwtService {
    JwtService::new(DEMO_SIGNING_KEY)
}

/// Mints a signed HS256 access token for `subject` with `roles`, valid for the
/// service's default lifetime (one hour).
pub fn mint_token(subject: &str, roles: &[&str]) -> String {
    jwt_service()
        .encode(json!({ "sub": subject, "roles": roles }))
        .expect("mint_token: HS256 encode")
}

```

`JwtService::new(secret)` builds an HS256 service. `encode` signs a JSON payload and — this is the load-bearing detail — injects an `exp` claim when the payload has none, defaulting to one hour out. Every token Lumen issues has a bounded lifetime; a token with no `exp` is rejected at `decode` time. The `mint_token` helper is what the HTTP tests call to obtain a credential the verifier will accept.

🍃 SPRING PARITY

`JwtService.encode / decode / to_authentication` mirror pyfly's `JwtService.encode / decode / to_security_context`, which in turn wrap Spring's `JwtEncoder / JwtDecoder`. The mandatory `exp` claim matches pyfly's `options={"require": ["exp"]}` and Spring's default decoder validation.

The Verifier and the Authentication

A `Verifier` is the resource-server port: validate a token, return an `Authentication` (the principal, username, roles, and the raw claims). `VerifierFn` adapts a plain async closure into one. Lumen's verifier delegates straight to `JwtService::to_authentication`, which maps `sub` → principal and `roles` → roles, then re-wraps any failure as a `SecurityError::Verification`:

Rust

```
/// Builds the resource-server Verifier: validates the token's HS256
/// signature + expiry, then maps `sub` → principal and `roles` → roles onto an
/// Authentication. A bad signature / expired token surfaces as a
/// SecurityError::Verification, which the BearerLayer renders as a
/// `401 application/problem+json`.
pub fn build_verifier() -> impl Verifier {
    VerifierFn(|token: String| async move {
        jwt_service()
            .to_authentication(&token)
            .map_err(|e: SecurityError| SecurityError::verification(format!(
                "invalid token: {e}")))
    })
}
```

`Authentication` carries the fields a handler or filter inspects:

Field	Type	From the claim
<code>principal</code>	<code>String</code>	<code>sub</code>
<code>username</code>	<code>String</code>	<code>sub</code> (or a friendlier claim)
<code>roles</code>	<code>Vec<String></code>	<code>roles</code>
<code>claims</code>	<code>HashMap<String, serde_json::Value></code>	every decoded claim

Its helpers — `has_role(r)`, `has_any_role(&[...])`, `has_authority(a)` (matches a role or a fine-grained permission/scope), and `Authentication::anonymous()` — cover the common checks. Lumen's unit test asserts the round-trip directly:

Rust

```
#[tokio::test]
async fn mint_then_verify_roundtrips_claims() {
    let token = mint_token("u-alice", &[CUSTOMER_ROLE]);
    let auth: Authentication = build_verifier().verify(&token).await.unwrap();
    assert_eq!(auth.principal, "u-alice");
    assert!(auth.has_role(CUSTOMER_ROLE));
}
```

A tampered token (`"not.a.jwt"`) or one signed with the wrong key is rejected with `SecurityError::Verification` — the two negative tests in `security.rs` prove it.

Production swap. Move to a real identity provider by replacing `build_verifier` with `firefly::security::JwksVerifier`, pointed at your IdP's JWKS URI (RS256, `kid` cache, `iss` / `aud` validation, `exp` required). The `Verifier` port is identical, so `security_layers` — and every handler — is untouched. That is the "swap the adapter, keep the code" promise applied to identity.

URL authorization – the FilterChain

`JwtService` answers *who is this caller?*; the `FilterChain` answers *may they do this?* It matches request paths against rules in declaration order — first match wins — and renders a 401 (no/invalid credential) or 403 (authenticated but under-privileged) as an RFC 9457 problem. Lumen composes the bearer layer and the chain in one place:

Rust

```
/// Builds the BearerLayer + FilterChain that protect the service.
///
/// | Route | Rule |
/// |-----|-----|
/// | `GET /api/v1/wallets/:id` | permit (public read) |
/// | `GET /actuator/*` | permit (management) |
/// | `POST /api/v1/wallets` | require `CUSTOMER` |
/// | `POST /api/v1/wallets/:id/deposit` / `withdraw` | require `CUSTOMER` |
/// | `POST /api/v1/transfers` | require `CUSTOMER` |
pub fn security_layers() -> (BearerLayer, FilterChain) {
    // `allow_anonymous` lets an unauthenticated request reach the chain; the
    // chain (not the bearer layer) then decides – a 401 on a `require` route
    // without a valid token, a pass on a permitted route.
    let bearer =
        BearerLayer::new(BearerConfig::new(build_verifier()).allow_anonymous(true));
    let chain = FilterChain::new()
        .permit_method("GET", "/api/v1/wallets")
        .permit("/actuator/")
        .require("/api/v1/wallets", &[CUSTOMER_ROLE])
        .require("/api/v1/transfers", &[CUSTOMER_ROLE])
        .any_request_permit();
    (bearer, chain)
}
```

Two design choices are worth dwelling on:

- `allow_anonymous(true)` on the bearer layer. With it set, a request with no `Authorization` header is *not* rejected at the bearer layer — it reaches the chain

carrying an anonymous `Authentication`. That keeps a single decision-maker: the `FilterChain` decides every route. A public `GET` passes; a `require` route with no valid token becomes a 401. Without `allow_anonymous`, the bearer layer would reject anonymous traffic before the chain could permit the public reads.

- **Order matters.** `permit_method("GET", "/api/v1/wallets")` and `permit("/actuator/")` come *first*, so the public reads and the management surface are decided before the broad `require("/api/v1/wallets", ...)` could catch them. `any_request_permit()` re-opens the unmatched tail.

⚠ WARNING

Once any rule is declared, a `FilterChain` is **fail-closed**: a request matching no rule is rejected with 403 (Spring Security 6 semantics). Re-open the unmatched tail explicitly with `any_request_permit()` / `any_request_authenticated()` / `any_request_deny()`. A chain with *no* rules is a no-op and passes everything (never a surprise blanket lockout).

Layering it onto the router

The chain is attached to the `WebStack` at build time, and the bearer layer is applied around the whole router so the chain sees a populated `Authentication`. This is the relevant slice of `LumenApp::router` and `build_app` in `src/web.rs`:

Rust

```
// in build_app():
let (_bearer, chain) = crate::security::security_layers();
let web = WebStack::new(firefly::starter_web::CoreConfig {
    app_name: APP_NAME.into(),
    app_version: VERSION.into(),
    ..Default::default()
})
.with_security(chain);

// in LumenApp::router():
let (bearer, _chain) = crate::security::security_layers();
// The WebStack carries the FilterChain (set in build_app); layer the
// bearer auth on the outside so the chain sees a populated Authentication.
self.web.apply_middleware(routes).layer(bearer)
```

`WebStack::with_security(chain)` stores the chain; `WebStack::apply_middleware` layers it *inside* the inherited correlation / security-headers / CORS edge, so even a 401 response carries those headers and a correlation id. The bearer layer goes on the outside (axum runs the last-added layer first): authenticate, then authorize.

Proving it end to end

The HTTP suite (`tests/http.rs`) drives the fully-wired router with `tower::oneshot` and asserts the security behavior directly. A mutation with no token is a 401 problem; a malformed body on an authenticated request is a 422 problem:

Rust

```
#[tokio::test]
async fn missing_token_is_401_problem_on_mutations() {
    let res = build_router()
        .await
        .oneshot(post(
            "/api/v1/wallets",
            serde_json::json!({ "owner": "mallory", "openingBalance": 10 }),
            false, // no Authorization header
        ))
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::UNAUTHORIZED);
    assert!(content_type(&res).contains("application/problem+json"));
}
```

The test helper builds the `Authorization` header from `mint_token`, so an authenticated request is just `post(path, body, true)`:

Rust

```
fn bearer() -> String {
    format!("Bearer {}", mint_token("u-alice", &[CUSTOMER_ROLE]))
}
```

Every other test in the suite — open, get, deposit/withdraw, transfer — runs as the minted `CUSTOMER`, so authentication is exercised on the happy path too.

The rest of firefly-security (Lumen's growth room)

Lumen uses the symmetric-key fast path. The same crate carries the production surface you reach for as a real wallet service matures:

- **JWKS verification.** `JwksVerifier` is a drop-in `Verifier` for RS256 tokens from an external IdP (Keycloak, Auth0, Cognito): `kid` cache, `iss` / `aud` checks, `exp` required, and the same `sub` / `roles` / `permissions` claim mapping.
- **Method guards.** For per-handler checks (the analog of `@PreAuthorize`), the `guards` module composes typed predicates: `has_role("CUSTOMER").or(has_authority("wallet:approve"))`, then `guard.authorize(Some(&auth))?` — 401 with no principal, 403 if the predicate is false.
- **Role hierarchy.** `RoleHierarchy::new()` parses `"ADMIN > CUSTOMER"` specs; attach it with `chain.with_role_hierarchy(..)` so granting `ADMIN` implies `CUSTOMER`.
- **Pattern rules.** Alongside the prefix rules Lumen uses, the chain offers an fnmatch-style glob DSL — `permit_pattern("/public/**")`, `require_pattern("/api/admin/**", &["ADMIN"])`, `require_authority("/api/reports/**", &["reports:read"])`, `authenticated("/api/**")`.
- **Sessions.** For browser flows where logout must mean logout, the `firefly-session` crate adds a `SessionLayer` over a `SessionStore` (`MemorySessionStore` for dev, a Redis-backed store for scale). A handler pulls the request's `Session` with the `SessionExt` extractor and calls `session.rotate_id()` after login (session-fixation defense), `session.set_attribute("user_id", id)`, and `session.invalidate()` on logout.
- **OAuth2.** The `oauth2` module covers both sides: `ClientRegistration` + `OAuth2LoginHandler` for the authorization-code login flow (state + nonce + PKCE S256, OIDC id-token validation), and an `AuthorizationServer` that issues tokens for `client_credentials` / `refresh_token`.

- **CSRF & passwords.** `CsrfLayer` implements the double-submit-cookie pattern for cookie-session flows; `BcryptPasswordEncoder` (default work factor 12) hashes credentials. The `$2b$` hashes interchange with the `firefly-idp-internal-db` adapter and every other port.

Rust

```
use firefly_security::BcryptPasswordEncoder;

let enc = BcryptPasswordEncoder::new(); // work factor 12 (the default)
let hash = enc.hash("s3cret").unwrap();
assert!(enc.verify("s3cret", &hash).unwrap());
assert!(!enc.verify("wrong", &hash).unwrap());
```

What changed in Lumen

This chapter closed Lumen's open front door without adding a dependency or a line of business logic to the handlers:

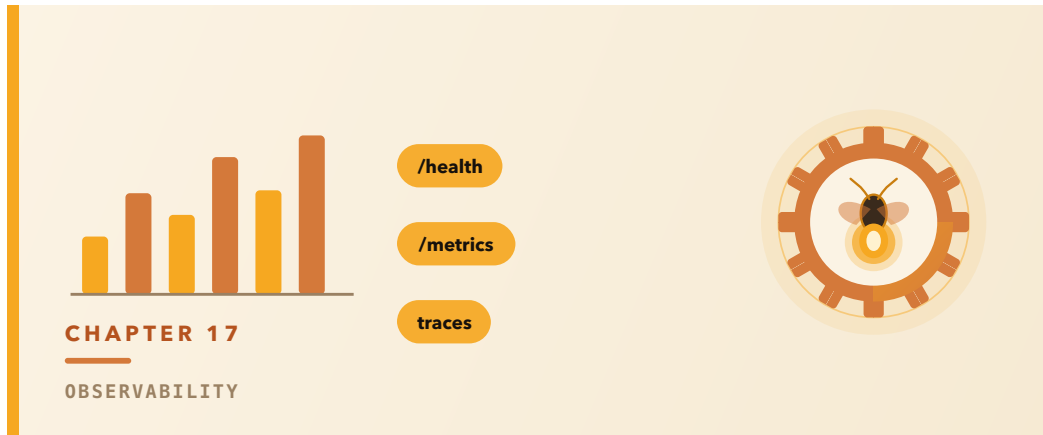
- A single HS256 `JwtService` (`src/security.rs`) both mints the demo tokens and verifies incoming ones; `encode` auto-stamps a one-hour `exp`, and a token without `exp` is rejected at decode time.
- `build_verifier` turns the service into a `Verifier` via `VerifierFn`, mapping `sub` → principal and `roles` → roles onto an `Authentication`, with a bad token surfacing as `SecurityError::Verification` → a 401 problem.
- `security_layers` composes a `BearerLayer` (with `allow_anonymous(true)`) and a path-ordered RBAC `FilterChain`: public `GET /api/v1/wallets/:id`, public `/actuator/*`, and `CUSTOMER`-only `POST /api/v1/wallets`, `/deposit`, `/withdraw`, and `/transfers`.
- `WebStack::with_security` + `apply_middleware` attach the chain inside the correlation/headers edge; `LumenApp::router` layers the bearer auth on the outside.

- The HTTP suite proves the contract: an unauthenticated mutation is a 401 problem, the happy paths run as a minted `CUSTOMER`, and the unit tests round-trip and reject tokens.

Exercises

1. **Add an ADMIN-only route.** Give Lumen a hypothetical `GET /api/v1/wallets` collection list and protect it with `require_pattern` so only an `ADMIN` may list every wallet, while `CUSTOMER` keeps access to the single-wallet read. Mint an `ADMIN` token in a test and assert a `CUSTOMER` token gets a 403.
2. **Role hierarchy.** Introduce a `SUPER` role that implies `CUSTOMER`. Build a `RoleHierarchy` from `"SUPER > CUSTOMER"`, attach it with `chain.with_role_hierarchy(..)`, mint a `SUPER`-only token, and assert it passes the `require("/api/v1/wallets", &["CUSTOMER"])` rule.
3. **Swap in JWKS.** Sketch a `build_verifier_jwks()` that returns a `JwksVerifier::new("https://idp.example.com/.well-known/jwks.json")` and confirm (by reading the `Verifier` trait) that `security_layers` needs no other change. Why does the rest of Lumen not care which verifier it got?
4. **Expiry.** Lower the token lifetime with `JwtService::new(KEY).expiration_seconds(1)`, mint a token, wait two seconds, and assert the verifier now returns `SecurityError::Verification`.

A secure service is only trustworthy if you can *see* what it is doing. The next chapter gives Lumen eyes and ears — structured logs, health, metrics, and the admin dashboard. Continue to [Observability](#).



CHAPTER 17

Observability & Health

In Chapter 14 you locked Lumen's mutating routes behind a JWT and a role filter. The service is now safe — but it is still a black box. When a deposit is slow in production you need to know *where* the time went; when the broker degrades you want a dashboard that turns red before your pager does; when an auditor asks why a transfer was rejected you need a structured log line with the context to reconstruct the decision.

By the end of this chapter Lumen will expose an **actuator admin surface** on a separate port — health, info, metrics, loggers, scheduled tasks — feed it an **info contributor** describing the build, emit **structured logs** with a correlation id on every request, and (as the next step it grows into) surface all of it in the embedded **admin dashboard** with its fifteen views, including the new **Beans** view. Lumen is **observable by default**: `WebStack::new` already wired the logging layer, the health composite, the metric registry, and the request-metrics middleware. This chapter turns those defaults into an admin surface and explains what each piece reports.

 SPRING PARITY

This is Spring Boot Actuator plus the `firefly-otel-spring-boot-starter`: structured logs with MDC-style correlation, `/actuator/health` + `/actuator/metrics` + `/actuator/loggers`, and a Spring-Boot-Admin-style dashboard. The endpoint paths and the `configuredLevel` / `effectiveLevel` logger shape match Spring's, so Actuator-aware tooling works unchanged.

The admin surface in main.rs

Lumen serves the public API on `127.0.0.1:8080` and the actuator on `127.0.0.1:8081` — a separate listener, so `/actuator/*` never leaks onto the public network. The `WebStack` `Deref`s to its `Core`, so `actuator_router` is reachable directly. This is the relevant slice of `src/main.rs`:

Rust

```
use firefly::starter_core::InfoContributor;

let api = app.router();
let contributor: InfoContributor = Box::new(|| {
    let mut info = serde_json::Map::new();
    info.insert(
        "sample".into(),
        serde_json::json!({ "name": APP_NAME, "store": "in-memory", "eventBus":
        "in-memory" }),
    );
    info
});
let admin = app.web.actuator_router(vec![contributor]);
```

`actuator_router(info_contributors)` returns an axum `Router` carrying the full management surface. Lumen mounts it on the admin listener in its lifecycle wiring (the production chapter covers that end to end); for now, the two routers are served on two ports.

The actuator endpoints

`actuator_router` exposes the management endpoints below. Lumen's reach the admin port at `http://127.0.0.1:8081/actuator/*`:

Endpoint	Returns
<code>/actuator/health</code>	the composite rollup (+ liveness/readiness probes)
<code>/actuator/info</code>	app metadata + your info contributors
<code>/actuator/metrics</code>	Micrometer-parity meter listing
<code>/actuator/metrics/:name</code>	one meter's detail
<code>/actuator/prometheus</code>	the Prometheus text-exposition scrape target
<code>/actuator/env</code>	masked, origin-attributed property sources
<code>/actuator/scheduledtasks</code>	scheduled-task descriptors
<code>/actuator/version</code>	the running version
<code>/actuator/loggers[:name]</code>	runtime log-level control
<code>/actuator/threaddump</code>	a thread/task dump
<code>/actuator/httpexchanges</code>	recent HTTP exchanges (when wired)
<code>/actuator/caches[:name]</code>	cache listing + eviction (when wired)
<code>/actuator/refresh</code>	reload config (the <code>ReloadableConfig</code> hook)

The info contributor

An `InfoContributor` is a boxed closure returning a `serde_json::Map`; each one adds a section to `/actuator/info`. Lumen's reports its store and event-bus kind, so an operator hitting `/actuator/info` sees that this instance is running the in-memory infrastructure:

JSONC

```
// GET /actuator/info
{
  "app": { "name": "lumen", "version": "26.6.3" },
  "sample": { "name": "lumen", "store": "in-memory", "eventBus": "in-memory" }
}
```

The `app` block is filled from the `CoreConfig.app_name` / `app_version` Lumen set in Chapter 3; the `sample` block is the contributor above.

Health

A health `Indicator` is an async probe returning a `HealthResult` (status + message + details); a `Composite` aggregates them into the canonical rollup — `DOWN` if any indicator is `DOWN`, else `DEGRADED` if any is `DEGRADED`, else `UP`. The `Core` already carries a `HealthComposite`; bridge an observability indicator onto it with `core.add_observability_indicator(..)`. A real Lumen deployment would add a broker-liveness indicator and a store probe:

Rust

```
use firefly_observability::{HealthResult, IndicatorFn};

// Wire a probe onto the WebStack's composite (it Derefs to Core):
app.web.add_observability_indicator(IndicatorFn::new("event-bus", || async {
    HealthResult::up() // a real probe would ping the broker
}));
```

`/actuator/health/liveness` and `/actuator/health/readiness` are the sub-paths your orchestrator's probes hit — separate so an in-flight migration that fails readiness need not trigger a liveness restart.

Request metrics – already on

`WebStack::new` turns on the request-metrics middleware by default (it fills in a `RequestMetricsConfig` if the caller left one unset). For every request it records the labeled `http_server_requests_seconds` timer plus a companion `_max` gauge, tagged `method` / templated `uri` (the axum matched path, so `/api/v1/wallets/:id` not the concrete id) / `status` / `outcome` / `exception`, and bridges them onto the actuator `MetricRegistry`. A clean request carries `exception="None"`. So the moment Lumen booted in Chapter 6 it was already emitting per-route latency; this chapter just exposes it at `/actuator/metrics`.

The Micrometer dot-case names map straight to Prometheus (`http_server_requests_seconds`), so pointing a Prometheus `scrape_config` at `/actuator/prometheus` lights up Grafana with no extra code.

Structured logging and correlation

`WebStack` installs a `tracing` layer that formats every event as one structured line and enriches it with the request's correlation id (set by the correlation middleware, on by default). Lumen calls `init_logging` once at startup, best-effort so a test harness that already owns the global subscriber does not panic:

Rust

```
let app = build_app().await;
// Best-effort: a test harness may already own the global subscriber.
let _ = app.web.init_logging();
```

After that, plain `tracing` macros produce enriched lines; fields recorded on an enclosing span merge into each event. The field names (`time`, `level`, `msg`, `service`, `correlationId`) are identical across the ports, so one log pipeline parses every Firefly service.

Because the correlation id lives in a task-local scope, it flows automatically into every log line, every event Lumen's ledger publishes (`Event::new` stamps it), and every outbound client call (the W3C `traceparent` is propagated). A request that opens a wallet, publishes `WalletOpened`, and projects it into the read model stitches together under one id with no manual threading.

SPRING PARITY

`init_logging` is the analog of Spring Boot's Logback + MDC setup; the task-local correlation id plays the role of the MDC `traceId` / `spanId`. PyFly's `get_logger` + `TransactionIdMiddleware` and Go's `CorrelationLayer` do the same — the wire field names are shared.

Tracing / OpenTelemetry

`firefly-observability` exposes the building blocks that compose with the `tracing` ecosystem and propagates W3C trace-context (`traceparent` / `tracestate`) on the HTTP edges and outbound client calls. The OpenTelemetry SDK wiring — exporters, sampling, resource attributes — is left to your `main.rs`, where you add your preferred OTel `tracing` layer alongside the correlation layer. Lumen ships without an exporter (it is teaching code with no external collector), but the trace-context propagation is already on the edges.

The admin dashboard

The actuator surface is JSON for machines. `firefly-admin` mounts a **Spring-Boot-Admin**-style single-page dashboard — vendored, no `npm` build — that ties health, metrics, loggers, beans, mappings, caches, CQRS handlers, sagas, traces, and a live log tail into one pane of glass with Server-Sent-Event streams. It is the next surface Lumen grows into; you enable the facade's `admin` feature and wire it from the `Core` accessors.

`mount(AdminConfig, AdminDeps)` returns the dashboard router. `AdminDeps::new` takes the required collaborators; the rest are optional fields you fill in with struct-update syntax. A Lumen wiring drawing on its `Core` looks like:

Rust

```

use std::sync::Arc;
use firefly::admin::{mount, AdminConfig, AdminDeps, LogBuffer, TraceBuffer};

let traces = Arc::new(TraceBuffer::new());
let logs = LogBuffer::new();

// `AdminDeps::new` takes the required collaborators; the optional fields that
// back the remaining views are set afterwards with struct-update syntax.
let deps = AdminDeps {
    scheduler: Some(app.web.scheduler()), // → Scheduled Tasks view
    bus: Some(app.web.cqrs_bus()),         // → CQRS view
    container: Some(container),           // → Beans view
    ..AdminDeps::new(
        APP_NAME,
        VERSION,
        app.web.health_composite(), // Arc<HealthComposite>
        app.web.metric_registry(), // Arc<MetricRegistry>
        Arc::clone(&traces),
        logs,
    )
};

let dashboard = mount(AdminConfig::default(), deps);

```

The dashboard auto-discovers what those collaborators expose and renders it in **fifteen built-in views**, grouped in the sidebar:

Section	Views
Dashboard	Overview, Health
Application	Beans, Environment, Configuration, Loggers
Monitoring	Metrics, Scheduled Tasks, HTTP Traces, Log Viewer
Infrastructure	Mappings, Caches, CQRS, Transactions
Fleet	Instances (server mode)

Each view is backed by a `/admin/api/*` JSON endpoint; the SSE streams (`/admin/api/sse/{health,metrics,traces,logfile,beans,runtime,server}`) push updates without your code polling. Admin and actuator paths are excluded from trace capture so they never pollute the trace panel.

The Beans view

The newest view is **Beans** — the dashboard's window onto the dependency-injection container. When you pass `with_container(...)`, the dashboard serves:

Endpoint	Returns
<code>GET /admin/api/beans</code>	every registered bean with its stereotype and scope
<code>GET /admin/api/beans/graph</code>	the dependency graph between beans
<code>GET /admin/api/beans/:name</code>	one bean's detail (type, scope, dependencies)
<code>GET /admin/api/sse/beans</code>	a live snapshot at each refresh interval

The Overview view also rolls up a `beans` block (`{ total, stereotypes }`) and a `wiring` block (live CQRS-handler and scheduled-task counts) drawn from the same container, so the landing page shows how much the service is wired without opening the full Beans view. Because Lumen wires its composition root explicitly rather than

scanning a container, the Beans view is sparse for Lumen itself — but the moment you adopt `#[derive(Component)]` + `firefly::scan` (Chapter 4), the graph fills in. Without a container the endpoints degrade gracefully to an empty `{ total: 0 }` block.

🌿 SPRING PARITY

The Beans view is Spring Boot Actuator's `/beans` endpoint and Spring Boot Admin's Beans panel. `firefly-admin` maps to Spring Boot Admin overall: server mode (`AdminServerConfig`) replaces `@EnableAdminServer`, the `AdminClient` self-registration replaces `spring.boot.admin.client.url`, and the vanilla-JS SPA + SSE streams replace the Vaadin/React frontend and its WebSocket notifications. The Python and Go ports expose the same fifteen views.

Custom views

To add your own sidebar view, implement the `AdminView` trait and push it onto `AdminDeps::views`; the dashboard lists it under `/admin/api/views/{:id}`. A Lumen "Treasury" view might surface the total custody balance across all wallets, queried from the read model.

What changed in Lumen

This chapter turned Lumen's always-on observability defaults into a working admin surface:

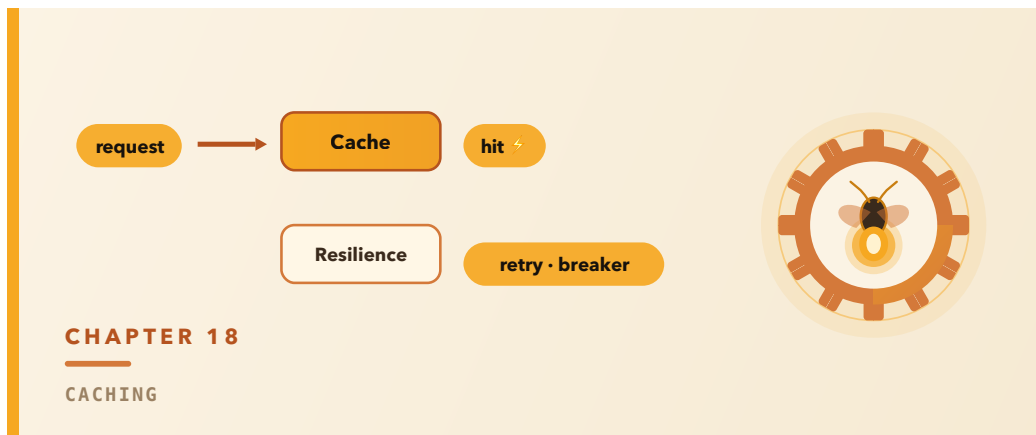
- `main.rs` builds an `InfoContributor` describing the in-memory store and event bus, and serves `app.web.actuator_router(vec![contributor])` on the admin port — `/actuator/health`, `/info`, `/metrics`, `/loggers`, `/scheduledtasks`, and the rest.

- **init_logging** (best-effort, so the test harness can own the subscriber) switches on structured, correlation-enriched logging; the correlation id flows into every log line, published event, and outbound call automatically.
- The **request-metrics** middleware — on since `WebStack::new` — records `http_server_requests_seconds` per templated route, now exposed at `/actuator/metrics` and `/actuator/prometheus`.
- The **admin dashboard** (the `firefly-admin` step Lumen grows into) ties it all together in fifteen views, including the new **Beans** view backed by the DI container, with live SSE streams.

Exercises

1. **Reach the actuator.** Run `cargo run --bin lumen`, then `curl http://127.0.0.1:8081/actuator/info` and confirm the `sample` block reports the in-memory store. Hit `/actuator/health` and `/actuator/metrics`.
2. **Add a health indicator.** Wire a `IndicatorFn::new("read-model", ..)` onto the composite with `add_observability_indicator` that returns `UP` when the read model holds at least one wallet view, and watch it appear under `/actuator/health`.
3. **A Lumen metric.** Record a counter — e.g. `lumen_transfers_total` — on the `metric_registry()` each time the transfer saga completes, and verify it appears at `/actuator/metrics`. (Recall the housekeeping heartbeat in Chapter 16 keeps an `AtomicU64` you could surface the same way.)
4. **Mount the dashboard.** Enable the facade's `admin` feature, mount the dashboard with `with_container`, and open `/admin` — note how the Beans view is sparse for Lumen's explicit root, then convert one collaborator to `#[derive(Component)]` and watch it appear.

With Lumen observable, the next chapter adds background work and the path to outbound notifications. Continue to [Scheduling & Notifications](#).



CHAPTER 18

Caching & Resilience

By the end of this chapter, you will understand exactly how Lumen's `GET /api/v1/wallets/:id` serves a wallet view from a 30-second cache — and why every deposit, withdrawal, and transfer *invalidates* that cache so a read after a write never lies. The cache itself was switched on back in `CQRS` with one annotation and one middleware; this chapter opens it up: the unified cache port behind it, the backends you can swap in, and the `firefly-resilience` decorators that protect the slow call a cache miss falls through to.

`firefly-cache` exposes a single port — `Adapter` — and ships in-process, no-op, and fallback implementations plus a typed memoization wrapper. Code against the port and select the backend (in-memory, Redis, Postgres) at wiring time. All of it reaches Lumen through the one `firefly` facade, as `firefly::cache` and `firefly::resilience`.

 SPRING PARITY

`Adapter` is the `Cache` abstraction; `Typed::get_or_set` is `@Cacheable`; the query-side `#[firefly(cache_ttl = "30s")]` is the declarative read-through Lumen uses. The resilience decorators are Resilience4j (circuit breaker, rate limiter, bulkhead, timeout) composed for reactive Rust.

Lumen's read-side cache

Lumen's `GetWallet` query carries its caching policy as a declaration sitting right next to the type — no cache code in the handler:

Rust

```
/// `GET /api/v1/wallets/:id` query. `#[firefly(cache_ttl = "30s")]` is
/// reflected
/// on the generated `Message::cache_ttl`, so a `QueryCache` memoises reads for
/// 30 seconds.
#[derive(Debug, Clone, Default, Serialize, Deserialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetWallet {
    pub id: String,
}
```

The `#[derive(Query)]` macro reads that attribute and emits a `cache_ttl()` on the generated `Message` impl — a fact the test pins so it can never silently disappear:

Rust

```
#[test]
fn get_wallet_carries_cache_ttl() {
    assert!(GetWallet::default().cache_ttl().is_some());
}
```

The TTL is inert until something *honors* it. That something is the `QueryCache` middleware, installed on the bus in Lumen's composition root (`build_app` in `src/web.rs`):

Rust

```
use firefly::cqrs::QueryCache;

// Read-side caching on the bus (honours GetWallet's 30s cache_ttl). The
// handle is kept so a mutation can invalidate it for read-after-write.
let query_cache = QueryCache::new();
bus.use_middleware(query_cache.middleware());
// Validation middleware enforces the `#[firefly(validate)]` checks.
bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());
```

`QueryCache::new()` builds the cache; `query_cache.middleware()` returns the bus middleware that consults it. When a `GetWallet` flows through the bus, the middleware checks the cache: a hit returns the memoized `WalletView` without ever reaching the handler; a miss runs the handler, stores the result under the query's key for 30 seconds, and returns it. The `query_cache` handle is kept on `LumenApp` precisely so the write side can reach back into it.

Why the handle, not just the middleware? The middleware reads and fills the cache; only a holder of the handle can invalidate it. Lumen keeps `query_cache` on `LumenApp` and passes a clone into the controller state, so the mutating handlers can drop stale entries the moment they change a balance.

Read-after-write invalidation

A 30-second TTL is great for a read-heavy view and a disaster for correctness if you never invalidate. Deposit `$2.50`, then read the balance within 30 seconds, and a naive cache would happily serve the *old* number. Lumen avoids that by invalidating the whole `GetWallet` family after every mutation. From the controller in `src/web.rs`:

Rust

```
#[post("/wallets/:id/deposit")]
async fn deposit(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    Json(body): Json<AmountBody>,
) -> WebResult<Json<WalletView>> {
    let cmd = Deposit { wallet_id: id, amount: body.amount };
    let view: WalletView = api.bus.send(cmd).await.map_err(cqrs_to_web)?;
    api.query_cache.invalidate_type::<GetWallet>();
    Ok(Json(view))
}
```

`invalidate_type::<GetWallet>()` drops every cached `GetWallet` entry, so the next read re-runs the handler and reflects the write. The withdraw handler does the same, and so does the transfer endpoint from [Sagas](#) — a transfer changes *two* balances, so it must invalidate the family too:

Rust

```
// In the transfer handler – a transfer touches both wallets' views.
api.query_cache.invalidate_type::<GetWallet>();
```

The end-to-end HTTP test proves the loop closes: deposit then withdraw, then read back, and the cached view reflects both writes rather than a stale balance.

Rust

```
// after a deposit(+250) and a withdraw(-50) on an opening balance of 100:
let view: WalletView = body_json(res).await;
assert_eq!(view.balance, 300); // read-after-write is honest
assert_eq!(view.version, 3);
```

SPRING PARITY

This is `@Cacheable` on the query plus `@CacheEvict` on the command, the read-through/evict pattern you know from Spring's cache abstraction. The declaration lives on the message (`#[firefly(cache_ttl)]`), the eviction is an explicit `invalidate_type` at the write boundary, and the backing store is the same swappable `Adapter` port every other consumer uses.

The cache port

Everything above runs on `MemoryAdapter` by default, but the `QueryCache` — like every other consumer (session store, OAuth2 token store) — depends on the abstract `Adapter` port, never on a concrete client. That is what lets you move Lumen's cache to Redis without touching a handler. The port:

Rust

```
#[async_trait]
pub trait Adapter: Send + Sync {
    async fn get(&self, key: &str) -> Result<Vec<u8>, CacheError>;
    async fn set(&self, key: &str, value: &[u8], ttl: Option<Duration>) ->
    Result<(), CacheError>;
    async fn delete(&self, key: &str) -> Result<(), CacheError>;
    async fn clear(&self) -> Result<(), CacheError>;
    async fn set_if_absent(&self, key, value, ttl) -> Result<bool, CacheError>;
    async fn exists(&self, key: &str) -> Result<bool, CacheError>;
    async fn delete_prefix(&self, prefix: &str) -> Result<u64, CacheError>;
    async fn health_check(&self) -> Result<(), CacheError>;
}
```

A miss is `CacheError::NotFound`; `ttl: None` (or zero) means no expiry.

Implementation	Backing	Use
<code>MemoryAdapter</code>	<code>HashMap</code> + <code>RwLock</code>	in-process, TTL-aware, the default
<code>NoOpAdapter</code>	none	tests / disabled-cache configs
<code>FallbackAdapter { primary, secondary }</code>	composite	primary-then-secondary, writes to both
<code>RedisAdapter (firefly-cache-redis)</code>	Redis (RESP)	distributed cache

Typed memoization

Lumen's `QueryCache` keys and serializes for you, but when you cache something *outside* the query bus — say, a wallet's risk score fetched from an external service — the `Typed<T>` wrapper is the primitive. It serializes values as `serde_json` bytes (wire-compatible with the other ports) and gives you `get_or_set`: consult the cache, call the loader on a miss, persist, and return the value. A caching error never masks a successful loader result:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly::cache::{MemoryAdapter, Typed};

#[derive(serde::Serialize, serde::Deserialize)]
struct WalletView { id: String }

#[tokio::main(flavor = "current_thread")]
async fn main() -> Result<(), firefly::cache::CacheError> {
    let cache = Arc::new(MemoryAdapter::new());
    let typed: Typed<WalletView> = Typed::new(cache);

    let view = typed
        .get_or_set("wallet:wlt_alice", Some(Duration::from_secs(60)), || async
    {
        // loaded from the ledger / read model on a miss
        Ok(WalletView { id: "wlt_alice".into() })
    })
    .await?;
    assert_eq!(view.id, "wlt_alice");
    Ok(())
}

```

Choosing and composing backends

`Core::new` (and therefore `WebStack`, which Lumen builds on) uses `MemoryAdapter` by default; pass an `Arc<dyn cache::Adapter>` in `CoreConfig.cache` to swap it. For high availability, compose Redis with an in-process fallback so a Redis blip degrades to local caching instead of failing:

Rust

```

use std::sync::Arc;
use firefly::cache::{FallbackAdapter, MemoryAdapter};

// Tries Redis first; on a transport error or miss, falls through to memory
// and writes to both.
let cache = FallbackAdapter::new(redis_adapter, Arc::new(MemoryAdapter::new()));

```

Because everything downstream depends on the port, swapping the backend changes one constructor — Lumen's handlers, the `QueryCache`, and the session store are untouched. A single-process Lumen keeps the default in-memory cache; a multi-node deployment swaps in `RedisAdapter` so a `GetWallet` cached on one node is seen on the next, and so an `invalidate_type` on any node clears the shared entry.

The in-memory → real-infra swap. This mirrors the event store and broker swap you have already seen: develop and test against the in-memory adapter, wire the distributed backend in production via `CoreConfig`. The teaching baseline stays a no-infra `cargo run`; the production path is one line of wiring.

Resilience decorators

`firefly-resilience` guards the calls a cache miss falls through to — and any outbound call, like the Payments settlement from `HTTP Clients`. Four primitives, composable into a `Chain`:

Decorator	Guards against	Error on trip
<code>CircuitBreaker</code>	cascading failure of a slow / failing dep	<code>ResilienceError::CircuitOpen</code>
<code>RateLimiter</code>	outbound rate overrun (token bucket)	<code>ResilienceError::RateLimited</code>
<code>Bulkhead</code>	resource exhaustion from runaway concurrency	<code>ResilienceError::BulkheadFull</code>
<code>Timeout</code>	stuck calls	<code>ResilienceError::Timeout</code>

A `Chain` composes them into a single guarded call:

Rust

```
use std::{sync::Arc, time::Duration};
use firefly::resilience::{Bulkhead, Chain, CircuitBreaker, CircuitConfig,
Timeout};

# async fn ex() -> Result<(), firefly::resilience::ResilienceError> {
let breaker = Arc::new(CircuitBreaker::new(CircuitConfig::default()));

let guarded = Chain::new()
    .with(Timeout::new(Duration::from_secs(2))) // per-call deadline
    (outermost)
    .with(breaker) // open the circuit on
    repeated failures
    .with(Bulkhead::new(20)); // cap concurrent in-flight
    calls

guarded.execute(|| async {
    // the protected operation – a cache loader, an upstream call, ...
    Ok(())
}).await?;
# Ok(())
# }
```

Each decorator also stands alone:

Rust

```
use std::time::Duration;
use firefly::resilience::{Bulkhead, CircuitBreaker, CircuitConfig, RateLimiter,
Timeout};

let cb = CircuitBreaker::new(CircuitConfig::default());
let _ = cb.execute(|| async { settle().await }).await;

let rl = RateLimiter::new(100.0, 200);           // 100 rps, burst 200
let _ = rl.execute(|| async { call().await }).await;

let bh = Bulkhead::new(20);
let _ = bh.try_execute(|| async { call().await }).await; // non-blocking;
BulkheadFull if full

let to = Timeout::new(Duration::from_secs(2));
let _ = to.execute(|| async { slow_call().await }).await;
```

🌱 SPRING PARITY

`Chain` is the composed Resilience4j decorator stack; the ordering (timeout outermost, breaker, bulkhead innermost) is the same call-wrap order you tune in `resilience4j.yaml`. `CircuitBreaker::execute` returns the operation's value (so a guarded read still hands you the `WalletView`), while `Chain::execute` is for guarded operations whose value you discard.

A complete read path

The pieces fit together as a cache-aside read protected by a circuit breaker — exactly the shape a multi-node Lumen would use to serve a wallet view from Redis, repairing from the read model (or the event stream) on a miss while a breaker protects that repair:

Rust

```

use std::sync::Arc;
use std::time::Duration;
use firefly::cache::{MemoryAdapter, Typed};
use firefly::resilience::{CircuitBreaker, CircuitConfig};

let typed: Typed<WalletView> = Typed::new(Arc::new(MemoryAdapter::new()));
let breaker = CircuitBreaker::new(CircuitConfig::default());

let view = typed
    .get_or_set("wallet:wlt_alice", Some(Duration::from_secs(30)), || async {
        // the loader is what the circuit protects: the read model / event
        store.
            breaker.execute(|| async { load_wallet_view("wlt_alice").await }).await
            .map_err(|e| firefly::cache::CacheError::Backend(e.to_string()))
    })
    .await?;

```

You now have a fast, resilient read path — the same one Lumen's declarative `#[firefly(cache_ttl = "30s")]` gives you for free on the query bus.

What changed in Lumen

- We opened up the read-side cache Lumen has used since CQRS: `#[firefly(cache_ttl = "30s")]` on `GetWallet` is honored by the `QueryCache` middleware that `build_app` installs on the bus with `bus.use_middleware(query_cache.middleware())`.
- We saw why the `query_cache` handle lives on `LumenApp`: so every mutating handler — deposit, withdraw, **and** transfer — can call `invalidate_type::<GetWallet>()` and keep read-after-write honest within the 30-second TTL.
- We traced the cache down to its `Adapter` port, the swappable backends (`MemoryAdapter` default → `RedisAdapter` for a multi-node deployment via

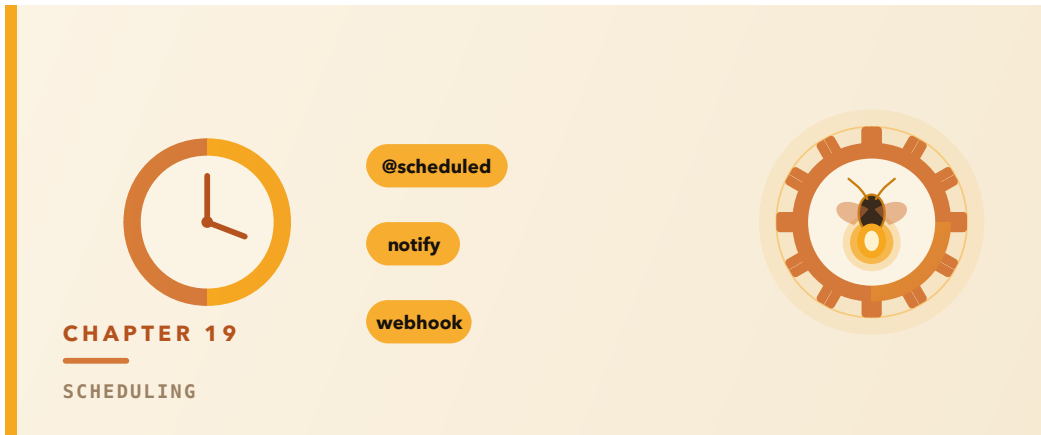
`CoreConfig.cache`), the `Typed<T>::get_or_set` memoization primitive, and the `FallbackAdapter` for Redis-with-local-fallback.

- We covered the `firefly-resilience` decorators (`CircuitBreaker`, `RateLimiter`, `Bulkhead`, `Timeout`) and the `Chain` that composes them — the guard that protects both a cache loader and the outbound Payments call from [HTTP Clients](#).

Exercises

1. **Prove the TTL.** Write a test that opens a wallet, reads it (priming the cache), then deposits *directly through the ledger* (bypassing the controller, so no invalidation runs) and reads again within 30 seconds. Assert you still see the *old* balance — demonstrating the TTL is real — then call `query_cache.invalidate_type::<GetWallet>()` and assert the read now reflects the deposit.
2. **Swap in a fallback adapter.** Build a `FallbackAdapter` whose primary always errors on `get/set` and whose secondary is a `MemoryAdapter`. Wire it into `CoreConfig.cache`, run the deposit/withdraw/read flow, and assert correctness is unaffected — the cache degrades to the in-process layer instead of failing.
3. **Guard a loader with a `Chain`.** Wrap a deliberately slow loader in a `Chain::new().with(Timeout::new(...))` and assert that a loader exceeding the deadline surfaces `ResilienceError::Timeout` rather than hanging. Then add a `CircuitBreaker` to the chain, trip it with repeated failures, and assert the next call fails fast with `ResilienceError::CircuitOpen`.

The remaining chapters fold Lumen back together through the declarative-macro lens, then cover shipping it. Continue to [Declarative Services with Macros](#).



CHAPTER 19

Scheduling, Notifications & Webhooks

A real wallet service does more than answer HTTP. It sweeps abandoned wallets overnight, recomputes interest, retries stuck transfers, and — the part a customer notices — emails a daily statement. Two framework concerns cover that back-office work: running code on a timer (`firefly-scheduling`) and delivering messages through swappable providers (`firefly-notifications`), with outbound webhooks (`firefly-callbacks`) and inbound webhooks (`firefly-webhooks`) rounding out the integration story.

By the end of this chapter Lumen will run a **scheduled housekeeping heartbeat**, declared with `#[scheduled]` and started from `main.rs`, and you will know exactly where a daily-statement notification, an outbound balance-changed webhook, and an inbound payment-provider callback would hang off it. Lumen keeps the heartbeat deliberately tiny — it records that it ran — so the macro is shown wired end to end without dragging in a provider SDK.

 SPRING PARITY

`#[scheduled]` is `@Scheduled` : cron, fixed rate, fixed delay. The `firefly-notifications Channel` + `Dispatcher` is the notification-sender abstraction; `firefly-callbacks` is the outbound-webhook subsystem; `firefly-webhooks` is the inbound receiver. Each swaps a provider for a one-line registration, never a code change.

The scheduled heartbeat

Lumen's `src/housekeeping.rs` is the whole feature. A zero-argument `async fn` carries `#[scheduled(...)]`; the macro generates a `schedule_<fn>(scheduler)` registration helper. Here is the file, end to end:

Rust

```

use std::sync::atomic::{AtomicU64, Ordering};
use firefly::prelude::*;

/// The number of times the heartbeat has run – observable from a test (and, in
/// a real service, a counter you would surface on `/actuator/metrics`).
static HEARTBEAT_TICKS: AtomicU64 = AtomicU64::new(0);

/// A periodic housekeeping heartbeat. `#[scheduled(fixed_rate = "60s")]`
/// generates `schedule_ledger_heartbeat(scheduler)`; the framework calls this
/// on every tick after the initial delay.
#[scheduled(fixed_rate = "60s", initial_delay = "5s")]
pub async fn ledger_heartbeat() -> Result<(), std::io::Error> {
    HEARTBEAT_TICKS.fetch_add(1, Ordering::Relaxed);
    Ok(())
}

/// Registers the heartbeat on a fresh scheduler and returns it – `main()`
/// starts it; tests assert it registered.
pub fn build_scheduler() -> std::sync::Arc<Scheduler> {
    let scheduler = std::sync::Arc::new(Scheduler::new());
    schedule_ledger_heartbeat(&scheduler);
    scheduler
}

/// How many heartbeat ticks have run so far.
pub fn heartbeat_ticks() -> u64 {
    HEARTBEAT_TICKS.load(Ordering::Relaxed)
}

```

Three things are happening:

- The macro generates the wiring. `#[scheduled(fixed_rate = "60s", initial_delay = "5s")]` on `ledger_heartbeat` emits a `schedule_ledger_heartbeat(&scheduler)` free function. You write the work; the macro writes the trigger + registration. `Scheduler` comes from `firefly::prelude::*`, so the one facade import covers it.

- `build_scheduler` is the composition root for timers. It makes a fresh `Scheduler`, registers the heartbeat, and hands it back. `main.rs` owns when it starts.
- The heartbeat is observable. It bumps an `AtomicU64`, which a test reads — and which, in a real deployment, you would surface as a counter on `/actuator/metrics` (Chapter 15).

`main.rs` starts the scheduler on a background task, because `Scheduler::start` runs until the scheduler is stopped:

Rust

```
// in main():
let scheduler = build_scheduler();
tokio::spawn(async move { scheduler.start().await });
```

`Scheduler::start` runs each task on its own tokio task with panic recovery, and `stop()` shuts down gracefully (in-flight runs finish first). The scheduled tasks also surface on `/actuator/scheduledtasks`.

What the tests assert

`housekeeping.rs`'s test module proves both halves — the task registered, and it ticks when called:

Rust

```

#[test]
fn scheduled_task_registers() {
    let scheduler = build_scheduler();
    let names: Vec<String> = scheduler.tasks().into_iter().map(|t|
t.name).collect();
    assert!(names.contains(&"ledger_heartbeat".to_string()));
}

#[tokio::test]
async fn heartbeat_runs() {
    let before = heartbeat_ticks();
    ledger_heartbeat().await.unwrap();
    assert_eq!(heartbeat_ticks(), before + 1);
}

```

`scheduler.tasks()` returns a `Vec<TaskDescriptor>` (each has a `name`), so a test can introspect the schedule without waiting for a tick — and the dashboard's Scheduled Tasks view (Chapter 15) reads the same list.

The trigger menu

`#[scheduled]` covers the everyday case; the underlying `Scheduler` exposes all four trigger kinds directly, which is what a real Lumen statement run would use:

Rust

```

use std::{sync::Arc, time::Duration};
use firefly::prelude::Scheduler;

let s = Arc::new(Scheduler::new());

// Cron: a 5-field expression (or the Spring 6-field form with leading seconds).
s.cron("daily-statements", "0 2 * * *", || async { Ok(()) }).unwrap();

// FixedRate: fire every period from a fixed anchor (slips on a slow run).
s.fixed_rate("metrics-emit", Duration::from_secs(30), || async { Ok(()) });

// FixedDelay: fire `delay` after the previous run finished.
s.fixed_delay("sweep-abandoned", Duration::from_secs(300), || async { Ok(()) });

```

Trigger	Behavior
<code>CronTrigger</code>	fires when the local wall clock matches the expression
<code>ZonedCronTrigger</code>	fires per the expression in an IANA time zone
<code>FixedRateTrigger</code>	fires every period from a fixed start anchor (slips)
<code>FixedDelayTrigger</code>	fires the delay after the previous run finished

The cron grammar is the canonical 5-field `minute hour day-of-month month day-of-week`, plus the Spring 6-field form with leading seconds, the Quartz `?` placeholder, and the `@daily` / `@hourly` / `@weekly` macros. For a time zone, build a `ZonedCronTrigger`:

Rust

```
use firefly::scheduling::{parse_cron, ZonedCronTrigger};

// Lumen would run statements at 9am in the customer's region.
let expr = parse_cron("0 9 * * MON-FRI").unwrap();
let trigger = ZonedCronTrigger::in_zone(expr, "America/New_York").unwrap();
```

🌱 SPRING PARITY

`Scheduler::cron / fixed_rate / fixed_delay` mirror `@Scheduled(cron=...)` / `fixedRate / fixedDelay`, and the 6-field leading-seconds form matches Spring's cron dialect. `#[scheduled]` is the annotation; `schedule_<fn>` is the registration the Spring container would do for you.

Notifications – where the daily statement hangs

The heartbeat is the hook for outbound messaging. `firefly-notifications` gives you a channel-agnostic `Notification` envelope, a `Channel` transport trait, and a `Dispatcher` that routes a message to the channel registered for its `Kind`. The default `MemoryChannel` records every message (ideal for tests); a real provider adapter slots in behind the same trait. A Lumen statement run, in sketch:

Rust

```

use std::sync::Arc;
use firefly_notifications::{Dispatcher, Kind, MemoryChannel, Notification};

let dispatcher = Dispatcher::new();
dispatcher.register(Arc::new(MemoryChannel::new(Kind::EMAIL)));

dispatcher
    .dispatch(Notification {
        channel: Kind::EMAIL,
        to: "alice@example.com".into(),
        subject: "Your Lumen statement".into(),
        body: "Closing balance: $42.00".into(),
        ..Notification::default()
    })
    .await
    .unwrap();

```

The `Dispatcher` routes by `Kind` (`Kind::EMAIL` / `Kind::SMS` / `Kind::PUSH`), or a custom `Kind::new("...")`; a message for an unregistered kind is a `NotificationError::NoChannel`. For production, register a real channel in place of `MemoryChannel` — same trait, real delivery:

Crate	Channel	Backing
<code>firefly-notifications-smtp</code>	email	<code>lettre</code> (real MIME, STARTTLS)
<code>firefly-notifications-twilio</code>	SMS	Twilio
<code>firefly-notifications-firebase</code>	push	Firebase Cloud Messaging
<code>firefly-notifications-sendgrid</code>	email	SendGrid
<code>firefly-notifications-resend</code>	email	Resend

Because every channel is an `Arc<dyn Channel>`, the heavy provider SDKs stay out of a service that does not select that channel: code against the `Channel` port, register the concrete adapter at wiring time. So Lumen's daily statement is "on the heartbeat tick, build a `Notification` per wallet and `dispatch` it" — the provider is a wiring decision, not a rewrite.

Outbound webhooks – telling other systems a balance changed

When another system needs to *react* to a Lumen event — a fraud monitor that wants every large deposit, say — you push it an outbound webhook with `firefly-callbacks`. Services register `Target` s; the `HmacDispatcher` signs each payload with HMAC-SHA256, retries with exponential backoff, and records every `Attempt` to a pluggable `Store` for audit:

Rust

```
use std::sync::Arc;
use firefly_callbacks::{CallbackEvent, DispatcherConfig, HmacDispatcher,
MemoryStore};

let store = Arc::new(MemoryStore::new());
let dispatcher = HmacDispatcher::new(store, DispatcherConfig::default()); // 3
attempts, 200ms, doubling

// On a large deposit, Lumen would publish a CallbackEvent; the dispatcher signs
// and POSTs it to every registered Target, headers and `sha256=<hex>` signature
// byte-identical to the Go/Java/.NET/Python ports.
```

Each delivery carries `X-Firefly-Event`, `X-Firefly-Event-Id`, `X-Firefly-Timestamp`, and an `X-Firefly-Signature: sha256=<hmac-hex>` keyed on the target's secret — wire-compatible with every other Firefly port, so a receiver written against any of them verifies Lumen's deliveries unchanged.

Inbound webhooks – receiving a provider callback

The mirror image is `firefly-webhooks`: when Lumen's external payment provider calls *back* (a charge settled, a payout failed), the inbound pipeline validates the signature, deduplicates, and dispatches to a processor. The signature validators ship for the common providers:

Validator	Header	Algorithm
<code>HmacValidator</code>	<code>X-Signature</code> (default)	HMAC-SHA256 hex (optional <code>sha256=</code> prefix)
<code>StripeValidator</code>	<code>Stripe-Signature</code>	<code>t=<unix>,v1=<hmac-hex></code> , 5-minute tolerance
<code>GitHubValidator</code>	<code>X-Hub-Signature-256</code>	HMAC-SHA256 hex
<code>TwilioValidator</code>	<code>X-Twilio-Signature</code>	HMAC-SHA1 base64 of URL + sorted form fields

Rust

```
use std::sync::Arc;
use firefly_webhooks::{web, MemoryDlq, Pipeline, StripeValidator};

let pipeline = Arc::new(Pipeline::new(Arc::new(MemoryDlq::new())));
pipeline.register_validator(StripeValidator::new(b"whsec_test"));
let app: axum::Router = web::router(pipeline); // mount under /api/webhooks/...
```

You test that receiver with the `firefly-testkit` signers — `sign_stripe`, `sign_github`, `sign_twilio`, `sign_hmac` — which produce header values byte-identical to what the validators expect, so a signed test request validates exactly as a real provider's would (Chapter 18).

What changed in Lumen

This chapter gave Lumen its first background work and mapped its integration surface:

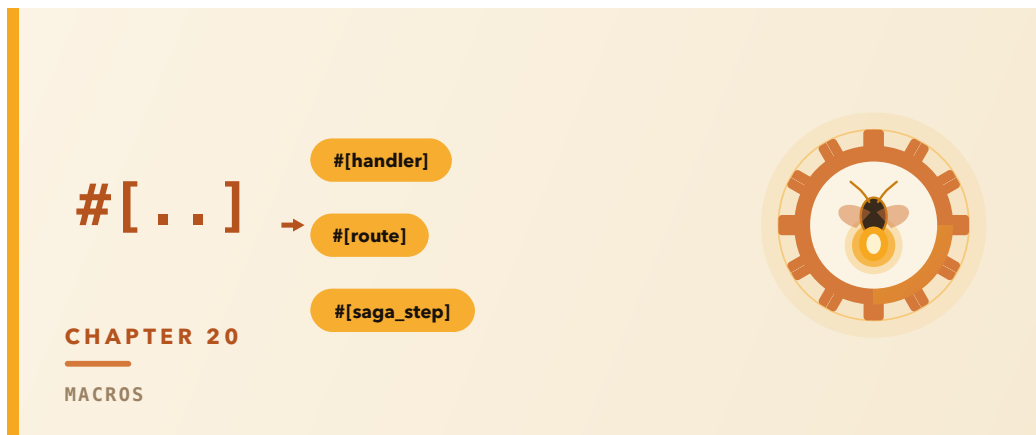
- `src/housekeeping.rs` declares the `ledger_heartbeat` with `#[scheduled(fixed_rate = "60s", initial_delay = "5s")]`; the macro generates `schedule_ledger_heartbeat(&scheduler)`, and `build_scheduler` registers it on a fresh `Scheduler`.
- `src/main.rs` starts the scheduler on a background tokio task (`(tokio::spawn(async move { scheduler.start().await }));`); the tasks surface on `/actuator/scheduledtasks` and in the dashboard's Scheduled Tasks view.
- The heartbeat's `AtomicU64` is the seam where a real **daily-statement notification** (`firefly-notifications`), an **outbound balance-changed webhook** (`firefly-callbacks`), and an **inbound provider callback** (`firefly-webhooks`) would attach — each a registration, not a rewrite.

Exercises

1. **Cron the statement run.** Replace the `fixed_rate` heartbeat with a `#[scheduled(cron = "0 2 * * *")]` task (or register `s.cron("statements", "0 2 * * *", ..)` directly) and assert it appears in `scheduler.tasks()` by name.
2. **Dispatch on the tick.** Inside `ledger_heartbeat`, build a `Dispatcher` with a `MemoryChannel::new(Kind::EMAIL)`, dispatch a one-line statement, and assert (via `MemoryChannel::messages`) that the message was recorded.
3. **Sign and receive.** Stand up a `Pipeline` with a `StripeValidator`, mount `web::router(..)`, and use `firefly_testkit::sign_stripe` to drive a signed request through it with a `TestClient` — assert it is accepted, then tamper with the body and assert it is rejected.

4. **Audit the outbound.** Wire an `HmacDispatcher` over a `MemoryStore`, dispatch a `CallbackEvent` to a `Target`, and read the recorded `Attempt` rows from the store to confirm the retry policy fired.

Lumen now does background work and knows where its messaging hangs. The next chapter deepens the read-side cache the CQRS layer introduced. Continue to [Caching](#).



CHAPTER 20

Declarative Services with Macros

Lumen is finished. Over twenty chapters it grew from an empty scaffold into a secure, observable, event-sourced CQRS service with a transfer saga and a streaming endpoint — and it depends on exactly **one** Firefly crate. This capstone re-reads the whole service through a single lens: the **declarative macros**. By the end of this chapter you will be able to point at every `#[derive(...)]` and `#[...]` in `samples/lumen` and say precisely what wiring it collapsed into a declaration next to the code. That is the thesis the running crate proves: *one facade + macros = the framework, with the boilerplate gone*.

🌱 SPRING PARITY

The declarative layer is the Rust answer to Spring's annotation set and pyfly's decorators. What changes is *when* the wiring is generated — at compile time by a `proc-macro`, not at runtime by reflection — so there is no startup scanning cost and no reflective surprises. The programming model is the same: a declaration sits next to the code it describes, and the framework generates the glue.

One dependency, one prelude

Every chapter began the same way. Lumen's `Cargo.toml` lists one Firefly crate:

TOML

[dependencies]

```
firefly = { workspace = true } # the whole framework + every macro
axum    = { workspace = true } # you author the controller handlers
serde   = { workspace = true } # your messages + event payloads
serde_json = { workspace = true }
tokio    = { workspace = true }
uuid     = { workspace = true }
chrono   = { workspace = true }
async-trait = { workspace = true }
```

and every module opens with one glob:

Rust

```
use firefly::prelude::*;
```

That glob brings the whole framework into scope — `Bus`, `Container`, `Scheduler`, `Saga / Step`, `Application / ShutdownHandle`, `Core / CoreConfig`, `WebResult / WebError`, `FireflyError / FireflyResult`, `Mono / Flux` — and every macro. There are also per-crate aliases (`firefly::cqrs::Bus`, `firefly::eventsourcing::EventStore`, `firefly::security::JwtService`, ...) for the types you name explicitly. `axum` and `serde` are the only two ecosystem crates Lumen writes against directly.

Staying lean

The default `firefly` build pulls only the framework's lean *port* crates — no heavy third-party drivers. Lumen needs none, so its build is minimal. Heavy adapters are opt-in cargo features (the swap path every chapter pointed at):

Feature	Pulls in
<code>data-sqlx</code>	relational repository adapter (Postgres / MySQL / SQLite)
<code>data-mongodb</code>	document repository adapter (MongoDB)
<code>eda-kafka</code> / <code>eda-rabbitmq</code> / <code>eda-redis</code> / <code>eda-postgres</code>	event-broker transports
<code>cache-redis</code> / <code>cache-postgres</code>	cache backends
<code>admin</code>	the admin dashboard
<code>full</code>	all of the above

The macros Lumen uses

Lumen exercises the full declarative set. Here is the catalogue, each mapped to the exact Lumen file it lands in:

Macro	Lumen file	Generates
<code>#[derive(Command)] /</code> <code>#[derive(Query)]</code>	<code>commands.rs</code>	the <code>Message</code> impl (<code>#[firefly(validate)]</code> , <code>#[firefly(cache_ttl = "...")]</code>)
<code>#[command_handler] /</code> <code>#[query_handler]</code>	<code>commands.rs</code>	a <code>register_<fn>(bus)</code> helper
<code>#[derive(DomainEvent)]</code>	<code>domain.rs</code>	<code>EVENT_TYPE</code> + <code>to_domain_event</code>
<code>#[derive(AggregateRoot)]</code>	<code>domain.rs</code>	<code>AGGREGATE_TYPE</code> + <code>aggregate()</code> / <code>aggregate_mut()</code>
<code>#[event_listener(topic =</code> <code>"...")]</code>	<code>ledger.rs</code>	a <code>subscribe_<fn>(broker)</code> helper
<code>#[rest_controller] +</code> <code>#[get/post]</code>	<code>web.rs</code>	a <code>routes(state) -> axum::Router</code>
<code>#[scheduled(fixed_rate =</code> <code>"...")]</code>	<code>housekeeping.rs</code>	a <code>schedule_<fn>(scheduler)</code> helper

The DI stereotype derives (`#[derive(Component/Service/Repository/Configuration/Controller)]`, `#[bean]`, `#[autowired]`, `register_all!`) and `#[derive(ConfigProperties)]` round out the set; Lumen wires its collaborators explicitly rather than through the container (chapter 4), so the [DI deep-dive](#) is where you saw those at work.

CQRS – messages and handlers (`commands.rs`)

`#[derive(Command)] / #[derive(Query)]` generate the `Message` impl. `#[firefly(validate)]` makes an empty / zero field fail validation before the handler runs; `#[firefly(cache_ttl = "...")]` feeds the query cache. These are verbatim Lumen:

Rust

```
#[derive(Debug, Clone, Default, Serialize, Deserialize, Command)]
#[serde(default)]
pub struct OpenWallet {
    #[firefly(validate)]
    pub owner: String,
    #[serde(rename = "openingBalance")]
    pub opening_balance: i64,
}

#[derive(Debug, Clone, Default, Serialize, Deserialize, Query)]
#[firefly(cache_ttl = "30s")]
pub struct GetWallet {
    pub id: String,
}
```

`#[command_handler]` / `#[query_handler]` mark a free `async fn` and generate a `register_<fn>(bus)` helper that installs it on the bus:

Rust

```
#[command_handler]
pub async fn open_wallet(cmd: OpenWallet) -> Result<WalletView, CqrsError> { /*
... */ }

#[query_handler]
pub async fn get_wallet(q: GetWallet) -> Result<WalletView, CqrsError> { /* ...
*/ }

// generated: register_open_wallet(&bus); register_get_wallet(&bus);
```

Lumen calls each generated helper in one `register(&bus)` fn. Because free fns cannot capture wiring state, the resolved `Ledger` + `ReadModel` are published once through a `bind` / `OnceLock` and reached from the handlers — the pattern chapter 9 introduced.

 SPRING PARITY

`#[derive(Command)]` + `#[command_handler]` is `@CommandHandler` on a typed handler;
`#[firefly(cache_ttl)]` is `@Cacheable` on a query. The `Bus` is the command/query gateway.

Event sourcing – domain events and the aggregate (`domain.rs`)

`#[derive(DomainEvent)]` stamps each payload with a stable `EVENT_TYPE` discriminator (its struct name) and a `to_domain_event` conversion;
`#[derive(AggregateRoot)]` finds the embedded `AggregateRoot` field and generates `Wallet::AGGREGATE_TYPE` plus `aggregate()` / `aggregate_mut()`:

Rust

```
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize, DomainEvent)]
pub struct WalletOpened {
    pub wallet_id: String,
    pub owner: String,
    pub opening_balance: i64,
}

#[derive(Debug, Clone, AggregateRoot)]
#[firefly(aggregate_type = "Wallet")]
pub struct Wallet {
    pub root: AggregateRoot, // the framework root – uncommitted-event buffer
    + version
    pub owner: String,
    pub balance: Money,
    pub opened: bool,
}
```

The only event-sourcing wiring Lumen writes by hand is the `apply` fold; the discriminators and the wire conversion are generated.

 SPRING PARITY

`#[derive(AggregateRoot)]` is the Axon-style event-sourced aggregate; the `EVENT_TYPE` discriminator is the `@Revision` / `@EventType` tagging that keeps the JSON wire-compatible across the ports.

Messaging – the projection listener (`ledger.rs`)

`#[event_listener(topic = "...")]` marks a free `async fn(Event) -> FireflyResult<()>` and generates a `subscribe_<fn>(broker)` helper that subscribes it to the topic. Lumen's read-model projection is one declaration:

Rust

```
#[event_listener(topic = "wallets.events")]
pub async fn project_wallet_event(ev: Event) -> FireflyResult<()> {
    // reload the wallet's stream, fold to a WalletView, upsert – idempotent.
    Ok(())
}
// generated: subscribe_project_wallet_event(broker)
```

The composition root calls `ledger::subscribe_project_wallet_event(broker)` after binding, closing the CQRS loop.

 SPRING PARITY

`#[event_listener(topic = "...")]` is `@KafkaListener` / `@RabbitListener` — the topic subscription is generated; you write only the handler body.

Web – the controller (`web.rs`)

`#[rest_controller(path = "...")]` turns an `impl` block into a generated `WalletApi::routes(state) -> axum::Router`. Each method carries one verb mapping and uses ordinary axum extractors, returning `WebResult<T>` so a handler error renders as RFC 9457 `application/problem+json`:

Rust

```
#[rest_controller(path = "/api/v1")]
impl WalletApi {
    #[post("/wallets")]
    async fn open(State(api): State<WalletApi>, Json(body): Json<OpenWallet>)
        -> WebResult<(axum::http::StatusCode, Json<WalletView>)> { /* dispatch
via api.bus */ }

    #[get("/wallets/:id")]
    async fn get(State(api): State<WalletApi>, Path(id): Path<String>)
        -> WebResult<Json<WalletView>> { /* ... */ }
    // ... deposit / withdraw / transfer
}
// generated: WalletApi::routes(state) -> axum::Router
```

The macro also submits each route into a link-time table, so the OpenAPI generator and the actuator `/mappings` endpoint can enumerate Lumen's routes without reparsing the source.

 SPRING PARITY

`#[rest_controller]` + `#[get]` / `#[post]` is `@RestController` + `@GetMapping` / `@PostMapping`; `WebResult<T>` rendering RFC 9457 is `@ControllerAdvice` returning a `ProblemDetail`.

Scheduling – the heartbeat (`housekeeping.rs`)

`#[scheduled(...)]` generates a `schedule_<fn>(scheduler)` helper that registers a zero-argument `async fn` on a `Scheduler`:

Rust

```
#[scheduled(fixed_rate = "60s", initial_delay = "5s")]
pub async fn ledger_heartbeat() -> Result<(), std::io::Error> { /* ... */ }
// generated: schedule_heartbeat(&scheduler)
```

 SPRING PARITY

```
#[scheduled(fixed_rate = "60s")] is @Scheduled(fixedRate = 60000);
#[scheduled(cron = "...")] is the cron form.
```

How it works: the `__rt` contract

A `proc-macro` crate cannot re-export runtime types, so macro-generated code references every runtime type through the facade's hidden `__rt` contract path — e.g. `::firefly::__rt::firefly_cqrs::Bus`. That is why Lumen, depending only on `firefly` (plus the `axum` / `serde` it writes against anyway), compiles whatever a macro expands to without listing the underlying `firefly-*` crates. You never write `__rt` yourself. If you rename or shim the facade, pass `#[firefly(crate = "my_firefly")]` to any macro to override the leading segment.

Rust has no reflective autowiring, so the declarative layer removes the *mechanical* boilerplate, not the explicitness: Lumen still calls `register(&bus)`, still calls `subscribe_project_wallet_event(broker)`, still hands `WalletApi::routes(state)` to the web stack. The macros generate the `impl`s, the routers, and the helper functions — the wiring you would otherwise hand-write.

The whole crate, declaratively

Read top to bottom, the macros tell Lumen's story:

Text

```

money.rs      (no macros – a pure value object; the no-thiserror promise)
domain.rs     #[derive(DomainEvent)] x3  #[derive(AggregateRoot)]
ledger.rs     #[event_listener(topic = "wallets.events")]
commands.rs   #[derive(Command)] x3     #[derive(Query)]
              #[command_handler] x3     #[query_handler]
transfer.rs   (Saga / Step builder – orchestration is a runtime API, not a
macro)
security.rs   (JwtService / BearerLayer / FilterChain – runtime APIs)
web.rs        #[rest_controller] + #[get] / #[post] x6
housekeeping.rs #[scheduled(fixed_rate = "60s", initial_delay = "5s")]

```

Note what is *not* a macro: the transfer saga and the security filter chain are built with runtime builders (`Saga::new(...).step(...)`, `FilterChain::new().require(...)`), because their shape is data, not a fixed declaration — and Lumen keeps them explicit so the control flow stays visible. Declarative where it collapses boilerplate, explicit where the graph is the point: that balance is the whole design.

Verifying the crate

Everything above compiles and is tested. From the workspace root:

Shell

```

cargo build -p firefly-sample-lumen
cargo test -p firefly-sample-lumen           # 34 unit + 7 HTTP +
1 doctest
cargo test -p firefly-sample-lumen --features streaming # + 3 streaming tests
cargo clippy -p firefly-sample-lumen --all-targets -- -D warnings

```

The HTTP tests drive the macro-generated `WalletApi::routes(...)` router in-process through `tower::ServiceExt::oneshot` (no socket bound), proving the generated routes, the CQRS handlers, validation (422), the not-found path (404), the auth

boundary (401), the transfer saga (happy + compensation), and the projection convergence all work end to end — every prose listing in this book is a slice of that running crate.

What changed in Lumen

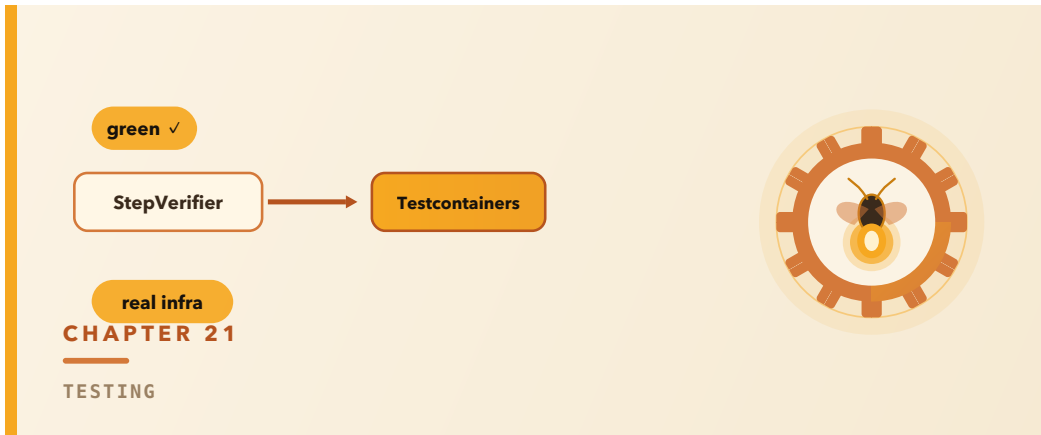
Nothing — this chapter is the retrospective, not a new feature. Re-read as a catalogue, Lumen's macros are: three `#[derive(DomainEvent)]` + one `#[derive(AggregateRoot)]` (`domain.rs`); one `#[event_listener]` (`ledger.rs`); three `#[derive(Command)]` + one `#[derive(Query)]` + three `#[command_handler]` + one `#[query_handler]` (`commands.rs`); one `#[rest_controller]` with six verb methods (`web.rs`); and one `#[scheduled]` (`housekeeping.rs`). Each replaced a chunk of hand-written wiring with a declaration next to the code — and all of it arrived through one dependency and one prelude glob.

Exercises

1. Trace one macro end to end. Pick `#[derive(Query)]` on `GetWallet`. Find where its generated `cache_ttl()` is read (the `QueryCache` middleware in `web.rs`) and the test that asserts it (`get_wallet_carries_cache_ttl` in `commands.rs`). Change the TTL to `"5s"` and re-run the tests.
2. Add a verb. Add a `#[get("/wallets/:id/events")]`-style read method to the `#[rest_controller]` impl (non-streaming: return the event list as JSON) and confirm `WalletApi::routes` picks it up with no other change.
3. Add a scheduled task. Write a second `#[scheduled(cron = "0 0 * * *")]` function in `housekeeping.rs`, register it in `build_scheduler`, and assert it appears in `scheduler.tasks()`.
4. Count the wiring you didn't write. For each macro in the catalogue, name the helper or impl it generated (`register_*`, `subscribe_*`, `schedule_*`, `routes`,

`EVENT_TYPE` , `AGGREGATE_TYPE`). That list is the boilerplate the declarative layer wrote for you.

That is Lumen, complete and declarative. The appendices that follow are reference: a [Spring-Boot → Firefly-Rust cheat sheet](#), a [module index](#), and a [glossary](#).



CHAPTER 21

Testing Firefly Applications

Every chapter so far has shown Lumen's listings *and* the tests that keep them honest — that is the whole point of the book: the prose is verified against a crate that compiles and passes its suite. This chapter steps back and looks at the test strategy as a whole, the way you would design it for your own service.

By the end of this chapter you will know how Lumen tests at three levels — pure unit tests with no I/O, in-process HTTP tests that drive the *real* router through `tower::oneshot`, and the `firefly-testkit` helpers (`TestClient`, `Slice`, `assert_event_published`) that make all of it terse — and how the same crate scales up to real-infrastructure integration tests when you need a live Postgres or Kafka. Lumen's gate is **34 unit tests + 7 HTTP tests + 1 doctest**, all hermetic; the streaming feature adds 3 more.

 SPRING PARITY

The three tiers mirror Spring's: `@Test` unit tests, `@WebMvcTest` / `WebTestClient` slice tests, and `@SpringBootTest` + Testcontainers integration tests. `TestClient` is the Rust spelling of `WebTestClient` / pyfly's `PyFlyTestClient`; `Slice` is a `@...Test` slice; the event assertions match pyfly's `assert_event_published`.

The in-process testing model

Lumen's default stack is entirely in-memory — `MemoryEventStore`, `InMemoryBroker`, a `Mutex<HashMap>` read model — so almost every test runs as a plain `#[tokio::test]` with **no socket and no external service**. The HTTP tests do not bind a port; they hand a `Request` to the router and `await` the `Response`. That is fast, deterministic, and CI-friendly.

One caveat is worth stating up front, because Lumen's tests are built around it. Lumen's free-function command handlers and its read-model projection publish their collaborators through process-global `OnceLock`s (the declarative-macro pattern from Chapter 9). So the *first* `build_router()` in a test binary wires the shared ledger that every later test in that binary then drives. The tests cope by giving each wallet a fresh owner and a server-assigned id, so there is no cross-test interference even though the ledger is shared.

Unit tests with no infrastructure

The domain and the value object are pure, so their tests need nothing. Lumen's `money.rs` and `domain.rs` assert invariants directly — exact-cents arithmetic, positive amounts, sufficient funds, owner required. The CQRS layer is just as direct: dispatch a command through a real `Bus` and assert the result. This is the heart of `commands.rs`'s test module:

Rust

```

#[tokio::test]
async fn handlers_dispatch_through_the_bus() {
    let ledger = Ledger::new(
        Arc::new(MemoryEventStore::new()),
        Arc::new(InMemoryBroker::new()),
    );
    bind(ledger, Arc::new(ReadModel::default()));
    let bus = Bus::new();
    // Validation middleware enforces the `#[firefly(validate)]` checks.
    bus.use_middleware(firefly::cqrs::ValidationMiddleware::new());
    register(&bus);

    let opened: WalletView = bus
        .send(OpenWallet { owner: "alice".into(), opening_balance: 100 })
        .await
        .unwrap();
    assert_eq!(opened.balance, 100);

    let after: WalletView = bus
        .send(Deposit { wallet_id: opened.id.clone(), amount: 50 })
        .await
        .unwrap();
    assert_eq!(after.balance, 150);
}

```

Validation is tested without ever touching HTTP — call `.validate()` on the command directly, because `#[derive(Command)]` generated it from the `#[firefly(validate)]` fields:

Rust

```
#[test]
fn open_wallet_validates_owner() {
    assert!(OpenWallet::default().validate().is_err()); // empty owner
    assert!(OpenWallet { owner: "alice".into(), opening_balance:
0 }.validate().is_ok());
}
```

Security (`security.rs`), the saga (`transfer.rs`), and the scheduled task (`housekeeping.rs`) each carry their own `#[cfg(test)] mod tests` in the same spirit — mint-then-verify a token, run the saga happy path and the compensation path, register the heartbeat and assert it ticks.

In-process HTTP tests with `tower::oneshot`

The end-to-end suite lives in `tests/http.rs` and drives the fully-wired `build_router()` — the real `#[rest_controller]` routes, the CQRS handlers, the event-sourced ledger, the read-model projection, the transfer saga, *and* the JWT/RBAC enforcement from Chapter 14. No mocks: every layer is the production layer, just over in-memory infrastructure.

The pattern is `Router` + `tower::ServiceExt::oneshot`. Lumen wraps it in two tiny helpers so each test reads as one HTTP round-trip:

Rust

```

use http_body_util::BodyExt;
use tower::ServiceExt;

fn post(path: &str, body: serde_json::Value, auth: bool) -> Request<Body> {
    let mut b = Request::post(path).header("content-type", "application/json");
    if auth {
        b = b.header("authorization", bearer()); // Bearer <minted CUSTOMER
token>
    }
    b.body(Body::from(serde_json::to_vec(&body).unwrap())).unwrap()
}

async fn body_json<T: serde::de::DeserializeOwned>(res: Response) -> T {
    let bytes = res.into_body().collect().await.unwrap().to_bytes();
    serde_json::from_slice(&bytes).unwrap()
}

```

A test then opens a wallet and asserts the projected read came back through CQRS:

Rust

```

#[tokio::test]
async fn open_then_get_round_trips_through_cqrs() {
    let opened = open_wallet("alice", 1_000).await; // POST /api/v1/wallets,
    asserts 201
    assert_eq!(opened.balance, 1_000);

    // GET dispatches the #[query_handler]; the projection has already folded
    // the WalletOpened event into the read model.
    let res = build_router()
        .await
        .oneshot(get(&format!("/api/v1/wallets/{}", opened.id)))
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::OK);
    let fetched: WalletView = body_json(res).await;
    assert_eq!(fetched.balance, 1_000);
}

```

The same file proves the saga

(`transfer_saga_happy_path_moves_funds_between_wallets`), the compensation path (`transfer_saga_overdraft_compensates_and_is_422`), and the problem-rendering for the three failure modes — a missing token is a 401, an empty owner is a 422, an unknown id is a 404 — each asserting the `application/problem+json` content type. That single suite is the proof that the whole stack composes.

The testkit: `TestClient`, `Slice`, event assertions

`firefly-testkit` packages the boilerplate above into reusable helpers. Lumen's own tests use the raw `tower::oneshot` form to show the mechanism with no magic, but in your service the testkit makes the same tests far shorter. Three pieces matter most.

TestClient – an in-process HTTP client (feature `web`)

`TestClient::new(router)` wraps any axum `Router` and gives you `get` / `post` / `put` / `patch` / `delete` plus a fluent assertion API on the `TestResponse`. The `open_then_get` test above, rewritten with `TestClient`:

Rust

```
use firefly_testkit::TestClient;

#[tokio::test]
async fn open_then_get_with_testclient() {
    let client = TestClient::new(build_router().await);

    let created = client
        .post("/api/v1/wallets", &serde_json::json!({ "owner": "alice",
"openingBalance": 1000 }))
        .await
        .assert_status(201);
    let id = created.json_path("$.id").unwrap();

    client
        .get(&format!("/api/v1/wallets/{}", id.as_str().unwrap()))
        .await
        .assert_status(200)
        .assert_json_path("$.balance", 1000);
}
```

`assert_status`, `assert_success`, `assert_header`, `assert_body_contains`, `assert_json_eq`, `assert_json_path` / `assert_json_path_exists` / `assert_json_path_absent`, and `json::<T>()` / `json_path("$.field")` cover the common assertions; the path grammar is the single-result JSONPath subset (a leading `$`, dotted or bracketed member access, array indexing). Each assertion returns `&Self` so they chain. (Blocking variants — `post_blocking`, `get_blocking`, ... — exist for non-async test contexts.)

Slice – a focused DI container for a test

`Slice` builds a minimal `firefly-container` for a slice test: register only the collaborators the unit under test needs, then resolve them. It is the analog of a Spring `@Test` slice — the container without the full application context:

Rust

```
use firefly_testkit::Slice;
use firefly_container::{Container, ContainerError, Scope};

let slice = Slice::new()
    .instance(ReadModel::default()) // a ready instance (a
    "mock_bean")
    .register::<MyService, _>(Scope::Singleton, |c: &Container| {
        Ok(MyService::new()) // a factory; resolve
        deps from `c`
    })
    .build();

let read_model: std::sync::Arc<ReadModel> = slice.get();
```

`register` / `register_named` take a factory `|c: &Container| -> Result<T, ContainerError>` (it may resolve its own dependencies from `c`); `instance` / `instance_named` install a ready value (the override/mock path); `bind` coerces a concrete type to a trait object; and `eager` forces construction at build time. `build()` returns a `BuiltSlice` you resolve from with `get::<T>()` / `get_named::<T>(name)`.

Asserting emitted events

`SpyBroker` records what a handler published; `assert_event_published(&spy, "Type")` asserts an event of that type was recorded and returns it (the `_with` variant also checks the payload contains a substring; `assert_no_events_published` asserts none). `must_encode` / `must_decode` are panic-on-failure JSON helpers. A Lumen-flavored example — proving an open emits a `WalletOpened`:

Rust

```

use firefly_testkit::{assert_event_published, must_encode, SpyBroker};

#[test]
fn open_emits_wallet_opened() {
    let spy = SpyBroker::new();
    // The ledger would publish through the broker; here we record the envelope
    // the projection would consume.
    spy.record("wallets.events", "WalletOpened",
               &must_encode(&serde_json::json!({ "id": "wlt_1", "owner":
"alice" })));

    let event = assert_event_published(&spy, "WalletOpened");
    assert_eq!(event.topic, "wallets.events");
}

```

Webhook signers

When Lumen grows an inbound webhook (Chapter 16), the testkit's HMAC signers — `sign_hmac`, `sign_stripe`, `sign_github`, `sign_twilio` — produce header values byte-identical to what each `firefly-webhooks` validator expects, so a signed test request validates exactly as a real provider's would:

Rust

```

use firefly_testkit::sign_stripe;

let sig = sign_stripe(b"whsec_test", br#"{"type":"charge.succeeded"}"#,
1_700_000_000);
// Attach `sig` as the `Stripe-Signature` header on a TestClient POST.

```

Testing reactive pipelines

The streaming endpoint (Chapter 20) builds a `Flux`. A reactive pipeline is tested by driving it to a terminal — `block()`, `collect_list()`, `count()` — and asserting the resolved value. This is the `firefly-reactive` analog of Reactor's `StepVerifier`:

Rust

```
use firefly_reactive::Flux;

#[tokio::test]
async fn pipeline_filters_and_maps() {
    let out = Flux::range(1, 5)
        .filter(|x| x % 2 == 1)
        .map(|x| x * 10)
        .collect_list()
        .block()
        .await
        .unwrap()
        .unwrap();
    assert_eq!(out, vec![10, 30, 50]);
}
```

Lumen's streaming tests (`tests/streaming.rs`, gated behind the `streaming` feature) take the HTTP route instead: open a wallet, deposit, then `GET /events` and assert two NDJSON lines (`WalletOpened` + `MoneyDeposited`) by default, `text/event-stream` with `?format=sse`, and a 404 for an unknown wallet.

Real-infrastructure integration tests

Lumen runs hermetically, but the production adapters need real services. The workspace ships a `docker-compose.yml` with Postgres, Redis, RabbitMQ, a Kafka-compatible Redpanda, Keycloak, S3/Blob emulators, and an SMTP capture. The

convention throughout the adapter crates is: a test reads a connection URL from the environment and **skips when it is unset**, so the default `cargo test` stays green on a bare machine while CI flips the full suite on:

Rust

```
#[tokio::test]
#[ignore = "requires postgres (DATABASE_URL)"]
async fn postgres_event_store_round_trips() {
    let Ok(url) = std::env::var("DATABASE_URL") else { return }; // skip on a
    bare machine
    // ... drive the Postgres-backed EventStore against the live database
}
```

Shell

```
docker compose up -d                # start the backing services
DATABASE_URL=postgres://firefly:firefly@localhost:5432/firefly \
REDIS_URL=redis://localhost:6379/0 \
  cargo test --workspace -- --ignored    # run the env-gated suite
docker compose down
```

Running Lumen's suite

From the workspace root (with `export PATH="/opt/homebrew/bin:$PATH"`):

Shell

```
cargo build -p firefly-sample-lumen
cargo test -p firefly-sample-lumen
# 34 unit + 7 HTTP + 1 doctest
cargo test -p firefly-sample-lumen --features streaming # + 3 streaming tests
cargo clippy -p firefly-sample-lumen --all-targets -- -D warnings
cargo fmt -p firefly-sample-lumen -- --check
```

If a snippet in any chapter drifts from the file, this gate fails — which is how the book stays honest.

What changed in Lumen

Nothing in `src/` — this chapter is the retrospective on the test code that grew alongside every feature:

- **Unit tests** per module: `money` and `domain` invariants, `commands` validation + bus dispatch, `security` mint/verify/reject, `transfer` happy + compensation, `housekeeping` registration + tick.
- The `tests/http.rs` end-to-end suite drives the real `build_router()` with `tower::oneshot`, covering open → get → deposit/withdraw → transfer (happy + compensated) → projection convergence → 401/422/404 problems.
- `tests/streaming.rs` (feature-gated) exercises the NDJSON / SSE endpoint.
- The `firefly-testkit` helpers — `TestClient`, `Slice`, `assert_event_published`, the HMAC signers — are the terse path to the same coverage in your own service.

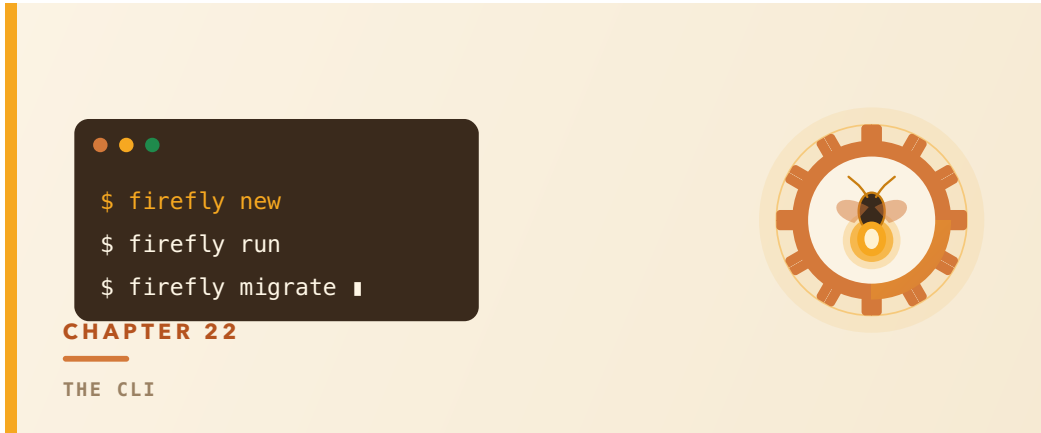
Exercises

1. Rewrite a test with `TestClient`. Take `deposit_and_withdraw_update_the_balance` from `tests/http.rs` and rewrite it using `TestClient` + `assert_json_path`. Add the `Authorization` header via `TestClient::request` so the mutation is authenticated.
2. A `Slice` test for the read model. Use `Slice` to register a `ReadModel::default()` instance, project a `WalletOpened` into it by hand, and assert `find` returns the view — all without the bus or the router.
3. Event assertion on the ledger. Wire a `SpyBroker` into a `Ledger` in a test, commit a deposit, and use

`assert_event_published_with(&spy, "MoneyDeposited", "50")` to prove the amount is on the wire.

4. **A skipping integration test.** Write an `#[ignore]` d test that reads `DATABASE_URL`, returns early when unset, and otherwise opens a wallet against a Postgres-backed event store. Confirm it skips with a plain `cargo test` and runs with the variable set.

With the stack proven at every level, the remaining chapters cover the CLI and shipping Lumen to production. Continue to [The CLI](#).



CHAPTER 22

The firefly CLI

So far you have built Lumen by hand — a file at a time, `cargo build` after each chapter. By the end of this chapter you will know the other way to start a service like Lumen: the `firefly` developer CLI scaffolds a project, generates the same artifacts the earlier chapters wrote by hand, runs the binary with profiles and overrides, manages migrations, exports an OpenAPI document, and introspects a running Lumen over its actuator surface. It is the Rust port of pyfly's `pyfly` CLI, adapted to a compiled Cargo workspace.

🌿 SPRING PARITY

`firefly` is to a Firefly-Rust service what the Spring Boot CLI / `spring init` and `mvn / gradle` goals are to a Spring Boot service: project scaffolding, run, build-info stamping, and actuator introspection in one binary. The command names track pyfly's `pyfly` group one-for-one (`new` / `run` / `generate` / `db` / `doctor` / `sbom` / `license`).

Installing

Shell

```
cargo install --path crates/cli  # installs the `firefly` binary
firefly --help                  # prints the banner + every command
firefly --version                # 26.6.3
```


Command overview

Command	Purpose
<code>firefly new <name></code>	scaffold a new firefly-rust project
<code>firefly generate <kind> <name></code> (alias <code>g</code>)	generate a code artifact
<code>firefly run</code>	<code>cargo run</code> with profile / override flags
<code>firefly build <info\ image></code>	stamp <code>build-info.json</code> / build an OCI image
<code>firefly info</code>	framework + environment information
<code>firefly doctor</code>	toolchain checks (rustc, cargo, git, ...)
<code>firefly db <init\ migrate\ upgrade\ status></code>	migration management
<code>firefly openapi --format json\ yaml [-o file]</code>	export an OpenAPI 3.1 document
<code>firefly actuator <endpoint> --url <base></code>	query a running app's <code>/actuator/*</code>
<code>firefly routes\ env\ health\ metrics --url <base></code>	remote introspection of a running app
<code>firefly beans\ conditions --url <base></code>	DI / auto-config report of a running app
<code>firefly completion <shell></code>	print a shell-completion script
<code>firefly sbom [--json]</code>	software bill of materials from <code>Cargo.lock</code>
<code>firefly license</code>	framework + dependency license report

Scaffolding a project

`firefly new` generates a workspace-less Cargo crate with a `src/` tree, a `firefly.yaml`, a `.gitignore`, a `README.md`, a `Dockerfile`, and `tests/` — roughly the shape Lumen started from in chapter 2:

Shell

```
firefly new lumen --archetype web-api --features web,data,cqrs --git
firefly new my-lib --archetype library --dep-path ../../crates # local dev
deps
firefly new --list # archetypes +
features
firefly new svc --dry-run # plan without
writing
```

Archetypes: `core`, `web-api`, `web`, `hexagonal`, `library`, `cli`. The generated `firefly-*` dependencies are git / path / version configurable (`--dep-path` / `--dep-version`, defaulting to the canonical GitHub repo). `--git` initializes a repository with an initial commit; `--force` overwrites an existing target directory.

🌿 SPRING PARITY

`firefly new --archetype web-api` is `spring init --dependencies=web`: it stamps the entry point, the controller, and the dependency set so the first `cargo run` boots. `--features` selects the same opt-in adapters the [facade chapter](#) lists (`web`, `data`, `cqrs`, `eda`, `cache`, `security`, ...).

Generating artifacts

`firefly generate` writes a code artifact into the current project, detecting the package, archetype, and feature flags from `Cargo.toml` + `firefly.yaml`. These are the same pieces you wrote by hand for Lumen — a command + handler, an aggregate, a saga:

Shell

```
firefly generate command OpenWallet      # command + handler in src/cqrs/
firefly generate query   GetWallet       # query + handler
firefly generate aggregate Wallet        # a #[derive(AggregateRoot)] skeleton
firefly generate saga    MoneyTransfer  --dry-run
firefly generate migration AddWallets    # V###__add_wallets.sql in migrations/
firefly g handler Deposit                # `g` is the alias
```

Artifact kinds: `handler`, `route`, `entity`, `repository`, `dto`, `aggregate`, `command`, `query`, `saga`, `migration`. Names are accepted in any case and converted as needed; `--force` overwrites, `--dry-run` plans without writing.

Running the app

`firefly run` is a thin wrapper over `cargo run` that maps profile and configuration flags to the `FIREFLY_*` environment variables the framework reads at startup, then exec's Cargo:

Shell

```

firefly run                                # cargo run
firefly run -p dev -p test                 # FIREFLY_PROFILES_ACTIVE=dev,test
firefly run -D server.port=9090            # FIREFLY_SERVER_PORT=9090
firefly run --env LUMEN_ADDR=0.0.0.0:8080  # a raw env var for the process
firefly run --debug                        # FIREFLY_LOGGING_LEVEL_ROOT=DEBUG
firefly run --release --bin lumen          # cargo run --release --bin lumen
firefly run --dry-run                      # print the resolved env + cargo
command

```

🌿 SPRING PARITY

`firefly run -p dev -D server.port=9090` is `mvn spring-boot:run -Dspring-boot.run.profiles=dev -Dserver.port=9090`. The flags become environment variables rather than JVM system properties, but the resolution order is the same: CLI flag → env → config file. There is no `--reload` / `--server` / `--workers` (pyfly's ASGI-server selection) — a Rust service is a single compiled binary, so those have no analog and are intentionally omitted.

For Lumen specifically, recall that `main.rs` reads `LUMEN_ADDR` / `LUMEN_ADMIN_ADDR` directly, so the equivalent of the two-port bind is:

Shell

```

firefly run --bin lumen \
  --env LUMEN_ADDR=127.0.0.1:8080 \
  --env LUMEN_ADMIN_ADDR=127.0.0.1:8081

```

Building for release

Plain compilation is `cargo build`; the `build` group adds the two artifacts a release pipeline needs:

Shell

```

firefly build info                # write build-info.json (git SHA + UTC
time)
firefly build info -o target/build-info.json
firefly build image -t lumen:1.0.0
# OCI image via Cloud Native Buildpacks (`pack`)
firefly build image --builder docker    # or a plain Dockerfile build

```

`build info` writes the `build-info.json` that `/actuator/info` surfaces — the admin-port `info` endpoint chapter 15 wired for Lumen reads it when present, so the git SHA and build time show up next to the `InfoContributor` block.

Database migrations

Lumen runs over an in-memory event store, so it ships no SQL migrations — but the moment you swap in the Postgres event store from chapter 20, `firefly db` manages the schema. It wraps the `firefly-migrations` forward-only runner:

Shell

```

firefly db init                    # migrations/ + starter
V001__init.sql
firefly db migrate -m "create wallets" # writes V002__create_wallets.sql
firefly db upgrade --url sqlite://app.db # apply pending migrations
firefly db status --url sqlite://app.db # show applied + pending

```

The database URL resolves from `--url`, then `$DATABASE_URL`, then `firefly.datasource.url` in `firefly.yaml`, defaulting to `sqlite://firefly.db`.

🌿 SPRING PARITY

`firefly db` mirrors pyfly's `pyfly db` group, which drives Alembic; this port drives the framework's own forward-only runner. The runner is forward-only, so `firefly db downgrade` is unsupported — write a corrective migration instead, exactly as you would with a forward-only Flyway setup.

📄 NOTE

The CLI migration backend is **SQLite** via `rusqlite`; a `postgres://` / `mysql://` URL returns a clear "not wired into the CLI" error. For another driver, adapt the `firefly_migrations::Database` port and call `run` directly from your build, rather than through the CLI.

Exporting OpenAPI

Shell

```
firefly openapi                                # OpenAPI 3.1 JSON to stdout
firefly openapi --format yaml -o openapi.yaml
```

The document metadata (`info.title` / `info.version` / `description`) is read from `firefly.yaml` then `Cargo.toml`. A compiled binary cannot boot an arbitrary app to enumerate live routes, so the CLI emits a metadata-stamped **skeleton**; to emit Lumen's real routes, build them with `firefly_openapi::Builder` (which reads the `#[rest_controller]` route table) and serve them with `Builder::router()`.

Introspecting a running app

These commands query a *running* Lumen over HTTP — a compiled binary has no offline DI context to boot, so `--url` is required. Point them at Lumen's admin port (the actuator surface from chapter 15):

Shell

```

firefly health --url http://localhost:8081 # -> /actuator/health
firefly env --url http://localhost:8081 # -> /actuator/env
firefly routes --url http://localhost:8081 # -> /actuator/mappings
firefly metrics requests --url http://localhost:8081
firefly actuator info --url http://localhost:8081 --json
firefly actuator metrics requests --url http://localhost:8081 --json
firefly beans --url http://localhost:8081 # the DI container's bean table
firefly conditions --url http://localhost:8081 # the auto-configuration report

```

 SPRING PARITY

`firefly beans` and `firefly conditions` mirror Spring Boot's `/actuator/beans` and `/actuator/conditions` — they render the DI container's bean table and the conditional-bean evaluation report of a running service. `firefly routes` reads `/actuator/mappings`, the Rust analog of Spring's `RequestMappingHandlerMapping` dump.

Diagnosing, completing, and auditing

Shell

```

firefly info # framework version + which optional adapters are
built
firefly doctor # checks rustc, cargo, git, clippy, rustfmt, docker
firefly completion zsh # > ~/.zfunc/_firefly (bash | zsh | fish |
powershell)
firefly sbom # a software bill of materials from Cargo.lock
firefly sbom --json # machine-readable, for a compliance pipeline
firefly license # the framework + dependency license report

```

`firefly doctor` is the first thing to run on a fresh machine: it reports your `rustc` / `cargo` versions and whether `git`, `clippy`, `rustfmt`, and `docker` are on the `PATH`, ending with "All checks passed!" or a list of what to fix.

Running through cargo

If you have not installed the binary, drive the CLI through cargo from a framework checkout:

Shell

```
make cli ARGS="doctor"
make cli ARGS="new orders --archetype web-api"
cargo run -p firefly-cli --bin firefly -- info
```

What changed in Lumen

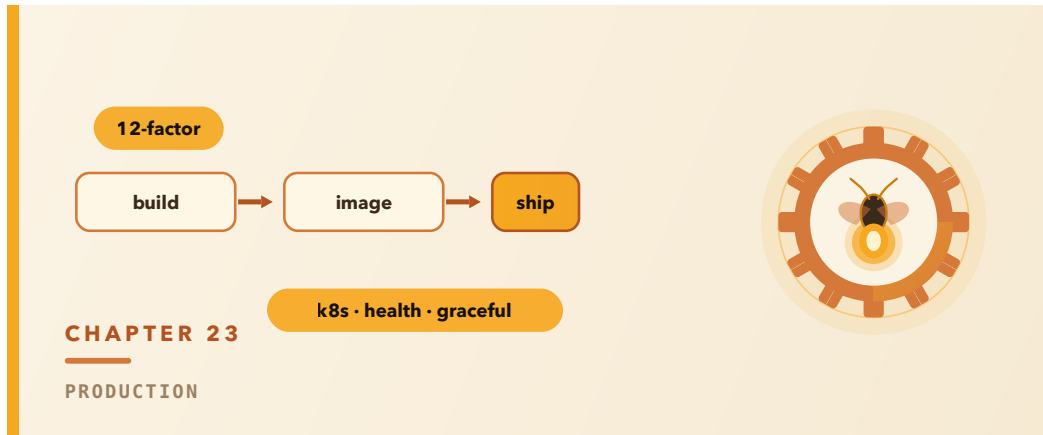
Nothing in `samples/lumen` itself — this chapter is operational. But you saw the CLI path to every artifact Lumen grew by hand: `firefly new --archetype web-api` scaffolds the chapter-2 skeleton, `firefly generate command/query/aggregate/saga` writes the CQRS, DDD, and orchestration pieces, `firefly run --bin lumen` launches it with `LUMEN_ADDR` overrides, and `firefly health/routes/beans --url :8081` introspects the actuator surface from chapter 15. The CLI never invents APIs — every command maps to a framework crate (`firefly-migrations`, `firefly-openapi`, the actuator endpoints) you have already met.

Exercises

1. **Scaffold a Lumen twin.** Run `firefly new lumen2 --archetype web-api --features web,cqrs --dry-run`, then without `--dry-run`. Compare the generated `src/` tree to Lumen's, and `cargo build` it.
2. **Generate the CQRS pieces.** In the scaffolded project, run `firefly generate command OpenWallet` and `firefly generate query GetWallet`. Inspect the generated handlers and note how they match the `#[command_handler]` / `#[query_handler]` shape from chapter 9.

3. **Run with a profile and an override.** Start the app with `firefly run -p dev -D server.port=9090 --dry-run` and read the resolved `FIREFLY_*` environment it would export. Then drop `--dry-run` and confirm the port.
4. **Introspect the real Lumen.** `cargo run --bin lumen`, then in another shell run `firefly health --url http://localhost:8081`, `firefly routes --url http://localhost:8081`, and `firefly beans --url http://localhost:8081`. Match the route table against the endpoint table in `web.rs`.

With a project scaffolded, generated, run, and introspected, the next chapter takes Lumen all the way to production. Continue to [Production & Deployment](#).



CHAPTER 23

Extending Firefly & Going to Production

Lumen has grown from a bare scaffold into a secure, observable, event-sourced CQRS service with a saga and a scheduled task. This last chapter of the build arc wires the **process entry point** — the thing that boots Lumen and keeps it running reliably — and turns on the optional **reactive streaming endpoint**. It also covers everything between "it works on my machine" and a service in production: graceful shutdown, the management split, configuration, packaging, and the swap from in-memory infrastructure to real Postgres + Kafka.

By the end of this chapter you will have read Lumen's `main.rs` end to end: a banner, two servers (public API + actuator) wired through the lifecycle `Application`, graceful SIGINT/SIGTERM shutdown, and the streaming endpoint (`GET /api/v1/wallets/:id/events` → NDJSON / SSE) behind the `streaming` feature.

 SPRING PARITY

The lifecycle `Application` is `SpringApplication.run()` : it traps the termination signals, drains in-flight work, and runs lifecycle hooks. Running the actuator on a second port is the Spring Boot `management.server.port` split. The reactive endpoint is WebFlux's streaming `Flux<T>` response.

The lifecycle and graceful shutdown

`firefly-lifecycle`'s `Application` traps SIGINT/SIGTERM, gives each server task its own drain signal, and grants a drain budget before exiting.

`Core::new_application()` (reachable through the `WebStack`'s `Deref`) builds one named after the app. This is the spine of Lumen's `src/main.rs` :

Rust

```

let api_addr = std::env::var("LUMEN_ADDR").unwrap_or_else(|_|
    DEFAULT_ADDR.to_owned());
let admin_addr =
    std::env::var("LUMEN_ADMIN_ADDR").unwrap_or_else(|_|
        DEFAULT_ADMIN_ADDR.to_owned());

let application = app
    .web
    .new_application()
    .on_server("api", move |shutdown| async move {
        let listener = tokio::net::TcpListener::bind(&api_addr).await?;
        axum::serve(listener, api)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    })
    .on_server("admin", move |shutdown| async move {
        let listener = tokio::net::TcpListener::bind(&admin_addr).await?;
        axum::serve(listener, admin)
            .with_graceful_shutdown(shutdown.wait())
            .await?;
        Ok(())
    });

if let Err(err) = application.run().await {
    if !err.is_cancelled() {
        eprintln!("application failed: {err}");
        std::process::exit(1);
    }
}

```

The key moves:

- **Two servers, two drains.** `on_server("api", ..)` and `on_server("admin", ..)` register the public API on `:8080` and the actuator on `:8081`. Each closure receives its own `shutdown` handle; passing `shutdown.wait()` to

`axum::serve(...).with_graceful_shutdown` drains that listener when the signal arrives. Multiple servers are allowed, each on its own task.

- **`run()` blocks until a signal.** It returns when SIGINT/SIGTERM is received and the drain completes. A clean shutdown surfaces as a *cancelled* error — `err.is_cancelled()` — which Lumen treats as success; any other error exits non-zero. (`on_start` / `on_stop` hooks and `app.shutdown_handle()` for programmatic shutdown round out the API; Lumen needs neither.)
- **Env-overridable binds.** `LUMEN_ADDR` and `LUMEN_ADMIN_ADDR` override the defaults, so a container reads its ports from the environment.

The full boot sequence

Read top to bottom, `main()` does exactly six things — build, log, assemble the two routers, start the scheduler, print the banner, run:

Rust

```

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let app = build_app().await;
    // Best-effort: a test harness may already own the global subscriber.
    let _ = app.web.init_logging();

    let api = app.router();
    let contributor: InfoContributor = Box::new(|| {
        let mut info = serde_json::Map::new();
        info.insert(
            "sample".into(),
            serde_json::json!({ "name": APP_NAME, "store": "in-memory",
"eventBus": "in-memory" }),
        );
        info
    });
    let admin = app.web.actuator_router(vec![contributor]);

    // Register and start the scheduled housekeeping task on a background task.
    let scheduler = build_scheduler();
    tokio::spawn(async move { scheduler.start().await });

    app.web.print_banner();
    println!("::: {APP_NAME} :: digital-wallet & ledger (v{VERSION})");

    // ... the lifecycle Application from the previous section ...
}

```

`build_app()` is the composition root from Chapter 4 — it wires the `WebStack`, the CQRS bus, the event-sourced ledger, the read model, the projection, and the security chain over in-memory infrastructure. `app.router()` (Chapter 14) adds the JWT bearer + RBAC layers. `print_banner()` emits the ASCII Firefly banner and the version tagline. Everything else you have already seen, chapter by chapter — `main.rs` just assembles it.

Teaching code runs with no dependencies. Lumen's binary boots with an in-memory event store and broker, so `cargo run --bin lumen` needs nothing external. The tests drive `build_router()` in-process rather than this binary.

The reactive streaming endpoint (feature `streaming`)

Lumen's last endpoint streams a wallet's event history. It is feature-gated so the teaching baseline stays lean — `Cargo.toml` declares it, and it needs nothing beyond the facade (`firefly::reactive::Flux` + `firefly::web::{NdJson, Sse}`):

TOML

```
[features]
default = []
streaming = []
```

The handler lives on a separate, feature-gated sub-router merged into `LumenApp::router`, so the macro-generated `routes()` never references a method that is compiled out. It loads the wallet's persisted events, maps them to the view shape, wraps them in a `Flux`, and returns NDJSON by default or SSE when `?format=sse` is passed:

Rust

```

async fn stream_events(
    State(api): State<WalletApi>,
    Path(id): Path<String>,
    axum::extract::Query(params): axum::extract::Query<StreamParams>,
) -> Response {
    use crate::domain::WalletEvent;
    use firefly::reactive::Flux;
    use firefly::web::{NdJson, Sse};

    // `load_events` returns `Err(NotFound)` for an absent wallet, so the 404 is
    // decided before the streaming response head is committed.
    let events = match api.ledger.load_events(&id).await {
        Ok(events) => events,
        Err(e) => return WebError::from(domain_to_web(e)).into_response(),
    };
    let items: Vec<WalletEvent> =
events.iter().map(WalletEvent::from_domain).collect();
    let flux = Flux::just(items);
    if params.format.as_deref() == Some("sse") {
        Sse(flux).into_response()
    } else {
        NdJson(flux).into_response()
    }
}

```

`NdJson(flux)` renders one JSON document per line (`application/x-ndjson`);
`Sse(flux)` renders Server-Sent Events (`text/event-stream`). The 404 for an unknown wallet is resolved *before* the response head is committed, because once a streaming body starts you can no longer change the status. `tests/streaming.rs` (run with `--features streaming`) proves all three behaviors:

Rust

```
#[tokio::test]
async fn events_stream_as_ndjson_by_default() {
    let id = open_with_deposit().await;           // two events: opened +
    deposited
    let res = build_router()
        .await
        .oneshot(Request::get(format!("/api/v1/wallets/{id}/
events"))).body(Body::empty()).unwrap()
        .await
        .unwrap();
    assert_eq!(res.status(), StatusCode::OK);
    // content-type contains "ndjson"; the body is two JSON lines:
    // one "WalletOpened", one "MoneyDeposited".
}
```

 SPRING PARITY

Returning a `Flux<T>` as `application/x-ndjson` or `text/event-stream` is exactly WebFlux's streaming response. `Flux::just` is Reactor's `Flux.just`; a production stream would use `Flux::from_stream` over a live subscription instead of a materialized `Vec`.

The management split in production

Always serve the actuator on a different listener from the public API so `/actuator/*` is reachable by your orchestrator but never on the public network — exactly the `:8080` / `:8081` split Lumen uses. Firewall the admin port. The health sub-paths feed your orchestrator's probes:

Probe	Endpoint
liveness	<code>/actuator/health/liveness</code>
readiness	<code>/actuator/health/readiness</code>
overall	<code>/actuator/health</code>

Production hardening middleware

Lumen's `WebStack::new` already turns on CORS, OWASP security headers, request metrics, and the access log (the web-tier batteries). The remaining pyfly-parity middleware is opt-in through `CoreConfig`, weaving in at the correct filter order:

Knob	Adds
<code>cors</code>	CORS preflight + simple-request decoration
<code>security_headers</code>	OWASP response headers (<code>nosniff</code> , <code>DENY</code> , HSTS, ...)
<code>csrf</code>	double-submit-cookie CSRF (for browser flows)
<code>request_log</code>	one structured access-log event per request
<code>request_metrics</code>	<code>http_server_requests_seconds</code> + <code>_max</code> (actuator)
<code>http_exchanges</code>	recent-exchange recorder + <code>/actuator/httpexchanges</code>
<code>loggers</code>	<code>/actuator/loggers</code> runtime log-level control

The effective chain (outermost → innermost) is CORS → Problem → SecurityHeaders → Correlation → Metrics → HttpExchanges → RequestLog → CSRF → Idempotency → your router. Idempotency stays innermost so a replayed request still passes every outer concern.

Configuration in production

Bind configuration from layered sources with environment overrides on top, so a container reads its settings from the environment (Chapter 3). For Lumen, the two bind addresses are already env-driven:

Shell

```
FIREFLY_PROFILE=prod \  
LUMEN_ADDR=0.0.0.0:8080 \  
LUMEN_ADMIN_ADDR=0.0.0.0:8081 \  
./lumen
```

`FIREFLY_*` variables beat the YAML files, secrets are masked in `/actuator/env`, and `${...}` placeholders resolve env-then-config-then-default. The JWT signing key (Chapter 14) is the obvious thing to inject this way rather than inline.

From in-memory to real infrastructure

This is the payoff of the whole architecture: Lumen swaps its in-memory defaults for real backends at the **composition root**, and nothing downstream changes. `build_app` constructs a `MemoryEventStore` and reads the `WebStack`'s in-memory broker; production replaces those two lines:

Rust

```
// build_app() today:
let store: Arc<dyn firefly::eventsourcing::EventStore> =
Arc::new(MemoryEventStore::new());
let broker = Arc::clone(&web.broker);

// production: a Postgres-backed event store and a Kafka broker, same ports.
// let store: Arc<dyn EventStore> = Arc::new(postgres_event_store); // firefly-
// eda-postgres
// let broker = Arc::new(kafka_broker); // firefly-
// eda-kafka
```

The **Ledger**, the projection, the CQRS handlers, the saga, and every test are written against the **EventStore** and **Broker** ports — so the wallet domain never learns it moved from a **HashMap** to Postgres + Kafka. That is "swap the adapter, keep the code," applied to the storage and messaging tiers.

Container packaging

A typical multi-stage build for Lumen:

Dockerfile

```
FROM rust:1.85 AS build
WORKDIR /app
COPY . .
RUN cargo build --release -p firefly-sample-lumen

FROM debian:bookworm-slim
COPY --from=build /app/target/release/lumen /usr/local/bin/lumen
EXPOSE 8080 8081
ENTRYPOINT ["/usr/local/bin/lumen"]
```

Because the lifecycle `Application` traps `SIGTERM` and drains, the container stops cleanly when the orchestrator sends a termination signal — no `--init` shim or signal-forwarding wrapper required.

A deployment checklist

- [] Actuator (`:8081`) on a separate, firewalled port from the API (`:8080`).
- [] Liveness/readiness probes pointed at `/actuator/health/{liveness,readiness}`.
- [] `security_headers`, `cors`, and (for browser flows) `csrf` on.
- [] `request_log` + `request_metrics` on; logs shipped as JSON, metrics scraped from `/actuator/prometheus`.
- [] Correlation propagation verified end-to-end across services.
- [] JWT signing key injected from the environment / a secret store, not inline.
- [] In-memory event store + broker swapped for Postgres + Kafka at `build_app`.
- [] Graceful-shutdown drain budget tuned for the slowest in-flight transfer.
- [] The verification gate green: `cargo test -p firefly-sample-lumen` and `--features streaming`, plus `clippy -D warnings` and `fmt --check`.

What changed in Lumen

This chapter wired the entry point and the optional stream — the last code the arc adds:

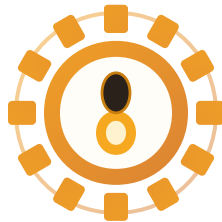
- `src/main.rs` boots the whole service: `build_app` → `init_logging` → assemble the API router and the actuator router (with the info contributor) → start the scheduler on a background task → `print_banner` → run the lifecycle `Application` with two `on_server` listeners and graceful `SIGINT`/`SIGTERM` drain. A clean shutdown is a *cancelled* error and exits zero.

- `Cargo.toml` declares the `streaming` feature; `src/web.rs` adds the feature-gated `streaming_router` / `stream_events` handler that returns a `Flux<WalletEvent>` as NDJSON (default) or SSE (`?format=sse`), with the 404 resolved before the response head.
- `tests/streaming.rs` proves the NDJSON default, the SSE switch, and the 404, all behind the `streaming` feature.
- The chapter showed the one-line `in-memory` → `Postgres` + `Kafka` swap at `build_app`, proving the port-and-adapter design end to end.

Exercises

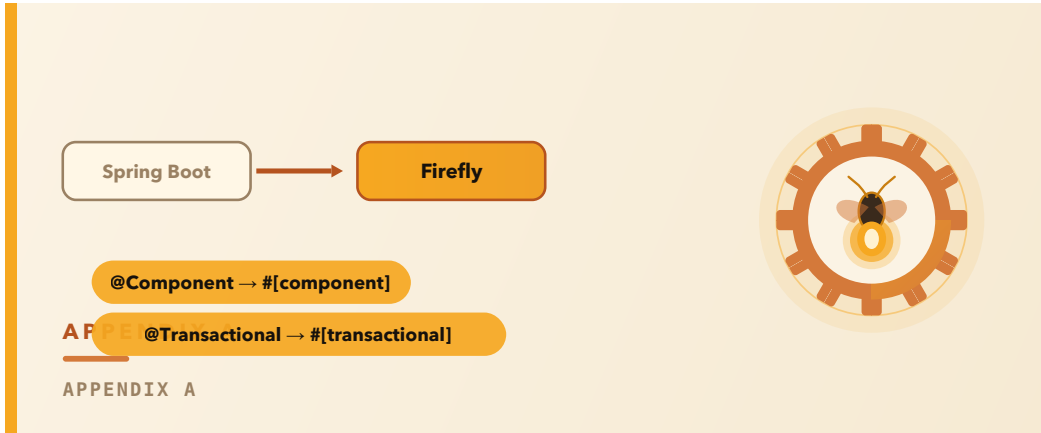
1. **Run and drain.** `cargo run --bin lumen`, open a wallet, then Ctrl-C and watch the graceful drain. Confirm the process exits zero.
2. **Override the ports.** Start Lumen with `LUMEN_ADDR=0.0.0.0:9000`
`LUMEN_ADMIN_ADDR=0.0.0.0:9001 cargo run --bin lumen` and confirm the API and actuator move.
3. **Stream the history.** Build with `--features streaming`, open a wallet and deposit, then `curl http://127.0.0.1:8080/api/v1/wallets/<id>/events` (NDJSON) and `...?format=sse` (SSE). Compare the two content types.
4. **Sketch the Postgres swap.** Write the two lines in `build_app` that would replace `MemoryEventStore` with a Postgres-backed `EventStore`, and explain in one sentence why `Ledger`, the projection, and the tests need no change.

That completes the guided tour of Lumen. The remaining chapters revisit the declarative macros as a capstone and provide reference material: a [Spring Boot migration map](#), the [Module Index](#), and the [Glossary](#).



Appendices





APPENDIX A

Spring Boot → Firefly Cheat-Sheet

If you are porting a Firefly Java/Spring Boot service to Rust — or simply come from a Spring background — this appendix is your translation table. Each Firefly Rust crate is wire-shape compatible with its Java counterpart but expressed in idiomatic Rust: explicit construction, `tower` middleware composition, and `async` / `await`.

Everything below is the same surface Lumen — the digital-wallet service this book builds — exercises. Where a row maps to a concrete Lumen file, that is the place to see it running.

Lumen at a glance

The fastest way to read this appendix is against Lumen's macros. Each Lumen declaration is a Spring annotation in idiomatic Rust:

Lumen (samples/lumen)	Spring / Firefly-Java
<code>#[rest_controller(path = "/api/v1")] + #[get] / #[post] (web.rs)</code>	<code>@RestController + @GetMapping / @PostMapping</code>
<code>#[derive(Command)] / #[derive(Query)] + #[command_handler] / #[query_handler] (commands.rs)</code>	<code>@CommandHandler / @QueryHandler</code>
<code>#[derive(AggregateRoot)] / #[derive(DomainEvent)] (domain.rs)</code>	Axon-style <code>@Aggregate / @EventSourcingHandler</code>
<code>#[event_listener(topic = "wallets.events")] (ledger.rs)</code>	<code>@KafkaListener / @RabbitListener</code>
<code>Saga::new(...).step(Step::with_compensation(...)) (transfer.rs)</code>	<code>@Saga + @SagaOrchestrationStart / compensation</code>
<code>JwtService + BearerLayer + FilterChain (security.rs)</code>	<code>spring-security resource server + @PreAuthorize</code>
<code>#[scheduled(fixed_rate = "60s")] (housekeeping.rs)</code>	<code>@Scheduled(fixedRate = 60000)</code>
<code>WebStack::actuator_router(...) + InfoContributor (main.rs)</code>	<code>spring-boot-starter- actuator + InfoContributor</code>

Module mapping

Java / Spring	Rust crate
<code>firefly-common</code>	<code>firefly-kernel</code>
<code>firefly-web</code> + <code>firefly-spring-utils</code>	<code>firefly-web</code>
<code>firefly-common-cache</code>	<code>firefly-cache</code>
<code>firefly-otel-spring-boot-starter</code>	<code>firefly-observability</code>
<code>firefly-common-data</code>	<code>firefly-data</code>
<code>firefly-common-cqrs</code>	<code>firefly-cqrs</code>
<code>firefly-common-eda</code>	<code>firefly-eda</code>
<code>firefly-event-sourcing-...</code>	<code>firefly-eventsourcing</code>
<code>firefly-common-domain</code> (orchestration)	<code>firefly-orchestration</code>
<code>firefly-service-client</code>	<code>firefly-client</code>
Spring WebFlux (Netty)	<code>axum</code> on <code>tokio</code> , <code>tower::Layer s</code>
Reactor <code>Mono</code> / <code>Flux</code>	<code>firefly_reactive::</code> { <code>Mono</code> , <code>Flux</code> }
<code>@ConfigurationProperties</code>	<code>firefly-config</code>
<code>SpringApplication</code>	<code>firefly-lifecycle</code>
<code>spring-boot-starter-actuator</code>	<code>firefly-actuator</code>
<code>@Scheduled</code>	<code>firefly-scheduling</code>
<code>resilience4j-*</code>	<code>firefly-resilience</code>
<code>spring-security</code>	<code>firefly-security</code>
Flyway	<code>firefly-migrations</code>
<code>springdoc-openapi</code>	<code>firefly-openapi</code>

Java / Spring	Rust crate
<code>MessageSource</code>	<code>firefly-i18n</code>
<code>ServerSentEvent</code>	<code>firefly-sse</code>
<code>@Transactional</code>	<code>firefly-transactional</code>
<code>spring-boot-starter-test</code>	<code>firefly-testkit</code>
<code>@Component</code> / <code>@Autowired</code> DI	<code>firefly-container</code> (opt-in)
<code>@Aspect</code> / Spring AOP	<code>firefly-aop</code>
<code>HttpSession</code> / Spring Session	<code>firefly-session</code>
Spring Shell <code>@ShellMethod</code>	<code>firefly-shell</code>
Spring WebSocket / <code>@ServerEndpoint</code>	<code>firefly-websocket</code>
Spring Boot Admin	<code>firefly-admin</code>
<code>@KafkaListener</code>	<code>firefly-eda-kafka</code>
Spring AMQP (RabbitMQ)	<code>firefly-eda-rabbitmq</code>
<code>spring-data-redis</code> cache	<code>firefly-cache-redis</code>
<code>JavaMailSender</code> (SMTP)	<code>firefly-notifications-smtp</code>

Reactive types

The Java framework uses Project Reactor — and so does the Rust port. `firefly-reactive` is a faithful `Mono` / `Flux` reimplementation, so a Reactor operator chain maps almost one-to-one (see [The Reactive Model](#)):

Project Reactor	firefly-reactive
<code>Mono<T></code>	<code>Mono<T></code>
<code>Flux<T></code>	<code>Flux<T></code>
<code>Throwable</code> (error signal)	<code>firefly_kernel::FireflyError</code> (fixed)
<code>Mono.empty()</code>	<code>Ok(None)</code> from a <code>Mono</code>
<code>Mono.error(new NotFound())</code>	<code>Mono::error(FireflyError::not_found("..."))</code>
<code>Schedulers.parallel()</code>	<code>Scheduler::Parallel</code>
<code>subscribeOn</code> / <code>publishOn</code>	<code>subscribe_on</code> / <code>publish_on</code>
<code>Mono.timeout(d)</code>	<code>Mono::timeout(d)</code>
<code>Flux.onBackpressureBuffer()</code>	<code>Flux::on_backpressure_buffer(n)</code>
<code>Retry.backoff(..)</code>	<code>Backoff</code> + <code>Mono::retry_backoff</code>
<code>Mono.toFuture()</code> / <code>await</code>	<code>Mono::into_future</code> / <code>.await</code>
<code>Flux.toStream()</code>	<code>Flux::to_stream</code>

NOTE

Ambient request metadata that Reactor carries in the subscriber context (`deferContextual`) travels in Rust task-local scopes instead — `firefly_kernel::correlation_id()`, `request_id()`, `tenant_id()` — set by the `CorrelationLayer` HTTP middleware. Cancellation, which Reactor models as subscription disposal, is future-drop in Rust (and a `CancellationToken` for the cooperative orchestration engines).

Annotation → macro cheat sheet

The declarative layer ([chapter 21](#)) maps the Spring annotation set onto `firefly-macros`. Every macro arrives through `use firefly::prelude::*;`:

Spring / pyfly annotation	Firefly-Rust macro	On
<code>@RestController</code> + <code>@GetMapping</code> / <code>@PostMapping</code>	<code>#[rest_controller(path)]</code> + <code>#[get]</code> / <code>#[post]</code> / <code>#[put]</code> / <code>#[delete]</code> / <code>#[patch]</code>	an <code>impl</code> block
<code>@CommandHandler</code> / <code>@QueryHandler</code>	<code>#[command_handler]</code> / <code>#[query_handler]</code>	<code>async fn(Msg) -> Result<R, CqrsError></code>
message + <code>@Valid</code> / <code>@Cacheable</code>	<code>#[derive(Command)]</code> / <code>#[derive(Query)]</code> (<code>#[firefly(validate)]</code> , <code>#[firefly(cache_ttl = "...")]</code>)	a message struct
<code>@KafkaListener</code> / <code>@RabbitListener</code>	<code>#[event_listener(topic = "...")]</code>	<code>async fn(Event) -> FireflyResult<()></code>
<code>@Scheduled(fixedRate = ...)</code> / <code>(cron = ...)</code>	<code>#[scheduled(fixed_rate = "...")]</code> / <code>#[scheduled(cron = "...")]</code>	a zero-arg <code>async fn</code>
Axon <code>@Aggregate</code> / event payloads	<code>#[derive(AggregateRoot)]</code> / <code>#[derive(DomainEvent)]</code>	a struct
<code>@Component</code> / <code>@Service</code> / <code>@Repository</code> / <code>@Configuration</code> / <code>@Controller</code>	<code>#[derive(Component/Service/Repository/Configuration/Controller)]</code>	a struct
<code>@Autowired</code> field/ctor injection	<code>#[autowired]</code> on an <code>Arc<T></code> / <code>Option<Arc<T>></code> / <code>Vec<Arc<T>></code> / <code>Provider<T></code> field	a component field
<code>@Bean</code> factory method	<code>#[bean]</code> (+ <code>#[bean(name/scope/primary/profile)]</code>)	a method on a <code>#[derive(Configuration)] impl</code>

Spring / pyfly annotation	Firefly-Rust macro	On
<code>@Primary</code> / <code>@Order</code> / <code>@Qualifier</code>	<code>#[firefly(primary)]</code> / <code>#[firefly(order = N)]</code> / <code>#[firefly(qualifier = "...")]</code>	a component / field
<code>@Profile</code> / <code>@ConditionalOnProperty</code> / <code>@ConditionalOnBean</code> / <code>@ConditionalOnMissingBean</code>	<code>#[firefly(profile = "...")]</code> / <code>condition_on_property</code> / <code>condition_on_bean</code> / <code>condition_on_missing_bean</code>	a component
<code>@PostConstruct</code> / <code>@PreDestroy</code>	<code>#[firefly(post_construct = "...")]</code> / <code>#[firefly(pre_destroy = "...")]</code>	a component
<code>@ConfigurationProperties(prefix)</code>	<code>#[derive(ConfigProperties)]</code> <code>#[firefly(prefix = "...")]</code>	a <code>serde::Deserialize</code> struct
<code>@ComponentScan</code> / <code>@EnableAutoConfiguration</code>	<code>Container::scan()</code> / <code>firefly::scan(&c)</code> (link-time inventory)	the container
explicit bean list	<code>register_all!(&c, [A, B, ...])</code>	—

See the [DI deep-dive](#) for the worked examples.

Service-tier mapping

Spring estate	Firefly-Rust starter
data/core service (owns the DB)	<code>firefly-starter-core</code> / <code>firefly-starter-data</code>
domain service (sagas, CQRS, ES)	<code>firefly-starter-domain</code>
edge / BFF service (<code>@Workflow</code> + <code>WebClient</code> aggregation)	<code>firefly-starter-experience</code> (<code>ExperienceStack</code> , <code>DomainClients</code> , <code>SignalService</code>)
<code>@WaitForSignal</code> workflow gate	<code>Node::wait_for_signal(...)</code> + <code>SignalService::deliver(...)</code>

See [The Experience Tier \(BFF\)](#).

Programming-model mapping

Spring concept	Rust entry point
<code>@SpringBootApplication</code> auto-config	<code>Core::new(CoreConfig { .. })</code>
<code>SpringApplication.run()</code>	<code>core.new_application().on_server(..).run().await</code>
<code>@Component</code> / <code>@Autowired</code>	explicit <code>Arc<dyn Trait></code> injection; <code>#[derive(Component)]</code> + <code>#[autowired]</code> for the container
<code>@RestController</code>	<code>#[rest_controller]</code> (or an axum handler on <code>core.apply_middleware(router)</code>)
<code>@RestController</code> returning <code>Mono<T></code>	a handler returning <code>MonoJson(Mono<T>)</code>
<code>@RestController</code> returning <code>Flux<T></code> (NDJSON)	a handler returning <code>NdJson(Flux<T>)</code> / <code>Sse(Flux<T>)</code>
<code>@ControllerAdvice</code> <code>ProblemDetail</code> handler	<code>ProblemLayer</code> + <code>WebResult<T></code> (? renders RFC 7807)
<code>@CommandHandler</code> / <code>@QueryHandler</code>	<code>bus.register(\ c: Cmd\ async move { .. })</code>
<code>@Cacheable</code>	<code>Typed::get_or_set(key, ttl, loader)</code>
<code>@Scheduled(cron = "0 9 * * MON-FRI")</code>	<code>scheduler.cron("name", "0 9 * * 1-5", run)</code>
<code>@CircuitBreaker</code> <code>@RateLimiter</code> <code>@Bulkhead</code>	<code>Chain::new().with(Timeout::...).with(CircuitBreaker::...)</code>
<code>@PreAuthorize("hasRole('ADMIN')")</code>	<code>FilterChain::new().require("/admin/", &["ADMIN"])</code> + <code>BearerLayer</code>
R2DBC <code>ReactiveCrudRepository<T, ID></code>	<code>firefly_data::ReactiveCrudRepository<T, ID></code>
<code>Pageable</code> / <code>Sort</code> / <code>PageRequest</code>	<code>firefly_data::{Pageable, Sort, Order}</code> + <code>Page<T></code>
Axon / event-sourced aggregates	<code>firefly_eventsourcing</code> (<code>AggregateRoot</code> , <code>EventStore</code>)
Temporal / Camunda workflows	<code>firefly_orchestration::Workflow</code> (DAG + compensation)

Spring concept	Rust entry point
<code>@Transactional</code>	<code>with_tx(ctx, &db, \ ctx\ { .. })</code>
Flyway <code>V001__init.sql</code>	<code>firefly_migrations::run</code> over a <code>Database</code> port
<code>WebClient</code>	<code>firefly_client::WebClientBuilder</code> → <code>body_to_mono</code> / <code>body_to_flux</code>
<code>RestTemplate</code> / <code>RestClient</code>	<code>firefly_client::RestBuilder</code> → <code>client.request(..)</code>
<code>MessageSource.getMessage(...)</code>	<code>bundle.t(locale, key, args)</code>
<code>@SpringBootTest</code>	<code>tower::ServiceExt::oneshot</code> + <code>firefly_testkit</code>

What changes, and what stays the same

Stays the same — the wire contracts. The `application/problem+json` shape, the `Idempotency-Key` semantics, the event envelope JSON, the saga step definitions, the HMAC webhook signatures, the `Page<T>` JSON, the `Authentication` claims mapping, and the configuration keys are byte-identical to the Java release line. A Java service and a Rust service interoperate on the wire with no adapter.

Changes — the wiring style. Spring's reflective DI and annotation scanning become explicit construction (`Core::new`, constructor injection, optional `Container`). Annotation-driven aspects become explicit weaving at the call site. There is no leading `ctx` parameter and no XML — ambient state lives in task-locals, and dependencies are `Arc<dyn Trait>` fields the compiler checks.

A worked port

A Spring WebFlux controller:

Java

```

@RestController
class OrderController {
    @GetMapping("/orders/{id}")
    Mono<Order> get(@PathVariable String id) {
        return service.find(id)
            .switchIfEmpty(Mono.error(new ResourceNotFound("order " + id)));
    }
}

```

becomes a Firefly Rust handler:

Rust

```

use axum::extract::Path;
use firefly_reactive::Mono;
use firefly_web::MonoJson;

async fn get_order(Path(id): Path<String>) -> MonoJson<Order> {
    // Mono::empty() (Ok(None)) renders a 404 problem automatically – no
    // switchIfEmpty needed; an Err(FireflyError) renders that error's problem.
    MonoJson(service.find(&id))
}

```

The `Mono<T>` model carries over directly; the empty-Mono → 404 behaviour is built into the `MonoJson` responder, so the `switchIfEmpty` boilerplate disappears.

For the full crate catalogue, see the [Module Index](#).



APPENDIX B

Crate & Module Index

The Firefly Rust workspace ships **78 members** — 73 framework crates under `crates/`, plus the cross-crate integration suite and four samples: the book's running example `lumen`, alongside `orders`, `reactive-banking`, and `macro-quickstart`. Each crate carries its own `README.md` with its full public surface, design rationale, and a runnable quick-start.

The canonical, always-current catalogue is `MODULES.md` in the repository root. This appendix reproduces it tier-by-tier as a navigable index.

Front door

Crate	What it provides
<code>firefly</code>	The one-dependency facade: <code>use firefly::prelude::*;</code> , per-crate aliases, the <code>__rt</code> macro contract, feature-gated adapters
<code>firefly-macros</code>	Declarative macros: <code>#[derive(Command/Query/Component/DomainEvent/AggregateRoot)]</code> , <code>#[command_handler]</code> / <code>#[query_handler]</code> , <code>#[scheduled]</code> , <code>#[rest_controller]</code> + verbs, <code>#[event_listener]</code>

Foundational

Crate	What it provides
<code>firefly-kernel</code>	RFC 7807 <code>ProblemDetail</code> , <code>FireflyResult<T></code> , <code>Clock</code> , the <code>FireflyError</code> hierarchy, task-local correlation/request/tenant scopes, the <code>ddd</code> kit (<code>Entity</code> / <code>Specification</code> / domain events)
<code>firefly-reactive</code>	The <code>Mono<T></code> / <code>Flux<T></code> reactive core — the Project Reactor analog and keystone of every reactive surface
<code>firefly-utils</code>	Try helpers, retry with exponential backoff, slug, AES-256-GCM, templates
<code>firefly-validators</code>	IBAN, BIC, Luhn, currency, phone, password, VAT, national/tax IDs
<code>firefly-web</code>	Problem renderer, correlation, idempotency, PII masking, CORS, CSRF, security headers, server metrics, the reactive <code>MonoJson</code> / <code>NdJson</code> / <code>Sse</code> responders, TLS bootstrap
<code>firefly-config</code>	Typed YAML+env+flag binding, profiles, <code>\${...}</code> placeholders, runtime reload, masked property sources, <code>ApplicationEventBus</code> , config-server client
<code>firefly-i18n</code>	Locale-aware message <code>Bundle</code> + Accept-Language picker
<code>firefly-session</code>	Server-side HTTP sessions + <code>SessionStore</code> + <code>SessionLayer</code>

Platform

Crate	What it provides
<code>firefly-cache</code>	<code>Adapter</code> port + Memory/NoOp/Fallback + typed <code>Typed<T></code> memoisation
<code>firefly-observability</code>	<code>tracing</code> + correlation enrichment, health composite, startup banner, W3C trace-context, metrics
<code>firefly-data</code>	Storage-agnostic ports: Filter DSL + <code>Specification</code> , <code>SqlDialect</code> (pg/mysql/sqlite) + <code>Specification::to_mongo()</code> , <code>Page<T></code> , <code>Repository<T, K></code> , <code>ReactiveCrudRepository</code> , <code>PostgresReactiveRepository</code> , auditing + soft-delete, derived queries, paging
<code>firefly-cqrs</code>	Command/query <code>Bus</code> with validation/cache/authorization middleware + reactive <code>send_mono</code> / <code>query_mono</code>
<code>firefly-eda</code>	<code>Event</code> envelope, <code>Publisher</code> / <code>Subscriber</code> / <code>Broker</code> ports, <code>InMemoryBroker</code> , glob topics, consumer groups, retry/DLQ, reactive <code>Flux</code> subscriptions
<code>firefly-eventsourcing</code>	Aggregate roots, event store, snapshots, projections, global stream, transactional outbox, multi-tenancy
<code>firefly-orchestration</code>	<code>Saga</code> , <code>Workflow</code> (DAG), <code>Tcc</code> — compensation, per-step retry, <code>StepContext</code>
<code>firefly-rule-engine</code>	YAML DSL → AST → evaluator with <code>between</code> / null / <code>regex</code> , <code>EvaluationMode</code> , validator + <code>ActionHandler</code>
<code>firefly-plugins</code>	Lifecycle SPI + composite registry
<code>firefly-lifecycle</code>	<code>Application::run()</code> orchestrator with signal trap + drain
<code>firefly-actuator</code>	<code>/actuator/{health,info,metrics,env,tasks,version}</code> + probes, loggers, httpexchanges, threaddump, refresh
<code>firefly-scheduling</code>	Cron + FixedRate + FixedDelay <code>Scheduler</code> with zones

Crate	What it provides
<code>firefly-resilience</code>	<code>CircuitBreaker</code> , <code>RateLimiter</code> , <code>Bulkhead</code> , <code>Timeout</code> , composable <code>Chain</code>
<code>firefly-security</code>	<code>BearerLayer</code> , <code>RBAC</code> , <code>FilterChain</code> , <code>JwksVerifier</code> , <code>oauth2</code> , <code>RoleHierarchy</code> , <code>CsrfLayer</code> , <code>BcryptPasswordEncoder</code>
<code>firefly-migrations</code>	Versioned forward-only SQL migrations over a <code>Database</code> port
<code>firefly-openapi</code>	OpenAPI 3.1 generator + Swagger-UI shim
<code>firefly-sse</code>	Server-Sent Events writer with heartbeat + Last-Event-Id
<code>firefly-transactional</code>	<code>with_tx(ctx, db, f)</code> over pluggable <code>Database</code> / <code>Transaction</code> ports
<code>firefly-testkit</code>	HMAC signers, <code>SpyBroker</code> , JSON test helpers
<code>firefly-aop</code>	Spring-style aspect advice — <code>Pointcut</code> , <code>JoinPoint</code> , <code>Aspect</code> , <code>intercept</code>
<code>firefly-shell</code>	Spring-Shell-style CLI framework + <code>CommandLineRunner</code> / <code>ApplicationRunner</code>
<code>firefly-websocket</code>	WebSocket server over axum + topic <code>BroadcastHub</code>

Adapters

Crate	Port → backend
<code>firefly-client</code>	REST <code>RestClient</code> + reactive <code>WebClient</code> + SOAP/gRPC/GraphQL/WS
<code>firefly-config-server</code>	Spring-Cloud-Config-compatible REST endpoint
<code>firefly-idp</code> + <code>idp-internal-db</code> / <code>idp-keycloak</code> / <code>idp-azure-ad</code> / <code>idp-aws-cognito</code>	Identity providers
<code>firefly-ecm</code> + <code>ecm-storage-aws</code> / <code>ecm-storage-azure</code> / <code>ecm-esignature-*</code>	Content management + e-signature
<code>firefly-notifications</code> + <code>notifications-smtp</code> / <code>-twilio</code> / <code>-firebase</code> / <code>-sendgrid</code> / <code>-resend</code>	Notification channels
<code>firefly-callbacks</code>	Outbound webhook subsystem (HMAC dispatcher + audit + admin)
<code>firefly-webhooks</code>	Inbound ingestion (Stripe / GitHub / Twilio / generic HMAC + DLQ)
<code>firefly-data-sqlx</code>	<code>firefly_data</code> ports → relational (Postgres / MySQL / SQLite over <code>sqlx</code>)
<code>firefly-data-mongodb</code>	<code>firefly_data</code> ports → document store (MongoDB)
<code>firefly-cache-redis</code> / <code>firefly-cache-postgres</code>	<code>cache::Adapter</code> → Redis (RESP) / Postgres key-value table
<code>firefly-eda-kafka</code> / <code>-rabbitmq</code> / <code>-postgres</code> / <code>-redis</code>	<code>eda::Broker</code> → Kafka / RabbitMQ / Postgres outbox / Redis Streams
<code>firefly-session-redis</code> / <code>firefly-session-postgres</code>	distributed <code>SessionRegistry</code> → Redis sorted set / Postgres table

Starters

Crate	What it bundles
<code>firefly-starter-core</code>	web + cache + observability + eda + cqrs + actuator + lifecycle + scheduling
<code>firefly-starter-application</code>	starter-core + plugins registry
<code>firefly-starter-domain</code>	starter-core + in-memory event-sourcing stores
<code>firefly-starter-data</code>	starter-core (you supply the DB)
<code>firefly-starter-web</code>	<code>WebStack</code> — <code>Core</code> + CORS + security headers + request metrics + access log
<code>firefly-starter-experience</code>	<code>ExperienceStack</code> (alias <code>Bff</code>) — <code>WebStack</code> + <code>DomainClients</code> (the <code>ClientFactory</code>) + <code>SignalService</code> gates + Redis-capable <code>WorkflowState</code> + <code>WorkflowQueryService</code> + <code>ChildWorkflowService</code> — the experience (BFF) tier
<code>firefly-backoffice</code>	starter-application + back-office context middleware

DI / Operations / Tooling

Crate	What it provides
<code>firefly-container</code>	Opt-in <code>TypeId</code> -keyed DI container (service locator)
<code>firefly-admin</code>	Spring-Boot-Admin-style embedded dashboard + JSON API + SSE
<code>firefly-cli</code>	The <code>firefly</code> developer binary (<code>new</code> / <code>generate</code> / <code>db</code> / <code>openapi</code> / <code>actuator</code> / <code>doctor</code> / <code>completion</code> / <code>sbom</code> / <code>license</code>)

For per-crate detail, open the crate's `README.md` in the [repository](#), or read the `MODULES.md` catalogue.

Glossary

Definitions of the terms and types that recur throughout this book, in the precise sense Firefly uses them.

Actuator

The management surface (`firefly-actuator`) exposing the endpoints `/actuator/health`, `/actuator/info`, `/actuator/metrics`, `/actuator/env`, `/actuator/tasks`, `/actuator/version`, `/actuator/loggers`, `/actuator/httpexchanges`, and `/actuator/refresh`. Mounted by `core.actuator_router(..)`, typically on a separate, firewalled port.

Adapter

A concrete implementation of a **port**. Selected at wiring time as an `Arc<dyn Port>` so heavy SDK dependencies stay out of services that do not use them. Examples:

`RedisAdapter` (a `cache::Adapter`), `KafkaBroker` (an `eda::Broker`).

Aggregate root

The consistency boundary in DDD. The non-event-sourced variant holds a `PendingEvents<E>` buffer (`firefly_kernel::ddd`); the event-sourced variant is an `AggregateRoot` whose state is rebuilt by replaying its `DomainEvent`s (`firefly-eventsourcing`).

Autowired

A component field the DI container resolves and injects by type (`#[autowired]`, `firefly-container`). The field type sets the shape: `Arc<T>` (required), `Option<Arc<T>>` (optional), `Vec<Arc<T>>` (all implementations), `Provider<T>` (deferred). The Rust analog of Spring's `@Autowired`.

Backpressure

A slow consumer throttling a fast producer. Firefly's reactive `Flux` streams (NDJSON/SSE responders, `WebClient::body_to_flux`, Postgres row streams) honour backpressure end-to-end so a large stream never lands fully in memory.

Bean

Any value the DI `Container` builds, wires, and owns. Declared with a `stereotype` derive (`#[derive(Component/Service/Repository/Configuration/ Controller)]`) or produced by a `#[bean]` factory method on a `#[derive(Configuration)]` holder. Keyed by `TypeId`; resolvable with `resolve::<T>()`.

BFF (Backend-for-Frontend)

See Experience tier.

Bus

The CQRS command/query dispatcher (`firefly_cqrs::Bus`). Handlers register by input type; dispatch is keyed on `std::any::TypeId`. `send` / `query` are async; `send_mono` / `query_mono` are the reactive twins.

Compensation

The undo step a `saga` runs in reverse order when a later step fails (`Step::with_compensation`). In Lumen's transfer saga, the debit's compensation is a refund deposit on the source wallet.

CompensationPolicy

How a saga/workflow rolls back on failure: `BestEffort` (continue compensating even if one compensation fails) or `StopOnError` (abort at the first compensation failure).

Component scanning

Link-time bean discovery (`Container::scan()` / `firefly::scan`): every non-generic stereotype derive submits an `inventory` thunk, and `scan` collects them across the crate graph, applies **conditions** and **profiles**, and registers the survivors. The Rust analog of Spring's `@ComponentScan` (link-time, not reflective). Generic beans use the `register_all!` fallback.

Conditional bean

A bean registered only when the environment matches — `#[firefly(profile = "...", condition_on_property = "k=v", condition_on_bean = "T", condition_on_missing_bean = "T", condition_on_class = "label", condition_on_single_candidate = "T")]`. Evaluated by `scan` in two passes (config/profile facts first, registry-dependent checks second). Spring's `@Profile` / `@ConditionalOn*`.

Container

The opt-in, `TypeId`-keyed DI service locator (`firefly-container`): registers beans, resolves by type or name, supports scopes, trait-object bindings, `primary` / `order` disambiguation, deferred `Provider<T>`, lifecycle hooks, and component scanning. Distinct from **Core** (the wired infrastructure bundle).

Core

The wired infrastructure bundle returned by `Core::new(CoreConfig)` (`firefly-starter-core`): cache, CQRS bus, event broker, health composite, metrics, scheduler, logging, and the middleware chain.

Correlation id

A per-request identifier carried in the `X-Correlation-Id` header and a kernel task-local scope. It auto-enriches every log line, every published event, and every outbound client call so a request stitches together across services.

DomainEvent

The event-sourced, versioned, wire-formatted event in `firefly-eventsourcing` (distinct from the transient `TransientDomainEvent` in `firefly_kernel::ddd`). Its JSON is byte-compatible across the ports.

Event (EDA)

The envelope every `firefly-eda` event flows through — `id`, `type`, `source`, `topic`, `correlationId`, `time`, `headers`, `payload`, `key`. Constructed with `Event::new`, wire-compatible across the ports.

Experience tier (BFF)

The top service tier (`firefly-starter-experience`, `ExperienceStack` / `Bff`): a Backend-for-Frontend that composes several **domain** SDKs into journey-specific, atomic REST endpoints. It owns no database and calls only domain services (`channel` → `experience` → `domain` → `core`). Built from `DomainClients` (the `ClientFactory`), `SignalService` gates, Redis-capable `WorkflowState`, and `WorkflowQueryService`.

FireflyError

The framework's error type (`firefly_kernel::FireflyError`). It renders as an RFC 7807 `application/problem+json` response, and it is the fixed error channel of the reactive `Mono` / `Flux` (their terminal `Err` signal).

FilterChain

The path-based authorization matcher in `firefly-security` (`permit` / `require` / glob `permit_pattern` / `require_pattern`). Fail-closed once any rule is declared (Spring Security 6 deny-by-default).

Flux

A reactive publisher of $0..N$ values plus a terminal completion-or-error (`firefly_reactive::Flux`). The Rust analog of Reactor's `Flux<T>`.

Idempotency

The replay behaviour applied to `POST` / `PUT` / `PATCH` requests carrying an `Idempotency-Key` header. A repeat replays the stored response (`Idempotent-Replay: true`); reuse with a different body is a 409.

Mono

A reactive publisher of *at most one* value plus a terminal error (`firefly_reactive::Mono`). The Rust analog of Reactor's `Mono<T>` . An empty `Mono` (`Ok(None)`) is the equivalent of `Mono.empty()` .

NDJSON

Newline-delimited JSON (`application/x-ndjson`) — one compact JSON document per line. The `NdJson(Flux<T>)` responder streams it with backpressure.

Outbox (transactional)

A pattern (`TransactionalOutbox` , `firefly-eda-postgres`) that writes events in the same transaction as the state change and delivers them to consumers afterward, giving at-least-once delivery without a separate broker.

Port

An object-safe `async_trait` trait defining an integration point — `cache::Adapter` , `eda::Broker` , `notifications::Channel` , `idp::Adapter` . Code depends on the port; an adapter implements it.

Primary

The disambiguator (`#[firefly(primary)]`) that picks one bean when several implementations are bound to the same port. Resolving with no primary among multiple candidates is a `NoUniqueBean` error naming every candidate. Spring's `@Primary` .

Problem (RFC 7807 / 9457)

The `application/problem+json` error envelope (`type` , `title` , `status` , `detail`) that every Firefly service renders for errors and panics, identical across the ports. RFC 9457 obsoletes and is wire-compatible with RFC 7807; the book uses both numbers interchangeably.

Profile

A named environment (`prod` , `dev` , `test`) that gates conditional beans (`#[firefly(profile = "expr")]`). The expression grammar supports `&` , `|` , `!` , comma-as-OR, and parentheses (Spring Boot 2.4+). Active profiles live on the `ApplicationContext` / `ConditionContext` .

Projection

A read-side handler that builds a read model from events (`firefly_eventsourcing::Projection`), driven per-aggregate (`replay`) or over the global stream (`drive_once` / `replay_all`).

Qualifier

A name used to select a specific bean when several share a type (`#[firefly(qualifier = "replica")]`) → `resolve_named`). Spring's `@Qualifier` .

Reactive

The `Mono` / `Flux` programming model (`firefly-reactive`) and everything built on it — reactive endpoints, repositories, the `WebClient` , reactive EDA/CQRS. The Rust analog of Project Reactor / Spring WebFlux.

Saga

A sequential distributed-transaction engine (`firefly_orchestration::Saga`) with reverse-order compensation on failure. See also `Workflow` (DAG) and `Tcc` .

Scheduler

The task runner (`firefly_scheduling::Scheduler`) owning Cron, FixedRate, and FixedDelay triggers, each on its own tokio task with panic recovery.

Scope

A bean's lifecycle (`#[firefly(scope = "...")]`): `singleton` (one cached instance, the default), `transient` (fresh per resolve), `request`, or `session` (both driven by a `ScopeHandler`). Spring's `singleton` / `prototype` (Firefly: `transient`) / `request` / `session`.

Signal

An external event that satisfies a parked workflow gate in the **experience tier** (`SignalService::deliver` / `Node::wait_for_signal`). Delivery is buffered, so a signal that arrives before the gate parks is not lost. Spring's `@WaitForSignal`.

SSE (Server-Sent Events)

A one-way streaming protocol (`text/event-stream`). The `Sse(Flux<T>)` responder and `firefly-sse`'s `SseWriter` emit it; `WebClient::body_to_flux` decodes it.

Specification

A composable business-rule predicate (`firefly_kernel::ddd::Specification<T>`) combined with `.and()`, `.or()`, `.not()`. Any `Fn(&T) -> bool` is one.

Starter

A crate that bundles a sensible default stack so a service depends on one crate. `firefly-starter-core` is the common starting point; `firefly-starter-domain` and `firefly-starter-experience` add the domain and BFF tiers.

Stereotype

The architectural-role label a DI bean carries (`component` , `service` , `repository` , `configuration` , `controller` , `bean`), set by which derive declared it. Functionally equivalent; the differences are the documented intent and the grouping shown in the admin `/beans` view. Spring's `@Component` family.

TCC (Try-Confirm-Cancel)

A two-phase distributed-transaction engine (`firefly_orchestration::Tcc`): Try all participants, Confirm all on success, Cancel the tried participants on any Try failure.

Value object

A domain type defined entirely by its attributes (no identity) and **immutable**: every operation returns a new value. Lumen's `Money` is the canonical example — exact integer-cent arithmetic, closed under `add` / `subtract` . The DDD counterpart of an **aggregate root** (which has identity).

Verifier

The async port (`firefly_security::Verifier`) that validates a bearer token and returns an `Authentication` . `JwksVerifier` , IDP adapters, and `VerifierFn` closures all satisfy it.

WebClient

The reactive HTTP client (`firefly_client::WebClient`) whose terminal operators return `Mono` / `Flux` (`body_to_mono` , `body_to_flux` , `exchange`). The Rust analog of WebFlux's `WebClient` .

Workflow

A DAG distributed-transaction engine (`firefly_orchestration::Workflow`): independent nodes run concurrently within a wave, with reverse-order compensation under a configurable `CompensationPolicy` . In the **experience tier**, a node may park on a **signal** gate.

WorkflowState

Redis-capable persisted journey state in the **experience tier** (`firefly_starter_experience::WorkflowState`): round-trips a workflow run's `StepContext` snapshot through the cache `Adapter` , keyed by correlation id, so a parked journey survives a client disconnect (`save` / `load` / `delete`).